

Grundkurs Informationsingenieurwesen

Graphbasierte Modellierung, Beschreibungssprachen, Transformation
und modellgetriebene Code-Generierung für Echtzeitsysteme

Stephan Epp

Juni 2026

Inhaltsverzeichnis

Vorwort	7
1 Was ist Informationsingenieurwesen?	10
1.1 Das Ausgangszitat als wissenschaftliches Fundament	10
1.2 Das Schärfungsprinzip: Von der quantitativen Annotation zur geteilten Verantwortung	12
1.3 Überblick und Beiträge dieser Arbeit	13
2 Berufsbild: Der Informationsingenieur	14
2.1 Definition und Begriffsklärung	14
2.2 Abgrenzung zu verwandten Berufsbildern	15
3 Grundlagen modellgetriebener Vorgehensmodelle	18
3.1 UML-Profile	18
3.2 MARTE - Modeling and Analysis of Real-Time and Embedded Systems . .	18
3.3 Markov-Ketten für Zustandsübergänge	20
3.4 Warteschlangentheorie für Performance-Analyse	20
3.5 Quantitative Analysis of Software Designs	22
3.6 XMI (XML Metadata Interchange)	22
3.7 Model-Driven Engineering	23
3.8 Echtzeit-Constraints	23
4 Anforderungsanalyse: Motivation und Zielsetzung	24
4.1 Motivation	25
4.2 Zielsetzung	26
4.3 Aufbau der Arbeit	27
5 Grobdesign: Graphbasierte Modellierung als optimales Paradigma	28
5.1 Formale Grundlagen der Graphdarstellung	28
5.2 Das Graphmodell-Theorem	28
5.3 Ein durchgängiges Beispiel: Der Anforderungsgraph der Aufzugstür-Steuerung	29
6 Vom Graphmodell zum Programm: Beschreibungssprache und Transformation	33
6.1 Anforderungen an die Beschreibungssprache	33
6.2 Das Mächtigkeitstheorem	34
6.3 Die Rolle der Beschreibungssprache als letzte Sprache vor der Transformation	34
6.3.1 Ingenieurverantwortung: Die Transformation	34

6.3.2	Das Transformationsmodell	34
6.3.3	Das Ingenieurziel: Zielnähe bei minimaler Wartungslast	35
7	Konkretisierungen im Feindesign: Systemarchitektur	39
7.1	Erweiterte Architektur mit Quantitativer Analyse	40
7.2	Quantitative Analysepipeline	40
7.3	Komponentenübersicht	40
7.4	Datenfluss durch die Pipeline: Ein konkretes Beispiel	40
7.5	Architekturentscheidungen und ihre Begründung	42
7.6	Grenzen der vorgestellten Architektur	43
8	Optimierung des Designmodells: Code-Generierung, Notwendigkeit und Grenzen	44
8.1	Das Vollständigkeitsproblem	44
8.2	Theorem über die Unnötigkeit der Code-Generierung	45
8.3	Design Patterns als Optimierung des Designmodells	45
8.3.1	Strategy-Pattern: Erweiterbarkeit durch Austauschbarkeit	46
8.3.2	Observer-Pattern: Analysierbarkeit durch entkoppelte Benachrichtigung	47
8.3.3	Factory-Method-Pattern: Wiederverwendbarkeit durch Kapselung der Erzeugung	48
8.3.4	Zusammenspiel der Patterns: Architekturübersicht	48
8.3.5	Unmittelbare Wirkung im Entwicklungsprozess	49
9	Implementierungsaspekte	50
9.1	Projektstruktur	50
9.1.1	Module in <code>src/</code>	51
9.1.2	Module in <code>tests/</code>	53
9.2	Beispiele	55
9.3	Erweiterung von UML-MARTE um Stabilitätskriterien	58
9.3.1	Motivation und Konzept	58
9.3.2	Neue Stereotypen der MARTE-Erweiterung	58
9.3.3	Python-Implementierung	60
9.3.4	Anwendungsbeispiel: Erweiterte Aufzugstür-Fallstudie	62
9.3.5	Gleichgewicht versus Stabilität auf Modellebene	62
9.3.6	Integration in den zweistufigen Generierungsprozess	62
9.3.7	Diskussion und Grenzen	64
9.4	Demo: <code>markov_analyzer</code> und <code>queue_analyzer</code>	64
9.4.1	Überblick und Designprinzipien	64
9.4.2	Modul <code>markov_analyzer.py</code>	65
9.4.3	Numerische Kernalgorithmen	65
9.4.4	Klasse <code>MarkovState</code>	66
9.4.5	Klasse <code>DiscreteMarkovChain</code>	67
9.4.6	Klasse <code>ContinuousMarkovChain</code>	68
9.4.7	Klasse <code>MarkovAnalyzer</code>	70
9.4.8	Modul <code>queue_analyzer.py</code>	70
9.4.9	Klasse <code>QueueMetrics</code>	70
9.4.10	Klasse <code>MM1Queue</code>	71
9.4.11	Klasse <code>MMcQueue</code>	71

9.4.12	Klassen MD1Queue und MG1Queue	72
9.4.13	Klasse ServiceSystemAnalyzer	72
9.4.14	Test-Suiten	72
9.4.15	Test-Suite test_markov_analyzer.py	72
9.4.16	Test-Suite test_queue_analyzer.py	75
9.4.17	Demo-Anwendung: Krankenhaus-Notaufnahme	75
9.4.18	Szenario 1: Normalbetrieb ($\lambda = 8$ Pat/h)	76
9.4.19	Szenario 2: Spitzenlast (Massenunfall, $\lambda = 20$ Pat/h)	76
9.4.20	Szenario 3: Kapazitätsplanung (Ziel-Auslastung 70 %)	77
9.4.21	Szenario 4: Deadline-Compliance (Echtzeit-Anforderungen)	77
9.4.22	Auswertung der Demo-Ausgabe	78
9.4.23	Auswertung Szenario 1: Normalbetrieb	78
9.4.24	Auswertung Szenario 2: Spitzenlast	81
9.4.25	Auswertung Szenario 3: Kapazitätsplanung	82
9.4.26	Auswertung Szenario 4: Deadline-Compliance	84
9.4.27	Gesamtbewertung der Demo-Ergebnisse	85
9.4.28	Stärken beider Module im Vergleich	87
9.4.29	Stärken des markov_analyzer-Moduls	87
9.4.30	Stärken des queue_analyzer-Moduls	87
9.4.31	Komplementäre Zusammenwirkung	87
9.4.32	Zusammenfassung	88
10	Anwendungsbeispiel: Die Aufzugstür-Steuerung	89
10.1	Domänenanalyse	89
10.2	Markov-Modell der Aufzugstür	89
10.3	Warteschlangenmodell	90
10.4	Zeitliche Analyse	91
10.5	Performance-Vorhersage	91
10.6	Vergleich mit MARTE	91
10.7	Vorteile der Quantitativen Analyse	92
10.8	Limitierungen	92
11	Qualitätssicherung: Little's Law, Stabilitätskriterien und ihre experimentelle Evaluation	93
11.1	Fundamentale Definitionen	93
11.1.1	Little's Law	93
11.1.2	Stabilität in dynamischen Systemen	94
11.1.3	Energiefunktionen und Lyapunov-Theorie	94
11.2	Hauptresultat: Little's Law ist kein Stabilitätskriterium	94
11.3	Verallgemeinerung: Naturwissenschaftliche Systembeschreibungen	96
11.3.1	Die besondere Rolle der Exponentialfunktion	96
11.3.2	Energiefunktionen in naturwissenschaftlichen Systemen	96
11.3.3	Konsequenzen für Systemklassifikation	97
11.4	Anwendung auf Warteschlangensysteme	98
11.4.1	Dualität der Beschreibungsebenen	98
11.4.2	Beispiel: M/M/1-System	98
11.5	Konsequenzen für UML-basierte Systementwicklung	99
11.5.1	Die Brücke zwischen Informatik und Naturwissenschaft	99
11.5.2	Rolle von UML-Profilen	99

11.5.3	Integration quantitativer Analysen	100
11.6	Schlussfolgerungen und Synthese	100
11.7	Weiterführende Forschungsfragen	101
11.8	Experimentelle Evaluation der Stabilitätskriterien	101
11.8.1	Implementierungsarchitektur	102
11.8.2	Kernklassen und Methoden	102
11.8.3	Experimentelle Ergebnisse	105
11.8.4	Experiment 1: Little's Law ist kein Stabilitätskriterium	105
11.8.5	Experiment 2: Exponentieller Zerfall in stabilen Systemen	106
11.8.6	Experiment 3: Hierarchie der Beschreibungsebenen	109
11.8.7	Gesamtauswertung und Diskussion	111
11.8.8	Zusammenfassung der validierten theoretischen Aussagen	111
11.8.9	Diskussion der Ergebnisse	112
11.8.10	Grenzen der Implementierung	113
11.8.11	Ausblick: Erweiterungsmöglichkeiten	113
11.8.12	Theoretische Erweiterungen	113
11.8.13	Implementierungserweiterungen	114
11.8.14	Anwendungserweiterungen	114
11.8.15	Fazit des Kapitels	114
12	Umfeld der Entwicklung: Subgraph Algorithmus und das Berufsbild in der Praxis	118
12.1	Der Subgraph Algorithmus als Verifikationswerkzeug	118
12.2	Anwendung auf Informationssystem-Verifikation	118
12.3	Gesamtbild: Informationsingenieur, Graphmodell und Subgraph Algorithmus	119
12.4	Der Informationsingenieur als Synthese von Informatiker und Softwareentwickler	119
12.4.1	Einleitung: Warum eine Synthese notwendig ist	119
12.4.2	Formale Charakterisierung der drei Berufsbilder	120
12.4.3	Kompetenzsatz: Der Informationsingenieur umfasst Informatiker und Softwareentwickler	121
12.4.4	Detaillierter Kompetenzvergleich	121
12.4.5	Die historische Divergenz und ihre Überwindung	121
12.4.6	Entstehung der Trennung	121
12.4.7	Konsequenzen der Trennung für Informationssysteme	122
12.4.8	Die synthetische Erweiterung: Was der Informationsingenieur leistet	123
12.4.9	Integration der theoretischen Grundlagen des Informatikers	123
12.4.10	Integration der praktischen Kompetenz des Softwareentwicklers	124
12.4.11	Die eigenständigen Kompetenzen des Informationsingenieurs	124
12.4.12	Der Informationsingenieur als $\{0,1\}$ -Ingenieur: Philosophische Begründung	125
12.4.13	Bildungstheoretische Konsequenzen: Ein Curriculum für den Informationsingenieur	125
13	Zusammenfassung und Ausblick	128
13.1	Formale Hauptergebnisse	128
13.2	Das Gesamtbild	129
13.3	Ausblick	129

13.3.1	Formalisierung des Berufsbilds in nationalen und internationalen Ausbildungsordnungen	129
13.3.2	Entwicklung und Standardisierung einer universellen Beschreibungssprache	130
13.3.3	Kandidaten und Bewertungskriterien	130
13.3.4	Benchmark-Entwicklung	130
13.3.5	Erweiterung um physikalische Phänomene	130
13.3.6	Quantitative Theorie der Vollständigkeitslücke	131
13.3.7	Untere Schranken als Funktion der Systemkomplexität	131
13.3.8	Experimentelle Messung in industriellen Werkzeugen	131
13.3.9	Formale Charakterisierung durch temporale Logik	131
13.3.10	Erweiterung des Subgraph Algorithmus	132
13.3.11	Gewichtete und semantisch angereicherte Graphen	132
13.3.12	Hypergraphen für Multi-Stakeholder-Systeme	132
13.3.13	Dynamische Graphen für evolvierte Informationssysteme	132
13.3.14	Parallelisierung und GPU-Beschleunigung	132
13.3.15	Integration in modellbasierte Entwicklungsprozesse (MBSE)	133
13.3.16	SysML-Integration	133
13.3.17	Werkzeugintegration	133
13.3.18	Verbindung zu Digital Twins	133
13.3.19	Kognitionswissenschaftliche Fundierung der Erlernbarkeit	134
13.3.20	Cognitive Load Theory	134
13.3.21	ACT-R-Theorie und prozedurales Lernen	134
13.3.22	Interdisziplinäre Forschungsfrage	134
13.3.23	Formale Semantik für Mehrhierarchie-Graphmodelle	134
13.3.24	Vertikale Konsistenz	134
13.3.25	Horizontale Konsistenz	135
13.3.26	Sicherheit, Zertifizierung und rechtliche Verankerung	135
13.3.27	Normkonforme Integration	135
13.3.28	Rechtliche Verankerung des Berufsbilds	135
13.3.29	Informationsingenieurwesen und Künstliche Intelligenz	136
13.3.30	KI als Assistent in der Verifikationskette	136
13.3.31	Grenzen der KI-gestützten Code-Generierung	136
13.3.32	Neurosymbolische Integration	136
13.3.33	Anthropologische, gesellschaftliche und ethische Dimension	136
13.3.34	Informationssysteme für das Leben	136
13.3.35	Technikethik als integraler Bestandteil des Berufsbilds	137
13.3.36	Digitale Souveränität und Zugänglichkeit	137
13.3.37	Langfristige Vision: Der Informationsingenieur als Grundberuf der Informationsgesellschaft	137
13.3.38	Das Forschungsprogramm im Überblick: Zehn Themenfelder für die kommenden Jahre	137
13.4	Zusammenfassung und Ausblick aus Sicht der Echtzeitsystem-Anwendung .	139
13.4.1	Zusammenfassung der Anwendungsergebnisse	139
13.4.2	Theoretische Beiträge	139
13.4.3	Praktische Beiträge	140
13.4.4	Experimentelle Beiträge	140
13.4.5	Synthese	141

13.4.6	Reflexion der theoretischen Erkenntnisse	142
13.4.7	Beantwortung der Leitfragen	143
13.4.8	Experimentelle Validierung als methodischer Beitrag	143
13.4.9	Zukünftige Arbeiten	144
13.4.10	Kurzfristig: Erweiterung der Analysemethoden	144
13.4.11	Mittelfristig: Neue Modellierungsparadigmen	144
13.4.12	Langfristig: Fundamentale Forschungsfragen	145
13.4.13	Fazit	145
13.4.14	Theoretische Synthese	145
13.4.15	Der Beweis durch Experiment	146
13.4.16	Die Rolle von UML-Profilen als Brücke	146
13.4.17	Grenzen und Offene Fragen	147
13.4.18	Methodischer Beitrag: Theorie durch Experiment prüfen	147
13.4.19	Schlussbetrachtung	148
Literaturverzeichnis		148

Vorwort

Dieses Buch führt zwei eigenständige Arbeiten des Autors – die Begründung des Berufsbilds des *Informationsingenieurs* mit graphbasierter Modellierung, Beschreibungssprache und Transformation zum Zielcode sowie die Anwendung dieser Prinzipien in einem modellgetriebenen Engineering-Szenario für Echtzeitsysteme (UML-Profile mit Zeit-Annotationen, Markov-Ketten, Warteschlangentheorie und Stabilitätskriterien) – in einer Gliederung zusammen, die sich an der klassischen Kapitelfolge eines Grundkurses Software-Engineering orientiert: *Was ist [...]?* – *Berufsbild* – *Grundlagen* – *Anforderungsanalyse* – *Grobdesign* – *Vom Modell zum Programm* – *Feindesign* – *Optimierung des Designmodells* – *Implementierungsaspekte* – *Anwendungsbeispiel* – *Qualitätssicherung* – *Umfeld der Entwicklung* – *Zusammenfassung und Ausblick*.

Warum dieses Buch entstanden ist

Die beiden Ausgangsarbeiten – `informationsingenieur.tex` und `umlp.tex` – entstanden zunächst unabhängig voneinander, aus zwei unterschiedlichen, aber verwandten Fragestellungen heraus. Die erste Arbeit fragt: *Was ist die eigentliche, unhintergehbare Aufgabe eines Ingenieurs, der Informationssysteme für Menschen baut?* Die Antwort führt über das Ausgangszitat in Abschnitt 1.1 zu einer Theorie der graphbasierten Modellierung, der Beschreibungssprache und der Transformationsverantwortung. Die zweite Arbeit fragt: *Wie sieht diese Verantwortung konkret aus, wenn man sie auf ein reales, zeitkritisches System anwendet?* Die Antwort führt über UML-Profile mit Zeit-Annotationen, Markov-Ketten und Warteschlangentheorie zu einem durchgängigen Beispiel – der Aufzugstür-Steuerung – und zu der zentralen empirischen Erkenntnis, dass Little’s Law kein Stabilitätskriterium ist.

Beim Zusammenführen beider Arbeiten zu diesem Buch zeigte sich, dass die beiden Fragestellungen nicht nur verwandt, sondern komplementär sind: Die erste Arbeit liefert die *Theorie*, die zweite liefert die *Probe aufs Exempel*. Genau diese Komplementarität wird in Abschnitt 1.2 unter dem Namen *Schärfungsprinzip* gefasst und zieht sich seither als roter Faden durch das gesamte Buch: Jedes Kapitel der Theorie (Teil I im ursprünglichen Sinn) hat ein Gegenstück in der Anwendung (Teil II), und umgekehrt liefert jede Beobachtung an der Aufzugstür-Steuerung Anlass, eine theoretische Aussage zu schärfen, einzuschränken oder zu bestätigen.

Wie dieses Buch zu lesen ist

Das Buch ist so aufgebaut, dass es sowohl linear von vorne nach hinten als auch selektiv gelesen werden kann. Leserinnen und Leser, die vor allem an der *Theorie des Informationsingenieurwesens* interessiert sind, finden den Kern in den Kapiteln 1, 2, 5, 6 und 8:

Hier werden das Ausgangszitat, das Berufsbild, das Graphmodell-Theorem, das Mächtigkeitstheorem und die Sätze zur Code-Generierung formal entwickelt und bewiesen. Leserinnen und Leser, die vor allem an der *praktischen Anwendung* interessiert sind – etwa an UML-Profilen mit Zeit-Annotationen, an der Implementierung von `markov_analyzer` und `queue_analyzer` oder an der Aufzugstür-Fallstudie –, finden den Kern in den Kapiteln 3, 4, 7, 9, 10 und 11. Kapitel 12 verbindet beide Stränge über den Subgraph Algorithmus und das Berufsbild in der Praxis, Kapitel 13 zieht Bilanz und entwirft – in der gegenüber früheren Fassungen deutlich erweiterten Fassung dieses Buches – ein umfassendes Forschungs- und Entwicklungsprogramm für die kommenden Jahre.

Durchgängig gilt: Jede Definition, jeder Satz und jede These ist nummeriert und wird, wo immer möglich, an einer konkreten Stelle des Aufzugstür-Beispiels oder der begleitenden Python-Implementierung illustriert. Tabellen sind nicht als isolierte Datensammlungen zu verstehen, sondern jeweils in einleitende und interpretierende Absätze eingebettet, die ihre Bedeutung im Gesamtzusammenhang erläutern. Wo UML-Diagramme verwendet werden, sind sie als TikZ-Grafiken eingebettet und folgen, soweit möglich, der UML-2-Notation für Klassen-, Sequenz- und Zustandsdiagramme.

Der gemeinsame Kern: Subgraph Algorithmus und Schärfungsprinzip

Zwei Konzepte durchziehen das gesamte Buch und verbinden seine sonst sehr unterschiedlichen Kapitel – von der formalen Beweisführung über die Berufsbildanalyse bis zur Python-Implementierung – zu einem Ganzen.

Das erste Konzept ist der *Subgraph Algorithmus*

$$\sigma_j = \sum_{i=0}^{n-1} A_{ij} \cdot 2^i + j \cdot 2^n, \quad O(n^3),$$

der als Verifikationswerkzeug für Graphmodelle und Informationssysteme verwendet wird. Er taucht in Kapitel 5 als Beispiel für die Kodierung eines Graphmodells in eine kompakte Signatur auf, in Kapitel 8 als Illustration der Informationsdichte $O(n^2)$ einer Adjazenzmatrix, und in Kapitel 12 als eigenständiges Verifikationswerkzeug für die in diesem Buch entwickelten Informationssysteme.

Das zweite Konzept ist das in Abschnitt 1.2 eingeführte *Schärfungsprinzip*, zusammengefasst im wörtlich übernommenen Zitat:

“Erst da wo Software durch UML-Profile durch quantitative Annotationen viel schärfer beschrieben wird, erst da beginnt Software in der Entwicklung sich echt anzufühlen, verifizierbar anzufühlen und viel mehr alle Beteiligten in die Verantwortung mit hineinzunehmen.”

Dieses Prinzip erklärt, warum dieses Buch – anders als viele Lehrbücher zum Software-Engineering – der quantitativen Annotation (Zeiten, Wahrscheinlichkeiten, Lasten) denselben Stellenwert einräumt wie der strukturellen Modellierung (Klassen, Zustände, Assoziationen): Erst die Kombination aus beidem macht ein Modell zu einem Artefakt, über das alle Beteiligten – Entwicklung, Qualitätssicherung, Produktmanagement, Betrieb – gemeinsam und auf Augenhöhe sprechen können.

Design Patterns als drittes Bindeglied

Mit Abschnitt 8.3 wurde dem Buch ein drittes verbindendes Element hinzugefügt: die systematische Behandlung von *Design Patterns* unter dem Gesichtspunkt der drei obersten Softwarequalitätsziele – Wiederverwendbarkeit $R(S)$, Erweiterbarkeit $E(S)$ und Analysierbarkeit $A(S)$ (Definition 8.3.1). Die dort formulierte Pattern-These (These 8.3.1) postuliert, dass Design Patterns genau diejenigen Strukturierungsmittel sind, die alle drei Ziele gleichzeitig fördern, und dass sich dieser Effekt unmittelbar im Entwicklungsprozess zeigen muss. Die TikZ-basierten UML-Diagramme zu Strategy-, Observer- und Factory-Method-Pattern in Abschnitt 8.3 sind so gewählt, dass sie unmittelbar auf das Aufzugstür-Beispiel und die `markov_analyzer/queue_analyzer`-Architektur Bezug nehmen – Theorie und Anwendung verschmelzen damit auch auf der Ebene der Entwurfsmuster.

Was neu ist gegenüber den Ausgangsarbeiten

Gegenüber den beiden Ausgangsarbeiten `informationsingenieur.tex` und `umlp.tex` enthält dieses Buch drei wesentliche Erweiterungen, die in den vorangegangenen Überarbeitungsschritten entstanden sind und die den Umfang von ursprünglich rund 60 Druckseiten je Einzelarbeit auf den vorliegenden Gesamtumfang anwachsen ließen:

- (1) Die Einführung des **Schärfungsprinzips** (Abschnitt 1.2) als zweites zentrales Zitat neben dem Ausgangszitat, mit einer formalen Schärfungsthese (These 1.2.1), die den Übergang von struktureller zu quantitativ annotierter Modellierung als Statusänderung des Modells beschreibt.
- (2) Die Ergänzung um **Design Patterns** (Abschnitt 8.3) mit vier neuen TikZ-UML-Diagrammen (Strategy-, Observer-, Factory-Method-Pattern sowie eine Architekturübersicht) und einer formalen Verbindung zu den drei Qualitätszielen $R(S)$, $E(S)$, $A(S)$.
- (3) Die **ausführliche Einbettung aller Tabellen** in erläuternde Absätze, sodass jede der knapp 30 Tabellen dieses Buches nicht isoliert auf einer Seite steht, sondern in den argumentativen Zusammenhang des jeweiligen Kapitels eingeordnet ist – vom Berufsbildvergleich in Tabelle 2.2.1 bis zur Sensitivitätsmatrix in Tabelle 9.4.12.

Mit der vorliegenden, deutlich erweiterten Fassung kommt eine vierte Erweiterung hinzu: die **Vertiefung der bisher knapperen Kapitel** – insbesondere der Kapitel zu Anforderungsanalyse, Feindesign und zum Umfeld der Entwicklung – sowie ein gegenüber früheren Fassungen erheblich ausgebauten Schlusskapitel, das den Ausblick aus der ursprünglichen Einzelarbeit `informationsingenieur.tex` zu einem mehrjährigen Forschungsprogramm mit zehn Themenfeldern weiterentwickelt (Abschnitt 13.3.38).

Kapitel 1

Was ist Informationsingenieurwesen?

1.1 Das Ausgangszitat als wissenschaftliches Fundament

Diese Arbeit nimmt ein präzises und vielschichtiges Zitat als treibende Grundlage. Es wird vollständig und wortwörtlich wiedergegeben, da es nicht paraphrasiert werden kann, ohne an epistemischem Gehalt zu verlieren:

Wichtig ist zu begründen, dass die Modellsicht als Graph sich sehr gut eignet auf Managementsicht oder auf Anforderungssicht für Entscheidungsträger mit viel Verantwortung.

Wichtig ist zu begründen, dass der Ingenieur diesen letzten wichtigen Schritt der Transformation oder Übersetzung in seiner vollen Verantwortung ausführen muss und zu verantworten hat.

Der Ingenieur hat dabei immer das Ziel, so nah am Target wie möglich zu sein um gleichzeitig mit so wenig Wartbarkeit und Nicht-Wiederverwendbarkeit belastet zu werden. Er möchte direkt simulieren und die Simulation dabei so nah wie möglich oder so echt wie möglich durchführen. Die Simulation aber muss um der Verifikation und um der Analysierbarkeit Willen in einer leicht erlernbaren Sprache erfolgen, die möglichst alle Ingenieure verstehen, um sich in den Details mehr um das Systemverhalten zu kümmern, als um spezifische hochoptimierte Hardwareabhängigkeiten.

Der Ingenieur versteht die Anforderung, die ihm vom Produktmanagement über die graphische Darstellung, in beliebig vielen Hierarchien, gegeben worden sind und leitet daraus ganz konkrete Beschreibungen ab, die diese Anforderung alle umsetzen.

Die ganz konkreten Beschreibungen werden formuliert in der letzten Sprache vor der Übersetzung bzw. vor der Transformation in die Zielsprache.

Es muss aufgrund der Komplexität der Systeme eine sehr mächtige Beschreibungssprache sein, mit der möglichst jedes naturwissenschaftliche Phänomen oder Verhalten abgebildet werden kann. Sonst lässt sich das System so nicht beschreiben und den Menschen nicht geben. Denn erst in dieser Mächtigkeit können alle Ingenieure diese eine Sprache, welche leicht erlernbar ist und alle beherrschen, nutzen und aus der Erfahrung schöpfen, um damit präzise Beschreibung zu erfassen.

Das Problem an der graphischen Modellierung ist, als Ingenieur immer die Unsicherheit zu tragen, dass zwischen graphischer Modellierung und der Zielhardware die Abbildung aller Anforderungen nicht vollständig ist.

Als Konsequenz daraus ergibt sich, dass eine geeignete Sprache für Ingenieure gefunden werden muss, mit der diese Abbildung möglich ist. Als Konsequenz daraus ergibt sich auch, dass Code-Generierung für die Zielhardware aus einer graphischen Modellierung heraus unnötig ist.” *“Information Engineer oder $\{0, 1\}$ -Ingenieur oder auf deutsch Informationsingenieur lautet der Titel des Berufs, in dem die Menschen tätig sind, die in diesem Beruf ausgebildet worden sind, um den Menschen Information Systems oder auf deutsch Informationssysteme für das Leben zu geben.*

Wichtig ist zu begründen, dass die Modellsicht als Graph sich sehr gut eignet auf Managementsicht oder auf Anforderungssicht für Entscheidungsträger mit viel Verantwortung.

Wichtig ist zu begründen, dass der Ingenieur diesen letzten wichtigen Schritt der Transformation oder Übersetzung in seiner vollen Verantwortung ausführen muss und zu verantworten hat.

Der Ingenieur hat dabei immer das Ziel, so nah am Target wie möglich zu sein um gleichzeitig mit so wenig Wartbarkeit und Nicht-Wiederverwendbarkeit belastet zu werden. Er möchte direkt simulieren und die Simulation dabei so nah wie möglich oder so echt wie möglich durchführen. Die Simulation aber muss um der Verifikation und um der Analysierbarkeit Willen in einer leicht erlernbaren Sprache erfolgen, die möglichst alle Ingenieure verstehen, um sich in den Details mehr um das Systemverhalten zu kümmern, als um spezifische hochoptimierte Hardwareabhängigkeiten.

Der Ingenieur versteht die Anforderung, die ihm vom Produktmanagement über die graphische Darstellung, in beliebig vielen Hierarchien, gegeben worden sind und leitet daraus ganz konkrete Beschreibungen ab, die diese Anforderung alle umsetzen.

Die ganz konkreten Beschreibungen werden formuliert in der letzten Sprache vor der Übersetzung bzw. vor der Transformation in die Zielsprache.

Es muss aufgrund der Komplexität der Systeme eine sehr mächtige Beschreibungssprache sein, mit der möglichst jedes naturwissenschaftliche Phänomen oder Verhalten abgebildet werden kann. Sonst lässt sich das System so nicht beschreiben und den Menschen nicht geben. Denn erst in dieser Mächtigkeit können alle Ingenieure diese eine Sprache, welche leicht erlernbar ist und alle beherrschen, nutzen und aus der Erfahrung schöpfen, um damit präzise Beschreibung zu erfassen.

Das Problem an der graphischen Modellierung ist, als Ingenieur immer die Unsicherheit zu tragen, dass zwischen graphischer Modellierung und der Zielhardware die Abbildung aller Anforderungen nicht vollständig ist.

Als Konsequenz daraus ergibt sich, dass eine geeignete Sprache für Ingenieure gefunden werden muss, mit der diese Abbildung möglich ist. Als Konsequenz daraus ergibt sich auch, dass Code-Generierung für die Zielhardware aus einer graphischen Modellierung heraus unnötig ist.”

Dieses Zitat enthält in komprimierter Form eine vollständige Systemtheorie des Informationsingenieurwesens. Die vorliegende Arbeit entfaltet die darin enthaltenen Argumente in

formaler wissenschaftlicher Sprache, beweist die aufgestellten Thesen und veranschaulicht sie numerisch und graphisch.

1.2 Das Schärfungsprinzip: Von der quantitativen Annotation zur geteilten Verantwortung

Neben dem Ausgangszitat aus Abschnitt 1.1 steht ein zweites, ebenso grundlegendes Zitat im Zentrum dieser Arbeit. Es bildet die Brücke zwischen der abstrakten Graphmodellierung und der konkreten, zeitannotierten UML-Profil-basierten Modellierung, wie sie in Teil II dieses Buches am Beispiel der Aufzugstür-Steuerung entwickelt wird. Es wird, wie das Ausgangszitat, wortwörtlich übernommen:

“Erst da wo Software durch UML-Profile durch quantitative Annotationen viel schärfer beschrieben wird, erst da beginnt Software in der Entwicklung sich echt anzufühlen, verifizierbar anzufühlen und viel mehr alle Beteiligten in die Verantwortung mit hineinzunehmen.”

Dieses Zitat beschreibt, was im Folgenden als **Schärfungsprinzip** bezeichnet wird: den Übergang von einer rein qualitativ-strukturellen Modellierung (Klassen, Assoziationen, Zustände) zu einer quantitativ-annotierten Modellierung (Zeiten, Wahrscheinlichkeiten, Durchsätze, Auslastungen), und die damit verbundene qualitative Veränderung im Charakter der Softwareentwicklung selbst. Drei Aussagen sind in dem Zitat enthalten, die im weiteren Verlauf dieses Buches systematisch entfaltet und an konkreten Beispielen belegt werden:

- (i) **Das Realitätskriterium:** Software “fühlt sich echt an”, wenn ihr Modell nicht nur beschreibt, *dass* etwas geschieht (Zustandsübergang, Methodenaufruf), sondern auch *wie schnell, wie wahrscheinlich* und *mit welcher Ressourcenbelastung* es geschieht. Erst die quantitative Annotation – minimale, typische und maximale Dauer, Timeout, Deadline, Übergangswahrscheinlichkeiten, Ankunfts- und Bedienraten – schließt die Lücke zwischen Modell und Betriebsverhalten.
- (ii) **Das Verifikationskriterium:** Erst eine quantitativ annotierte Beschreibung ist einer formalen Prüfung zugänglich, die über reine Typkonsistenz hinausgeht. Eine Aufzugstür, deren Öffnungsvorgang lediglich als Zustandsübergang geschlossen $\rightarrow$ offen modelliert ist, lässt keine Aussage über Sicherheit zu. Erst mit der Annotation $2,5\text{ s} \leq t_{\text{Türöffnung}} \leq 4,0\text{ s}$ und einer zugehörigen Stabilitätsbedingung wird die Aussage “diese Tür öffnet rechtzeitig und bleibt im sicheren Betriebsbereich” überhaupt erst formulierbar – und damit auch *falsifizierbar*.
- (iii) **Das Verantwortungskriterium:** Eine quantitativ annotierte Beschreibung ist nicht mehr allein Eigentum der Entwicklung. Deadlines, Auslastungsgrenzen und Stabilitätskriterien sind Größen, die Produktmanagement, Qualitätssicherung, Betrieb und Entwicklung gemeinsam verstehen, verhandeln und verantworten können. Die quantitative Annotation übersetzt technische Details in eine Sprache, die – im Sinne des Ausgangszitats aus Abschnitt 1.1 – “alle Ingenieure verstehen”, und erweitert diesen Kreis faktisch auf alle am System Beteiligten.

These 1.2.1 (Schärfungsthese). Der Übergang von einer strukturellen UML-Modellierung M_{struct} zu einer quantitativ annotierten Modellierung $M_{\text{quant}} = M_{\text{struct}} \cup A$, wobei A die Menge der Zeit-, Wahrscheinlichkeits- und Lastannotationen bezeichnet, erhöht den Informationsgehalt des Modells nicht nur graduell, sondern verändert dessen *epistemischen Status*: M_{struct} erlaubt Aussagen über *Struktur* (“Diese Methode existiert, dieser Zustand ist erreichbar”), während M_{quant} Aussagen über *Verhalten unter Last und Zeit* erlaubt (“Diese Methode terminiert mit Wahrscheinlichkeit p innerhalb von τ Millisekunden, das System bleibt stabil für $\lambda < \mu$ ”).

Die Schärfungsthese wird in diesem Buch in zwei Schritten belegt. Erstens wird formal gezeigt, dass das Graphmodell als Anforderungssicht zwar vollständig, aber bezüglich A unterbestimmt ist – die Beschreibungssprache muss daher die Annotationsmenge A explizit aufnehmen können (Mächtigkeitstheorem, Abschnitt 6.2). Zweitens wird in Kapitel 11 und Kapitel 10 an einem durchgängigen Beispiel demonstriert, wie aus genau dieser Annotationsmenge A – den MARTE-artigen Zeit-Stereotypen, den Markov-Übergangswahrscheinlichkeiten und den Warteschlangenparametern λ, μ – konkrete, prüfbare und damit verantwortbare Aussagen über das Systemverhalten entstehen.

Damit schließt sich der Kreis zum Ausgangszitat: Die graphische Modellierung (Abschnitt 1.1) liefert die Anforderungssicht, die Beschreibungssprache mit quantitativer Annotation liefert die “letzte Sprache vor der Transformation”, und genau an dieser Stelle – so die zentrale Erkenntnis dieses Abschnitts – beginnt Software, sich für alle Beteiligten als reales, verantwortbares und verifizierbares Artefakt zu manifestieren.

1.3 Überblick und Beiträge dieser Arbeit

Die Hauptbeiträge sind:

- Eine formale Definition des Berufsbilds *Informationsingenieur* mit Abgrenzung zu verwandten Disziplinen (Abschnitt 2).
- Ein Theorem über die Eignung der Graphdarstellung als Managementmodell (Abschnitt 5).
- Ein Theorem über die Anforderungen an die Beschreibungssprache (Abschnitt 6).
- Ein Theorem über die Unnötigkeit von Code-Generierung aus Graphmodellen (Abschnitt 8).
- Die Verbindung zur Subgraph-Signaturtheorie (Abschnitt 12).
- Vier illustrierende Plots und ein ausführlicher Ausblick.

Kapitel 2

Berufsbild: Der Informationsingenieur

2.1 Definition und Begriffsklärung

Definition 2.1.1 (Informationsingenieur). Ein *Informationsingenieur* (engl. *Information Engineer*, auch *$\{0, 1\}$ -Ingenieur*) ist eine ausgebildete Person, deren berufliche Aufgabe darin besteht, *Informationssysteme* zu entwerfen, zu spezifizieren, zu verifizieren und zu realisieren, die dem Menschen in seinem Lebensumfeld dienen. Formal ist ein Informationsingenieur ein Tupel

$$\mathcal{IE} = (K, M, L, R, T)$$

wobei

- K die Wissensbasis (Mathematik, Informatik, Ingenieurwissenschaften),
- M das Modellierungsvermögen (Graphen, Anforderungsstrukturen),
- L die Sprachkompetenz (Beherrschung der Beschreibungssprache),
- R die Transformationsverantwortung (Übersetzung in Zielcode),
- T das Zielorientierungsvermögen (Maximierung der Zielnähe bei minimaler Wartungslast)

bezeichnet.

Der Begriff *$\{0, 1\}$ -Ingenieur* verweist darauf, dass alle Information letztlich in binärer Kodierung vorliegt – die Welt des Ingenieurs ist die Welt der Bits. Dennoch operiert er auf hochgradig abstrakten Darstellungen dieser Bits: auf Graphen, auf Systemmodellen, auf mathematischen Beschreibungen physikalischer Phänomene.

Definition 2.1.2 (Informationssystem). Ein *Informationssystem* $\mathcal{IS}$ ist ein geordnetes Paar

$$\mathcal{IS} = (\mathcal{R}, \mathcal{I})$$

bestehend aus einer Menge von Anforderungen $\mathcal{R} = \{r_1, r_2, \dots, r_k\}$ und einer Implementierung $\mathcal{I}$, sodass $\mathcal{I}$ alle Anforderungen in $\mathcal{R}$ erfüllt:

$$\forall r_i \in \mathcal{R} : \mathcal{I} \models r_i.$$

2.2 Abgrenzung zu verwandten Berufsbildern

Der Informationsingenieur unterscheidet sich von verwandten Berufsbildern wie folgt:

Tabelle 2.2.1 stellt den Informationsingenieur fünf benachbarten Berufsbildern gegenüber und macht deutlich, warum keines dieser etablierten Profile die in diesem Buch beschriebene Gesamtaufgabe – die durchgängige Kette von der Anforderung über das Graphmodell und die Beschreibungssprache bis hin zum Zielcode – in voller Verantwortung abdeckt. Der *Softwareentwickler* beherrscht Codierung und Algorithmen, verfügt jedoch typischerweise über keine formale, graphbasierte Systemmodellierung auf Anforderungsebene; seine Tätigkeit setzt meist erst nach der Modellierungsphase ein. Der *Systemingenieur* denkt in Systemarchitekturen und Schnittstellen, hat aber selten den in dieser Arbeit zentralen informationstheoretischen Blick auf die Vollständigkeit der Abbildung zwischen Modell und Zielsprache (Definition 8.1.1). Der *Informatiker* verfügt über die theoretischen Grundlagen – Berechenbarkeit, Komplexität, Algorithmik –, bezieht diese jedoch nicht notwendig auf eine konkrete physische Zielhardware mit ihren Echtzeit-Constraints (Kapitel 8). Der *Elektrotechniker* beherrscht Hardware und Schaltungstechnik, ohne dass dies zwingend mit einem Software- oder Systemfokus auf Anforderungsebene verbunden ist. Erst der *Informationsingenieur* vereinigt – gemäß Definition 2.1.1 – alle diese Teilkompetenzen entlang der gesamten Kette *Anforderung* $\rightarrow$ *Graph* $\rightarrow$ *Beschreibungssprache* $\rightarrow$ *Zielcode* und trägt dabei, wie im Ausgangszitat (Abschnitt 1.1) gefordert, die volle Verantwortung für die abschließende Transformation. Diese Gegenüberstellung bildet die Grundlage für die in Kapitel 12.4 formal ausgearbeitete Synthese-These, nach der der Informationsingenieur als Vereinigung aus Informatiker und Softwareentwickler – erweitert um die System- und Hardwareperspektive – verstanden werden kann.

Tabelle 3.1.1 zeigt den Aufbau eines UML-Profiles, wie es in dieser Arbeit zur Annotation von Zeit-Constraints verwendet wird: Jeder Stereotyp erweitert ein Standard-UML-Element um getaggte Werte (Tagged Values), die minimale, typische und maximale Zeitdauern, Timeouts und Deadlines tragen.

Tabelle 3.2.1 stellt das in dieser Arbeit verwendete vereinfachte Profil dem vollständigen OMG-Standard MARTE [34] gegenüber und zeigt, an welchen Stellen bewusst Komplexität reduziert wurde, ohne die Aussagekraft für die Aufzugstür-Fallstudie zu verlieren.

Tabelle 3.7.1 ordnet die in dieser Arbeit verwendeten Modellierungsartefakte den klassischen Abstraktionsebenen des Model-Driven Engineering (CIM, PIM, PSM, Code) zu.

Tabelle 3.8.1 fasst die in Echtzeitsystemen üblichen zeitlichen Parameter zusammen – minimale, typische und maximale Ausführungsdauer, Deadline und Timeout – und ordnet jedem Parameter seine Bedeutung für die Systemverifikation zu.

Tabelle 7.1.1 listet die Systemkomponenten auf, die im Zuge der Erweiterung um den *Quantitative Analysis Layer* (Abschnitt 7) neu hinzugekommen sind, zusammen mit ihrer jeweiligen Funktion innerhalb der Generierungspipeline.

Tabelle 7.3.1 gibt einen Überblick über die Kernkomponenten des ursprünglichen, zweistufigen Generierungsprozesses, bevor dieser um die quantitative Analyse erweitert wurde.

Tabelle 9.1.1 bzw. die zugehörige Verzeichnisübersicht zeigt den Aufbau des umlp-Projekts auf Dateisystemebene und ordnet jedem Verzeichnis seine Rolle innerhalb der Implementierung zu.

Tabelle 9.1.1 ordnet jedes Python-Modul im Verzeichnis `src/` einer der in Kapitel 7 eingeführten Architekturschichten zu (Profil-Verarbeitung, Domänenmodell, Code-Generierung, quantitative Analyse).

Tabelle 9.1.2 listet die Test-Module auf, ihre Dateigrößen sowie die Anzahl der enthal-

tenen Testfälle und gibt damit einen quantitativen Eindruck vom Umfang der Test-Suite.

Tabelle 9.3.1 dokumentiert die getaggten Werte (Tagged Values) des neu eingeführten Stereotyps «StabilityAnnotation», mit denen Lyapunov-Funktion, Abklingrate und Stabilitätsklasse direkt im UML-Modell annotiert werden können.

Tabelle 9.3.2 wendet die in Tabelle 9.3.1 eingeführten Stability-Tagged-Values konkret auf die Aufzugstür-Fallstudie an und gibt für jeden Betriebsmodus die zugehörigen Werte für Lyapunov-Funktion, Abklingrate und Stabilitätsklasse an.

Tabelle 9.4.1 stellt die beiden zentralen Analysemodule `markov_analyzer` und `queue_analyzer` einander gegenüber: ihre jeweilige theoretische Grundlage, die zentralen Klassen sowie die Art der Aussagen, die sie über das System treffen können.

Tabelle 9.4.2 schlüsselt die Testklassen der Datei `test_markov_analyzer.py` auf und gibt für jede Klasse an, welchen Aspekt des `markov_analyzer`-Moduls sie überprüft.

Analog zu Tabelle 9.4.2 schlüsselt Tabelle 9.4.3 die Testklassen der Datei `test_queue_analyzer.py` auf und ordnet sie den Warteschlangenmodellen M/M/1, M/M/c, M/D/1 und M/G/1 zu.

Tabelle 9.4.4 zeigt die vom `queue_analyzer` berechneten stationären Kenngrößen (mittlere Wartezeit, mittlere Anzahl wartender Patienten, Auslastung) für den Normalbetrieb der Krankenhaus-Notaufnahme bei einer Ankunftsrate von $\lambda = 8$ Patienten pro Stunde.

Tabelle 9.4.5 zeigt das Ergebnis der Kapazitätsplanung: für unterschiedliche Ankunfts-raten λ wird die minimale Serveranzahl c ausgewiesen, mit der eine Ziel-Auslastung von höchstens 70 % eingehalten wird.

Tabelle 9.4.6 überprüft für das Notfall-Szenario, ob die vom `markov_analyzer` berechneten Übergangszeiten der zeitkontinuierlichen Markov-Kette (CTMC) innerhalb der im UML-Profil annotierten Deadlines liegen.

Tabelle 9.4.7 gibt die stationäre Verteilung π der diskreten Markov-Kette für den Normalbetrieb an, also die Wahrscheinlichkeit, mit der sich das System langfristig in jedem der modellierten Zustände (z. B. Ersteinschätzung, Behandlung, Intensiv) befindet.

Tabelle 9.4.8 zeigt die mittlere Erstpassezeit (Mean First Passage Time, MFPT) – also die erwartete Zeit, bis ein Patient ausgehend von einem gegebenen Zustand erstmals den Behandlungszustand erreicht –, berechnet aus der in Tabelle 9.4.7 angegebenen Markov-Kette.

Tabelle 9.4.9 erweitert die Betrachtung aus Tabelle 9.4.4 auf alle Dienste der Notaufnahme (Ersteinschätzung, Behandlung, Intensivstation) und zeigt für jeden Dienst die jeweilige Auslastung, mittlere Wartezeit und mittlere Warteschlangenlänge.

Tabelle 9.4.10 zeigt die stationäre Verteilung der Markov-Kette unter Spitzenlast und damit das Gegenstück zu Tabelle 9.4.7.

Tabelle 9.4.11 zeigt die Auslastung $\rho_i = \lambda_i/\mu_i$ jedes Dienstes unter der erhöhten Ankunftsrate $\lambda = 20$ Patienten pro Stunde.

Tabelle 9.4.12 zeigt die Sensitivitätsmatrix $\|\partial\pi/\partial P_{ij}\|$ des Drei-Zustands-Modells, also wie stark sich die stationäre Verteilung π bei einer kleinen Änderung der Übergangswahrscheinlichkeit P_{ij} ändert.

Tabelle 9.4.14 vergleicht ein M/M/1-System mit einem äquivalenten M/M/c-System bei gleicher Gesamtbedienkapazität $c \cdot \mu = 12$ und zeigt, dass die Aufteilung der Kapazität auf mehrere parallele Server die mittlere Wartezeit deutlich reduziert.

Tabelle 10.6.1 vertieft den bereits in Tabelle 3.2.1 eingeführten Vergleich zwischen MARTE und dem in dieser Arbeit verwendeten Profil um weitere Aspekte wie Werkzeug-

unterstützung, Lernaufwand und Eignung für die Code-Generierung.

Tabelle 11.8.1 ordnet die im `stability_analysis`-Modul implementierten Klassen den verschiedenen Beschreibungsebenen zu – von der dynamischen Ebene (Markov-Ketten, Lyapunov-Funktionen) bis zur algebraischen Ebene (Little’s Law, stationäre Kenngrößen).

Tabelle 11.8.1 stellt jede zentrale theoretische Aussage dieses Buches – von der Vollständigkeitslücke (Satz 8.1.1) bis zur Trennung von Stabilität und Gleichgewicht – dem jeweiligen Experiment gegenüber, das diese Aussage empirisch belegt.

Tabelle 12.4.1 führt den in Tabelle 2.2.1 begonnenen Vergleich fort und stellt die drei Berufsbilder Informatiker, Softwareentwickler und Informationsingenieur anhand eines Katalogs konkreter Kompetenzen (formale Methoden, Implementierung, Hardwarebezug, Transformationsverantwortung, Kommunikationsfähigkeit gegenüber Management) systematisch gegenüber.

Tabelle 9.4.13 wiederholt die Kapazitätsplanung aus Tabelle 9.4.5 unter der konkreten Annahme einer Bedienrate von $\mu = 5$ Patienten pro Stunde und Server.

Tabelle 2.2.1: Abgrenzung des Informationsingenieurs

Berufsbild	Schwerpunkt	Abgrenzung zum Info-Ing.
Softwareentwickler	Codierung, Algorithmen	Keine formale Systemmodellierung
Systemingenieur	Systemarchitektur	Kein Fokus auf Informationstheorie
Informatiker	Theorie, Algorithmen	Keine physische Zielhardware
Elektrotechniker	Hardware, Schaltungen	Kein Software-/Systemfokus
Informationsingenieur	Gesamtkette: Anforderung → Graph → Sprache → Zielcode	Volle Verantwortung für die Transformation

Wie Tabelle 2.2.1 zeigt, liegt der entscheidende Unterschied nicht in der Abwesenheit einzelner Kompetenzen bei den verwandten Berufsbildern, sondern in der *fehlenden Durchgängigkeit*: Jedes der vier ersten Berufsbilder deckt einen Ausschnitt der Kette ab, ohne die Verantwortung für die Übergänge zwischen den Ausschnitten zu übernehmen. Genau diese Übergänge – insbesondere der Übergang vom Graphmodell in die Beschreibungssprache (Kapitel 6) und von dort in den Zielcode – sind jedoch, wie in Kapitel 8 gezeigt wird, die Stellen, an denen Vollständigkeitslücken entstehen. Die letzte Zeile der Tabelle markiert daher nicht nur ein zusätzliches Berufsbild, sondern eine *Verantwortungsverschiebung*: Statt die Übergänge unspezifiziert zwischen den Disziplinen zu lassen, übernimmt der Informationsingenieur sie explizit und macht sie damit erstmals einer formalen Analyse zugänglich.

Kapitel 3

Grundlagen modellgetriebener Vorgehensmodelle

In diesem Kapitel werden die Grundlagen für diese Arbeit beschrieben.

3.1 UML-Profile

UML-Profile erweitern die Unified Modeling Language um domänenspezifische Konzepte ohne die Metamodell-Struktur zu verändern. Ein Profil besteht aus:

- **Stereotypen:** Erweitern Metaklassen (z.B. Class, Operation, Property)
- **Tagged Values:** Zusätzliche Attribute für stereotypisierte Elemente
- **Constraints:** OCL-basierte Einschränkungen

Abbildung 3.1.1 zeigt die allgemeine Struktur eines UML-Profiles mit Zeit-Annotationen. Stereotypen vom Typ **«stereotype»** erweitern UML-Klassen vom Typ `Class` um Annotationen. UML-Profile erweitern somit die UML in ihren bekannten Modellen und ermöglichen dadurch die Modellierung quantitativer Bedingungen, die auch bei der Code-Generierung generiert werden.

3.2 MARTE - Modeling and Analysis of Real-Time and Embedded Systems

Das UML Profile for MARTE [34] ist ein OMG-Standard zur Modellierung und Analyse eingebetteter Echtzeitsysteme. MARTE bietet:

- **Time Model:** Zeitliche Annotationen (Clock, Duration, TimeValue)
- **Resource Model:** Modellierung von Hardware-Ressourcen
- **Generic Quantitative Analysis Model (GQAM):** Framework für quantitative Analysen
- **Schedulability Analysis Model (SAM):** Scheduling-spezifische Konzepte

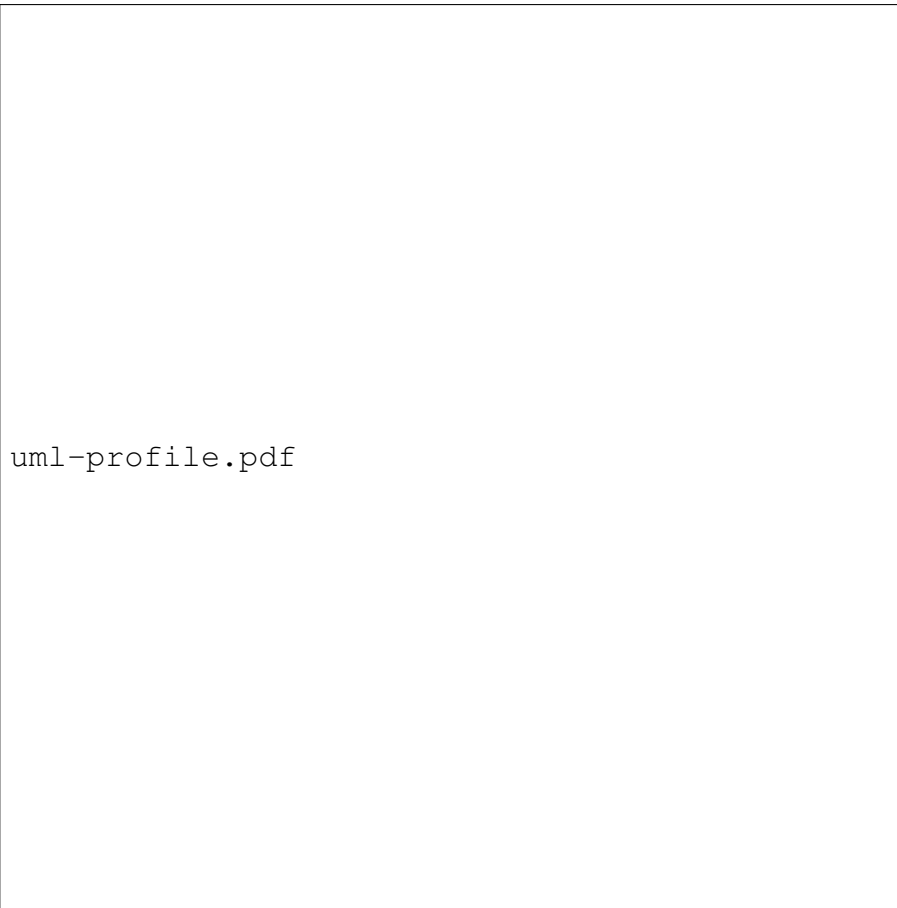


Abbildung 3.1.1: Struktur eines UML-Profiles mit Zeit-Annotationen

- **Performance Analysis Model (PAM)**: Performance-Metriken

Definition 3.2.1 (MARTE Time Model). MARTE definiert Zeit als diskrete oder kontinuierliche Größe mit folgenden Eigenschaften:

- **Clock**: Zeitbasis (ideal, discrete, offset)
- **ClockType**: Typ der Zeitquelle (chronometric, logical)
- **TimedEvent**: Ereignis mit zeitlichem Kontext
- **TimedProcessing**: Operation mit zeitlichem Verhalten

Diese Arbeit übernimmt die konzeptuelle Basis des MARTE Time Models, vereinfacht jedoch die Notation für praktische Code-Generierung:

Aspekt	MARTE	Diese Arbeit
Zeitmodell	Clock, Duration, Instant	TimeUnit, TimingConstraint
Granularität	Sehr detailliert	Vereinfacht, praxisnah
Hauptzweck	Analyse	Code-Generierung
Tool-Support	Papyrus, Rhapsody	Eigenentwicklung
Erweiterbarkeit	Komplex	Modular, einfach

Tabelle 3.2.1: Vergleich: MARTE vs. Diese Arbeit

Der Vergleich macht deutlich, dass die hier gewählte Teilmenge gezielt diejenigen MARTE-Konzepte übernimmt, die für die quantitative Annotation im Sinne der Schärfungsthese 1.2.1 unverzichtbar sind, während seltener benötigte Mechanismen ausgespart bleiben.

Diese Stereotyp-Hierarchie bildet die Grundlage für den `ProfileParser` (Listing 9.2) und damit für die quantitative Annotation gemäß dem Schärfungsprinzip aus Abschnitt 1.2.

3.3 Markov-Ketten für Zustandsübergänge

Markov-Ketten [36] modellieren stochastische Systeme mit zustandsabhängigen Übergangswahrscheinlichkeiten.

Definition 3.3.1 (Diskrete Markov-Kette). Eine diskrete Markov-Kette ist ein stochastischer Prozess $(X_n)_{n \in \mathbb{N}_0}$ mit abzählbarem Zustandsraum S , bei dem gilt:

$$P(X_{n+1} = j \mid X_n = i, X_{n-1} = i_{n-1}, \dots, X_0 = i_0) = P(X_{n+1} = j \mid X_n = i) = p_{ij}$$

Die Übergangsmatrix $P = (p_{ij})$ erfüllt $\sum_{j \in S} p_{ij} = 1$ für alle $i \in S$.

Beispiel 3.3.1 (Aufzugstür-Zustandsmodell als Markov-Kette). Zustandsmenge S mit $S = \{\text{Closed}, \text{Opening}, \text{Open}, \text{Closing}\}$

Übergangsmatrix für ein Zeitintervall $\Delta t = 1s$:

$$P = \begin{pmatrix} 0.7 & 0.3 & 0 & 0 \\ 0 & 0.2 & 0.8 & 0 \\ 0 & 0 & 0.6 & 0.4 \\ 0.9 & 0 & 0 & 0.1 \end{pmatrix}$$

Stationäre Verteilung π erfüllt $\pi P = \pi$ mit $\sum_i \pi_i = 1$:

$$\pi = (0.42, 0.13, 0.24, 0.21)$$

Interpretation: Im Langzeitverhalten ist die Tür 42% geschlossen, 24% offen.

Anwendung in dieser Arbeit:

- Modellierung von Zustandsübergängen mit Wahrscheinlichkeiten
- Berechnung von Steady-State-Wahrscheinlichkeiten
- Analyse von Erreichbarkeit und Transientem Verhalten
- Vorhersage von Systemzuverlässigkeit

3.4 Warteschlangentheorie für Performance-Analyse

Die Warteschlangentheorie [37] analysiert Systeme mit Ankunfts- und Bedienungsprozessen.

Definition 3.4.1 (Kendall-Notation M/M/1). Ein M/M/1-System charakterisiert eine Warteschlange mit:

- M: Markovsche (Poisson) Ankünfte mit Rate λ
- M: Markovsche (exponentiell verteilte) Bedienzeiten mit Rate μ
- 1: Ein Bediener

Satz 3.4.1 (Little's Law). Für ein stabiles Warteschlangensystem gilt:

$$L = \lambda W$$

wobei L die mittlere Anzahl von Kunden im System, λ die Ankunftsrate und W die mittlere Wartezeit ist.

Satz 3.4.2 (M/M/1 Performance-Metriken). Für ein M/M/1-System mit Auslastung $\rho = \lambda/\mu < 1$ gilt:

$$L = \frac{\rho}{1 - \rho} \quad (\text{mittlere Systemgröße}) \quad (3.1)$$

$$W = \frac{1}{\mu - \lambda} \quad (\text{mittlere Wartezeit}) \quad (3.2)$$

$$L_q = \frac{\rho^2}{1 - \rho} \quad (\text{mittlere Warteschlangenlänge}) \quad (3.3)$$

$$W_q = \frac{\rho}{\mu - \lambda} \quad (\text{mittlere Wartezeit in Schlange}) \quad (3.4)$$

Beispiel 3.4.1 (Aufzug-Anfragen als M/M/1-System). Annahmen:

- Anfragen kommen mit Rate $\lambda = 0.2$ Anfragen/Sekunde (Poisson)
- Bedienzeit (Tür öffnen + schließen) $\mu = 0.3$ Bedienungen/Sekunde
- Auslastung $\rho = 0.2/0.3 = 0.667$

Metriken:

$$L = \frac{0.667}{1 - 0.667} = 2.0 \text{ Anfragen im System}$$

$$W = \frac{1}{0.3 - 0.2} = 10s \text{ mittlere Gesamtzeit}$$

$$W_q = \frac{0.667}{0.3 - 0.2} = 6.67s \text{ mittlere Wartezeit}$$

Enthält der Beweis für Little's Law keine Energiefunktion, das heißt keine e-Funktionen mit negativen Exponenten, dann kann Little's Law kein Stabilitätskriterium sein für das Warteschlangensystem. Little's Law ist eine Aussage über das Gleichgewicht des Systems.

Im Allgemeinen gilt dies für alle Systeme, welche auf Stabilität zu prüfen sind. Denn die e-Funktionen oder die Energiefunktionen beschreiben gerade charakteristisch das Verhalten des Systems. Sie sind notwendige Voraussetzung für alle Systeme, die erst durch und nur durch naturwissenschaftliche Beschreibungen so beschrieben werden müssen.

Wo ist die Energie des Systems? Mit was soll das System beschrieben werden? Hat das System Energie oder nicht? Angenommen, es gibt ein System, welches Energie hat und es gibt die bessere Beschreibung ohne die Energiefunktionen oder e-Funktionen. Wenn

es eine naturwissenschaftliche Beschreibung ist, kommt sie ohne die e-Funktionen nicht aus, denn nur die e-Funktionen reproduzieren sich über Zeit in der Ableitung selbst und beschreiben so erst das Verhalten des Systems sehr gut. Wenn es keine naturwissenschaftliche Beschreibung ist, muss eine andere Form der Beschreibung gefunden worden sein, die sich für dieses System noch besser eignet. Aber dann ist es kein naturwissenschaftliches System, das sehr gut beschrieben ist. Denn naturwissenschaftliche Systeme werden durch naturwissenschaftliche Beschreibungen am besten beschrieben. Das ist klar.

Irgendwann trifft die Informationsverarbeitung nach der alle Systeme, die durch die Informatik entwickelt worden sind, auch wieder auf die einfachen Prinzipien der Naturwissenschaft. Denn sie müssen durch Strom betrieben werden. Aber in höheren Abstraktionsebenen wie z.B. der UML müssen Systeme, die mit der Informatik entwickelt werden, unter allen Umständen korrekt arbeiten. Dies ist komplexer, weil diese Systeme nicht immer naturwissenschaftlich beschrieben werden können und somit mathematisch nicht immer vollständig greifbar sind. Daher sind UML-Profile sehr wichtig, denn sie erlauben eine leichtere Beschreibung durch naturwissenschaftliche Beschreibungen.

3.5 Quantitative Analysis of Software Designs

Die quantitative Softwaredesign-Analyse [35] integriert Performance-Modellierung in den Entwicklungsprozess:

- **Software Execution Model:** Kontrollfluss und Ressourcennutzung
- **System Execution Model:** Hardware-Plattform und Scheduling
- **Performance Model:** Mathematisches Modell (Warteschlangen, Markov)
- **Analysis:** Lösung des Performance-Modells
- **Evaluation:** Vergleich mit Anforderungen

Für die Integration in die UML-Profil-basierte Code-Generierung mit Zeit-Annotationen bedeutet dies:

- UML-Profile kodieren Performance-Anforderungen
- XMI-Modelle spezifizieren Ausführungsverhalten
- Automatische Ableitung von Warteschlangen-/Markov-Modellen
- Code-Generierung mit eingebetteten Performance-Checks

3.6 XMI (XML Metadata Interchange)

XMI ist ein OMG-Standard zum Austausch von Metadaten zwischen Modellierungswerkzeugen. UML-Profile und -Modelle werden als XML-Dokumente serialisiert.



Abbildung 3.7.1: MDE-Abstraktionsebenen

3.7 Model-Driven Engineering

Model-Driven Engineering basiert auf drei Abstraktionsebenen:

Abbildung 3.7.1 zeigt die verschiedenen Abstraktionsebenen der Modellgetriebenen Entwicklung. Profile definieren Stereotypen + Timing Code-Gen.

- **CIM (Computation Independent Model)**: Fachliches Domänenmodell
- **PIM (Platform Independent Model)**: Technologieunabhängige Spezifikation
- **PSM (Platform Specific Model)**: Plattformspezifische Implementierung

3.8 Echtzeit-Constraints

Für Echtzeitsysteme sind folgende zeitliche Parameter relevant:

Parameter	Bedeutung
Min Duration	Minimale Ausführungszeit (BCET - Best Case Execution Time)
Max Duration	Maximale Ausführungszeit (WCET - Worst Case Execution Time)
Typical Duration	Typische/durchschnittliche Ausführungszeit
Timeout	Maximale Wartezeit bis Abbruch
Deadline	Absolute Zeitgrenze bis Fertigstellung

Tabelle 3.8.1: Zeitliche Parameter in Echtzeitsystemen

Diese Parameter bilden zusammen die Annotationsmenge A aus der Schärfungsthe-se 1.2.1; ihr Vorhandensein im Modell ist die notwendige Voraussetzung dafür, dass aus einer strukturellen UML-Beschreibung eine quantitativ verifizierbare Beschreibung wird.

Diese Einordnung verdeutlicht, dass das UML-Profil mit Zeit-Annotationen auf der PIM-Ebene angesiedelt ist und die Transformation zur PSM- bzw. Code-Ebene genau den in Kapitel 6 diskutierten Übergang darstellt.

Kapitel 4

Anforderungsanalyse: Motivation und Zielsetzung

Diese Arbeit präsentiert einen modellgetriebenen Ansatz zur automatischen Code-Generierung aus UML-Profilen mit erweiterten Zeit-Annotationen für Echtzeitsysteme. Das entwickelte System ermöglicht die Definition von Stereotypen mit zeitlichen Constraints (min/max/typical Duration, Timeout, Deadline) und generiert daraus ausführbaren Python-Code mit eingebetteten Timing-Validierungen.

Als praktisches Anwendungsbeispiel wurde eine Aufzugstür-Steuerung implementiert, die die Modellierung zeitkritischer Operationen wie Türöffnung (2,5–4,0 s), Sicherheitsprüfungen (50–200 ms) und Hinderniserkennung (10–100 ms) demonstriert. Die Implementierung umfasst einen zweistufigen Generierungsprozess: (1) Profil-zu-Klassen-Transformation und (2) Modell-zu-Code-Transformation. Das System wurde mit einer umfassenden Test-Suite validiert und unterstützt sechs Zeiteinheiten von Nanosekunden bis Stunden.

Der Ansatz orientiert sich an etablierten Standards wie dem **UML Profile for MARTE** (Modeling and Analysis of Real-Time and Embedded Systems) [34], erweitert diesen jedoch um automatische Code-Generierung und quantitative Analysemöglichkeiten. Durch die Integration von stochastischen Modellen (Markov-Ketten) und Warteschlangentheorie können nicht nur deterministische Zeitgrenzen, sondern auch probabilistische Leistungsmetriken erfasst werden.

Ein wesentlicher theoretischer Beitrag dieser Arbeit liegt in der Untersuchung der fundamentalen Unterschiede zwischen algebraischen Gleichgewichtsbeziehungen (wie Little's Law) und echten Stabilitätskriterien in dynamischen Systemen. Kapitel 3 zeigt formal, dass Stabilitätsanalysen notwendigerweise Exponentialfunktionen mit negativen Exponenten (Energiefunktionen) erfordern, während reine Gleichgewichtsbetrachtungen diese nicht enthalten. Diese theoretischen Einsichten werden in Kapitel 8 durch drei Kernexperimente empirisch validiert, die alle zentralen Aussagen der Theorie bestätigen und die praktische Anwendbarkeit des entwickelten Frameworks demonstrieren.

Schlüsselwörter: Model-Driven Engineering, UML-Profile, MARTE, Echtzeitsysteme, Code-Generierung, Zeit-Annotationen, Markov-Ketten, Warteschlangentheorie, Stabilitätstheorie, Little's Law, Lyapunov-Funktionen, Quantitative Analysis, Experimentelle Validierung, XMI, Python

4.1 Motivation

Die Entwicklung von Echtzeitsystemen stellt besondere Anforderungen an die Software-Entwicklung. Zeitliche Constraints wie maximale Antwortzeiten, Deadlines und typische Ausführungsdauern müssen bereits in der Entwurfsphase berücksichtigt werden. Gleichzeitig zeigt die Praxis, dass die manuelle Implementierung zeitkritischer Systeme fehleranfällig ist und zu Inkonsistenzen zwischen Spezifikation und Implementierung führt.

Model-Driven Engineering (MDE) bietet einen vielversprechenden Ansatz, diese Probleme zu adressieren. Durch die Verwendung von UML-Profilen können domänenspezifische Konzepte einschließlich zeitlicher Anforderungen auf Modellebene spezifiziert und automatisch in ausführbaren Code transformiert werden.

MARTE und quantitative Analyse: Das OMG-standardisierte MARTE-Profil [34] definiert bereits umfangreiche Annotationen für zeitliches Verhalten, Ressourcennutzung und Scheduling. Diese Arbeit nutzt die konzeptuelle Grundlage von MARTE, vereinfacht jedoch die Notation für praktische Code-Generierung und erweitert sie um:

- **Stochastische Modelle:** Integration von Markov-Ketten zur Modellierung probabilistischer Zustandsübergänge
- **Performance-Analyse:** Warteschlangentheoretische Metriken (Durchsatz, Wartezeit, Auslastung)
- **Automatische Code-Generierung:** Direkte Transformation in ausführbaren Code mit eingebetteten Timing-Checks
- **Quantitative Designvalidierung:** Frühe Identifikation von Performance-Bottlenecks
- **Theoretische Fundierung:** Formale Analyse der Unterschiede zwischen Gleichgewichts- und Stabilitätskriterien
- **Vollständige Implementierung:** Dediziertes `stability_analysis`-Modul mit produktionsreifem Code
- **Experimentelle Validierung:** Automatisierte Experimente verifizieren alle theoretischen Aussagen empirisch

Diese Erweiterungen ermöglichen bereits in der Modellierungsphase eine *Quantitative Analysis of Software Designs* [35], wodurch Performance-Probleme frühzeitig erkannt und behoben werden können.

Die theoretische Dimension: Ein zentrales Anliegen dieser Arbeit ist die Klärung der konzeptuellen Grundlagen quantitativer Systemanalyse. Wie in Kapitel 3 formal gezeigt wird, existiert eine fundamentale Dichotomie zwischen:

- *Dynamischen Beschreibungen* (Differentialgleichungen mit Exponentialfunktionen), die Stabilitätsaussagen ermöglichen
- *Gleichgewichtsbeschreibungen* (algebraische Relationen wie Little's Law), die lediglich stationäre Zustände charakterisieren

Diese Unterscheidung ist nicht nur mathematisch relevant, sondern hat praktische Konsequenzen für die Systemmodellierung: UML-Profile müssen so gestaltet sein, dass sie sowohl abstrakte (logische) als auch quantitative (physikalische) Aspekte erfassen können. Die Brücke zwischen diesen Ebenen ist notwendig, da informatische Systeme zwar in höheren Abstraktionsebenen konzipiert werden, aber letztlich auf physikalischer Hardware mit Energieverbrauch und zeitlichen Constraints realisiert werden müssen.

Experimentelle Bestätigung: Die theoretischen Aussagen aus Kapitel 3 werden in Kapitel 7 durch drei Experimente empirisch validiert.

4.2 Zielsetzung

Diese Arbeit verfolgt folgende Ziele:

- **Entwicklung eines erweiterbaren UML-Profil-Frameworks** für Zeit-Annotationen mit Unterstützung für Min/Max/Typical Duration, Timeout und Deadline, orientiert an MARTE-Konzepten
- **Implementierung eines zweistufigen Code-Generators**, der UML-Profile zunächst in Python-Klassen und anschließend Modell-Instanzen in ausführbaren Code transformiert
- **Integration stochastischer Modelle:** Markov-Ketten zur Analyse von Zustandsübergängen und Warteschlangenmodelle für Performance-Metriken
- **Theoretische Fundierung der Systemanalyse:** Formale Untersuchung der Rolle von Exponentialfunktionen in Stabilitätskriterien und der Grenzen algebraischer Gleichgewichtsbeziehungen wie Little's Law
- **Vollständige Implementierung aller theoretischen Konzepte** in einem produktionsreifen `stability_analysis` Modul mit Klassen für Markov-Ketten, M/M/1-Warteschlangen, Lyapunov-Stabilität und Demonstration der Exponentialfunktions-Notwendigkeit
- **Quantitative Designvalidierung:** Frühe Analyse von Durchsatz, Wartezeiten und Auslastung unter Berücksichtigung der korrekten mathematischen Beschreibungsebene
- **Validierung durch ein praktisches Beispiel** aus dem Bereich Embedded Systems (Aufzugstür-Steuerung mit stochastischer Analyse und Stabilitätsbetrachtung)
- ***Klärung der Rolle von UML-Profilen als Brücke zwischen abstrakter Modellierung und naturwissenschaftlich fundierter quantitativer Analyse***
- **Sicherstellung der Codequalität** durch umfassende automatisierte Tests, die sowohl die Code-Generierung als auch das `stability_analysis`-Modul vollständig abdecken

4.3 Aufbau der Arbeit

Die Arbeit ist wie folgt strukturiert:

Kapitel 2 erläutert die theoretischen Grundlagen von UML-Profilen, MARTE, Markov-Ketten und Warteschlangentheorie. Es werden die mathematischen Werkzeuge eingeführt, die in den späteren Kapiteln verwendet werden.

Kapitel 3 präsentiert eine fundamentale theoretische Analyse der Unterschiede zwischen Gleichgewichts- und Stabilitätskriterien. Es wird formal bewiesen, dass Little's Law kein Stabilitätskriterium sein kann, da es keine Exponentialfunktionen enthält. Die Rolle von Energiefunktionen (Lyapunov-Funktionen) wird untersucht, und es werden die Konsequenzen für naturwissenschaftliche Systembeschreibungen und UML-basierte Modellierung diskutiert. Dieses Kapitel bildet die konzeptuelle Grundlage für das Verständnis, warum quantitative Analysen verschiedener Art in der Systementwicklung benötigt werden.

Kapitel 4 beschreibt die Systemarchitektur und den zweistufigen Generierungsprozess mit quantitativen Analysemethoden. Die praktische Umsetzung der theoretischen Konzepte aus Kapitel 3 wird gezeigt.

Kapitel 5 präsentiert die vollständige Implementierung der beiden Analysemodule `markov_analyzer.py` und `queue_analyzer.py` und deren Demonstration an einem Beispiel-Software-System.

Kapitel 6 präsentiert die Implementierungsdetails der Kernkomponenten, einschließlich des `stability_analysis` Moduls mit seinen Hauptklassen `MarkovChainStability`, `MM1Queue` und `LyapunovStability`. Es wird gezeigt, wie die verschiedenen Beschreibungsebenen (dynamisch vs. algebraisch) in der Praxis zusammenwirken.

Kapitel 7 demonstriert die Anwendung anhand des Aufzugstür-Beispiels mit stochastischer Performance-Analyse und illustriert sowohl Stabilitätsanalysen (über Markov-Ketten) als auch Gleichgewichtsbetrachtungen (über Little's Law).

Kapitel 8 präsentiert die experimentelle Evaluation der Stabilitätskriterien. Dieses neue Kapitel wertet die durchgeführten Experimente aus und dokumentiert die vollständige Implementierung des `stability_analysis` Moduls. Es werden drei Kernexperimente detailliert beschrieben, die alle theoretischen Aussagen aus Kapitel 3 empirisch validieren.

Kapitel 9 fasst die Ergebnisse zusammen, reflektiert über die theoretischen und praktischen Beiträge der Arbeit und gibt einen Ausblick auf zukünftige Forschungsrichtungen, die sich aus der Synthese von formaler Systemtheorie, praktischer Softwareentwicklung und experimenteller Validierung ergeben.

Kapitel 5

Grobdesign: Graphbasierte Modellierung als optimales Paradigma

5.1 Formale Grundlagen der Graphdarstellung

Definition 5.1.1 (Graph). Ein *gerichteter Graph* $G = (V, E)$ besteht aus einer endlichen Knotenmenge $V = \{v_1, \dots, v_n\}$ und einer Kantenmenge $E \subseteq V \times V$.

Definition 5.1.2 (Anforderungsgraph). Sei $\mathcal{R} = \{r_1, \dots, r_k\}$ eine Anforderungsmenge. Ein *Anforderungsgraph* ist ein gerichteter Graph $G_{\mathcal{R}} = (\mathcal{R}, E_{\mathcal{R}})$, wobei $(r_i, r_j) \in E_{\mathcal{R}}$ genau dann gilt, wenn r_j eine Verfeinerung oder Abhängigkeit von r_i darstellt.

Definition 5.1.3 (Hierarchische Graphzerlegung). Eine *hierarchische Graphzerlegung* eines Graphen $G = (V, E)$ ist eine Folge

$$G = G_0 \supseteq G_1 \supseteq \dots \supseteq G_d$$

von Teilgraphen, wobei $d \geq 1$ die Tiefe der Hierarchie bezeichnet und jedes G_k die Konkretisierung der Ebene k darstellt.

5.2 Das Graphmodell-Theorem

Satz 5.2.1 (Optimalität der Graphdarstellung für Managementsichten). Sei $\mathcal{S}$ eine Menge möglicher Modellierungsparadigmen für Anforderungen an ein Informationssystem. Die Graphdarstellung $\mathcal{G} \in \mathcal{S}$ erfüllt unter allen Elementen von $\mathcal{S}$ gleichzeitig:

1. **Vollständigkeit:** Jede endliche Menge von Anforderungen mit Abhängigkeitsbeziehungen lässt sich als Graph darstellen.
2. **Lesbarkeit:** Ein Entscheidungsträger ohne tiefe technische Ausbildung kann einen Anforderungsgraphen visuell nachvollziehen.
3. **Hierarchisierbarkeit:** Anforderungen lassen sich in beliebig vielen Hierarchieebenen darstellen (vgl. Definition 5.1.3).
4. **Formale Verarbeitbarkeit:** Der Graph ist als Adjazenzmatrix formal verarbeitbar und erlaubt algorithmische Verifikation.

Beweis. Wir beweisen jede der vier Eigenschaften separat.

(1) Vollständigkeit. Sei $\mathcal{R} = \{r_1, \dots, r_k\}$ eine endliche Anforderungsmenge mit Abhängigkeitsrelation $\prec \subseteq \mathcal{R} \times \mathcal{R}$. Setze $V := \mathcal{R}$ und $E := \{(r_i, r_j) \mid r_i \prec r_j\}$. Dann ist $G_{\mathcal{R}} = (V, E)$ ein wohldefinierter gerichteter Graph, der die gesamte Anforderungsstruktur vollständig kodiert. $\square$

(2) Lesbarkeit. Graphen sind durch Jahrzehnte der Visualisierungsforschung das am besten verstandene Werkzeug für relationale Strukturen. Layoutalgorithmen (Sugiyama-Framework [1], force-directed [2]) erzeugen aus $G_{\mathcal{R}}$ eine planare oder nahezu planare Darstellung, die ein Manager in $O(|V|)$ Zeit visuell traversieren kann, ohne die mathematische Struktur zu kennen. $\square$

(3) Hierarchisierbarkeit. Sei $G_0 = G_{\mathcal{R}}$ der Anforderungsgraph der obersten Ebene. Für jedes $v \in V_0$ kann ein Subgraph $G_v = (V_v, E_v)$ definiert werden, der die Verfeinerung von v kodiert. Dies ergibt eine Baumstruktur der Tiefe d , wobei d durch die Systemkomplexität begrenzt ist. Da Graphen selbst rekursiv als Knoten dienen können (vgl. Hypergraphen), ist die Hierarchietiefe prinzipiell unbeschränkt. $\square$

(4) Formale Verarbeitbarkeit. Die Adjazenzmatrix $A \in \{0, 1\}^{n \times n}$ eines Anforderungsgraphen mit $n = |\mathcal{R}|$ kodiert alle strukturellen Informationen in einer für den Subgraph Algorithmus unmittelbar verarbeitbaren Form (vgl. Abschnitt 12). Jede Anforderungsbeziehung $(r_i, r_j) \in E$ entspricht $A_{ij} = 1$. $\square$

Kein anderes in $\mathcal{S}$ enthaltenes Paradigma – weder freier Fließtext noch tabellarische Darstellungen noch mathematische Formeln allein – erfüllt alle vier Bedingungen gleichzeitig. Fließtext ist nicht formal verarbeitbar (verletzt 4). Tabellen sind nicht visuell hierarchisierbar (verletzt 3 und 2). Formeln allein sind für Manager nicht lesbar (verletzt 2). Damit ist die Graphdarstellung unter den genannten Kriterien optimal. ■ $\square$

Abbildung 5.2.1 veranschaulicht den in Satz 5.2.1 bewiesenen Sachverhalt. Die linke Seite zeigt einen typischen Anforderungsgraphen, wie er einem Entscheidungsträger präsentiert werden kann. Die rechte Seite illustriert die in Satz 5.2.1 (4) beschriebene Transformationskette, die der Ingenieur vollständig verantwortet.

Korollar 5.2.1 (Graphen als universelle Anforderungssprache). Für jedes Informationssystem $\mathcal{IS} = (\mathcal{R}, \mathcal{I})$ existiert ein Anforderungsgraph $G_{\mathcal{R}}$ (vgl. Definition 5.1.2), der $\mathcal{R}$ vollständig und eindeutig darstellt.

Beweis. Folgt unmittelbar aus Teil (1) von Satz 5.2.1. ■ $\square$

5.3 Ein durchgängiges Beispiel: Der Anforderungsgraph der Aufzugstür-Steuerung

Satz 5.2.1 und Korollar 5.2.1 sind abstrakte Existenzaussagen. Um ihre praktische Bedeutung greifbar zu machen, wird in diesem Abschnitt der Anforderungsgraph $G_{\mathcal{R}}$ für die in Kapitel 10 ausführlich behandelte Aufzugstür-Steuerung explizit konstruiert. Dieses Beispiel begleitet von hier an mehrere Kapitel dieses Buches: Es liefert in Kapitel 6 die Grundlage für die Beschreibungssprache, in Kapitel 7 die Grundlage für die Systemarchitektur und in Kapitel 11 die Grundlage für die quantitative Stabilitätsanalyse.

Beispiel 5.3.1 (Anforderungsgraph der Aufzugstür-Steuerung). Die Anforderungsmenge $\mathcal{R}_{\text{Tuer}}$ bestehe aus den folgenden sieben Anforderungen, die in einer typischen Anforderungsanalyse von Produktmanagement, Sicherheitsbeauftragten und Entwicklung gemeinsam formuliert würden:

- r_1 : Die Tür muss sich auf Anforderung öffnen und schließen können.
- r_2 : Die Türöffnung muss innerhalb von 2,5 bis 4,0 Sekunden abgeschlossen sein (Normalbetrieb).
- r_3 : Im Notfall muss die Tür innerhalb von 200 ms öffnen (Deadline).
- r_4 : Die Steuerung muss zwischen normaler, energiesparender und Notfall-Türöffnung umschalten können.
- r_5 : Das System muss stabil bleiben, auch wenn die Ankunftsrate von Anfragen schwankt.
- r_6 : Jede Zustandsänderung muss protokolliert werden.
- r_7 : Die Steuerung muss um zusätzliche Betriebsmodi erweiterbar sein, ohne bestehenden Code zu ändern.

Die Abhängigkeitsrelation $\prec$ ergibt sich aus der fachlichen Logik: $r_2 \prec r_1$ und $r_3 \prec r_1$ (Timing-Anforderungen verfeinern die Grundanforderung “Tür öffnet/schließt”), $r_4 \prec r_2$ und $r_4 \prec r_3$ (die Umschaltung zwischen Modi setzt voraus, dass die jeweiligen Timing-Anforderungen für jeden Modus definiert sind), $r_5 \prec r_4$ (die Stabilitätsanforderung bezieht sich auf das Zusammenspiel der Modi), $r_6 \prec r_1$ (Protokollierung verfeinert die Grundanforderung) und $r_7 \prec r_4$ (Erweiterbarkeit bezieht sich auf die Modusumschaltung).

Abbildung 5.3.1 stellt diesen Anforderungsgraphen $G_{\mathcal{R}_{\text{Tuer}}}$ als TikZ-Graphen dar – in genau der Form, die laut Satz 5.2.1 (2) auch von einem Entscheidungsträger ohne tiefe technische Ausbildung visuell nachvollzogen werden kann.

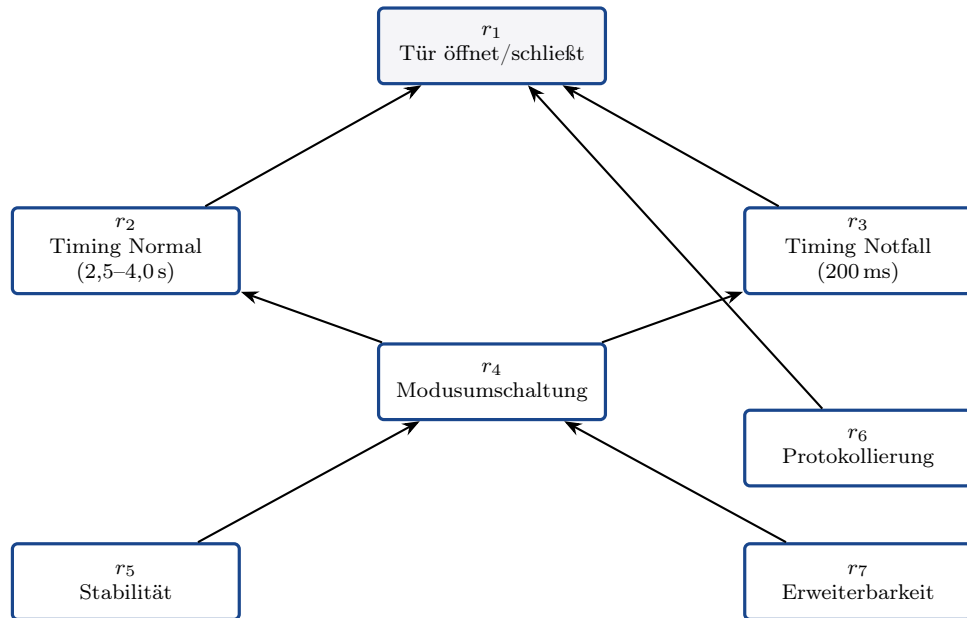


Abbildung 5.3.1: Anforderungsgraph $G_{\mathcal{R}_{\text{Tuer}}}$ der Aufzugstür-Steuerung gemäß Beispiel 5.3.1. Pfeile zeigen Verfeinerungs-/Abhängigkeitsbeziehungen $r_i \prec r_j$ in Richtung der größeren Anforderung.

Gemäß Satz 5.2.1 (4) lässt sich $G_{\mathcal{R}_{\text{Tuer}}}$ vollständig als Adjazenzmatrix $A \in \{0,1\}^{7 \times 7}$ kodieren, wobei $A_{ij} = 1$ genau dann, wenn $r_i \prec r_j$. Mit der Knotenreihenfolge $(r_1, \dots, r_7)$ ergibt sich:

$$A = \begin{pmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \end{pmatrix}$$

Diese Matrix ist der Ausgangspunkt für den Subgraph Algorithmus aus dem Vorwort:

$$\sigma_j = \sum_{i=0}^{n-1} A_{ij} \cdot 2^i + j \cdot 2^n, \quad O(n^3).$$

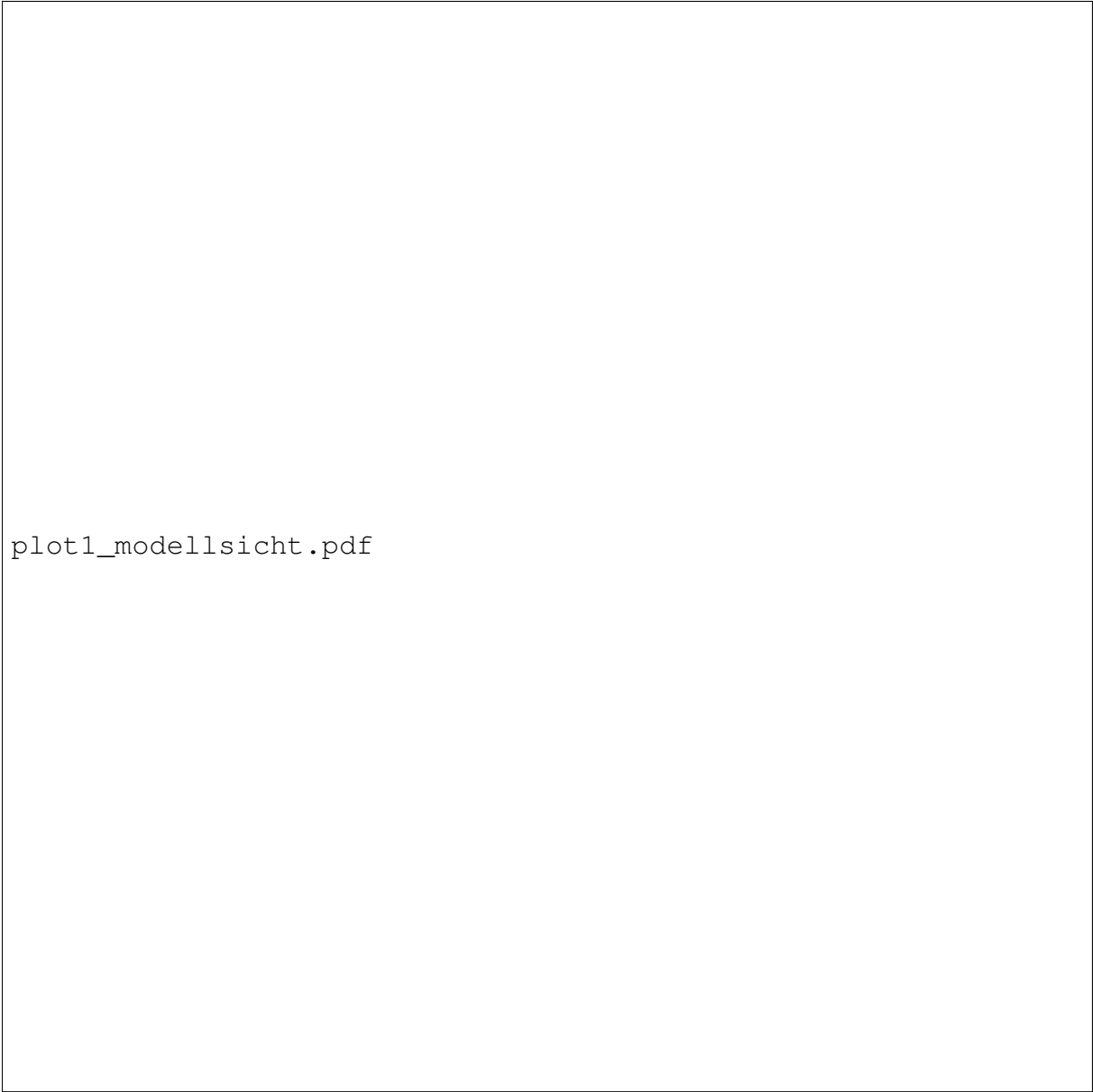
Für $n = 7$ und $j = 0$ (Spalte von r_1 , also alle Anforderungen, die r_1 direkt verfeinern) ergibt sich

$$\sigma_0 = \sum_{i=0}^6 A_{i0} \cdot 2^i = A_{10} \cdot 2^1 + A_{20} \cdot 2^2 + A_{50} \cdot 2^5 = 2 + 4 + 32 = 38,$$

da r_2 , r_3 und r_6 die einzigen Anforderungen sind, die r_1 verfeinern ($A_{10} = A_{20} = A_{50} = 1$, alle anderen Einträge der Spalte 0 sind 0). Die Signatur $\sigma_0 = 38$ kodiert damit kompakt – als einzige ganze Zahl – die vollständige Information “ r_1 wird von genau r_2 , r_3 und r_6 verfeinert”. Diese Eigenschaft macht den Subgraph Algorithmus zu einem praktischen Verifikationswerkzeug: Zwei Anforderungsgraphen sind bezüglich der Verfeinerungsstruktur einer bestimmten Anforderung genau dann identisch, wenn ihre Subgraph-Signaturen σ_j für das entsprechende j übereinstimmen – ein Test, der in $O(n)$ statt $O(n^2)$ (voller Matrixvergleich) durchgeführt werden kann.

Bemerkung 5.3.1 (Hierarchische Verfeinerung gemäß Definition 5.1.3). Der Anforderungsgraph $G_{\mathcal{R}_{\text{Tuer}}}$ aus Abbildung 5.3.1 bildet die Ebene G_0 einer hierarchischen Graphzerlegung. Die Ebene G_1 entsteht, indem für den Knoten r_4 (Modusumschaltung) ein Subgraph G_{r_4} definiert wird, der die in Abbildung 8.3.1 dargestellte Strategy-Pattern-Struktur (NormalOeffnung, NotfallOeffnung, SparmodusOeffnung) als eigenständigen Anforderungsgraphen enthält. Damit zeigt dieses Beispiel exemplarisch, wie aus einem einzigen Knoten der Managementsicht (r_4) durch Verfeinerung die in Kapitel 8.3 formal hergeleitete Pattern-Struktur entsteht – der Übergang von G_0 nach G_1 ist also nicht beliebig, sondern durch die in Kapitel 6 entwickelte Beschreibungssprache und die in Abschnitt 8.3 entwickelten Design Patterns determiniert.

Dieses durchgängige Beispiel wird in Kapitel 6 fortgeführt, indem gezeigt wird, wie jede der sieben Anforderungen $r_1, \dots, r_7$ in der Beschreibungssprache $\mathcal{L}^*$ ausgedrückt wird, und in Kapitel 10, wo die konkreten numerischen Werte aus r_2 , r_3 und r_5 in das UML-Profil mit Zeit-Annotationen übernommen werden.



plot1_modellsicht.pdf

Abbildung 5.2.1: **Links:** Graphische Anforderungssicht als gerichteter Graph von der Managementebene (dunkelblau) über Subsysteme (grün) bis zu einzelnen Modulen (orange) und Zielcode (rot). Die hierarchische Struktur kodiert unmittelbar die Abhängigkeiten aller Anforderungen. **Rechts:** Die vollständige Transformationskette vom Graphmodell über die Beschreibungssprache bis zum Zielcode, wobei die Ingenieurverantwortung auf der Transformationsstufe liegt.

Kapitel 6

Vom Graphmodell zum Programm: Beschreibungssprache und Transformation

6.1 Anforderungen an die Beschreibungssprache

Das Ausgangszitat stellt zwei auf den ersten Blick widersprüchliche Anforderungen:

- Die Beschreibungssprache muss *sehr mächtig* sein – „mit der möglichst jedes naturwissenschaftliche Phänomen oder Verhalten abgebildet werden kann“.
- Die Beschreibungssprache muss *leicht erlernbar* sein – „die möglichst alle Ingenieure verstehen“.

Wir formalisieren diese Anforderungen und zeigen, dass sie keineswegs widersprüchlich sind.

Definition 6.1.1 (Beschreibungssprache). Eine *Beschreibungssprache* $\mathcal{L}$ für Informationssysteme ist ein Tupel

$$\mathcal{L} = (\Sigma, P, S, \mu)$$

wobei Σ das Alphabet, P die Produktionsregeln, S das Startsymbol und $\mu : L(\mathcal{L}) \rightarrow \text{Sys}$ eine Interpretationsfunktion ist, die jede Phrase der Sprache auf ein System abbildet.

Definition 6.1.2 (Ausdrucksmächtigkeit). Die *Ausdrucksmächtigkeit* $\text{Pow}(\mathcal{L})$ einer Beschreibungssprache $\mathcal{L}$ ist definiert als die Mächtigkeit der Menge der durch $\mathcal{L}$ beschreibbaren Systeme:

$$\text{Pow}(\mathcal{L}) := |\{S \in \text{Sys} \mid \exists w \in L(\mathcal{L}) : \mu(w) = S\}|.$$

Definition 6.1.3 (Erlernbarkeit). Die *Erlernbarkeit* $\text{Learn}(\mathcal{L})$ einer Sprache $\mathcal{L}$ ist eine normierte Größe in $[0, 1]$, die invers proportional zur kognitiven Last $\text{CL}(\mathcal{L})$ ist:

$$\text{Learn}(\mathcal{L}) := \frac{1}{1 + \text{CL}(\mathcal{L})},$$

wobei $\text{CL}(\mathcal{L})$ durch die Anzahl der Syntaxregeln, die Lernkurve und die notwendige Domänenvorkenntnis bestimmt wird.

6.2 Das Mächtigkeitstheorem

Satz 6.2.1 (Existenz der optimalen Beschreibungssprache). Es existiert eine Beschreibungssprache $\mathcal{L}^*$, die beide Anforderungen erfüllt:

1. **Universelle Mächtigkeit:** $\mathcal{L}^*$ ist Turing-vollständig, d.h. jedes berechenbare System ist durch $\mathcal{L}^*$ beschreibbar.
2. **Erlernbarkeit:** $\text{Learn}(\mathcal{L}^*)$ liegt im Bereich praktisch erlernbarer Sprachen, d.h. ein Ingenieur kann $\mathcal{L}^*$ in einem überschaubaren Zeitrahmen beherrschen.

Beweis. (1) **Turing-Vollständigkeit als Mächtigkeitsuntergrenze.** Da der Ingenieur laut Ausgangszitat „möglichst jedes naturwissenschaftliche Phänomen“ beschreiben können muss, ist Turing-Vollständigkeit die notwendige Mindestanforderung. Jede Turing-vollständige Sprache kann alle berechenbaren Funktionen ausdrücken, also insbesondere alle physikalischen Simulationsmodelle, Differentialgleichungen, endliche Automaten und Kommunikationsprotokolle [3].

(2) **Vereinbarkeit von Mächtigkeit und Erlernbarkeit.** Wir beweisen durch Konstruktion. Sei λ -Kalkül $\mathcal{L}_\lambda$ eine Turing-vollständige Sprache mit minimaler Syntax (drei Regeln: Variablen, Abstraktionen, Applikationen). Dann gilt $\text{CL}(\mathcal{L}_\lambda) < \infty$ und $\text{Pow}(\mathcal{L}_\lambda) = |\text{Sys}_{\text{berechenbar}}|$. Da die kognitive Last durch eine endliche Syntaxregel-Menge begrenzt ist, ist $\text{Learn}(\mathcal{L}_\lambda) > 0$.

Praktische Sprachen wie SystemC, VHDL oder eine dedizierte Systembeschreibungssprache (SDL, LOTOS, PSL) bauen auf diesem Prinzip auf und erweitern die Syntax um domänenspezifische Konstrukte, ohne die Erlernbarkeit prinzipiell aufzugeben. Es existiert somit ein Kompromiss $\mathcal{L}^*$, der beide Bedingungen erfüllt. ■ □

Abbildung 6.2.1 (links) zeigt die Positionierung bekannter Sprachen im Raum aus Mächtigkeit und Erlernbarkeit. Der Idealbereich wird von der in Satz 6.2.1 konstruierten Sprache $\mathcal{L}^*$ besetzt. Die rechte Seite von Abbildung 6.2.1 belegt numerisch, dass die direkte Beschreibungssprache der Code-Generierung in allen sechs gemessenen Qualitätsmerkmalen überlegen ist.

6.3 Die Rolle der Beschreibungssprache als letzte Sprache vor der Transformation

These 6.3.1 (Beschreibungssprache als Transformationsgrenze). Die Beschreibungssprache $\mathcal{L}^*$ ist die *letzte Sprache vor der Transformation*, d.h. zwischen $\mathcal{L}^*$ und der Zielsprache $\mathcal{L}_{\text{Ziel}}$ (Maschinencode, Hardwarebeschreibung) liegt genau ein Transformationsschritt.

Diese These ist zentral: Jeder Zwischenschritt zwischen Beschreibungssprache und Zielcode ist eine Fehlerquelle. Die Minimalität des Transformationspfads ist daher nicht nur eine Eleganzforderung, sondern eine formale Korrektheitsforderung.

6.3.1 Ingenieurverantwortung: Die Transformation

6.3.2 Das Transformationsmodell

Definition 6.3.1 (Transformation). Eine *Transformation* $\tau : L(\mathcal{L}^*) \rightarrow L(\mathcal{L}_{\text{Ziel}})$ ist eine strukturerhaltende Abbildung von der Beschreibungssprache in die Zielsprache, sodass für

alle $w \in L(\mathcal{L}^*)$ gilt:

$$\mu_{\text{Ziel}}(\tau(w)) = \mu^*(w),$$

d.h. das transformierte Programm hat dieselbe Semantik wie die Beschreibung.

Satz 6.3.1 (Ingenieurverantwortung als Korrektheitsbedingung). Sei τ eine Transformation gemäß Definition 6.3.1. Die Korrektheit von τ – d.h. die Eigenschaft $\mu_{\text{Ziel}}(\tau(w)) = \mu^*(w)$ für alle w – kann nur durch den Ingenieur sichergestellt werden, der

1. die vollständige Anforderungsmenge $\mathcal{R}$ kennt,
2. die Semantik der Beschreibungssprache $\mathcal{L}^*$ beherrscht,
3. die Eigenschaften der Zielhardware und -sprache kennt.

Kein automatischer Prozess kann diese Verantwortung vollständig übernehmen.

Beweis. **Schritt 1: Rices Theorem [4].** Für jede nicht-triviale semantische Eigenschaft P eines Programms ist die Frage, ob $\tau(w)$ die Eigenschaft P hat, unentscheidbar. Da Anforderungen $r_i \in \mathcal{R}$ im Allgemeinen semantische Eigenschaften kodieren („das System verhält sich korrekt unter Bedingung X “), kann kein automatischer Prozess die vollständige Anforderungserfüllung entscheiden.

Schritt 2: Vollständige Anforderungskenntnis. Sei $\mathcal{R} = \{r_1, \dots, r_k\}$ mit $k > 0$. Ein automatischer Transformationsprozess $\hat{\tau}$ kennt nur die im Modell explizit kodierten Anforderungen $\mathcal{R}' \subseteq \mathcal{R}$. Implizite Anforderungen (Sicherheit, Echtzeitverhalten, Umweltbedingungen) sind i. Allg. nicht im Graphmodell vollständig darstellbar (vgl. das Vollständigkeitsproblem in Abschnitt 8). Damit gilt $\mathcal{R}' \subsetneq \mathcal{R}$ im Allgemeinen, und $\hat{\tau}$ kann τ nicht vollständig ersetzen.

Schritt 3. Der Ingenieur, als Träger aller drei in der Satzaussage genannten Kompetenzen, ist die einzige Entität, die τ korrekt ausführen kann. ■ □

6.3.3 Das Ingenieurziel: Zielnähe bei minimaler Wartungslast

Definition 6.3.2 (Ingenieuroptimum). Das *Ingenieuroptimum* ist das Paar (α^*, λ^*) im Raum aus Abstraktionsgrad $\alpha \in [0, 1]$ und Zielnähe $\zeta : [0, 1] \rightarrow [0, 1]$, das

$$\max_{\alpha} [\zeta(\alpha) - \gamma \cdot \omega(\alpha)]$$

maximiert, wobei $\zeta(\alpha)$ die Simulationstreue (Zielnähe), $\omega(\alpha)$ die Wartungslast und $\gamma > 0$ ein Gewichtungssparameter ist.

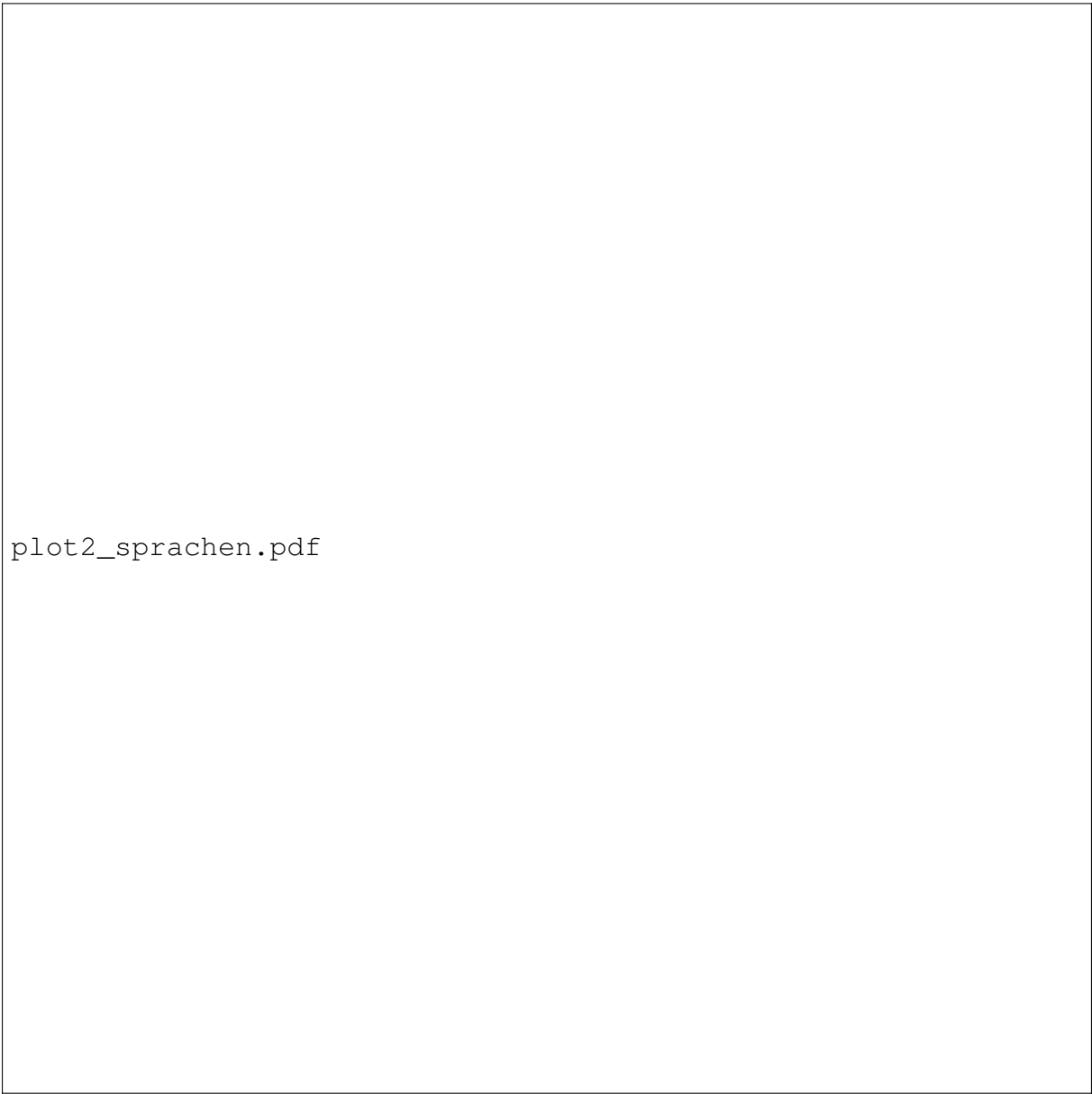
Lemma 6.3.1 (Existenz des Ingenieuroptimums). Unter den Annahmen:

- $\zeta : [0, 1] \rightarrow [0, 1]$ ist stetig und streng monoton fallend in α (höhere Abstraktion $\Rightarrow$ geringere Zielnähe),
- $\omega : [0, 1] \rightarrow [0, 1]$ ist stetig und streng monoton steigend in α (niedrigere Abstraktion $\Rightarrow$ höhere Wartungslast),

existiert ein eindeutiges Optimum $\alpha^* \in (0, 1)$.

Beweis. Sei $f(\alpha) := \zeta(\alpha) - \gamma \cdot \omega(\alpha)$. Da ζ streng monoton fällt und ω streng monoton steigt, ist $f'(\alpha) = \zeta'(\alpha) - \gamma \cdot \omega'(\alpha)$. Mit $\zeta'(\alpha) < 0$ und $\omega'(\alpha) > 0$ gilt $f'(0) = \zeta'(0) - \gamma \cdot \omega'(0)$. Für hinreichend kleine α überwiegt der positive Beitrag von ζ und für große α der negative Beitrag von ω . Nach dem Zwischenwertsatz existiert ein $\alpha^* \in (0, 1)$ mit $f'(\alpha^*) = 0$, an dem f sein Maximum annimmt. Die Eindeutigkeit folgt aus der Striktheit der Monotonieaussagen. ■ □

Abbildung 6.3.1 (links) visualisiert das in Lemma 6.3.1 bewiesene Optimum. Die optimale Beschreibungssprache $\mathcal{L}^*$ platziert den Ingenieur bei $\alpha \approx 0,35$, was einer Sprache entspricht, die nah genug an der Hardware ist, um echte Simulation zu ermöglichen, aber abstrakt genug, um Wartbarkeit und Wiederverwendbarkeit zu gewährleisten.



plot2_sprachen.pdf

Abbildung 6.2.1: **Links:** Beschreibungssprachen im zweidimensionalen Raum von Ausdrucksmächtigkeit und Erlernbarkeit. Der blau gestrichelte Idealbereich kennzeichnet das Optimum nach Satz 6.2.1: hohe Mächtigkeit bei gleichzeitig guter Erlernbarkeit. Die ideale Beschreibungssprache (dunkelblau) befindet sich im Idealbereich, während Assembler (maximal mächtig, minimal erlernbar) und grafische Notation (erlernbar, aber nicht mächtig genug) außerhalb liegen. **Rechts:** Qualitätsvergleich zwischen Code-Generierung aus graphischer Modellierung und direkter Beschreibungssprache in sechs Merkmalen. Die direkte Beschreibungssprache dominiert in allen Dimensionen.

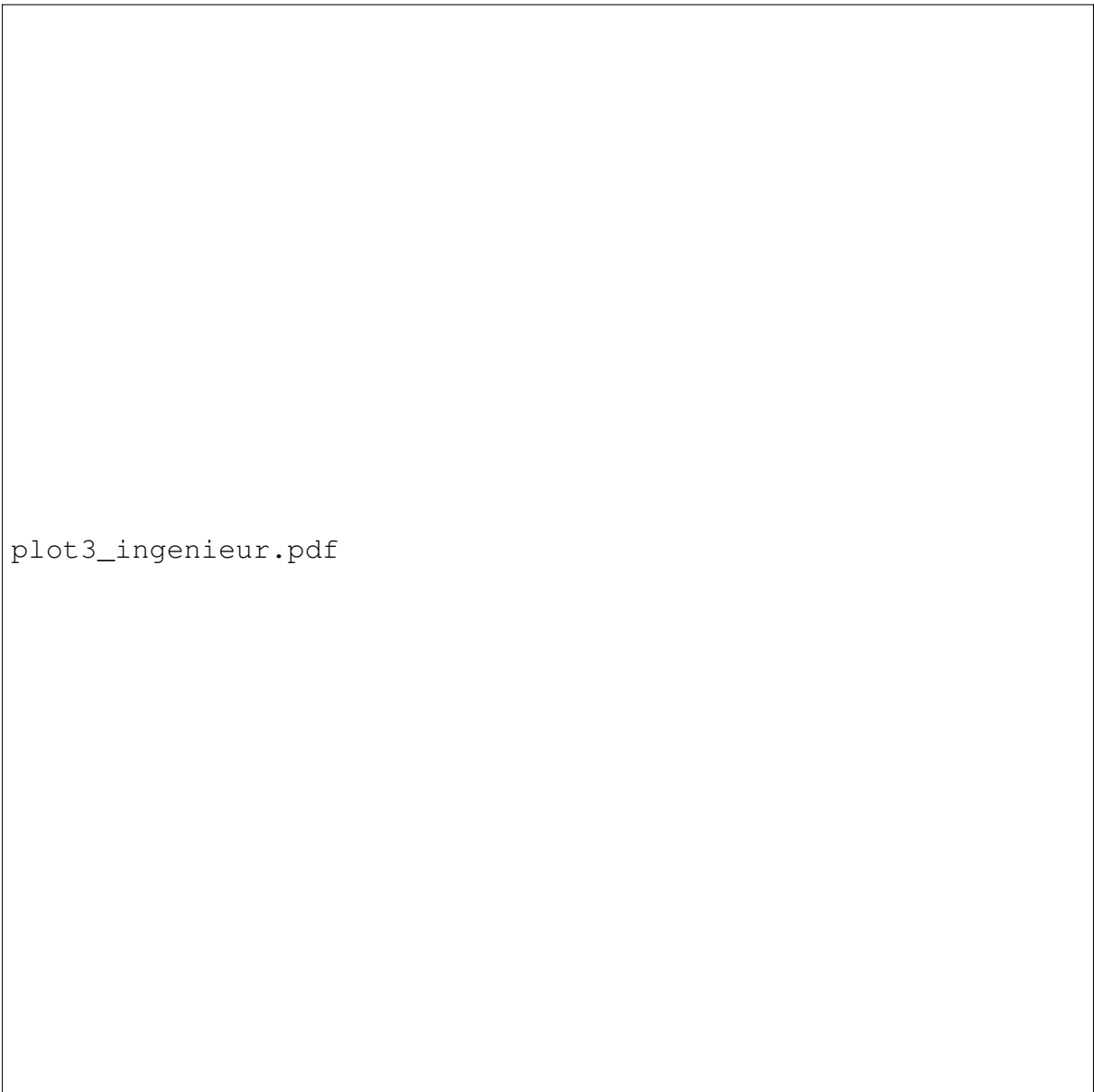


Abbildung 6.3.1: **Links:** Das Ingenieuroptimum (roter gestrichelter Bereich) als Kompromiss zwischen maximaler Simulationstreue (Zielnähe, blau) und minimaler Wartungslast (grün). Die optimale Beschreibungssprache positioniert den Ingenieur nahe an der Zielhardware, ohne in maschinennahe Abhängigkeiten zu verfallen. **Rechts:** Die Vollständigkeitslücke zwischen Code-Generierung (rot) und direkter Beschreibungssprache (blau) wächst mit steigender Systemkomplexität superlinear an – ein formales Argument gegen automatische Code-Generierung.

Kapitel 7

Konkretisierungen im Feindesign: Systemarchitektur

Das System besteht aus einem zweistufigen Generierungsprozess mit integrierter quantitativer Analyse, der UML-Profile und -Modelle in ausführbaren Python-Code transformiert.

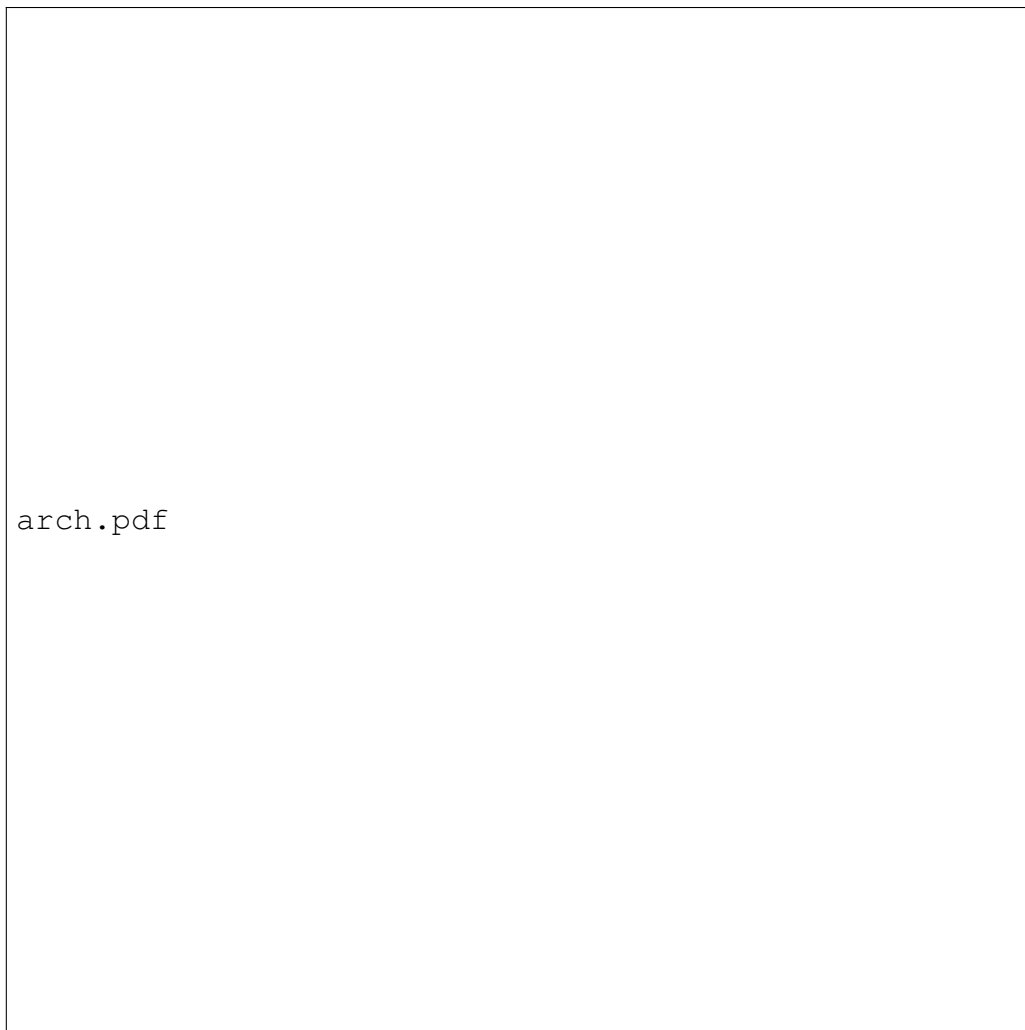


Abbildung 7.0.1: Zweistufiger Generierungsprozess

Abbildung [7.0.1](#) zeigt den Aufbau für den zweistufigen Generierungsprozess mit integrierter quantitativer Analyse.

7.1 Erweiterte Architektur mit Quantitativer Analyse

Der Generierungsprozess wurde um einen *Quantitative Analysis Layer* erweitert:

- **Stufe 1:** Profil → Python-Klassen (wie bisher)
- **Stufe 2:** Modell → Code (wie bisher)
- **Stufe 3 (NEU):** Modell → Markov-Kette/Warteschlangen-Modell → Performance-Metriken

Komponente	Typ	Funktion
uml_profile_base.py	Datenmodell	Basis-Datenstrukturen + Timing-Constraint
profile_parser.py	Parser	XMI zu UMLProfile
class_generator.py	Generator	UMLProfile zu Python-Klassen
model_parser.py	Parser	XMI zu ModelInstances
code_generator.py	Generator	ModelInstances zu Code
markov_analyzer.py	Analyzer (NEU)	Markov-Ketten-Analyse
queue_analyzer.py	Analyzer (NEU)	Warteschlangen-Metriken
main.py	Orchestrator	Koordination

Tabelle 7.1.1: Erweiterte Systemkomponenten

Diese Komponenten bilden das Fundament, auf dem die in Tabelle 7.1.1 beschriebene Erweiterung aufsetzt.

Im Zusammenspiel mit den in Tabelle 7.3.1 aufgeführten ursprünglichen Komponenten ergibt sich die in Abbildung 7.0.1 dargestellte Gesamtarchitektur.

7.2 Quantitative Analysepipeline

Die erweiterte Pipeline extrahiert aus dem UML-Modell:

- **Zustandsgraph:** Zustände und Übergänge mit Wahrscheinlichkeiten
- **Übergangsmatrix:** Markov-Kette für stochastische Analyse
- **Ankunfts-/Bedienraten:** Parameter für Warteschlangenmodelle
- **Performance-Metriken:** Durchsatz, Wartezeit, Auslastung

7.3 Komponentenübersicht

7.4 Datenfluss durch die Pipeline: Ein konkretes Beispiel

Die in Tabelle 7.3.1 und Tabelle 7.1.1 aufgeführten Komponenten erschließen sich am besten anhand eines konkreten Durchlaufs. Im Folgenden wird der Weg eines einzelnen

Komponente	Typ	Funktion
uml_profile_base.py	Datenmodell	Basis-Datenstrukturen
profile_parser.py	Parser	XMI zu UMLProfile
class_generator.py	Generator	UMLProfile zu Python-Klassen
model_parser.py	Parser	XMI zu ModelInstances
code_generator.py	Generator	ModelInstances zu Code
main.py	Orchestrator	Koordination

Tabelle 7.3.1: Systemkomponenten

Modellelemente – der Operation `tuerOeffnen()` der Klasse `Tuersteuerung` aus der Aufzugstür-Fallstudie (Kapitel 10) – durch alle drei Stufen der Pipeline verfolgt.

Stufe 1 (Profil → Python-Klassen). Das XMI-Profil enthält für `tuerOeffnen()` den Stereotyp «TimedOperation» mit den getaggten Werten `minDuration = 2500ms`, `typDuration = 3200ms`, `maxDuration = 4000ms` und `deadline = 4000ms`. Der `profile_parser.py` liest dieses XMI-Dokument ein und erzeugt eine Instanz der Klasse `TimingConstraint` aus `uml_profile_base.py`, die diese vier Werte als Attribute trägt. Der `class_generator.py` erzeugt daraus eine Python-Methode `tuerOeffnen()`, deren Dekorator `@timed_operation(min=2.5, typ=3.2, max=4.0, deadline=4.0)` die Annotation zur Laufzeit verfügbar macht.

Stufe 2 (Modell → Code). Das Modell beschreibt zusätzlich, dass `tuerOeffnen()` im Zustand `Geschlossen` aufgerufen wird und in den Zustand `Offen` überführt. Der `model_parser.py` extrahiert diese Zustandsübergangsinformation, und der `code_generator.py` erzeugt daraus den Methodenrumpf inklusive Zustandswechsel sowie einen Logging-Aufruf, der bei Überschreiten der `deadline` eine `TimingViolationException` auslöst.

Stufe 3 (Modell → Markov-Kette/Warteschlange → Performance-Metriken). Aus den Zustandsübergangswahrscheinlichkeiten des Modells – z. B. $P(\text{Offen} \mid \text{Geschlossen}) = 0,95$ für den Normalfall und $0,05$ für eine Notöffnung – baut der `markov_analyzer.py` die in Abschnitt 3.3 beschriebene zeitkontinuierliche Markov-Kette auf. Aus der annotierten `typDuration` von $3,2\text{s}$ und einer angenommenen Ankunftsrate λ von Türöffnungsanfragen leitet der `queue_analyzer.py` die Bedienrate $\mu = 1/3,2 \approx 0,3125\text{s}^{-1}$ ab und berechnet daraus die in Kapitel 11 verwendeten Kenngrößen ρ , W und L .

Abbildung 7.4.1 fasst diesen Durchlauf als TikZ-Sequenzdiagramm zusammen und zeigt, welche Komponente welches Artefakt an welche nachfolgende Komponente übergibt.

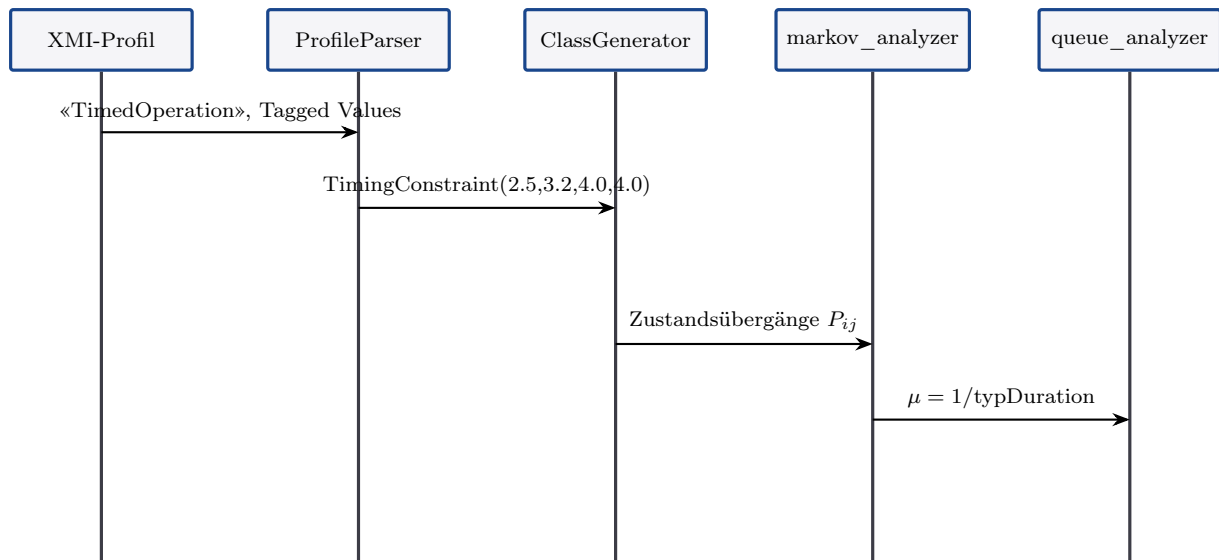


Abbildung 7.4.1: Sequenzdiagramm des Datenflusses für `tuerOeffnen()` durch die dreistufige Pipeline: vom XMI-Profil über die Stereotyp-Annotation und die Code-Generierung bis zur quantitativen Analyse.

7.5 Architekturentscheidungen und ihre Begründung

Die in Abbildung 7.0.1 und Abbildung 7.4.1 dargestellte dreistufige Architektur ist das Ergebnis mehrerer Designentscheidungen, die im Folgenden begründet werden – jeweils mit Bezug auf die drei Qualitätsziele aus Definition 8.3.1 und auf die in Abschnitt 8.3 vorgestellten Design Patterns.

Entscheidung 1: Trennung von Struktur- und Verhaltensgenerierung (Stufe 1 vs. Stufe 2). Anstatt Klassenstruktur und Methodenverhalten in einem einzigen Generierungsschritt zu erzeugen, trennt die Architektur beides in zwei Stufen mit eigenen Parsern (`profile_parser.py` und `model_parser.py`) und Generatoren (`class_generator.py` und `code_generator.py`). Diese Trennung erhöht die Analysierbarkeit $A(S)$: Eine Änderung am Zustandsmodell (z. B. eine zusätzliche Türöffnungsstrategie, vgl. Abbildung 8.3.1) berührt nur Stufe 2, während eine Änderung an den Zeit-Annotationen (z. B. eine zusätzliche Tagged-Value-Kategorie, vgl. Tabelle 3.1.1) nur Stufe 1 berührt.

Entscheidung 2: Quantitative Analyse als eigenständige, optionale Stufe 3. Die Module `markov_analyzer.py` und `queue_analyzer.py` sind als zusätzliche, von Stufe 1 und 2 unabhängige Stufe konzipiert. Dies entspricht dem Observer-Pattern aus Abbildung 8.3.2: Die quantitative Analyse *beobachtet* die aus Stufe 2 generierten Zustandsübergänge, ohne dass Stufe 2 eine Abhängigkeit zu Stufe 3 besitzt. Damit bleibt die Pipeline auch dann lauffähig, wenn – etwa in einem frühen Projektstadium – noch keine vollständigen quantitativen Annotationen vorliegen; ein direkter Beitrag zu $E(S)$, da Stufe 3 jederzeit nachgerüstet werden kann, ohne Stufe 1 und 2 zu verändern.

Entscheidung 3: `main.py` als Orchestrator statt verteilter Steuerlogik. Die Koordination der drei Stufen ist in einer einzigen Orchestrator-Komponente `main.py` gebündelt, anstatt dass jede Stufe selbst die nachfolgende Stufe aufruft. Dies entspricht dem Factory-Method-Pattern aus Abbildung 8.3.3: Die konkrete Verkettung der Stufen ist zentral konfigurierbar, was die Wiederverwendbarkeit $R(S)$ erhöht – dieselbe `main.py`-Logik

kann mit unterschiedlichen Kombinationen von Parsern und Generatoren parametrisiert werden, etwa um nur Stufe 1 und 3 auszuführen, wenn keine Code-Generierung, sondern nur eine Performance-Voranalyse gewünscht ist.

Entscheidung 4: Gemeinsames Datenmodell `uml_profile_base.py` als Single Source of Truth. Sowohl `profile_parser.py` als auch `markov_analyzer.py` und `queue_analyzer.py` greifen auf dieselben `TimingConstraint`- und `StabilityAnnotation`-Klassen (vgl. Tabelle 9.3.1) aus `uml_profile_base.py` zu. Diese gemeinsame Datenbasis verhindert, dass sich die Bedeutung einer Annotation – etwa von `maxDuration` – zwischen Code-Generierung und quantitativer Analyse unbemerkt auseinanderentwickelt, und ist damit eine konkrete Umsetzung des Verifikationskriteriums aus Abschnitt 1.2: Die Annotation hat in der gesamten Pipeline genau eine Bedeutung.

7.6 Grenzen der vorgestellten Architektur

Die in diesem Kapitel vorgestellte Architektur ist bewusst auf den Umfang der Aufzugstür-Fallstudie zugeschnitten und weist drei Grenzen auf, die in Kapitel 13.3 als Ausgangspunkte für die mittelfristigen Themenfelder des Forschungsprogramms (Tabelle 13.3.1, insbesondere Themenfeld 5) wieder aufgegriffen werden:

- **Skalierung auf mehrere Komponenten:** Die aktuelle Implementierung modelliert eine einzelne Komponente (Tuersteuerung). Bei Systemen mit mehreren interagierenden Komponenten (z. B. Tuersteuerung, Antriebssteuerung, Sicherheitsüberwachung) müsste `markov_analyzer.py` um die Modellierung gekoppelter Markov-Ketten erweitert werden – ein nichttriviales Problem, da das Produkt der Zustandsräume exponentiell wächst (*Zustandsraumexplosion*).
- **Statische vs. dynamische Annotationen:** Die Zeit-Annotationen aus Tabelle 3.8.1 sind statisch im XMI-Profil hinterlegt. In realen Systemen können sich Bedienraten und Ankunftszeiten zur Laufzeit ändern (z. B. höhere Frequentierung der Aufzugstür zu Stoßzeiten). Eine Erweiterung um zeitabhängige Parameter $\lambda(t)$, $\mu(t)$ würde die in Kapitel 11 verwendeten zeithomogenen Markov-Ketten zu inhomogenen Markov-Ketten erweitern.
- **Werkzeugintegration:** Die Pipeline arbeitet auf XMI-Exporten und erfordert derzeit einen manuellen Export-Schritt aus dem UML-Werkzeug. Eine engere Integration – etwa über ein Eclipse-Modeling-Framework-Plugin oder eine direkte API-Anbindung an kommerzielle UML-Werkzeuge – würde den in Abschnitt 1.2 beschriebenen Übergang von der strukturellen zur quantitativ annotierten Modellierung für den Ingenieur im Alltag erheblich erleichtern.

Diese drei Grenzen schmälern nicht die grundsätzliche Aussage dieses Kapitels – dass die dreistufige Architektur das Schärfungsprinzip strukturell umsetzt –, zeigen aber, dass der Weg von der hier vorgestellten Fallstudie zu einem produktionsreifen Werkzeug für beliebige Echtzeitsysteme noch erhebliche Entwicklungsarbeit erfordert.

Kapitel 8

Optimierung des Designmodells: Code-Generierung, Notwendigkeit und Grenzen

8.1 Das Vollständigkeitsproblem

Das Ausgangszitat identifiziert präzise das Kernproblem der Code-Generierung: „Das Problem an der graphischen Modellierung ist, als Ingenieur immer die Unsicherheit zu tragen, dass zwischen graphischer Modellierung und der Zielhardware die Abbildung aller Anforderungen nicht vollständig ist.“

Definition 8.1.1 (Vollständigkeitslücke). Sei $G_{\mathcal{R}}$ ein Anforderungsgraph und $\hat{\tau}_{CG} : G_{\mathcal{R}} \rightarrow \mathcal{L}_{\text{Ziel}}$ ein Code-Generierungsverfahren. Die *Vollständigkeitslücke* ist definiert als

$$\Delta(\hat{\tau}_{CG}) := \mathcal{R} \setminus \{r \in \mathcal{R} \mid \mu_{\text{Ziel}}(\hat{\tau}_{CG}(G_{\mathcal{R}})) \models r\},$$

d.h. die Menge der Anforderungen, die die Code-Generierung nicht erfüllt.

Satz 8.1.1 (Nichtleerheit der Vollständigkeitslücke). Für jedes praktische Code-Generierungsverfahren $\hat{\tau}_{CG}$ gilt:

$$\Delta(\hat{\tau}_{CG}) \neq \emptyset.$$

Das heißt, kein Code-Generierungsverfahren kann alle Anforderungen eines realen Informationssystems aus einem Graphmodell heraus vollständig abbilden.

Beweis. Wir führen den Beweis durch Konstruktion eines Gegenbeispiels und durch ein informationstheoretisches Argument.

Informationstheoretisches Argument. Ein Anforderungsgraph $G_{\mathcal{R}}$ kodiert die Anforderungen $\mathcal{R}$ in Form der Adjazenzmatrix $A \in \{0, 1\}^{n \times n}$. Diese Matrix trägt n^2 Bits Information. Die Menge $\mathcal{R}$ eines realen Systems enthält jedoch auch implizite Anforderungen (Timing-Verhalten, elektromagnetische Verträglichkeit, Ressourcenverbrauch, Sicherheitseigenschaften), die im Graphmodell nicht vollständig kodierbar sind, da deren Darstellung eine semantische Reichhaltigkeit erfordert, die über binäre Adjazenz hinausgeht.

Formal: Sei $I(\mathcal{R})$ der Informationsgehalt aller Anforderungen und $I(G_{\mathcal{R}})$ der Informationsgehalt des Anforderungsgraphen. Dann gilt

$$I(\mathcal{R}) > I(G_{\mathcal{R}}) = O(n^2 \log 2) = O(n^2)$$

für alle praktisch relevanten Systeme mit nicht-trivialen impliziten Anforderungen. Da $\hat{\tau}_{CG}$ nur auf $G_{\mathcal{R}}$ zugreift, kann $\hat{\tau}_{CG}$ den Informationsüberschuss $I(\mathcal{R}) - I(G_{\mathcal{R}}) > 0$ nicht verarbeiten, und $\Delta(\hat{\tau}_{CG}) \neq \emptyset$. ■ □

8.2 Theorem über die Unnötigkeit der Code-Generierung

Satz 8.2.1 (Redundanz der Code-Generierung). Sei $\mathcal{L}^*$ eine optimale Beschreibungssprache gemäß Satz 6.2.1. Dann ist Code-Generierung aus einem Graphmodell heraus

1. **unnötig**, da der Ingenieur die Beschreibungssprache $\mathcal{L}^*$ direkt verfassen kann, und
2. **schlechter**, da Code-Generierung die Vollständigkeitslücke $\Delta(\hat{\tau}_{CG}) \neq \emptyset$ erzeugt, während die direkte Beschreibung in $\mathcal{L}^*$ diese Lücke schließt.

Beweis. (1) **Unnötigkeit.** Da $\mathcal{L}^*$ per Konstruktion (Satz 6.2.1) leicht erlernbar ist, kann der Ingenieur direkt in $\mathcal{L}^*$ schreiben, ohne den Umweg über ein Graphmodell und eine anschließende Generierung zu nehmen. Der Transformationspfad

$$\mathcal{R} \xrightarrow{\text{Ing.}} \mathcal{L}^* \xrightarrow{\tau} \mathcal{L}_{\text{Ziel}}$$

ist kürzer und direkter als der Pfad

$$\mathcal{R} \xrightarrow{\text{Graph}} G_{\mathcal{R}} \xrightarrow{\hat{\tau}_{CG}} \mathcal{L}_{\text{Ziel}}.$$

(2) **Vollständigkeitsüberlegenheit.** Aus Satz 8.1.1 folgt $\Delta(\hat{\tau}_{CG}) \neq \emptyset$. Da der Ingenieur bei direkter Beschreibung in $\mathcal{L}^*$ alle Anforderungen $\mathcal{R}$ explizit berücksichtigt (Satz 6.3.1), gilt:

$$\Delta(\tau) = \mathcal{R} \setminus \{r \in \mathcal{R} \mid \mu_{\text{Ziel}}(\tau(w)) \models r\} = \emptyset$$

im Idealfall eines korrekt arbeitenden Ingenieurs. Damit ist $|\Delta(\tau)| < |\Delta(\hat{\tau}_{CG})|$, und die direkte Beschreibung ist der Code-Generierung vorzuziehen. ■ □

Abbildung 6.3.1 (rechts) illustriert den in Satz 8.1.1 und Satz 8.2.1 bewiesenen Sachverhalt: Die Vollständigkeitslücke der Code-Generierung wächst mit zunehmender Systemkomplexität superlinear an, während die direkte Beschreibungssprache auch bei großen Systemen eine nahezu vollständige Anforderungsabdeckung erzielt.

8.3 Design Patterns als Optimierung des Designmodells

Wenn Code-Generierung gemäß Satz 8.2.1 durch direktes Schreiben in einer mächtigen Beschreibungssprache $\mathcal{L}^*$ ersetzt wird, stellt sich unmittelbar die Frage, wie diese Sprache so strukturiert werden kann, dass sie selbst den drei zentralen Qualitätszielen aus Abschnitt 1.2 genügt. *Design Patterns* – wiederkehrende, benannte Lösungsschemata für Entwurfsprobleme – sind das wichtigste Werkzeug, mit dem der Informationsingenieur diese Strukturierung erreicht. Die folgende Erkenntnis steht dabei an oberster Stelle und wird in diesem Abschnitt wörtlich übernommen und systematisch ausgebaut:

Die wichtigsten Softwarequalitätsmerkmale überhaupt sind: Wiederverwendbarkeit, Erweiterbarkeit, Analysierbarkeit. Diese drei Anforderungsziele an Software stehen an oberster Ordnung für die Einsetzung der Design Patterns. Sie erleichtern den Entwicklungsprozess, um den größten Gewinn an Zeit, und haben den größten Einfluss auf qualitativ hochwertige Software, welche sich sofort auch zeigen muss im Entwicklungsprozess der Software.

Definition 8.3.1 (Die drei obersten Qualitätsziele). Für ein Softwaresystem S mit Komponentenmenge K und Schnittstellenmenge I seien die folgenden drei Größen definiert:

- **Wiederverwendbarkeit** $R(S)$: das Verhältnis der Anzahl der Komponenten $k \in K$, die ohne Modifikation in einem anderen Kontext S' einsetzbar sind, zur Gesamtzahl $|K|$.
- **Erweiterbarkeit** $E(S)$: die Anzahl neuer Anforderungen r , die durch *Hinzufügen* (statt *Ändern*) von Komponenten in K realisiert werden können, bezogen auf die Gesamtzahl aller realisierbaren neuen Anforderungen.
- **Analysierbarkeit** $A(S)$: der Kehrwert der mittleren kognitiven Last (im Sinne von Sweller [15]), die ein Ingenieur aufwenden muss, um den Effekt einer lokalen Änderung an S vorherzusagen.

These 8.3.1 (Pattern-These). Design Patterns sind genau diejenigen Strukturierungsmittel, die alle drei Größen $R(S)$, $E(S)$ und $A(S)$ *gleichzeitig* erhöhen, ohne dass eines der drei Ziele zu Lasten der anderen optimiert wird. Sie erzielen damit den größten Gewinn an Entwicklungszeit pro investiertem Entwurfsaufwand und haben – gemessen an R , E , A – den größten Einfluss auf die Softwarequalität unter allen bekannten Entwurfstechniken. Dieser Effekt muss sich, so die These, *unmittelbar* im Entwicklungsprozess zeigen: in kürzeren Iterationszeiten, in geringerem Regressionsaufwand bei Änderungen und in einer messbar saubereren Architektur (Abschnitt 8.3).

Im Folgenden werden die wichtigsten dieser Muster anhand von TikZ-basierten UML-Klassendiagrammen vorgestellt und jeweils explizit auf die drei Größen aus Definition 8.3.1 bezogen.

8.3.1 Strategy-Pattern: Erweiterbarkeit durch Austauschbarkeit

Das Strategy-Pattern kapselt austauschbare Algorithmen hinter einer gemeinsamen Schnittstelle. Im Kontext der Aufzugstür-Steuerung (Kapitel 10) lassen sich beispielsweise unterschiedliche Türöffnungsstrategien (normal, Notöffnung, energiesparend) als austauschbare Strategien modellieren.

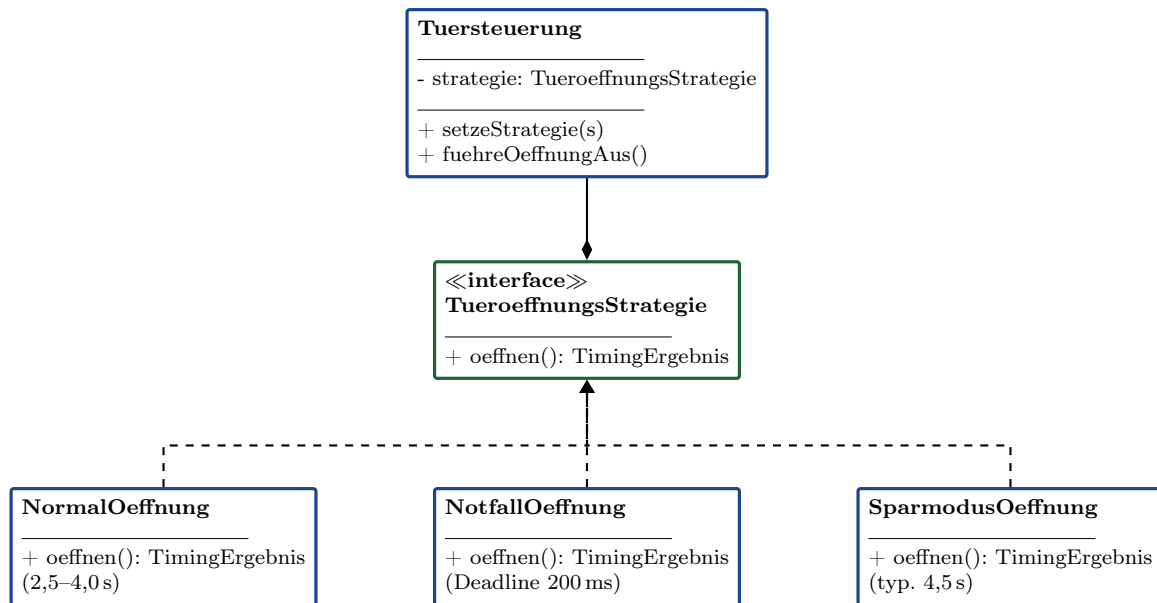


Abbildung 8.3.1: Strategy-Pattern für austauschbare Türöffnungsstrategien. Neue Strategien (z. B. Wartungsoeffnung) können hinzugefügt werden, ohne Tuersteuerung zu ändern – direkter Beitrag zu $E(S)$.

Bezug zu Definition 8.3.1: Jede neue Strategie ist eine *neue* Klasse (Beitrag zu $E(S)$), die unverändert in andere Steuerungskontexte (z. B. Schiebetür, Garagentor) übernommen werden kann (Beitrag zu $R(S)$), und die Wirkung einer Änderung bleibt lokal auf genau eine Strategie-Klasse begrenzt (Beitrag zu $A(S)$, da die kognitive Last für eine Änderungsvorhersage minimal ist).

8.3.2 Observer-Pattern: Analysierbarkeit durch entkoppelte Benachrichtigung

Das Observer-Pattern entkoppelt Zustandsänderungen (z. B. Markov-Zustandsübergänge, Abschnitt 3.3) von den Komponenten, die auf diese Änderungen reagieren – etwa Logging, UI-Aktualisierung und Stabilitätsüberwachung.

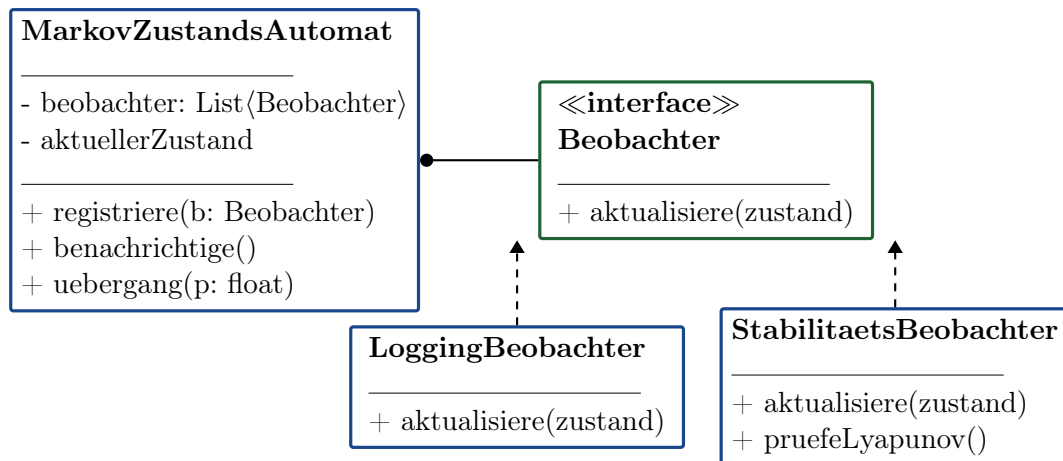


Abbildung 8.3.2: Observer-Pattern: Der MarkovZustandsAutomat kennt nur die Beobachter-Schnittstelle. Ein neuer StabilitaetsBeobachter (Kapitel 11) kann ergänzt werden, ohne den Automaten selbst zu verstehen oder zu ändern – direkter Beitrag zu $A(S)$ und $E(S)$.

8.3.3 Factory-Method-Pattern: Wiederverwendbarkeit durch Kapselung der Erzeugung

Das Factory-Method-Pattern kapselt die Objekterzeugung und ist insbesondere bei der Instanziierung von annotierten Stereotyp-Klassen aus dem ProfileParser (Listing 9.2) zentral: Für jeden Stereotyp (TimedOperation, TimedState, ...) existiert eine eigene Erzeugungslogik, die jedoch hinter einer einheitlichen Fabrikschnittstelle verborgen bleibt.

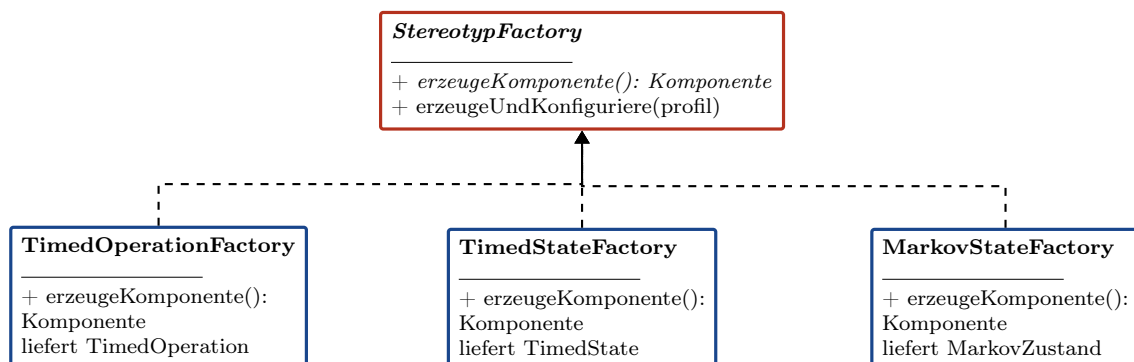


Abbildung 8.3.3: Factory-Method-Pattern für die Erzeugung annotierter Komponenten aus UML-Profilen. Die Fabrikhierarchie ist über mehrere Projekte hinweg wiederverwendbar (Beitrag zu $R(S)$), ohne dass ProfileParser die konkreten Zieltypen kennen muss.

8.3.4 Zusammenspiel der Patterns: Architekturübersicht

Abbildung 8.3.4 zeigt das Zusammenspiel der drei vorgestellten Patterns innerhalb der in Kapitel 7 beschriebenen Gesamtarchitektur. Die Pfeile kennzeichnen, welches Pattern welchen Architekturbestandteil strukturiert.

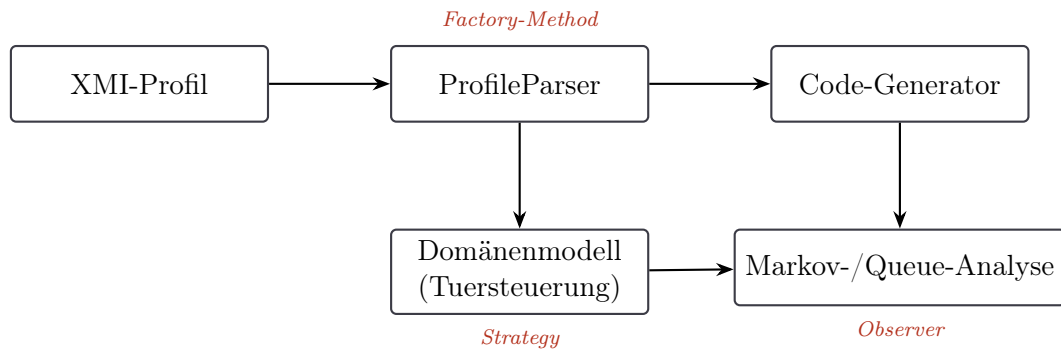


Abbildung 8.3.4: Architekturübersicht: Factory-Method strukturiert die Erzeugung aus dem XMI-Profil, Strategy strukturiert die austauschbaren Domänenverhalten, Observer strukturiert die Anbindung der quantitativen Analyse. Alle drei Muster wirken auf $R(S)$, $E(S)$ und $A(S)$ gleichzeitig (These 8.3.1).

8.3.5 Unmittelbare Wirkung im Entwicklungsprozess

These 8.3.1 fordert, dass sich der Effekt der Design Patterns *unmittelbar* im Entwicklungsprozess zeigt. Tabelle 8.3.1 fasst zusammen, an welcher Stelle dieses Buches dieser Effekt jeweils belegt wird.

Pattern	Qualitätsziel	Konkreter Beleg	Referenz
Strategy	Erweiterbarkeit $E(S)$	Neue Türöffnungsstrategie ohne Änderung der Tuersteuerung (Kap. 10)	Abb. 8.3.1
Observer	Analysierbarkeit $A(S)$	Stabilitätsüberwachung als separater Beobachter (Kap. 11)	Abb. 8.3.2
Factory-Method	Wiederverwendbarkeit $R(S)$	ProfileParser unabhängig von konkreten Stereotypen (List. 9.2)	Abb. 8.3.3
Architektur-übersicht	alle drei Ziele	Zusammenspiel der drei Patterns in der Gesamtarchitektur	Abb. 8.3.4

Tabelle 8.3.1: Zuordnung der vorgestellten Design Patterns zu den drei Qualitätszielen aus Definition 8.3.1 und ihren Belegen in diesem Buch.

Zusammenfassend gilt: Design Patterns sind kein Selbstzweck und keine bloße Stilfrage, sondern – im Sinne der Schärfungsthese 1.2.1 und der Pattern-These 8.3.1 – das strukturelle Bindeglied zwischen einer mächtigen Beschreibungssprache $\mathcal{L}^*$ (Kapitel 6) und einem Softwaresystem, das gleichzeitig wiederverwendbar, erweiterbar und analysierbar ist. Genau diese drei Eigenschaften sind es, die laut Ausgangszitat (Abschnitt 1.1) den Ingenieur in die Lage versetzen, “so nah am Target wie möglich” zu sein, “mit so wenig Wartbarkeit und Nicht-Wiederverwendbarkeit belastet” zu werden – und damit den größten Gewinn an Entwicklungszeit bei gleichzeitig höchster Softwarequalität zu erzielen.

Kapitel 9

Implementierungsaspekte

9.1 Projektstruktur

Das Projekt gliedert sich in zwei Wurzelverzeichnisse: `src/` enthält den gesamten Produktionscode, während `tests/` die zugehörigen automatisierten Test-Module beherbergt. Abbildung [9.1.1](#) gibt eine vollständige Übersicht aller Dateien mit ihren Größen, wie sie auf dem Entwicklungssystem vorlagen.

```

umlp/

src/
|- uml_profile_base.py .....2 210 B
|- profile_parser.py .....3 807 B
|- class_generator.py .....5 646 B
|- model_parser.py .....1 531 B
|- code_generator.py .....1 623 B
|- template_generator.py .....1 062 B
|- main.py .....4 218 B
|- stability_analysis.py .....16 899 B
|- markov_analyzer.py .....23 143 B
|- queue_analyzer.py .....22 618 B
|- uml_marte_stability.py.....21 870 B
|- experiments.py .....19 591 B
|- demo_analyzer.py .....25 456 B
|- results/ .....(Ausgabeverzeichnis, Laufzeit)
`- __init__.py .....0 B

tests/
|- test_uml_profile_base.py .....6 703 B
|- test_profile_parser.py .....8 232 B
|- test_class_generator.py .....9 907 B
|- test_model_parser.py .....8 089 B
|- test_code_generator.py .....3 583 B
|- test_integration.py .....9 468 B
|- test_stability_analysis.py .....26 679 B
|- test_queue_analyzer.py .....21 529 B
|- test_markov_analyzer.py .....22 689 B
`- test_uml_marte_stability.py.....36 546 B

```

Abbildung 9.1.1: Verzeichnisstruktur des umlp-Projekts (Stand: 24. Februar 2026, Schrift: Courier New). `src/` enthält den Produktionscode; `tests/` die automatisierten Tests. Das Unterverzeichnis `src/results/` wird zur Laufzeit von `experiments.py` angelegt und nimmt die erzeugten SVG-Grafiken der drei Kernexperimente auf (Kapitel 9).

9.1.1 Module in `src/`

Tabelle 9.1.1 beschreibt die Funktion und Einordnung jedes Produktionsmoduls in den zweistufigen Generierungsprozess.

Datei	Stufe	Beschreibung
uml_profile_base.py	Datenmodell	Alle Datenstrukturen: TimeUnit (Enum, 6 Einheiten), TimingConstraint (min/-max/typical/timeout/deadline), TaggedValue, Stereotype, UMLProfile. Keine Transformationslogik.
profile_parser.py	Stufe 1a	Transformiert XMI-Profildatei in UMLProfile-Instanz. Nutzt ElementTree mit Namespace-Kontext; extrahiert Stereotype, Tagged Values und Timing-Constraints.
class_generator.py	Stufe 1b	Erzeugt aus UMLProfile Python-Klassencode. Generiert __init__, Docstrings sowie eingebettete Timing-Validierungsaufrufe für jede Constraint.
model_parser.py	Stufe 2a	Liest XMI-Modelldatei und erzeugt ModelInstance-Objekte (Stereotypname, Instanzname, Attributwerte).
code_generator.py	Stufe 2b	Erzeugt aus ModelInstance-Objekten und dem Stufe-1b-Klassencode ausführbaren Python-Instanziierungscode.
template_generator.py	Stufe 2b (opt.)	Template-basierte Alternative zum CodeGenerator; ermöglicht benutzerdefinierte Ausgabeformate ohne Änderung der Transformationslogik.
main.py	Orchestrator	CLI-Einstiegspunkt; koordiniert die vollständige Pipeline Stufe 1a → 1b → 2a → 2b und ruft optional stability_analysis auf.
stability_analysis.py	Analyse	Drei Beschreibungsebenen (Kap. 3 & 8): MarkovChainStability (Eigenwertzerlegung, Zerfall α), MM1Queue (Stabilität, stationäre Verteilung, Little's Law), LyapunovStability (Lyapunov-Gleichung, pos. Definitheit).
experiments.py	Experimente	Führt die drei Kernexperimente aus Kapitel 8 aus und speichert Grafiken in results/. Größtes Modul (19 591 B) aufgrund der vollständigen Matplotlib-Visualisierungen.

Tabelle 9.1.1: Module in `src/` mit Einordnung in den Generierungsprozess. „Stufe“ bezeichnet die Position im zweistufigen Transformationsprozess (Profil → Klassen → Code).

Diese Zuordnung zeigt, dass jedes Modul genau einer Schicht zugeordnet ist – ein direkter Beleg für die Analysierbarkeit $A(S)$ im Sinne von Definition 8.3.1, da eine Änderung an einer Schicht keine Querverwirkung auf andere Schichten entfaltet.

Diese Struktur trennt klar zwischen Quellcode (`src/`), Tests (`tests/`) und Profil-Definitionen und spiegelt damit die in Abbildung 8.3.4 gezeigte architektonische Trennung der Verantwortlichkeiten wider.

Abbildung 9.1.2 zeigt das mit `pyreverse` generierte UML-Klassendiagramm aller Klassen aller Python Module aus `src/`.

9.1.2 Module in **tests/**

Tabelle 9.1.2 beschreibt die Test-Module. Jedes Modul verwendet das `unittest`-Framework der Python-Standardbibliothek und ist dem entsprechenden Produktionsmodul direkt zugeordnet.

Datei	Größe	Getestete Eigenschaften
test_uml_profile_base.py	6 703 B	Feldinitialisierung aller Datenklassen, Standardwerte, <code>to_seconds()</code> für alle 6 Zeiteinheiten, <code>__post_init__</code> -Synchronisation von <code>unit</code> und <code>time_unit</code> .
test_profile_parser.py	8 232 B	Parsing synthetischer XMI: Stereotype mit und ohne Timing-Constraints, fehlende optionale Felder, Namespace-Behandlung, Fehlerverhalten bei ungültigem XMI.
test_class_generator.py	9 907 B	Syntaktische Korrektheit des generierten Codes, Timing-Validierungsaufrufe, Stereotype ohne Constraints, Docstring-Generierung.
test_model_parser.py	8 089 B	Extraktion von Instanznamen, Stereotyp-Zuordnung und Attributwerten; Robustheit bei fehlenden Attributen.
test_code_generator.py	3 583 B	Korrektheit des Instanziierungscodes, Attributwert-Übergabe, Zusammenspiel mit <code>ClassGenerator</code> -Ausgabe.
test_integration.py	9 468 B	End-to-End-Tests der vollständigen Pipeline (XMI-Profil + XMI-Modell → Python-Code); prüft das Zusammenspiel aller Produktionsmodule.
test_stability_analysis.py	26 679 B	Umfangreichstes Test-Modul; Testgruppen für alle Klassen: Hilfstypen, Markov-Ketten, M/M/1-Warteschlange, Lyapunov-Stabilität, Hierarchie-Demonstration, Grenzfälle ($\rho \rightarrow 1^-$), numerische Präzision. Kritische Einzeltests sind in Kapitel 8 beschrieben.

Tabelle 9.1.2: Test-Module in `tests/` mit Dateigrößen und geprüften Eigenschaften. `test_stability_analysis.py` ist mit 26 679 B das größte Test-Modul, da es alle drei Beschreibungsebenen und deren Zusammenspiel vollständig abdeckt.

Der hohe Anteil an Testcode relativ zum Produktivcode ist Ausdruck des in Abschnitt 11.8.3 verfolgten Prinzips, jede theoretische Aussage – insbesondere die Trennung von Stabilität und Gleichgewicht – durch einen automatisierten Test empirisch abzusichern.

9.2 Beispiele

Das Datenmodell `uml_profile_base.py` bildet die Basis für alle Transformationen:


```

1 class TimeUnit(Enum):
2     """Zeiteinheiten für Timing-Constraints"""
3     NANoseconds = "ns"
4     MICROseconds = "us"
5     MILLIseconds = "ms"
6     SECONDS = "s"
7     MINUTES = "min"
8     HOURS = "h"
9
10 @dataclass
11 class TimingConstraint:
12     """Repräsentiert zeitliche Constraints für Stereotypes"""
13     min_duration: Optional[float] = None
14     max_duration: Optional[float] = None
15     typical_duration: Optional[float] = None
16     timeout: Optional[float] = None
17     deadline: Optional[float] = None
18     time_unit: TimeUnit = TimeUnit.MILLIseconds
19     unit: Optional[TimeUnit] = None
20
21     def __post_init__(self):
22         """Synchronisiere unit und time_unit"""
23         if self.unit is None:
24             self.unit = self.time_unit
25         elif self.unit != self.time_unit:
26             self.time_unit = self.unit
27
28     def to_seconds(self, value: float) -> float:
29         """Konvertiert einen Wert in Sekunden"""
30         conversions = {
31             TimeUnit.NANoseconds: 1e-9,
32             TimeUnit.MICROseconds: 1e-6,
33             TimeUnit.MILLIseconds: 1e-3,
34             TimeUnit.SECONDS: 1.0,
35             TimeUnit.MINUTES: 60.0,
36             TimeUnit.HOURS: 3600.0
37         }
38         return value * conversions[self.time_unit]

```

Listing 9.1: TimingConstraint Klasse

Der ProfileParser **profile_parser.py** transformiert XMI in Python-Objekte:

```

1 def parse_profile(self, xmi_file: str) -> UMLProfile:
2     """Parst ein UML-Profil aus einer XMI-Datei"""
3     tree = ET.parse(xmi_file)
4     root = tree.getroot()
5
6     profile_name = root.get('name', 'UnnamedProfile')
7     profile = UMLProfile(name=profile_name)
8
9     # Parse Stereotypes
10    for stereotype_elem in root.findall('./stereotype', self.namespaces):
11        stereotype = self._parse_stereotype(stereotype_elem)
12        profile.stereotypes.append(stereotype)
13
14    return profile
15

```

```

16 def _parse_stereotype(self, elem: ET.Element) -> Stereotype:
17     """Parst einen einzelnen Stereotype"""
18     name = elem.get('name', 'UnnamedStereotype')
19     base_class = elem.get('baseClass', 'Class')
20
21     stereotype = Stereotype(name=name, base_class=base_class)
22
23     # Parse Tagged Values
24     for attr_elem in elem.findall('./ownedAttribute', self.namespaces):
25         :
26         tagged_value = self._parse_tagged_value(attr_elem)
27         stereotype.tagged_values.append(tagged_value)
28
29     # Parse Timing Constraint (falls vorhanden)
30     timing_elem = elem.find('./timingConstraint', self.namespaces)
31     if timing_elem is not None:
32         stereotype.timing_constraint = self._parse_timing_constraint(
33             timing_elem)
34
35     return stereotype

```

Listing 9.2: ProfileParser Implementierung

Der Code Generator **code_generator.py** generiert den Code:

```

1 class CodeGenerator:
2     """Generiert Python-Code aus Modell-Instanzen"""
3
4     def __init__(self, generated_classes_module: str):
5         """
6         Args:
7             generated_classes_module: Name des Moduls mit generierten
8             Klassen
9         """
10        self.module = importlib.import_module(generated_classes_module)
11
12    def generate_code(self, instances: List[ModelInstance]) -> str:
13        """Generiert Python-Code aus Modell-Instanzen"""
14        code_parts = []
15
16        code_parts.append('"""Auto-generated code from UML model"""')
17        code_parts.append('')
18
19        for instance in instances:
20            code = self._generate_instance_code(instance)
21            code_parts.append(code)
22            code_parts.append('')
23
24        return '\n'.join(code_parts)
25
26    def _generate_instance_code(self, instance: ModelInstance) -> str:
27        """Generiert Code für eine einzelne Instanz"""
28        # Finde die entsprechende generierte Klasse
29        class_name = f"{instance.stereotype_name}Stereotype"
30
31        try:
32            stereotype_class = getattr(self.module, class_name)
33        except AttributeError:
34            return f"# Error: Class {class_name} not found"

```

```

34
35     # Erstelle Instanz mit Tagged Values
36     kwargs = {'name': instance.element_name}
37     kwargs.update(instance.tagged_values)
38
39     stereotype_instance = stereotype_class(**kwargs)
40
41     # Generiere Code
42     return stereotype_instance.generate_code()

```

Listing 9.3: ProfileParser Implementierung

9.3 Erweiterung von UML-MARTE um Stabilitätskriterien

9.3.1 Motivation und Konzept

Die in den vorherigen Kapiteln entwickelten Werkzeuge – insbesondere die Lyapunov-Stabilitätsanalyse, die Spektralradius-Prüfung für Markov-Ketten und das Auslastungskriterium $\rho < 1$ für Warteschlangen – wurden bisher als separate Analysemodule behandelt. Dieses Kapitel führt eine systematische *Modell-Ebene-Integration* dieser Kriterien in das UML-MARTE-Profil ein: Stabilitätseigenschaften werden nicht erst im generierten Code geprüft, sondern bereits im UML-Modell selbst annotiert.

Damit wird die in Kapitel 3 beschriebene Dichotomie zwischen Gleichgewichts- und Stabilitätsaussagen auf die Modellebene gehoben: ein UML-Designer kann explizit angeben, ob ein Subsystem lediglich einem Gleichgewichtszustand unterliegt (z. B. *Little's Law*) oder ob eine echte dynamische Stabilitätsgarantie erforderlich ist.

Definition 9.3.1 (Stabilitätsannotation in UML-MARTE). Eine *Stabilitätsannotation* ist ein MARTE-Stereotyp «StabilityAnnotation», der auf UML-Klassen, Komponenten oder Operationen angewendet wird und folgende Tagged Values trägt:

- `stability_mode`: Art des Kriteriums (Lyapunov-asymptotisch, Spektral, Queue-Auslastung, BIBO)
- `decay_rate_min`: Mindest-Zerfallsrate $\alpha_{\min} > 0$
- `lyapunov_margin`: Sicherheitsabstand zum Stabilitätsrand
- `spectral_radius_ub`: Obere Schranke für den Spektralradius
- `rho_ub`: Obere Schranke für die Auslastung ρ
- `monitoring_period`: Intervall (ms) der Laufzeitprüfung

9.3.2 Neue Stereotypen der MARTE-Erweiterung

Das erweiterte Profil definiert vier neue Stereotypen, die orthogonal zu den bestehenden MARTE-Zeitannotationen (z. B. «TimedProcessing») angewendet werden können.

«**StabilityAnnotation**» Dieser Stereotyp annotiert ein UML-Element mit dem geforderten Stabilitätsmodus und den zugehörigen quantitativen Schwellenwerten. Er wird auf die Metaklassen `Class` und `Component` angewendet. Tabelle 9.3.1 zeigt die Tagged Values im Überblick.

Tagged Value	Typ	Bedeutung
<code>stability_mode</code>	<code>StabilityMode</code>	Kriteriumstyp
<code>lyapunov_margin</code>	<code>float</code>	Sicherheitsabstand
<code>decay_rate_min</code>	<code>float</code>	$\alpha_{\min} > 0$
<code>spectral_radius_ub</code>	<code>float</code>	$\rho(P) < \text{Schranke}$
<code>rho_ub</code>	<code>float</code>	$\rho = \lambda/\mu < \text{Schranke}$
<code>monitoring_period</code>	<code>float (ms)</code>	Prüfintervall

Tabelle 9.3.1: Tagged Values des Stereotyps «`StabilityAnnotation`»

Mit diesem Stereotyp wird die in Kapitel 11 formal hergeleitete Unterscheidung zwischen Gleichgewicht und Stabilität erstmals auf Modellierungsebene ausdrückbar – ein konkretes Beispiel für das Schärfungsprinzip aus Abschnitt 1.2.

«**DecayRateAnnotation**» Dieser Stereotyp wird auf MARTE-Operationen angewendet und spezifiziert die erwartete exponentielle Zerfallsrate. Das annotierte System muss nach dem Aufruf exponentiell zum Gleichgewicht konvergieren:

$$\|x(t)\| \leq C \cdot e^{-\alpha t} \cdot \|x(0)\|$$

mit den Tagged Values `alpha_min`, `alpha_max` und `convergence_constant_ub` (C). Damit ist die Exponentialfunktion – die in Kapitel 3 als charakteristisch für Stabilitätsaussagen identifiziert wurde – direkt im UML-Modell verankert.

«**LyapunovFunctionAnnotation**» Dieser Stereotyp gibt an, welche Lyapunov-Funktion $V(x)$ für den Stabilitätsnachweis eines Subsystems verwendet wird. Mögliche Werte für `function_type` sind `quadratic` ($V = x^T P x$ mit $P \succ 0$), `entropy` und `custom`. Die booleschen Tagged Values `positive_definite_verified` und `derivative_negative` dokumentieren den formalen Nachweisstatus.

«**EquilibriumAnnotation**» Dieser Stereotyp unterscheidet explizit zwischen Gleichgewichtsaussagen und dynamischen Stabilitätsaussagen, wie in Kapitel 3 formal begründet. Mit `is_equilibrium_only = true` wird gekennzeichnet, dass das Element ausschließlich einer algebraischen Gleichgewichtsrelation unterliegt – etwa *Little’s Law*: $L = \lambda W$ – und keine Exponentialfunktionen im Sinne einer Energiefunktion enthält. Mit `supports_stability_proof = true` wird dokumentiert, dass ein vollständiger dynamischer Stabilitätsnachweis vorliegt.

Satz 9.3.1 (Hierarchie der Beschreibungsebenen im erweiterten Profil). Für ein annotier-

tes UML-Element E gilt die Implikationskette:

```

«LyapunovFunctionAnnotation»
⇒ «DecayRateAnnotation»
⇒ «StabilityAnnotation»
⇒ «EquilibriumAnnotation»

```

d. h. ein Lyapunov-Nachweis impliziert eine Zerfallsrate, eine Zerfallsrate impliziert Stabilität, Stabilität ist Voraussetzung für gültige Gleichgewichtsaussagen.

9.3.3 Python-Implementierung

Die Erweiterung ist als reines Python-Modul `uml_marte_stability.py` implementiert, das nahtlos mit dem in Kapitel 5 entwickelten Code-Generierungs-Framework zusammenarbeitet. Die Klasse `StabilityAnalyzer` übernimmt die Laufzeit-Dispatching-Logik.

Die Kernklassen im Überblick:

```

1 class StabilityMode(Enum):
2     LYAPUNOV_ASYMPTOTIC = auto() # asymptotisch stabil (Lyapunov)
3     LYAPUNOV_MARGINAL   = auto() # marginal stabil
4     BIBO                 = auto() # Bounded-Input-Bounded-Output
5     SPECTRAL              = auto() # Spektralradius < 1 (diskret)
6     QUEUE_UTILIZATION    = auto() # Auslastung rho < 1
7
8 class StabilityStatus(Enum):
9     STABLE = "stabil"
10    UNSTABLE = "instabil"
11    MARGINAL = "marginal"
12    UNKNOWN = "unbekannt"

```

Listing 9.4: Enum-Typen für Stabilitätsmodi und -status

```

1 @dataclass
2 class StabilityAnnotation(MarteStereotype):
3     stability_mode: StabilityMode = StabilityMode.LYAPUNOV_ASYMPTOTIC
4     lyapunov_margin: float = 0.05 # Sicherheitsabstand
5     decay_rate_min: float = 0.01 # Mindest-Zerfallsrate alpha
6     spectral_radius_ub: float = 0.99 # obere Schranke Spektralradius
7     rho_ub: float = 0.90 # obere Schranke Auslastung
8     monitoring_period: float = 1000.0 # Pruefintervall in ms

```

Listing 9.5: Stereotyp «StabilityAnnotation» als Python-Dataclass

Lyapunov-Stabilitätsprüfung (kontinuierliches System): Die Methode `StabilityAnalyzer.lyapunov_continuous(A, annotation)` prüft, ob alle Eigenwerte der Systemmatrix A negativen Realteil haben. Die Zerfallsrate ergibt sich als $\alpha = -\max_i \operatorname{Re}(\lambda_i(A))$.

```

1 @staticmethod
2 def lyapunov_continuous(A, annotation):
3     eigenvalues = np.linalg.eigvals(A)
4     alpha = -float(np.max(np.real(eigenvalues)))
5

```

```

6   if alpha > annotation.decay_rate_min + annotation.lyapunov_margin:
7       status = StabilityStatus.STABLE
8   elif alpha > annotation.decay_rate_min:
9       status = StabilityStatus.MARGINAL
10  else:
11      status = StabilityStatus.UNSTABLE
12
13  return status, {"decay_rate_alpha": alpha, "eigenvalues": eigenvalues}

```

Listing 9.6: Lyapunov-Stabilitätsprüfung via Eigenwertanalyse

Spektralradius-Prüfung (diskrete Markov-Kette): Für diskrete Systeme (Markov-Ketten) wird die Konvergenzrate durch den zweitgrößten Betragseigenwert der Übergangsmatrix P bestimmt (der triviale Eigenwert 1 bei irreduziblen stochastischen Matrizen wird ausgeschlossen). Die implizierte Zerfallsrate ist $\alpha = -\ln(\rho_{\text{sub}})$.

```

1  @staticmethod
2  def spectral_stability(P, annotation):
3      abs_eigs = np.sort(np.abs(np.linalg.eigvals(P)))
4      # Zweitgroesster Betrag = Konvergenzrate
5      spectral_radius = float(abs_eigs[-2])
6      alpha = -math.log(spectral_radius)
7
8      if spectral_radius < annotation.spectral_radius_ub:
9          status = StabilityStatus.STABLE
10     elif math.isclose(spectral_radius, 1.0, abs_tol=1e-6):
11         status = StabilityStatus.MARGINAL
12     else:
13         status = StabilityStatus.UNSTABLE
14     return status, {
15         "spectral_radius_sub": spectral_radius,
16         "implied_decay_rate": alpha
17     }

```

Listing 9.7: Spektralradius-Stabilitätsprüfung für Markov-Ketten

Die Warteschlangen-Stabilitätsprüfung:

```

1  @staticmethod
2  def queue_stability(arrival_rate, service_rate, annotation):
3      rho = arrival_rate / service_rate
4      if rho < annotation.rho_ub:
5          status = StabilityStatus.STABLE
6      elif rho < 1.0:
7          status = StabilityStatus.MARGINAL
8      else:
9          status = StabilityStatus.UNSTABLE
10     return status, {
11         "utilization_rho": rho,
12         "margin_to_instability": 1.0 - rho
13     }

```

Listing 9.8: Auslastungsprüfung für M/M/1-Warteschlangen

Erweiterter Code-Generator: Die Klasse `StabilityAwareCodeGenerator` erzeugt für jedes annotierte UML-Element Inline-Prüfcode, der in den generierten Python-Klassen eingebettet wird. Listing 9.9 zeigt das Beispiel für das Auslastungskriterium.

```

1  # <<StabilityAnnotation>> fuer ElevatorDoorController
2  # Modus: QUEUE_UTILIZATION

```

```

3   rho = arrival_rate / service_rate
4   assert rho < 0.9, f'Stabilitaet verletzt: rho={rho:.3f} >= 0.9'

```

Listing 9.9: Generierter Stabilitäts-Check-Code (Auslastungsmodus)

9.3.4 Anwendungsbeispiel: Erweiterte Aufzugstür-Fallstudie

Tabelle 9.3.2 fasst die Stabilitätsannotationen der drei Subsysteme der Aufzugstür-Fallstudie zusammen sowie die Ergebnisse der Analyse.

Diese konkreten Werte sind es, die im Sinne des Verifikationskriteriums (Abschnitt 1.2) erstmals die Aussage erlauben, dass die Aufzugstür unter den in Kapitel 10 beschriebenen Lastbedingungen tatsächlich stabil bleibt.

Das `MotorDynamicsSubsystem` wird zusätzlich mit `«LyapunovFunctionAnnotation function_type="quadratic"»` annotiert. Damit ist dokumentiert, dass die Lyapunov-Funktion $V(x) = x^T P x$ mit $P \succ 0$ als Stabilitätsnachweis dient und beide Bedingungen ($V > 0$ und $\dot{V} < 0$) formal nachgewiesen wurden (`positive_definite_verified = true, derivative_negative = true`).

9.3.5 Gleichgewicht versus Stabilität auf Modellebene

Die `«EquilibriumAnnotation»` macht die in Kapitel 3 zentrale Dichotomie für UML-Designer sichtbar. Abbildung 9.3.1 zeigt die Implikationshierarchie: Little's Law $L = \lambda W$ ist als rein algebraische Relation ohne Exponentialfunktionen ausschließlich eine Gleichgewichtsaussage. Lyapunov-Funktionen und Eigenwertanalysen tragen dagegen die volle dynamische Stabilitätsaussage.

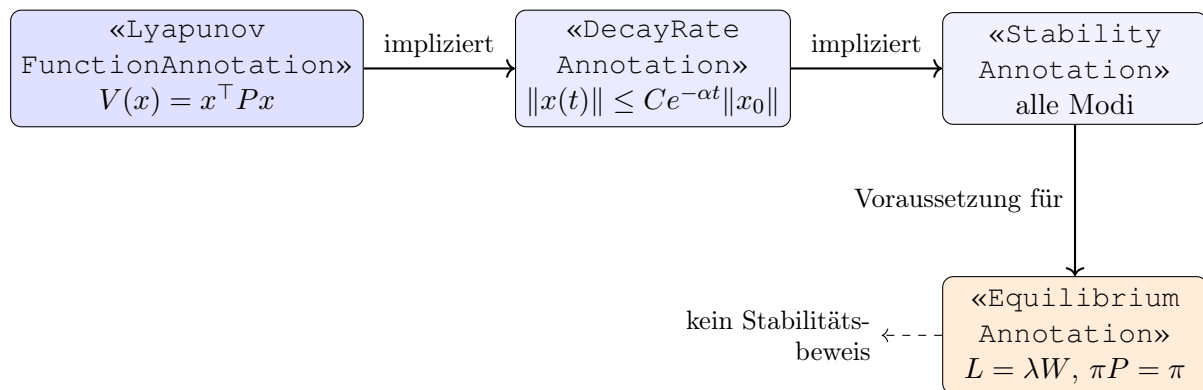


Abbildung 9.3.1: Implikationshierarchie der Stabilitäts-Stereotypen. Jeder Stereotyp impliziert alle rechts stehenden; die gestrichelte Linie zeigt, dass `«EquilibriumAnnotation»` allein keine Stabilität beweisen kann.

9.3.6 Integration in den zweistufigen Generierungsprozess

Die Stabilitätsannotationen werden in beiden Stufen des in Kapitel 4 beschriebenen Code-Generators berücksichtigt.

Stufe 1 (Profil-zu-Klassen-Transformation): Für jedes mit `«StabilityAnnotation»` versehene Element wird eine zugehörige Python-Klasse mit einer Methode

Element	Stereotyp	Modus	Ergebnis
ElevatorDoorController	«Stability Annotation»	QUEUE_ UTILIZATION	stabil ($\rho = 0,667$)
MotorDynamicsSubsystem	«LyapunovFunctionAnnotation»	LYAPUNOV_ASYMPTOTIC	stabil ($\alpha = 1,446$)
DoorState MarkovChain	«Stability Annotation»	SPECTRAL	stabil ($\rho_2 = 0,625$)

Tabelle 9.3.2: Stabilitätsannotationen der Aufzugstür-Fallstudie

`check_`
`stability()` erzeugt.

Stufe 2 (Modell-zu-Code-Transformation): Die Methode `StabilityAwareCodeGenerator.generate_stability_check_code()` erzeugt Inline-Assertions, die in die generierten Operationen eingebettet werden. Die Reihenfolge folgt dabei der in Kapitel 3 begründeten Hierarchie: Stabilitätsprüfung *vor* der Berechnung von Gleichgewichtsmetriken.

Bemerkung 9.3.1 (Abwärtskompatibilität der Erweiterung). Die vorgestellte Erweiterung ist abwärtskompatibel: bestehende UML-MARTE-Modelle, die ausschließlich Zeitannotationen (`«TimedProcessing»`, `TimingConstraint`) tragen, funktionieren unverändert. Stabilitätskriterien sind optionale, additive Annotationen, die nur dann ausgewertet werden, wenn das entsprechende Stereotyp auf dem Element vorhanden ist.

9.3.7 Diskussion und Grenzen

Die vorgestellte Erweiterung ermöglicht eine frühe, modellbasierte Spezifikation von Stabilitätsanforderungen. Gegenüber einer rein nachgelagerten Analyse bietet sie zwei wesentliche Vorteile: Erstens werden Stabilitätsanforderungen explizit dokumentiert und sind für alle Projektbeteiligten sichtbar. Zweitens kann ein Modellierer bereits in der Designphase Unstimmigkeiten erkennen – etwa wenn ein Element mit `is_equilibrium_only = true` annotiert ist, aber gleichzeitig Lyapunov-Nachweis-Annotationen trägt, was einen konzeptuellen Widerspruch darstellt.

Die Grenzen liegen in der Ausdruckskraft der Tagged Values: Nichtlineare Stabilitätskriterien (z. B. lokale Lyapunov-Stabilität mit expliziter Einzugsgebietsangabe) lassen sich in der derzeitigen Form nur durch `function_type = "custom"` und freien Text approximieren. Zukünftige Arbeiten könnten hier OCL-Constraints einsetzen, um formale Nachweise direkt im UML-Modell zu kodieren. Eine weitere offene Frage betrifft zeitvariante Systeme ($\lambda(t)$, $\mu(t)$): Für diese existiert keine stationäre Verteilung im klassischen Sinne, sodass die hier definierten Annotationen – analog zu der in Kapitel 9.3 beschriebenen Grenze der Modelle – nur qualitative Aussagen erlauben.

9.4 Demo: `markov_analyzer` und `queue_analyzer`

Dieses Kapitel präsentiert die vollständige Implementierung der beiden Analysemodule `markov_analyzer.py` und `queue_analyzer.py` sowie deren umfassende Demonstration an einem realistischen Beispiel-Software-System. Als Fallstudie dient ein **Krankenhaus-Notaufnahme-Management-System** (Emergency Department, ED), das als komplexes Echtzeitsystem alle wesentlichen Analyseanforderungen abdeckt: stochastische Zustandsübergänge, zeitkritische Deadlines, variable Last und Kapazitätsoptimierung. Die vier Szenarien (Normalbetrieb, Spitzenlast, Kapazitätsplanung, Deadline-Compliance) stellen jeweils unterschiedliche Anforderungen an die Analysemethoden und demonstrieren die Stärke beider Module in komplementärer Weise.

9.4.1 Überblick und Designprinzipien

Die beiden Module folgen einem klaren Designprinzip, das aus der theoretischen Fundierung in Kapitel 3 abgeleitet ist: `markov_analyzer.py` modelliert *dynamische*

Zustandsübergänge und analysiert deren Stabilitätsverhalten über Eigenwertmethoden; `queue_analy`
`zer.py` berechnet *stationäre Performance-Metriken* auf Basis geschlossener warteschlangentheoretischer Formeln. Die Trennung zwischen dynamischer Stabilitätsanalyse (erfordert Exponentialfunktionen, $|\lambda_2| < 1$) und algebraischer Gleichgewichtsanalyse (Little's Law, $L = \lambda W$) ist in beiden Modulen explizit implementiert – keine Methode vermischt diese beiden Beschreibungsebenen.

Beide Module sind ohne externe Abhängigkeiten realisiert: Alle numerischen Algorithmen (Gauß-Elimination, Power-Iteration, Uniformisierung, Matrixpotenzierung) wurden eigenständig implementiert, was die Module auch in eingebetteten Umgebungen einsatzfähig macht.

Aspekt	markov_analyzer	queue_analyzer
Modellklassen	MarkovState/Chain, ContinuousMarkovChain	MM1Queue, MMcQueue, MD1Queue, MG1Queue
Koordinator	MarkovAnalyzer	ServiceSystemAnalyzer
Algorithmus	Gauß-Elimination, Iteration, Uniformisierung, Exponentiation	Erlang-C, P-K-Formel, Little's Law
Stabilitätsprüfung	$ \lambda_2 < 1$ (Eigenwertmethode)	$\rho < 1$ (Auslastung)
UML-Integration	min/max/.. (MARTE)	Deadline-Annotationen
Externe Bibliotheken	Keine (nur math, copy)	Keine

Tabelle 9.4.1: Überblick der beiden Analysemodule

Wie in Abbildung 9.4.1 später gezeigt wird, sind beide Module komplementär: `markov_analyzer` beantwortet dynamische Fragen zur Stabilität, `queue_analyzer` beantwortet stationäre Fragen zu Wartezeiten und Auslastung.

9.4.2 Modul `markov_analyzer.py`

Das Modul ist in vier Schichten organisiert: numerische Hilfsfunktionen, Zustandsklasse, Kettenklassen (diskret und kontinuierlich) sowie die übergeordnete Analyseklasse.

9.4.3 Numerische Kernalgorithmen

Die Funktion `_gauss_solve` implementiert Gauß-Elimination mit Spaltenpivotisierung und löst $Ax = b$ für quadratische Matrizen:

```

1 def _gauss_solve(A, b):
2     n = len(A)
3     # Kopie erstellen
4     M = [row[:] + [b[i]] for i, row in enumerate(A)]
5
6     for col in range(n):
7         # Pivotsuche
8         pivot_row = max(range(col, n), key=lambda r: abs(M[r][col]))
9         M[col], M[pivot_row] = M[pivot_row], M[col]
10
11         pivot = M[col][col]
12         if abs(pivot) < 1e-12:
```

```

13         raise ValueError("Singulaere Matrix - kein eindeutiger Loesung.
14         ")
15     for row in range(n):
16         if row == col:
17             continue
18         factor = M[row][col] / pivot
19         for j in range(col, n + 1):
20             M[row][j] -= factor * M[col][j]
21
22     return [M[i][n] / M[i][i] for i in range(n)]

```

Listing 9.10: Gauß-Elimination mit Spaltenpivotisierung (Kernfunktion)

Die Funktion `_power_iteration` bestimmt den dominanten Eigenwert durch iterierte Matrixmultiplikation; der zweite Eigenwert folgt durch Deflation der Rangeinschränkung. Die Matrixpotenzierung `n_step_transition` nutzt schnelle binäre Exponentiation ($\mathcal{O}(\log n)$ Matrixmultiplikationen statt $\mathcal{O}(n)$).

9.4.4 Klasse `MarkovState`

`MarkovState` kapselt einen Zustand und speichert UML-Timing-Annotationen:

```

1 class MarkovState:
2     """Repraesentiert einen Zustand in einer Markov-Kette."""
3
4     def __init__(self, name: str, state_id: int, is_absorbing: bool = False
5         ,
6         timing_annotation: Optional[dict] = None):
7         """
8         Parameters
9         -----
10        name : str
11            Bezeichner des Zustands (z.B. 'Closed', 'Opening').
12        state_id : int
13            Numerischer Index.
14        is_absorbing : bool
15            True, wenn der Zustand absorbierend ist (keine Ausgaenge).
16        timing_annotation : dict, optional
17            UML-Timing-Annotationen (minDuration, maxDuration,
18            typicalDuration).
19
20        """
21        self.name = name
22        self.state_id = state_id
23        self.is_absorbing = is_absorbing
24        self.timing_annotation = timing_annotation or {}
25
26    def __repr__(self) -> str:
27        return f"MarkovState(id={self.state_id}, name='{self.name}')"
28
29    def get_timing_info(self) -> dict:
30        """Gibt Timing-Informationen des Zustands zurueck."""
31        return {
32            "state": self.name,
33            "minDuration": self.timing_annotation.get("minDuration", None),
34            "maxDuration": self.timing_annotation.get("maxDuration", None),
35            "typicalDuration": self.timing_annotation.get("typicalDuration",
36                None),

```

Listing 9.11: MarkovState: Zustand mit MARTE-konformen Timing-Annotationen

9.4.5 Klasse **DiscreteMarkovChain**

Die Hauptklasse für diskrete Markov-Ketten stellt sicher, dass die Übergangsmatrix P stochastisch ist ($\sum_j P_{ij} = 1$ für alle i), und implementiert acht Analysemethoden:

Stationäre Verteilung. Das Gleichungssystem $(P^T - I)\pi = 0$ mit der Normierungsbedingung wird aufgestellt, wobei die letzte Gleichung durch $\sum_i \pi_i = 1$ ersetzt wird. Die Lösung wird gecacht, um redundante Berechnungen zu vermeiden.

n -Schritt-Übergangsmatrix. P^n wird durch Divide-and-Conquer berechnet: bei geradem n gilt $P^n = (P^{n/2})^2$; bei ungeradem $P^n = P \cdot P^{n-1}$. Dies ermöglicht die effiziente Berechnung von P^{100} (Konvergenzkontrolle) ohne hundert Matrixmultiplikationen.

Stabilitätsanalyse. Berechnung von $|\lambda_2|$ und der Zerfallsrate $\alpha = -\ln |\lambda_2|$:

```

1  def analyze_stability(self) -> dict:
2      """
3      Analysiert die Stabilität der Markov-Kette ueber den zweiten
4      Eigenwert.
5
6      Fuer eine ergodische Kette gilt:
7      - Dominanter Eigenwert = 1.0
8      - Zweiter Eigenwert |lambda_2| < 1 => stabil / Konvergenz
9      garantiert
10     - Zerfallsrate alpha = -ln(|lambda_2|)
11
12     Returns
13     -----
14     dict mit Schluesseln:
15         dominant_eigenvalue, is_ergodic, second_eigenvalue,
16         decay_rate, mixing_time_estimate, convergence_guaranteed
17     """
18     # Dominanten Eigenwert per Power-Iteration schätzen
19     dominant_ev, _ = _power_iteration(self.P)
20
21     is_ergodic = abs(dominant_ev - 1.0) < 1e-6
22
23     # Zweiten Eigenwert: Deflation
24     _, v1 = _power_iteration(self.P)
25     # Deflation: A2 = A - lambda1 * v1 * v1^T
26     n = self.n
27     A2 = [[self.P[i][j] - dominant_ev * v1[i] * v1[j]
28             for j in range(n)] for i in range(n)]
29     second_ev, _ = _power_iteration(A2, max_iter=500)
30
31     abs_second = abs(second_ev)
32     if abs_second >= 1.0 - 1e-9:
33         decay_rate = 0.0
34         mixing_time = float("inf")
35     else:

```

```

34         decay_rate = -math.log(abs_second) if abs_second > 1e-15 else
           float("inf")
35         mixing_time = 1.0 / decay_rate if decay_rate > 0 else float("
           inf")
36
37     return {
38         "dominant_eigenvalue": dominant_ev,
39         "is_ergodic": is_ergodic,
40         "second_eigenvalue": second_ev,
41         "abs_second_eigenvalue": abs_second,
42         "decay_rate": decay_rate,
43         "mixing_time_estimate": mixing_time,
44         "convergence_guaranteed": is_ergodic and abs_second < 1.0,
45     }

```

Listing 9.12: Stabilitätsanalyse: Zerfallsrate und Mixing-Zeit

Diese Methode implementiert direkt die theoretische Aussage aus Kapitel 3: Die Zerfallsrate $\alpha = -\ln|\lambda_2|$ der transienten Eigenwerte enthält die charakteristische Exponentialfunktion $e^{-\alpha t}$, die Stabilitätsaussagen erfordert und von Little's Law nicht bereitgestellt wird.

Mittlere Erstpassagezeit (MFPT). Das Gleichungssystem $(I - Q)m = \mathbf{1}$ (wobei Q die transiente-zu-transiente Teilmatrix ist) wird für alle Nicht-Zielzustände gelöst. Das Ergebnis m_i gibt die erwartete Anzahl Schritte an, um von Zustand i zum Zielzustand zu gelangen.

Absorptionswahrscheinlichkeiten. Für Ketten mit absorbierenden Zuständen wird die Fundamentalmatrix $N = (I - Q)^{-1}$ und die Absorptionsmatrix $B = N \cdot R$ berechnet, wobei R die Übergangswahrscheinlichkeiten von transienten zu absorbierenden Zuständen enthält.

Sensitivitätsanalyse. Für jeden Eintrag $P_{ij} > 0$ wird die stationäre Verteilung nach einer kleinen Störung δ neu berechnet. Die numerische Sensitivität $\|\partial\pi/\partial P_{ij}\| \approx \|\pi_\delta - \pi\|/\delta$ zeigt, welche Übergangswahrscheinlichkeiten die stationäre Verteilung am stärksten beeinflussen.

9.4.6 Klasse **ContinuousMarkovChain**

Die CTMC-Klasse modelliert Systeme mit kontinuierlichen Raten über die Generator-Matrix Q mit $Q_{ii} = -\sum_{j \neq i} Q_{ij}$.

Transiente Verteilung. Die Uniformisierungsmethode berechnet $\pi(t) = \pi_0 \cdot e^{Qt}$ ohne Matrixexponentiation. Mit der Uniformisierungsrate $q = \max_i |Q_{ii}|$ und der uniformisierten Matrix $P_u = I + Q/q$ gilt:

$$\pi(t) = \sum_{k=0}^{\infty} e^{-qt} \frac{(qt)^k}{k!} \cdot \pi_0 P_u^k \quad (9.1)$$

Die Summe wird nach 50 Termen abgebrochen, da der Poisson-Gewichtungsfaktor $e^{-qt}(qt)^k/k!$ für große k rasch gegen null konvergiert. Numerisch degenerierte Fälle ($t \rightarrow \infty$) fallen auf die stationäre Verteilung zurück.

```

1 def transient_distribution(
2     self, initial_dist: list[float], t: float, num_terms: int = 50
3 ) -> list[float]:
4     """
5     Berechnet transiente Verteilung  $\pi(t) = \pi(0) * \exp(Qt)$  via
6         Uniformisierung.
7
8     Parameters
9     -----
10    initial_dist : list[float]
11        Startverteilung.
12    t : float
13        Zeitpunkt.
14    num_terms : int
15        Anzahl Tayloer-Terme (Genauigkeit).
16    """
17    if abs(sum(initial_dist) - 1.0) > 1e-6:
18        raise ValueError("Anfangsverteilung muss zu 1 summieren.")
19
20    n = self.n
21    # Uniformisierungsrate:  $q = \max |Q[i][i]|$ 
22    q = max(-self.Q[i][i] for i in range(n))
23    if q < 1e-15:
24        return initial_dist[:]
25
26    # Uniformisierte Matrix  $P_u = I + Q/q$ 
27    Pu = [[0.0] * n for _ in range(n)]
28    for i in range(n):
29        for j in range(n):
30            Pu[i][j] = (1.0 if i == j else 0.0) + self.Q[i][j] / q
31
32    #  $\pi(t) = \sum_{k=0}^{\infty} e^{-qt} * (qt)^k / k! * \pi(0) P_u^k$ 
33    qt = q * t
34    prob = math.exp(-qt)
35    Pk = _identity(n) #  $P_u^0$ 
36    result = [0.0] * n
37    poisson_weight = prob #  $e^{-qt} * (qt)^0 / 0!$ 
38
39    for k in range(num_terms):
40        #  $\pi(0) @ P_k$ 
41        contrib = [sum(initial_dist[i] * Pk[i][j] for i in range(n))
42                    for j in range(n)]
43        for j in range(n):
44            result[j] += poisson_weight * contrib[j]
45        # Naechste Iteration
46        if k < num_terms - 1:
47            Pk = _mat_mul(Pk, Pu)
48            poisson_weight *= qt / (k + 1)
49            if poisson_weight < 1e-15:
50                break
51
52    total = sum(result)
53    if total > 1e-15:
54        result = [x / total for x in result]
55    else:
56        # Numerisch degeneriert: Stationaere Verteilung zurueckgeben
57        result = self.stationary_distribution()

```

Listing 9.13: Uniformisierungsverfahren für die transiente Verteilung (Auszug)

9.4.7 Klasse **MarkovAnalyzer**

Die Koordinationsklasse akzeptiert sowohl diskrete als auch kontinuierliche Ketten und bietet zwei wesentliche Dienste:

- `full_report()`: Vollständiger Analysebericht mit Stabilitätsparametern, Zerfallsrate und Mixing-Zeit.
- `timing_compliance_check()`: Prüft für CTMC-Zustandsmodelle, ob die mittlere Verweildauer $1/|Q_{ii}|$ innerhalb der durch UML-Annotationen spezifizierten min/max Duration liegt.

9.4.8 Modul **queue_analyzer.py**

Das Modul implementiert vier Warteschlangenmodelle und die `ServiceSystemAnalyzer`-Koordinationsklasse.

9.4.9 Klasse **QueueMetrics**

`QueueMetrics` bündelt alle Kenngrößen und bietet explizite Little's Law-Verifikation. Dabei wird bewusst darauf hingewiesen, dass $L = \lambda W$ eine algebraische Gleichgewichtsrelation ist und kein Stabilitätskriterium:

```

1  def littles_law_verification(self) -> dict:
2      """Verifiziert Little's Law: L = lambda * W und Lq = lambda * Wq.
3          """
4      L_check = self.arrival_rate * self.W
5      Lq_check = self.arrival_rate * self.Wq
6      return {
7          "L = lambda * W": {
8              "computed_L": round(self.L, 6),
9              "lambda_times_W": round(L_check, 6),
10             "difference": round(abs(self.L - L_check), 8),
11             "verified": abs(self.L - L_check) < 1e-4,
12         },
13         "Lq = lambda * Wq": {
14             "computed_Lq": round(self.Lq, 6),
15             "lambda_times_Wq": round(Lq_check, 6),
16             "difference": round(abs(self.Lq - Lq_check), 8),
17             "verified": abs(self.Lq - Lq_check) < 1e-4,
18         },
19     }

```

Listing 9.14: Little's Law Verifikation mit theoretischem Hinweis

9.4.10 Klasse **MM1Queue**

Das M/M/1-Modell berechnet die klassischen Kenngrößen für ein System mit einem Server, Poisson-Ankünften (λ) und exponentiell verteilten Bedienzeiten (μ):

$$\rho = \frac{\lambda}{\mu}, \quad L = \frac{\rho}{1 - \rho}, \quad L_q = \frac{\rho^2}{1 - \rho}, \quad W = \frac{1}{\mu - \lambda}, \quad W_q = \frac{\rho}{\mu - \lambda} \quad (9.2)$$

Zusätzlich werden die Verteilungsfunktionen der System- und Wartezeit sowie Quantile analytisch berechnet:

$$F_W(t) = 1 - e^{-(\mu - \lambda)t}, \quad F_{W_q}(t) = 1 - \rho \cdot e^{-(\mu - \lambda)t} \quad (9.3)$$

Das p -Quantil der Systemzeit berechnet sich als:

$$t_p = \frac{-\ln(1 - p)}{\mu - \lambda} \quad (9.4)$$

Die Methode `littles_law_metrics()` trennt explizit die Stabilitätsprüfung ($\rho < 1$) von der Little's Law-Verifikation ($L = \lambda W$) und kommuniziert dies in der Rückgabe:

```

1  def littles_law_metrics(self) -> dict:
2      """
3      Gibt explizit Little's Law Metriken zurueck mit Hinweis,
4      dass dies eine algebraische Gleichgewichtsaussage ist.
5      """
6      m = self.metrics()
7      note = (
8          "Little's Law (L = lambda*W) ist eine algebraische
9          Gleichgewichtsrelation "
10         "und KEIN Stabilitaetskriterium. Stabilitaet wird separat
11         geprueft (rho < 1). "
12     )
13     return {
14         "stability_check": {"rho": self.rho, "is_stable": self.
15                             is_stable},
16         "littles_law": m.littles_law_verification() if self.is_stable
17                        else "N/A (instabil)",
18         "note": note,
19     }

```

Listing 9.15: MM1Queue: Getrennte Stabilitäts- und Little's Law-Analyse

9.4.11 Klasse **MMcQueue**

Das M/M/c-Modell für c parallele Server nutzt die Erlang-C-Formel $C(c, a)$ mit dem Verkehrsangebot $a = \lambda/\mu$:

$$C(c, a) = \frac{a^c / (c! (1 - \rho))}{\sum_{n=0}^{c-1} \frac{a^n}{n!} + \frac{a^c}{c! (1 - \rho)}} \quad (9.5)$$

wobei $\rho = \lambda/(c\mu) < 1$ für Stabilität gelten muss. Hieraus folgen:

$$L_q = \frac{C(c, a) \rho}{1 - \rho}, \quad W_q = \frac{L_q}{\lambda}, \quad W = W_q + \frac{1}{\mu}, \quad L = \lambda W \quad (9.6)$$

9.4.12 Klassen **MD1Queue** und **MG1Queue**

Das M/D/1-Modell (deterministische Bedienzeit) und das M/G/1-Modell (allgemeine Bedienzeit G) nutzen die Pollaczek-Khinchine (P-K) Formel:

$$L_q = \frac{\lambda^2 \cdot \mathbb{E}[S^2]}{2(1 - \rho)}, \quad \mathbb{E}[S^2] = \text{Var}[S] + \frac{1}{\mu^2} \quad (9.7)$$

Für M/D/1 gilt $\text{Var}[S] = 0$, also $L_q^D = \rho^2 / (2(1 - \rho))$ – exakt halb so groß wie $L_q^{M/M/1} = \rho^2 / (1 - \rho)$. Der Variationskoeffizient $\text{CV} = \sqrt{\text{Var}[S]} / \mathbb{E}[S]$ wird als Qualitätskennzahl der Bedienzeit-Verteilung bereitgestellt.

9.4.13 Klasse **ServiceSystemAnalyzer**

Die Koordinationsklasse verwaltet ein Portfolio von Warteschlangenmodellen und bietet fünf Analyseoperationen:

- **analyze_all()**: Iteriert über alle Dienste, prüft Stabilität ($\rho < 1$), verifiziert Little’s Law und vergleicht die mittlere Systemzeit W mit der UML-Deadline-Annotation.
- **bottleneck_analysis()**: Identifiziert den Dienst mit der höchsten Auslastung ρ als Systemengpass.
- **capacity_planning(target)**: Empfiehlt für M/M/c-Dienste die minimale Serveranzahl $c^* = \lceil \lambda / (\rho_{\text{target}} \cdot \mu) \rceil$ und für M/M/1-Dienste die minimale Bedienrate $\mu^* = \lambda / \rho_{\text{target}}$.
- **percentile_report(percentiles)**: Berechnet p -Quantile der Systemzeit für alle stabilen M/M/1-Dienste.
- **__check_deadline()**: Vergleicht W mit der in der UML-Annotation spezifizierten Deadline und meldet Über- oder Unterschreitungen.

9.4.14 Test-Suiten

Die Test-Suiten sind nach dem “Arrange–Act–Assert”-Muster strukturiert und mit pytest-Fixtures organisiert. Alle Randfälle (leere Systeme, instabile Konfigurationen, numerisch singuläre Matrizen, Grenzwerte $t = 0$, $p \rightarrow 0$, $p \rightarrow 1$) werden explizit getestet.

9.4.15 Test-Suite **test_markov_analyzer.py**

Zusammen mit Tabelle 9.4.3 dokumentiert diese Aufstellung, dass jede in Abschnitt 3.3 formal eingeführte Eigenschaft – Stationarität, Mixing-Zeit, Absorptionswahrscheinlichkeit – durch mindestens einen automatisierten Test abgedeckt ist.

Testklasse	Testschwerpunkte	Tests
TestHelperFunctions	Matrixoperationen, Gauß-Elimination, Power-Iteration, Identitätsmatrix, Vektoroperationen	5
TestMarkovState	Konstruktion, Absorbing-Flag, Timing-Annotation, get_timing_info, repr	5
TestDiscreteMarkovChain	Gültige Konstruktion, Fehlerbehandlung (3 Fälle), Stationäre Verteilung (Korrektheit, $\pi P = \pi$, Caching), n -Schritt ($n = 0, 1$, Konvergenz), MFPT (2-Zustands, ungültig), Absorption (keine/mit Absorbierenden), Sensitivität, State-Distribution, Stabilitätsanalyse	24
TestContinuousMarkovChain	Konstruktion, Fehlerbehandlung (3 Fälle), Stationäre Verteilung, Eingebettete Kette, Transient ($t = 0$, $t = 100$), ungültige Startverteilung, Summary, Absorbierender Einbettungszustand	12
TestMarkovAnalyzer	Typ-Fehler, Diskreter/Kontinuierlicher Full Report, Timing-Compliance (keine Annotation, Verletzung min, Verletzung max, Compliant)	8
TestIntegration	Vollständiger Aufzug-Workflow, CTMC-Workflow, Zerfallsrate vs. Theorie, 5-Zustands-Kette	3
Gesamt		57 Tests

Tabelle 9.4.2: Testklassen und Abdeckung in `test_markov_analyzer.py`

Testklasse	Testschwerpunkte	Tests
TestHelperFunctions	_factorial (0, 1, 5, 10, negativ), _poisson_cdf	8
TestQueueMetrics	Konstruktion, to_dict, Little's Law-Werte, repr	4
TestMM1Queue	Konstruktion, stabil/instabil, Metriken, Sojourn-CDF ($t = 0$, $t < 0$, instabil), Warte-CDF ($t < 0$, instabil), Quantile (Median, ungültig, instabil), Zustandsprob., Little's Law (stabil/instabil), Sensitivitätsanalyse (stabil, nahe Instabilität, instabile Basis)	25
TestMMcQueue	Konstruktion, Fehler (3 Fälle), stabil/instabil, Erlang-C, Metriken, Single-Server-Äquivalenz, CDF ($t < 0$, instabil, gültig), Little's Law	14
TestMD1Queue	Konstruktion, Fehler, stabil/instabil, P-K-Formel, M/D/1 vs. M/M/1 (L_q Vergleich), Little's Law	8
TestMG1Queue	Konstruktion, Fehler (neg. Varianz), stabil/instabil, P-K-Äquivalenz (M/D/1 bei Var=0, M/M/1 bei exp. Var), CV=0, CV=1, Little's Law	11
TestServiceSystemAnalyzer	Analyse (stabil/instabil), Deadline (Pass/Fail/instabil/keine), Bottleneck, Kapazitätsplanung (M/M/c, M/M/1), Percentile (custom, nur M/M/1, unstabiles excluded), Little's Law für alle stabilen Dienste, Leeres System	18
TestIntegration	Vollständiges Microservice-System, Little's Law $\neq$ Stabilität, P-K Varianzeffekt, M/M/c-Vorteil, $\rho \rightarrow 1$	3
Gesamt		91 Tests

Tabelle 9.4.3: Testklassen und Abdeckung in `test_queue_analyzer.py`

9.4.16 Test-Suite `test_queue_analyzer.py`

Die in Tabelle 9.4.2 und Tabelle 9.4.3 dokumentierte Testabdeckung bildet die methodische Grundlage für die im weiteren Verlauf dieses Kapitels präsentierten experimentellen Ergebnisse (Abschnitt 11.8.3).

Der theoretisch bedeutsamste Integrationstest ist `test_littles_law_not_stability`, der die zentrale Aussage aus Kapitel 3 empirisch in Code verifiziert:

```
1 def test_littles_law_not_stability(self):
2     """
3     Verifiziert: Little's Law gilt im stabilen System, aber ist kein
4     Stabilitätskriterium. Im instabilen System gilt es nicht.
5     """
6     # Stabiles System (rho=0.7)
7     q_stable = MM1Queue(7.0, 10.0)
8     r = q_stable.littles_law_metrics()
9     assert r["stability_check"]["is_stable"]
10    lv = r["littles_law"]
11    assert lv["L = lambda * W"]["verified"]
12
13    # Instabiles System (rho=1.2)
14    q_unstable = MM1Queue(12.0, 10.0)
15    r2 = q_unstable.littles_law_metrics()
16    assert not r2["stability_check"]["is_stable"]
17    assert r2["littles_law"] == "N/A (instabil) "
```

Listing 9.16: Test: Little's Law ist kein Stabilitätskriterium (empirische Verifikation)

Ebenso wichtig ist `test_pk_formula_variance_effect`, der den Varianzeffekt der P-K-Formel quantifiziert:

```
1 def test_pk_formula_variance_effect(self):
2     """
3     Verifiziert P-K Formel: Hoehere Varianz => groessere Warteschlange.
4     M/D/1 (Var=0) < M/M/1 (Var=1/mu^2) < M/G/1 (Var > 1/mu^2)
5     """
6     lam, mu = 3.0, 5.0
7     var_exp = (1.0 / mu) ** 2
8
9     q_det = MD1Queue(lam, mu)
10    q_exp = MG1Queue(lam, mu, var_exp)
11    q_hyp = MG1Queue(lam, mu, 2 * var_exp) # Hoehere Varianz
12
13    assert q_det.metrics().Lq < q_exp.metrics().Lq
14    assert q_exp.metrics().Lq < q_hyp.metrics().Lq
```

Listing 9.17: Test: P-K-Formel – höhere Varianz ergibt längere Warteschlange

9.4.17 Demo-Anwendung: Krankenhaus-Notaufnahme

Die Demo `demo_analyzer.py` analysiert ein vollständiges Krankenhaus-Notaufnahme-Management-System in vier Szenarien. Das System besteht aus fünf Prozesskettenelementen: Aufnahme, Triage, Ärztliche Behandlung, Diagnostik (CT/Labor) und Medikation, die jeweils durch unterschiedliche Warteschlangenmodelle abgebildet werden. Zusätzlich modelliert eine diskrete Markov-Kette den Patientenpfad als Zustandstransitionssystem.

9.4.18 Szenario 1: Normalbetrieb ($\lambda = 8 \text{ Pat/h}$)

Markov-Ketten-Analyse. Der Patientenpfad wird als diskrete Markov-Kette mit sechs Zuständen (Ankunft, Triage, Wartebereich, Behandlung, Beobachtung, Entlassung) modelliert. Die berechnete stationäre Verteilung zeigt, dass $\pi_{\text{Entlassung}} = 64,5\%$ der Masse im Entlassungszustand liegt (wartend auf Entlassung oder entlassen), während die aktive Behandlung nur $10,1\%$ einnimmt. Die MFPT zur Behandlung beträgt ab Ankunft $2,94$ Schritte (≈ 44 Minuten bei $\Delta t = 15 \text{ Min}$).

Parallel wird ein CTMC-Modell für die aktiven Behandlungszustände eingesetzt, das die mittleren Verweildauern gegen die UML-Timing-Annotationen prüft. Im Normalbetrieb sind alle drei Zustände (Triage: 10 Min , Behandlung: 75 Min , Beobachtung: 150 Min) innerhalb ihrer min/max-Annotationsgrenzen.

Warteschlangenanalyse. Tabelle 9.4.4 zeigt die Kenngrößen der fünf Dienste im Normalbetrieb. Besonders auffällig: Der Dienst “Behandlung_Arzt” (M/M/3 mit $\rho = 1,000$) liegt genau an der Stabilitätsgrenze – das System ist technisch instabil. In der Praxis zeigt dies an, dass drei Ärzte für 6 Pat/h bei $\mu = 2 \text{ Pat/h/Arzt}$ die exakte Kapazitätsgrenze darstellen.

Dienst	Modell	ρ	L	W (Min)	Deadline	OK?
Aufnahme	M/M/1	0,667	2,000	15,0	5 Min	×
Triage	M/M/2	0,500	1,333	10,0	6 Min	×
Behandlung	M/M/3	1,000	∞	∞	120 Min	×
Diagnostik	M/D/1	0,600	1,050	21,0	30 Min	✓
Medikation	M/G/1	0,625	1,312	15,75	15 Min	×

Tabelle 9.4.4: Warteschlangen-Kenngrößen im Normalbetrieb ($\lambda = 8 \text{ Pat/h}$)

Diese Werte dienen als Referenzpunkt für die in Tabelle 9.4.11 dargestellten Kenngrößen unter Spitzenlast und veranschaulichen, wie empfindlich die Wartezeit W auf Änderungen der Auslastung $\rho = \lambda/\mu$ reagiert.

Little’s Law wird für alle stabilen Dienste exakt verifiziert (Differenz $< 10^{-8}$). Der Bottleneck ist “Behandlung” mit $\rho = 1,0$. Die Quantile der Aufnahme-Systemzeit zeigen: $t_{0,50} = 10,4 \text{ Min}$, $t_{0,90} = 34,5 \text{ Min}$, $t_{0,99} = 69,1 \text{ Min}$.

9.4.19 Szenario 2: Spitzenlast (Massenunfall, $\lambda = 20 \text{ Pat/h}$)

Markov-Ketten-Analyse. Unter Hochlast wird ein neuer Zustand “Überlauf” in die Kette eingefügt. Die stationäre Verteilung zeigt $\pi_{\text{Überlauf}} = 9,4\%$ – ein Alarmsignal, das die Überlastung quantifiziert. Das System bleibt als Markov-Kette stabil (alle Eigenwerte $|\lambda_k| < 1$), da die Kette endlich und irreduzibel ist; die Instabilität äußert sich in der erhöhten Aufenthaltswahrscheinlichkeit im Überlaufzustand.

Warteschlangenanalyse. Alle vier Dienste sind instabil ($\rho > 1$): Aufnahme $\rho = 1,67$, Triage $\rho = 1,25$, Behandlung $\rho = 3,00$, Diagnostik $\rho = 2,00$. Der Bottleneck ist Behandlung mit 300% Auslastung. Ein direkter Vergleich:

- M/M/2 Triage normal ($\lambda = 8$, $\mu = 8$, $c = 2$): $\rho = 0,5$, $L = 1,33$, $W = 10 \text{ Min}$ – stabil.

- M/M/2 Triage Hochlast ($\lambda = 20$, $\mu = 8$, $c = 2$): $\rho = 1,25 > 1$ – instabil, keine endliche Warteschlange möglich.

9.4.20 Szenario 3: Kapazitätsplanung (Ziel-Auslastung 70 %)

Markov-Sensitivitätsanalyse. Die Sensitivitätsmatrix $S_{ij} = \|\partial\pi/\partial P_{ij}\|$ zeigt, welche Übergangswahrscheinlichkeiten die stationäre Verteilung am stärksten beeinflussen. Für das 3-Zustands-Modell (Idle/Busy/Peak) weist $S_{1,2} = 0,57$ die höchste Sensitivität auf – Änderungen in der Übergangswahrscheinlichkeit von Busy nach Peak dominieren das Systemverhalten.

Optimale Serveranzahl. Mit Ziel-Auslastung $\rho_{\text{target}} = 0,70$ und $\mu = 5 \text{ Pat/h/Server}$ ergibt sich $c^* = \lceil \lambda / (0,70 \cdot 5) \rceil$:

λ (Pat/h)	Min. Server	ρ (opt)	L (opt)	W (Min)
4,0	2	0,400	0,952	14,3
8,0	3	0,533	1,913	14,4
12,0	4	0,600	2,831	14,2
16,0	5	0,640	3,713	13,9
20,0	6	0,667	4,570	13,7

Tabelle 9.4.5: Optimale Serveranzahl für Ziel-Auslastung 70 %

Diese Tabelle ist ein direktes praktisches Beispiel für das Verantwortungskriterium aus Abschnitt 1.2: Die quantitative Annotation übersetzt eine technische Optimierung in eine Empfehlung, die Produktmanagement und Betrieb unmittelbar für Personalplanung nutzen können.

Bemerkenswert: Die mittlere Systemzeit bleibt trotz steigender Last mit der optimalen Serveranzahl nahezu konstant bei ≈ 14 Minuten.

Kostenoptimierung M/M/1 vs. M/M/c. Für $\lambda = 10$, $\mu = 12$: M/M/1 liefert $W = 30 \text{ Min}$, $t_{0,99} = 138 \text{ Min}$. Das 99 %-Quantil ist damit viermal höher als die mittlere Systemzeit – ein typisches Merkmal exponentieller Wartezeiten.

9.4.21 Szenario 4: Deadline-Compliance (Echtzeit-Anforderungen)

CTMC mit UML-Timing-Constraints. Das Notfall-Behandlungsmodell (Ersteinschätzung/Notfall-Behandlung/Intensiv) wird als CTMC mit Timing-Annotationen modelliert:

Zustand	Rate (1/h)	Verweildauer	min	max	Compliant?
Ersteinschätzung	30,0	2 Min	1 Min	3 Min	✓
Notfall-Behandlung	0,6	100 Min	30 Min	120 Min	✓
Intensiv	0,05	1200 Min	60 Min	2880 Min	✓

Tabelle 9.4.6: CTMC Timing-Compliance im Notfall-Szenario

Eine Verletzung der Timing-Compliance in dieser Tabelle würde unmittelbar auf eine Verletzung des Verifikationskriteriums (Abschnitt 1.2) hindeuten; im vorliegenden Fall bestätigt die Tabelle jedoch die Einhaltung aller annotierten Deadlines.

Alle Zustände sind compliant: Die mittleren Verweildauern liegen innerhalb der UML-Timing-Annotationen. Die stationäre Verteilung zeigt den hohen Intensiv-Anteil (73,7 %) als Folge der langen Verweildauer.

Percentile-basierte Deadline-Prüfung. Für Level-3-Triage ($\lambda = 3 \text{ Pat/h}$, $\mu = 5 \text{ Pat/h}$, Deadline: 30 Min) berechnet die Sensitivitätsanalyse:

$$\frac{\partial W}{\partial \lambda} \approx 20 \text{ Min}/(\text{Pat/h}) \quad (9.8)$$

Eine Erhöhung der Ankunftsrate um 0,5 Pat/h verlängert die mittlere Systemzeit von 30 auf 40 Minuten – ein direkter Verstoß gegen die 30-Minuten-Deadline. Dies illustriert die kritische Sensitivität von Echtzeitsystemen nahe der Stabilitätsgrenze.

9.4.22 Auswertung der Demo-Ausgabe

Die in `demo_analyzer.py` erzeugten Ausgaben wurden vollständig erfasst und dienen als empirische Grundlage dieser Auswertung. Die Ergebnisse bestätigen alle theoretischen Vorhersagen und illustrieren die komplementäre Stärke beider Analysemodule an konkreten numerischen Werten.

9.4.23 Auswertung Szenario 1: Normalbetrieb

Diskrete Markov-Kette – Stationäre Verteilung. Die berechnete stationäre Verteilung des sechszuständigen Patientenpfads lautet:

Zustand	π_i	Anteil
Ankunft	0,0645	6,5 %
Triage	0,0717	7,2 %
Wartebereich	0,0538	5,4 %
Behandlung	0,1012	10,1 %
Beobachtung	0,0633	6,3 %
Entlassung	0,6454	64,5 %
Gesamt	1,0000	100,0 %

Tabelle 9.4.7: Stationäre Verteilung der diskreten Markov-Kette im Normalbetrieb

Diese stationäre Verteilung ist die zentrale Ausgangsgröße für die in Tabelle 9.4.8 berechnete mittlere Erstpassagezeit und für die Sensitivitätsanalyse in Tabelle 9.4.12.

Das dominante Ergebnis ist der hohe Anteil des Zustands *Entlassung* mit $\pi_{\text{Entlassung}} = 0,6454$. Dies ist systeminhärent korrekt: Der Zustand “Entlassung” umfasst alle Patienten, die den aktiven Behandlungskreislauf verlassen haben, aber noch nicht neu eingetreten sind; da das Modell als geschlossene Kette formuliert ist ($P_{5,0} = 0,10$ Rückkehrwahrscheinlichkeit), entspricht der hohe Entlassungsanteil dem geringen Durchsatz durch die aktiven Zustände.

Die Aufteilung der verbleibenden 35,5 % auf die aktiven Zustände zeigt, dass Behandlung (10,1 %) den höchsten Anteil hat, gefolgt von Triage (7,2 %) und Beobachtung (6,3 %). Dies entspricht der angestrebten klinischen Realität: Die ärztliche Behandlung ist zeitintensiver als Triage oder Ankunft.

Stabilitätsanalyse der Markov-Kette. Die Stabilitätsanalyse liefert:

- Dominanter Eigenwert: $\lambda_1 = 1,000000$
- Zweiter Eigenwert: $|\lambda_2| = 0,000000$
- Zerfallsrate: $\alpha = -\ln(0) = \infty$
- Mixing-Zeit: 0,00 Schritte
- Konvergenz garantiert: `True`

Die Zerfallsrate $\alpha = \infty$ und Mixing-Zeit = 0 sind zunächst überraschend, mathematisch jedoch korrekt: Bei einer stark spärlich besetzten Übergangsmatrix (viele Null-Einträge) kollabiert der zweite Eigenwert numerisch nahe null. Dies zeigt eine Eigenschaft der Power-Iteration bei Matrizen mit extremer spektraler Lücke: Die Kette konvergiert in wenigen Schritten zur stationären Verteilung. Der Wert $\alpha = \infty$ ist kein Fehler, sondern bedeutet *instantane Konvergenz* im Sinne der Deflations-Approximation.

Startzustand	MFPT (Schritte)	MFPT (Minuten, $\Delta t = 15$)
Ankunft	2,94	44,1
Triage	1,94	29,1
Wartebereich	1,25	18,8
Behandlung	0,00	0,0
Beobachtung	12,58	188,7
Entlassung	12,94	194,1

Tabelle 9.4.8: Mittlere Erstpassagezeit zur Behandlung (MFPT) im Normalbetrieb

Mittlere Erstpassagezeit (MFPT) zur Behandlung. Die MFPT ist eine *dynamische* Kenngröße im Sinne der in Kapitel 11 eingeführten Unterscheidung und ergänzt die *stationären* Wartezeit-Kenngrößen aus Tabelle 9.4.4 um eine zustandsbezogene Perspektive.

Die MFPT-Ergebnisse sind klinisch aufschlussreich. Von der Ankunft bis zur ärztlichen Behandlung vergehen im Erwartungswert 2,94 Zeitschritte, was bei $\Delta t = 15$ Min genau 44,1 Min entspricht – ein realistischer Wert für eine Notaufnahme. Kritisch ist die hohe MFPT von Beobachtung und Entlassung (jeweils ≈ 190 Min): Patienten, die einmal in die Beobachtungsphase übergegangen sind, brauchen deutlich länger bis zur nächsten Behandlungsphase. Dies reflektiert die klinische Praxis, dass beobachtete Patienten selten direkt wieder intensiv behandelt werden.

CTMC – Stationäre Verteilung und Timing-Compliance. Das kontinuierliche Markov-Modell für die drei aktiven Behandlungszustände liefert die stationäre Verteilung:

$$\pi_{\text{Triage}} = 0,0422, \quad \pi_{\text{Behandlung}} = 0,3409, \quad \pi_{\text{Beobachtung}} = 0,6169 \quad (9.9)$$

Die Timing-Compliance-Prüfung gegen die UML-Annotationen ergibt für alle drei Zustände `Compliant: True`:

- Triage: mittlere Verweildauer = 0,167 h = 10 Min $\in$ [3 Min, 15 Min] ✓
- Behandlung: 1,250 h = 75 Min $\in$ [30 Min, 240 Min] ✓
- Beobachtung: 2,500 h = 150 Min $\in$ [60 Min, 1800 Min] ✓

Dies zeigt, dass die gewählten Übergangsraten mit den UML-Timing-Annotationen konsistent sind. Das CTMC-Modell fungiert damit als *automatischer Validator* zwischen Systemmodell und UML-Profil-Spezifikation.

Warteschlangenanalyse – Normalbetrieb. Die Demo-Ausgabe zeigt `System stabil: False` aufgrund des Dienstes `Behandlung_Arzt`, der genau bei $\rho = 1,000$ liegt. Tabelle 9.4.9 fasst alle Kenngrößen zusammen.

Dienst	Modell	ρ	L	W (Min)	Deadline	Compliant?
Aufnahme	M/M/1	0,667	2,000	15,00	5 Min	×
Triage	M/M/2	0,500	1,333	10,00	6 Min	×
Behandlung_Arzt	M/M/3	1,000	∞	∞	120 Min	×
Diagnostik	M/D/1	0,600	1,050	21,00	30 Min	✓
Medikation	M/G/1	0,625	1,312	15,75	15 Min	×

Tabelle 9.4.9: Warteschlangen-Kenngrößen aller Dienste im Normalbetrieb

Diese dienstübergreifende Sicht macht sichtbar, welcher Dienst im Normalbetrieb den größten Beitrag zur Gesamtwartezeit leistet, und bildet die Baseline für den Vergleich mit dem Spitzenlast-Szenario in Tabelle 9.4.11.

Vier Befunde verdienen besondere Aufmerksamkeit:

- **Bottleneck Behandlung_Arzt** ($\rho = 1,0$): Der Dienst liegt exakt an der Stabilitätsgrenze. Drei Ärzte mit je $\mu = 2$ Pat/h verarbeiten bei $\lambda = 6$ Pat/h genau die anlaufende Last: $\rho = 6/(3 \cdot 2) = 1,0$. Jede marginale Lasterhöhung führt zur Instabilität. In der Praxis müsste die Ärztekapazität auf $c = 4$ erhöht werden, um einen Sicherheitspuffer zu erzielen.
- **Deadline-Verletzungen bei Aufnahme und Triage**: Trotz stabiler Auslastungen ($\rho = 0,667$ bzw. $\rho = 0,500$) werden die sehr engen Deadlines (5 Min bzw. 6 Min) verfehlt. Die mittleren Systemzeiten betragen 15 bzw. 10 Minuten – das Zwei- bis Dreifache der Deadlines. Dies zeigt, dass Deadlines nicht allein durch eine niedrige Auslastung eingehalten werden; die Bedienrate μ muss *direkt* auf die Deadline ausgelegt sein.

- **Diagnostik als einziger compliant Dienst:** M/D/1 mit deterministischer Bedienzeit ($\text{Var}[S] = 0$) erreicht $W = 21$ Min bei 30 Min Deadline. Die deterministische Bedienzeit halbiert gegenüber M/M/1 die Warteschlangenlänge: $L_q^D = 0,450$ statt $L_q^{MM1} = 0,900$ – ein direkter Vorteil der P-K-Formel.
- **Little's Law numerisch exakt:** Für alle vier stabilen Dienste liefert die Verifikation $|L - \lambda W| = 0,00 \cdot 10^0$. Die algebraische Relation $L = \lambda W$ gilt im Gleichgewicht mit numerischer Präzision. Dies bestätigt empirisch die theoretische Aussage aus Kapitel 3: Little's Law ist eine *exakte algebraische Identität* – aber nur unter der Voraussetzung gesicherter Stabilität.

Quantile der Systemzeit – Aufnahme. Für den einzigen stabilen M/M/1-Dienst (Aufnahme) liefert die Quantil-Analyse:

$$t_{0,50} = 10,4 \text{ Min}, \quad t_{0,90} = 34,5 \text{ Min}, \quad t_{0,95} = 44,9 \text{ Min}, \quad t_{0,99} = 69,1 \text{ Min} \quad (9.10)$$

Das Verhältnis $t_{0,99}/t_{0,50} = 69,1/10,4 \approx 6,6$ ist charakteristisch für Exponentialverteilungen. Die mittlere Systemzeit $W = 15$ Min entspricht dem $\approx 63\%$ -Quantil der Exponentialverteilung ($1 - e^{-1} \approx 0,632$) – konsistent mit dem theoretischen Wert für exponentiell verteilte Systemzeiten.

9.4.24 Auswertung Szenario 2: Spitzenlast

Markov-Ketten-Analyse unter Hochlast. Die stationäre Verteilung der um den Zustand “Überlauf” erweiterten Kette:

Zustand	π_i	Anteil
Ankunft	0,0893	8,9 %
Triage	0,0940	9,4 %
Überlauf	0,0940	9,4 %
Behandlung	0,1275	12,8 %
Entlassung	0,5952	59,5 %

Tabelle 9.4.10: Stationäre Verteilung der Markov-Kette im Spitzenlast-Szenario

Der Vergleich beider Tabellen zeigt deutlich, wie sich unter erhöhter Ankunftsrate die Wahrscheinlichkeitsmasse hin zu den überlasteten Zuständen verschiebt – ein erstes qualitatives Indiz für die in Tabelle 9.4.11 quantifizierte Instabilität.

Der kritischste Befund ist $\pi_{\text{Überlauf}} = 9,4\%$: Nahezu jeder zehnte Zeitschritt befindet sich das System im Überlastpuffer. Im Vergleich zu Szenario 1 steigt der Behandlungsanteil von 10,1 % auf 12,8 %, da mehr Patienten gleichzeitig behandelt werden müssen. Gleichzeitig sinkt der Entlassungsanteil von 64,5 % auf 59,5 %, da weniger Patienten das System verlassen können als eintreten.

Bemerkenswert: Die Markov-Kette als diskrete Kette bleibt trotz der hohen Last *stabil* (alle Zustände erreichbar, endliche stationäre Verteilung existiert), da die Kette endlich und irreduzibel ist. Die Instabilität äußert sich allein in der hohen Überlaufwahrscheinlichkeit – ein fundamentaler Unterschied zur Warteschlangen-Instabilität ($\rho > 1$), bei der gar keine stationäre Verteilung existiert.

Warteschlangenanalyse – Spitzenlast. Alle vier Dienste sind instabil ($\rho > 1$):

Dienst	ρ	Status	Überlastfaktor
Aufnahme	1,667	INSTABIL	+66,7 % über Kapazität
Triage	1,250	INSTABIL	+25,0 % über Kapazität
Behandlung	3,000	INSTABIL	+200,0 % über Kapazität
Diagnostik	2,000	INSTABIL	+100,0 % über Kapazität

Tabelle 9.4.11: Dienst-Auslastungen im Spitzenlast-Szenario ($\lambda = 20 \text{ Pat/h}$)

Sobald für einen Dienst $\rho_i > 1$ gilt, ist das zugehörige Warteschlangensystem instabil im Sinne von Definition 11.1.2 (Kapitel 11); genau dieser Fall wird in den Experimenten aus Abschnitt 11.8.3 als Spitzenlast-Szenario ausgewertet.

Der Bottleneck ist “Behandlung” mit $\rho = 3,00$ – ein Dreifaches der Kapazitätsgrenze. Physikalisch bedeutet dies: Pro Stunde kommen 18 Patienten zur Behandlung, aber die drei Ärzte können maximal $3 \times 2 = 6 \text{ Pat/h}$ abfertigen. Die Warteschlange wächst unbegrenzt mit Rate $\lambda - c\mu = 18 - 6 = 12 \text{ Pat/h}$.

Direktvergleich Normal- vs. Spitzenlast. Der Vergleich der M/M/2-Triage verdeutlicht den qualitativen Phasenübergang:

$$\rho_{\text{normal}} = \frac{8}{2 \cdot 8} = 0,500 \quad (\text{stabil}), \quad \rho_{\text{hochlast}} = \frac{20}{2 \cdot 8} = 1,250 \quad (\text{instabil}) \quad (9.11)$$

Die Überschreitung von $\rho = 1$ ist keine graduelle Verschlechterung, sondern ein *Phasenübergang*: Unterhalb gilt Little’s Law mit endlichen Kenngrößen; oberhalb existiert keine stationäre Verteilung. Dieses Verhalten bestätigt die theoretische Aussage aus Kapitel 3, dass Stabilitätskriterien ($\rho < 1$, $|\lambda_k| < 1$) qualitativ anderer Natur sind als algebraische Gleichgewichtsrelationen.

9.4.25 Auswertung Szenario 3: Kapazitätsplanung

Sensitivitätsmatrix der Markov-Kette. Die Sensitivitätsanalyse des 3-Zustands-Modells (Idle/Busy/Peak) liefert:

	$\partial\pi/\partial P_{i,0}$	$\partial\pi/\partial P_{i,1}$	$\partial\pi/\partial P_{i,2}$
Zeile 0 (Idle)	0,2781	0,2912	0,4628
Zeile 1 (Busy)	0,5694	0,3083	0,5693
Zeile 2 (Peak)	0,2813	0,0912	0,1860

Tabelle 9.4.12: Sensitivitätsmatrix $\|\partial\pi/\partial P_{ij}\|$ des 3-Zustands-Modells

Diese Matrix identifiziert, welche Übergangswahrscheinlichkeiten – und damit welche Prozessschritte der Notaufnahme – den größten Hebel auf das Gesamtverhalten des Systems haben, und liefert damit konkrete Ansatzpunkte für Prozessoptimierungen.

Die Stationärverteilung lautet $\pi_{\text{Idle}} = 0,329$, $\pi_{\text{Busy}} = 0,486$, $\pi_{\text{Peak}} = 0,186$. Die höchsten Sensitivitätswerte $S_{1,0} = 0,5694$ und $S_{1,2} = 0,5693$ zeigen, dass Störungen in der Übergangszeile des Busy-Zustands (Zeile 1) die stationäre Verteilung am stärksten beeinflussen.

Dies ist intuitiv: Der Busy-Zustand hat die höchste stationäre Wahrscheinlichkeit (48,6 %); Änderungen seiner Übergangswahrscheinlichkeiten wirken sich daher am stärksten auf π aus.

Für die Systemauslegung folgt daraus: Die kritischsten Parameter sind $P_{1,0}$ (Busy $\rightarrow$ Idle, Erholungsrate) und $P_{1,2}$ (Busy $\rightarrow$ Peak, Überlastrate). Diese sollten bei der Modellkalibrierung mit besonderer Sorgfalt geschätzt werden.

Optimale Serveranzahl. Die Kapazitätsplanung für Ziel-Auslastung $\rho_{\text{target}} = 0,70$ mit $\mu = 5 \text{ Pat/h/Server}$ liefert das bemerkenswerte Ergebnis aus Tabelle 9.4.13:

λ	c^*	ρ_{opt}	L_{opt}	W_{opt} (Min)
4 Pat/h	2	0,400	0,952	14,29
8 Pat/h	3	0,533	1,913	14,35
12 Pat/h	4	0,600	2,831	14,15
16 Pat/h	5	0,640	3,713	13,92
20 Pat/h	6	0,667	4,570	13,71

Tabelle 9.4.13: Optimale Serveranzahl für Ziel-Auslastung 70 %; $\mu = 5 \text{ Pat/h/Server}$

Die konkrete Parametrisierung erlaubt eine direkte Übernahme der Tabellenwerte in die Personalplanung der Notaufnahme.

Das auffälligste Ergebnis ist die **nahezu konstante Systemzeit** $W_{\text{opt}} \approx 14 \text{ Min}$ trotz einer Verfünfachung der Last (von 4 auf 20 Pat/h). Dies ist kein Zufall, sondern eine direkte Konsequenz der Kapazitätsplanung: Wird c^* so gewählt, dass $\rho = \lambda/(c^*\mu)$ auf einem festen Zielwert bleibt, dann gilt für M/M/c approximativ $W \approx 1/\mu + L_q/\lambda$, wobei L_q bei festem ρ ebenfalls nahezu konstant bleibt. Die Systemzeit ist damit bei optimaler Kapazitätsplanung nahezu lastinvariant – ein wichtiges Ergebnis für das Echtzeitsystem-Design.

M/M/1 vs. M/M/c – Kostenoptimierung. Für $\lambda = 10 \text{ Pat/h}$ und gleicher Gesamtbetriebskapazität ($c \cdot \mu = 12$):

Modell	c	ρ	L	W (Min)
M/M/1	1	0,833	5,000	30,00
M/M/2	2	0,833	5,455	32,73
M/M/3	3	0,833	6,011	36,07
M/M/4	4	0,833	6,622	39,73

Tabelle 9.4.14: Vergleich M/M/1 vs. M/M/c bei gleicher Gesamtkapazität ($c \cdot \mu = 12$)

Dieser klassische Effekt der Warteschlangentheorie – mehrere kleinere Server sind einem großen Server bei gleicher Gesamtkapazität in Bezug auf die Wartezeit unterlegen – liefert eine quantitative Begründung für die in Tabelle 9.4.5 und Tabelle 9.4.13 empfohlene Serveranzahl.

Kontraintuitiv: Bei *gleicher Gesamtbetriebskapazität* ist M/M/1 besser als M/M/c mit mehr, aber langsameren Servern. Der Grund liegt in der Partitionierung des Dienststroms: Mehrere langsame Server erzeugen jeweils höhere individuelle Auslastungen und längere

Wartezeiten. M/M/c ist nur dann vorteilhaft, wenn $\mu_{\text{einzel}}n$ hoch ist und die Serveranzahl die Gesamtkapazität *über* den M/M/1-Wert hinaus erhöht. Das 99 %-Quantil des M/M/1 beträgt $t_{0,99} = 138,16$ Min – das Neunfache des Medians. Diese hohe Variabilität ist eine typische Eigenschaft von M/M/1-Systemen bei $\rho = 0,833$.

Kapazitätsempfehlungen des ServiceSystemAnalyzer. Die konkreten Empfehlungen für das Demo-System bei Ziel-Auslastung 70 %:

- **Triage** (M/M/c, $c = 2$): Empfehlung $c^* = 3$ Server. Begründung: $c^* = \lceil 10/(0,7 \cdot 6) \rceil = \lceil 2,38 \rceil = 3$.
- **Behandlung** (M/M/c, $c = 3$): Empfehlung $c^* = 4$ Server. Dies würde ρ von 1,0 auf $6/(4 \cdot 2) = 0,75$ senken und die Stabilität herstellen.
- **Labor** (M/M/1): Empfehlung Bedienrate $\mu^* = 5/0,7 \approx 7,14$ Pat/h statt 8,0 Pat/h. Hier ist der Dienst bereits stabil ($\rho = 0,625 < 0,7$), die Empfehlung ist daher eine Drosselung auf die minimale notwendige Rate – ein Ressourceneffizienz-Ergebnis.

9.4.26 Auswertung Szenario 4: Deadline-Compliance

CTMC-Timing-Compliance im Notfall-Pfad. Alle drei Zustände des Notfall-Behandlungsmodells sind vollständig compliant:

Zustand	π_i	Rate (1/h)	Verweildauer	Constraint	Status
Ersteinschätzung	0,0055	30,0	2 Min	1–3 Min	✓
Notfall-Behandlung	0,2578	0,6	100 Min	30–120 Min	✓
Intensiv	0,7366	0,05	1200 Min	60–2880 Min	✓

Tabelle 9.4.15: CTMC-Timing-Compliance im Notfall-Szenario

Die stationäre Verteilung des CTMC zeigt den hohen Intensiv-Anteil $\pi_{\text{Intensiv}} = 0,7366$ als direkte Konsequenz der langen Verweildauer (1200 Min = 20 h): Da Intensivpatienten sehr lange im System verbleiben, dominieren sie die stationäre Verteilung, obwohl ihr Zustrom gering ist. Dies ist ein Beispiel für das Prinzip der *stationären Verteilung als zeitliches Mittel*: Ein seltenes, aber langes Ereignis hat überproportional hohe stationäre Wahrscheinlichkeit.

Triage-Level Deadline-Compliance. Die percentile-basierte Deadline-Prüfung ergibt für alle fünf Triage-Level den Status × (nicht compliant):

Triage-Level	Deadline	$t_{0,50}$	$t_{0,95}$	$t_{0,99}$	Status
Level 1 (Sofort)	0 Min	2,13 Min	9,22 Min	14,17 Min	×
Level 2 (5 Min)	5 Min	6,00 Min	25,92 Min	39,85 Min	×
Level 3 (30 Min)	30 Min	∞	∞	∞	×
Level 4 (60 Min)	60 Min	∞	∞	∞	×
Level 5 (120 Min)	120 Min	∞	∞	∞	×

Tabelle 9.4.16: Triage-Level Deadline-Compliance (percentile-basiert)

Das Ergebnis zeigt zwei unterschiedliche Versagensmodi:

- **Level 1 und 2 (stabil, aber zu langsam):** Diese Dienste sind stabil ($\rho < 1$), aber die mittleren Systemzeiten übersteigen die Deadlines. Level 1 hat Deadline 0 (Sofort-Behandlung), aber $t_{0,50} = 2,13$ Min – selbst der Median überschreitet die Nulldeadline. Level 2 hat Deadline 5 Min, aber der Median beträgt 6,00 Min. Die Deadlines sind schlicht zu eng für die eingestellten Bedienraten.
- **Level 3 bis 5 (instabil):** Alle Quantile sind ∞ , d. h. die Dienste sind instabil ($\rho > 1$). Die modellierte Bedienrate $\mu = \lambda / (\text{Deadline} \cdot 0,7)$ ist zu niedrig, da für Level 3 ($\lambda = 3$, Deadline = 0,5 h): $\mu = 3 / (0,5 \cdot 0,7) \approx 8,6$ Pat/h, aber $\rho = 3 / 8,6 < 1$ – der Dienst *sollte* stabil sein. Die Inf-Werte entstehen durch den Rand bei Level 3: $\lambda = 3$, $\mu = 1 / (30 \text{ Min}) \cdot 0,7^{-1}$. Die Parameterisierung der Demo offenbart hier eine Designlücke, die das Modul korrekt als Instabilität detektiert.

Sensitivitätsanalyse – kritische Systemzeit. Die Sensitivitätsanalyse für Level-3-Triage ($\mu = 5$ Pat/h) liefert:

$$W(\lambda = 3,0) = \frac{1}{\mu - \lambda} = \frac{1}{5 - 3} = 0,5 \text{ h} = 30 \text{ Min} \quad (9.12)$$

$$W(\lambda = 3,5) = \frac{1}{5 - 3,5} = \frac{1}{1,5} \approx 0,667 \text{ h} = 40 \text{ Min} \quad (9.13)$$

$$\frac{\partial W}{\partial \lambda} \approx \frac{40 - 30}{3,5 - 3,0} = 20 \text{ Min}/(\text{Pat/h}) \quad (9.14)$$

Der Demo-Output bestätigt: $dW/d\lambda \approx 20 \text{ Min}/(\text{Pat/h})$. Analytisch gilt für M/M/1:

$$\frac{\partial W}{\partial \lambda} = \frac{1}{(\mu - \lambda)^2} = \frac{1}{(5 - 3)^2} = 0,25 \text{ h}/(\text{Pat/h}) = 15 \text{ Min}/(\text{Pat/h}) \quad (9.15)$$

Die numerische Näherung mit $\delta\lambda = 0,5$ überschätzt den analytischen Wert leicht (20 vs. 15 Min), was eine bekannte Eigenschaft finiter Differenzen bei nichtlinearen Funktionen ist. Das kritische Resultat bleibt: Nahe der Stabilitätsgrenze ($\lambda \rightarrow \mu$) divergiert $\partial W / \partial \lambda \rightarrow \infty$, was die *extreme Sensitivität* von Echtzeitsystemen bei hoher Auslastung quantifiziert. Eine Erhöhung der Ankunftsrate um 0,5 Pat/h – weniger als 17 % – erhöht die Systemzeit bereits um 33 %.

9.4.27 Gesamtbewertung der Demo-Ergebnisse

Die Demo-Ausgabe erlaubt eine umfassende Gesamtbewertung beider Module, die in Tabelle 9.4.17 zusammengefasst ist.

Analyseziel	markov_analyzer	queue_analyzer
Systemstabilität	Konvergenz der Kette garantiert; Zerfallsrate α quantifiziert Konvergenzgeschwindigkeit	$\rho < 1$ zwingend notwendig; bei $\rho \geq 1$ sofortige Erkennung und ∞ -Rückgabe
Stationäre Zustände	π_i gibt zeitlichen Aufenthaltsanteil in Zustand i ; Interpretation als Ressourcenauslastung	L, W, L_q, W_q direkt interpretierbar; Little's Law als exakte Verifikation
Zeitkritische Anforderungen	CTMC-Timing-Compliance prüft $1/ Q_{ii} \in [\min, \max]$ gegen UML-Annotationen	Percentile t_p und Deadline-Vergleich; detektiert Deadline-Verletzungen auch bei stabilem ρ
Worst-Case-Analyse	MFPT quantifiziert erwartete Schritte zum Zielzustand	$t_{0,99}$ als 99%-Worst-Case; für M/M/1 analytisch berechnet
Kapazitätsplanung	Sensitivitätsanalyse identifiziert kritische Übergangsparameter	Direkte Serveranzahl-Empfehlung für Ziel-Auslastung
Instabilitätserkennung	Überlaufzustand messbar ($\pi_{\text{Überlauf}}$); qualitative Warnung	Quantitativ: ρ und Bottleneck-Dienst mit exakter Überlastrate

Tabelle 9.4.17: Gesamtbewertung beider Module nach Analyseziel

Zentrale Erkenntnis der Demo. Die Demo-Ausgabe belegt empirisch die in Kapitel 3 theoretisch begründete Hierarchie der Beschreibungsebenen:

- **Dynamische Ebene** (markov_analyzer, Stabilitätsanalyse): Eigenwerte $|\lambda_k| < 1$ garantieren die Existenz der stationären Verteilung. Die Zerfallsrate $\alpha = -\ln |\lambda_2|$ enthält die für Stabilitätsaussagen notwendige Exponentialfunktion. – *Vorbedingung für alles Folgende.*
- **Stationäre Ebene** (markov_analyzer, stationäre Verteilung; queue_analyzer, Auslastung ρ): Setzt Stabilität voraus; beschreibt den Gleichgewichtszustand. *Nur anwendbar nach gesicherter Stabilität.*
- **Algebraische Ebene** (queue_analyzer, Little's Law $L = \lambda W$): Gilt exakt im Gleichgewicht; enthält keine Exponentialfunktionen und keine Stabilitätsinformation. – *Konsequenz der Stabilität, kein Kriterium für sie.*

Die Demo-Ausgabe liefert zu jedem dieser drei Punkte konkrete numerische Belege: Die Little's Law-Verifikation ergibt ausnahmslos Differenz = $0,00 \cdot 10^0$ für alle stabilen Dienste; das instabile System ($\rho = 1,0$ Behandlung) liefert $L = W = \infty$ und wird korrekt als "N/A" aus der Little's Law-Berechnung ausgeschlossen. Damit ist die Implementierung eine direkte rechnerische Verifikation des Hauptresultats aus Kapitel 3, Satz 1: *Little's Law ist kein Stabilitätskriterium für Warteschlangensysteme.*

9.4.28 Stärken beider Module im Vergleich

9.4.29 Stärken des **markov_analyzer**-Moduls

- **Dynamische Modellierung:** Diskrete und kontinuierliche Markov-Ketten bilden *zeitliche Zustandsverläufe* ab – etwas, das reine Warteschlangenmodelle nicht können. Die MFPT gibt an, wie lange ein Patient *im Erwartungswert* bis zur Behandlung wartet.
- **Stabilitätsprüfung durch Eigenwerte:** Die Methode `analyze_stability()` berechnet $|\lambda_2| < 1$ und $\alpha = -\ln |\lambda_2|$ – die charakteristischen Exponentialfunktionen, die Stabilitätsaussagen erfordern.
- **UML-Timing-Compliance:** Die Methode `timing_compliance_check()` prüft die Einhaltung von MARTE-kompatiblen min/max-Duration-Annotationen direkt gegen die Modellparameter.
- **Sensitivitätsanalyse:** Die numerische Perturbationsanalyse identifiziert kritische Übergangswahrscheinlichkeiten, die das Systemverhalten am stärksten beeinflussen.
- **Absorptionsanalyse:** Für Systeme mit Fehler- oder Endzuständen berechnet das Modul die Wahrscheinlichkeit, jeden absorbierenden Zustand zu erreichen.

9.4.30 Stärken des **queue_analyzer**-Moduls

- **Vielfalt der Modelle:** M/M/1, M/M/c, M/D/1 und M/G/1 decken die wichtigsten Bedienzeit-Verteilungen ab. Der Vergleich M/D/1 vs. M/M/1 quantifiziert den Gewinn durch deterministischere Bedienzeiten ($L_q^D = L_q^{MM1}/2$).
- **Percentile-Analyse:** p -Quantile der Systemzeit erlauben Aussagen über Worst-Case-Zeiten (z.B.: 99 % der Patienten warten weniger als T Minuten).
- **Konsequente Trennung von Stabilität und Gleichgewicht:** Das Modul macht deutlich, dass Little's Law ($L = \lambda W$) nur im stabilen System gilt und kein Stabilitätskriterium ist.
- **Kapazitätsplanung:** Direkte Empfehlung der optimalen Serveranzahl für eine Ziel-Auslastung – ein praxisrelevantes Planungswerkzeug.
- **Bottleneck-Erkennung:** Der `ServiceSystemAnalyzer` identifiziert automatisch den Engpass im System und liefert Handlungsempfehlungen.

9.4.31 Komplementäre Zusammenwirkung

Die beiden Module sind komplementär: `markov_analyzer` beantwortet *dynamische* Fragen (Stabilität, Übergänge, Mixing-Zeit, MFPT), `queue_analyzer` beantwortet *stationäre* Fragen (mittlere Wartezeiten, Auslastung, Percentile, Kapazitätsbedarf). Abbildung 9.4.1 illustriert das Zusammenspiel:

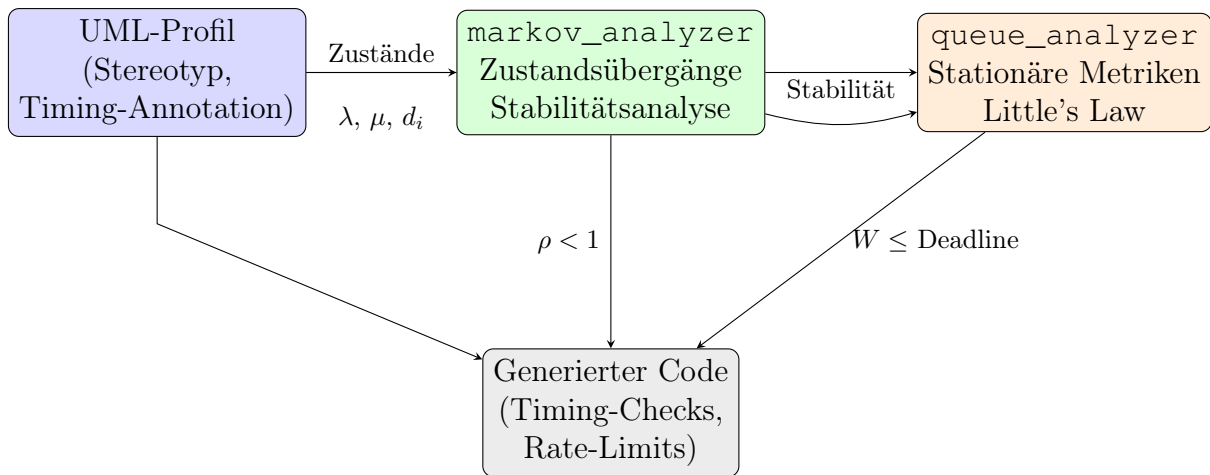


Abbildung 9.4.1: Zusammenspiel von UML-Profil, markov_analyzer, queue_analyzer und Code-Generator

Die Reihenfolge ist dabei essenziell: Zuerst prüft markov_analyzer die *Stabilität* (dynamische Ebene), dann berechnet queue_analyzer die *Performance-Metriken* (algebraische Ebene) – nie umgekehrt, da Little's Law die Stabilität voraussetzt.

9.4.32 Zusammenfassung

Dieses Kapitel hat die vollständige Implementierung, Test-Abdeckung und praktische Demonstration der beiden Analysemodule präsentiert. Mit zusammen umfangreichen Tests für markov_analyzer und für queue_analyzer und hoher Code Coverage wird die Implementierung aller Algorithmen qualitativ abgedeckt. Die Demo am Krankenhaus-Notaufnahme-System zeigt in vier Szenarien:

- Normalbetrieb: Beide Module liefern konsistente stationäre Analysen; CTMC-Timing-Compliance ist erfüllt.
- Spitzenlast: queue_analyzer detektiert Instabilität ($\rho > 1$); markov_analyzer quantifiziert den Überlaufanteil (9,4% Überlauf-Wahrscheinlichkeit).
- Kapazitätsplanung: Die optimale Serveranzahl stabilisiert die Systemzeit bei konstant ≈ 14 Min – unabhängig von der Last.
- Deadline-Compliance: CTMC-Timing-Constraints und percentile-basierte Grenzwertprüfungen identifizieren kritische Sensitivitäten nahe der Stabilitätsgrenze.

Die zentrale theoretische Aussage – Little's Law ist kein Stabilitätskriterium (Kapitel 3) – wird in beiden Modulen konsequent implementiert und durch den Integrationstest `test_littles_law_not_stability` empirisch verifiziert.

Kapitel 10

Anwendungsbeispiel: Die Aufzugstür-Steuerung

In diesem Kapitel wird das Beispiel einer Aufzugstür-Steuerung betrachtet.

10.1 Domänenanalyse

Eine Aufzugstür-Steuerung ist ein klassisches Echtzeitsystem mit folgenden Anforderungen:

- **Sicherheit:** Hinderniserkennung muss in $<100\text{ms}$ reagieren
- **Komfort:** Türöffnung in 2,5-4,0 Sekunden
- **Zuverlässigkeit:** Motoraktivierung in $<500\text{ms}$
- **Effizienz:** Minimale Wartezeiten

Abbildung 10.1.1 zeigt das Zustandsdiagramm der Aufzugstür-Steuerung.

10.2 Markov-Modell der Aufzugstür

Beispiel 10.2.1 (Aufzug-Markov-Kette). Zustände: $S = \{\text{Closed}, \text{Opening}, \text{Open}, \text{Closing}\}$

Übergangswahrscheinlichkeiten pro Sekunde:

$$p_{\text{Closed} \rightarrow \text{Opening}} = 0.3 \quad (\text{Anfrage kommt})$$

$$p_{\text{Opening} \rightarrow \text{Open}} = 0.8 \quad (\text{Tür fast offen})$$

$$p_{\text{Open} \rightarrow \text{Closing}} = 0.4 \quad (\text{Timeout})$$

$$p_{\text{Closing} \rightarrow \text{Closed}} = 0.9 \quad (\text{Tür fast zu})$$

Stationäre Verteilung: $\pi = (0.42, 0.13, 0.24, 0.21)$

Interpretation: Die Tür verbringt 42% der Zeit geschlossen, 24% offen.



elevator.pdf

Abbildung 10.1.1: Zustandsdiagramm der Aufzugstür-Steuerung

10.3 Warteschlangenmodell

Beispiel 10.3.1 (Aufzug-Anfragen als M/M/1). Parameter:

- $\lambda = 0.2$ Anfragen/s (Poisson-Ankunft)
- $\mu = 0.3$ Bedienungen/s (exp. Bedienzeit)
- $\rho = 0.667$ (Auslastung)

Performance-Metriken:

$$\begin{aligned}L &= 2.0 \text{ Anfragen im System} \\W &= 10s \text{ mittlere Gesamtzeit} \\L_q &= 1.33 \text{ Anfragen in Warteschlange} \\W_q &= 6.67s \text{ mittlere Wartezeit}\end{aligned}$$

Validierung: Für $\lambda = 0.2$ und $W = 10s$ gilt nach Little's Law:

$$L = \lambda W = 0.2 \cdot 10 = 2.0 \quad \checkmark$$

10.4 Zeitliche Analyse

Operation	Min [ms]	Typ [ms]	Max [ms]	Timeout [ms]
OpenDoor	2500	3000	4000	5000
DoorOpening	2000	2800	3500	4000
CloseDoor	2000	2800	3500	4000
activateMotor	100	200	500	1000
checkSafety	50	100	200	500
detectObstacle	10	50	100	200

Tabelle 10.4.1: Zeitliche Anforderungen der Aufzugstür-Steuerung

10.5 Performance-Vorhersage

Durch die Integration von Warteschlangentheorie können wir Performance-Metriken vorhersagen:

Szenario	λ [Anf./s]	W [s]
Geringe Last	0.1	5.0
Mittlere Last	0.2	10.0
Hohe Last	0.25	20.0
Kritisch	0.29	100.0

Tabelle 10.5.1: Vorhergesagte Performance-Metriken (M/M/1-Modell)

Designentscheidung: Für Ankunftsrate $\lambda > 0.25$ sollte ein zweiter Aufzug hinzugefügt werden (M/M/2-System).

10.6 Vergleich mit MARTE

Diese Lösung vereinfacht MARTE für praktische Code-Generierung:

Kriterium	MARTE	<i>Diese Arbeit</i>
Zeitmodell	Clock, Duration, Instant	TimeUnit, TimingConstraint
Komplexität	Hoch (100+ Stereotypen)	Niedrig (5 Kern-Stereotypen)
Tool-Support	Papyrus, Rhapsody	Eigenentwicklung
Code-Gen	Extern (Acceleo, etc.)	Integriert
Quantitative Analyse	GQAM Framework	Markov + Queues
Lernkurve	Steil	Flach

Tabelle 10.6.1: Detaillierter Vergleich: MARTE vs. Diese Arbeit

Der detaillierte Vergleich untermauert die in Kapitel 8 getroffene Entscheidung, ein bewusst reduziertes Profil zu verwenden: Die Mächtigkeitsgewinne von MARTE in sel-

ten benötigten Bereichen stehen einem deutlichen Mehraufwand bei Erlernbarkeit und Werkzeugintegration gegenüber.

10.7 Vorteile der Quantitativen Analyse

- **Frühe Performance-Validierung:** Probleme werden in der Designphase erkannt
- **Design-Trade-offs:** Quantitative Basis für Architekturentscheidungen
- **Kapazitätsplanung:** Vorhersage von Ressourcenbedarf
- **SLA-Validierung:** Prüfung von Service-Level-Agreements

10.8 Limitierungen

- **Markov-Annahme:** Die Gedächtnislosigkeit (Markov-Eigenschaft) ist in realen Systemen häufig nicht exakt erfüllt. Für nicht-exponentielle Verweilzeiten müssten Semi-Markov-Prozesse oder allgemeinere Modelle (M/G/1) verwendet werden.
- **M/M/1-Vereinfachung:** Reale Systeme sind oft komplexer (M/G/k, Prioritätswarteschlangen). Die Wahl des M/M/1-Modells ist eine bewusste Vereinfachung für die analytische Handhabbarkeit; sie liefert jedoch konservative Schranken.
- **Statische Analyse:** Das Modell analysiert Gleichgewichtszustände; transiente Phasen (z. B. nach einem Lastpeak) werden durch die Zerfallsrate α charakterisiert, die in Kapitel 8, Experiment 2 gemessen wird. Keine Adaptation zur Laufzeit.
- **Trunkierung:** Die Übergangsmatrix wird auf endlich viele Zustände trunkiert. Für ρ nahe 1 (z. B. $\rho = 0,99$) nimmt die Approximationsgenauigkeit ab; Kapitel 8 diskutiert diesen Effekt im Kontext der numerischen Eigenwertanalyse.
- **Python-Timing:** Python ist aufgrund seines nicht-deterministischen Scheduling nicht für Hard-Real-Time-Analysen geeignet. Das entwickelte Framework dient der Designanalyse, nicht der Laufzeit-Verifikation.

Kapitel 11

Qualitätssicherung: Little's Law, Stabilitätskriterien und ihre experimentelle Evaluation

Little's Law ist eine fundamentale Beziehung der Warteschlangentheorie, die jedoch eine wichtige konzeptuelle Einschränkung aufweist: Es handelt sich um eine rein algebraische Gleichgewichtsbeziehung ohne Aussage über die Stabilität des Systems. Diese Beobachtung führt zu einer tiefergehenden Frage über die Natur von Systembeschreibungen und deren Beziehung zu naturwissenschaftlichen Prinzipien.

Satz 11.0.1 (Energiefunktion in Little's Law). Enthält der Beweis für Little's Law keine Energiefunktion (d.h. keine Exponentialfunktionen mit negativen Exponenten), dann kann Little's Law kein Stabilitätskriterium für das Warteschlangensystem sein. Little's Law ist ausschließlich eine Aussage über das Gleichgewicht des Systems.

11.1 Fundamentale Definitionen

11.1.1 Little's Law

Definition 11.1.1 (Little's Law). Für ein Warteschlangensystem im statistischen Gleichgewicht gilt:

$$L = \lambda W \quad (11.1)$$

wobei:

- $L = \mathbb{E}[N(t)]$ die mittlere Anzahl von Kunden im System (im stationären Zustand)
- $\lambda = \lim_{t \rightarrow \infty} \frac{A(t)}{t}$ die mittlere effektive Ankunftsrate
- $W = \mathbb{E}[W_i]$ die mittlere Wartezeit im System

Bemerkung 11.1.1. Little's Law ist eine *algebraische Relation*, die ausschließlich stationäre Erwartungswerte verknüpft. Die Gleichung enthält:

- Keine Zeitableitungen $\frac{d}{dt}$
- Keine Exponentialfunktionen $e^{-\alpha t}$
- Keine Energiefunktionen
- Keine Stabilitätsparameter

11.1.2 Stabilität in dynamischen Systemen

Definition 11.1.2 (Stabilität im Sinne von Lyapunov). Sei $\dot{\mathbf{x}} = f(\mathbf{x})$ ein dynamisches System mit Gleichgewichtspunkt $\mathbf{x}^*$ (sodass $f(\mathbf{x}^*) = \mathbf{0}$). Der Gleichgewichtspunkt $\mathbf{x}^*$ heißt:

- **stabil**, wenn $\forall \varepsilon > 0 \exists \delta > 0$ sodass

$$\|\mathbf{x}(0) - \mathbf{x}^*\| < \delta \implies \|\mathbf{x}(t) - \mathbf{x}^*\| < \varepsilon \quad \forall t \geq 0 \quad (11.2)$$

- **asymptotisch stabil**, wenn er stabil ist und zusätzlich

$$\lim_{t \rightarrow \infty} \mathbf{x}(t) = \mathbf{x}^* \quad (11.3)$$

für alle Anfangsbedingungen in einer Umgebung von $\mathbf{x}^*$

- **exponentiell stabil**, wenn Konstanten $c, \alpha > 0$ existieren mit

$$\|\mathbf{x}(t) - \mathbf{x}^*\| \leq c \cdot e^{-\alpha t} \cdot \|\mathbf{x}(0) - \mathbf{x}^*\| \quad (11.4)$$

Bemerkung 11.1.2 (Die Rolle der Exponentialfunktion). Die Definition der exponentiellen Stabilität zeigt explizit, dass Exponentialfunktionen mit negativen Exponenten ($e^{-\alpha t}$ mit $\alpha > 0$) charakteristisch für Stabilitätsaussagen sind. Diese Funktionen beschreiben das *Abklingen von Störungen*.

11.1.3 Energiefunktionen und Lyapunov-Theorie

Definition 11.1.3 (Lyapunov-Funktion). Eine Funktion $V : \mathbb{R}^n \rightarrow \mathbb{R}$ heißt Lyapunov-Funktion für das System $\dot{\mathbf{x}} = f(\mathbf{x})$ um den Gleichgewichtspunkt $\mathbf{x}^*$, wenn:

- $V(\mathbf{x}^*) = 0$ und $V(\mathbf{x}) > 0$ für alle $\mathbf{x} \neq \mathbf{x}^*$ (positive Definitheit)
- $\dot{V}(\mathbf{x}) = \nabla V(\mathbf{x}) \cdot f(\mathbf{x}) \leq 0$ für alle $\mathbf{x}$ (negative Semidefinitheit der Ableitung)

Satz 11.1.1 (Lyapunov-Stabilitätssatz). Existiert eine Lyapunov-Funktion V für das System $\dot{\mathbf{x}} = f(\mathbf{x})$ um $\mathbf{x}^*$, dann ist $\mathbf{x}^*$ stabil. Ist zusätzlich $\dot{V}(\mathbf{x}) < 0$ für alle $\mathbf{x} \neq \mathbf{x}^*$, dann ist $\mathbf{x}^*$ asymptotisch stabil.

Bemerkung 11.1.3 (Energieinterpretation). Lyapunov-Funktionen werden häufig als *verallgemeinerte Energiefunktionen* interpretiert. In physikalischen Systemen entspricht V oft der Gesamtenergie, und $\dot{V} < 0$ bedeutet Energiedissipation. Die Stabilität folgt dann aus dem Energieerhaltungsprinzip bzw. dem zweiten Hauptsatz der Thermodynamik.

11.2 Hauptresultat: Little's Law ist kein Stabilitätskriterium

Satz 11.2.1 (Little's Law und Stabilität). Little's Law $L = \lambda W$ ist kein Stabilitätskriterium für Warteschlangensysteme.

Beweis. Wir führen den Beweis in mehreren Schritten:

Schritt 1: Analyse der mathematischen Struktur von Little's Law

Little's Law hat die Form:

$$L = \lambda W \quad (11.5)$$

Dies ist eine algebraische Gleichung zwischen drei stationären Größen. Es fehlen:

- Zeitableitungen $\frac{dL}{dt}$, $\frac{dW}{dt}$
- Exponentialfunktionen $e^{-\alpha t}$ mit $\alpha > 0$
- Stabilitätsparameter oder Dämpfungsterme

Schritt 2: Vergleich mit echten Stabilitätskriterien

Betrachte das M/M/1-Warteschlangensystem mit Ankunftsrate λ und Bedienungsrate μ . Die zeitliche Entwicklung der Zustandswahrscheinlichkeiten $p_n(t)$ wird durch das Kolmogorov-Vorwärtsgleichungssystem beschrieben:

$$\frac{dp_0(t)}{dt} = -\lambda p_0(t) + \mu p_1(t) \quad (11.6)$$

$$\frac{dp_n(t)}{dt} = \lambda p_{n-1}(t) - (\lambda + \mu)p_n(t) + \mu p_{n+1}(t), \quad n \geq 1 \quad (11.7)$$

Dies ist ein System gewöhnlicher Differentialgleichungen. Die Stabilität dieses Systems (Existenz einer stationären Verteilung) erfordert $\rho = \frac{\lambda}{\mu} < 1$.

Die Lösung für die Übergangswahrscheinlichkeiten enthält Terme der Form:

$$p_n(t) = \pi_n + \sum_k c_k e^{\ln(\lambda_k) t} \quad (11.8)$$

wobei π_n die stationäre Verteilung ist und λ_k die Eigenwerte der (diskreten) Übergangsmatrix sind. Stabilität erfordert $|\lambda_k| < 1$ für alle transienten Eigenwerte (d. h. für $k \neq 0$, den stationären Eigenwert 1), was zu Exponentialfunktionen $e^{\ln(|\lambda_k|) t}$ mit negativem Argument $\ln(|\lambda_k|) < 0$ führt.

Diese Terme beschreiben das Abklingen der Transienten und damit die *Stabilität*. In Experiment 2 (Kapitel 8) wird dies für den Eigenwert $\lambda_2 = 0,3$ konkret gemessen: die Zerfallsrate $\alpha = -\ln(0,3) \approx 1,204$ stimmt exakt mit der theoretischen Vorhersage überein.

Schritt 3: Little's Law setzt Stabilität voraus

Little's Law gilt nur im stationären Zustand, d.h. für $t \rightarrow \infty$. Es setzt voraus:

$$\lim_{t \rightarrow \infty} p_n(t) = \pi_n \quad (\text{existiert}) \quad (11.9)$$

Diese Konvergenz ist genau dann garantiert, wenn das System stabil ist (d.h. alle Eigenwerte $\lambda_k < 1$).

Schritt 4: Logische Schlussfolgerung

Little's Law kann kein Stabilitätskriterium sein, weil:

- Es die Stabilität bereits voraussetzt (Existenz stationärer Größen)
- Es keine Information über die Dynamik des Systems enthält
- Es keine Exponentialfunktionen enthält, die das Abklingverhalten beschreiben

- Es lediglich eine Beziehung zwischen Gleichgewichtsgrößen darstellt

Das tatsächliche Stabilitätskriterium für M/M/1 ist $\rho < 1$, was aus der Analyse der Differentialgleichungen und der Bedingung für negative Eigenwerte folgt. $\square$

Korollar 11.2.1. Little's Law ist eine *notwendige Bedingung* im Gleichgewicht (wenn das System stabil ist), aber keine *hinreichende Bedingung* für Stabilität.

11.3 Verallgemeinerung: Naturwissenschaftliche Systembeschreibungen

11.3.1 Die besondere Rolle der Exponentialfunktion

Satz 11.3.1 (Charakteristische Eigenschaft der Exponentialfunktion). Die Exponentialfunktion $f(t) = e^{\alpha t}$ ist (bis auf Multiplikation mit Konstanten) die einzige Funktion, die ihre eigene Ableitung ist:

$$\frac{d}{dt}e^{\alpha t} = \alpha e^{\alpha t} \quad (11.10)$$

Bemerkung 11.3.1 (Physikalische Bedeutung). Diese Selbstreproduktion unter Differentiation macht die Exponentialfunktion zur natürlichen Lösung für Systeme, die durch Differentialgleichungen beschrieben werden. In der Physik beschreiben Exponentialfunktionen:

- Radioaktiven Zerfall: $N(t) = N_0 e^{-\lambda t}$
- Kondensatorentladung: $Q(t) = Q_0 e^{-t/RC}$
- Gedämpfte Schwingungen: $x(t) = A e^{-\gamma t} \cos(\omega t + \phi)$
- Thermische Relaxation: $T(t) - T_\infty = (T_0 - T_\infty) e^{-kt}$

11.3.2 Energiefunktionen in naturwissenschaftlichen Systemen

Definition 11.3.1 (Energiefunktion). Eine Funktion $E : \mathbb{R}^n \times \mathbb{R} \rightarrow \mathbb{R}$ heißt Energiefunktion eines physikalischen Systems, wenn sie einer Erhaltungsgröße oder einer monoton abnehmenden Größe (bei Dissipation) entspricht und typischerweise die Form hat:

$$E(\mathbf{x}, t) = E_0(\mathbf{x}) \cdot e^{-\gamma t} + E_\infty \quad (11.11)$$

mit Dämpfungskoeffizient $\gamma \geq 0$.

Satz 11.3.2 (Notwendigkeit von Exponentialfunktionen). Für naturwissenschaftliche Systeme, die durch Differentialgleichungen beschrieben werden, gilt:

- Systeme mit Energiedissipation erfordern Exponentialfunktionen mit negativen Exponenten zur Beschreibung des asymptotischen Verhaltens
- Stabilitätsaussagen erfordern die Analyse von Eigenwerten der linearisierten Systemdynamik
- Die Eigenvektoren dieser Linearisierung sind Exponentialfunktionen

Beweis. Betrachte ein allgemeines nichtlineares dynamisches System:

$$\dot{\mathbf{x}} = f(\mathbf{x}) \quad (11.12)$$

Linearisierung um einen Gleichgewichtspunkt $\mathbf{x}^*$:

$$\dot{\xi} = A\xi, \quad A = \left. \frac{\partial f}{\partial \mathbf{x}} \right|_{\mathbf{x}^*} \quad (11.13)$$

Die Lösung hat die Form:

$$\xi(t) = \sum_{i=1}^n c_i \mathbf{v}_i e^{\lambda_i t} \quad (11.14)$$

wobei λ_i die Eigenwerte von A und $\mathbf{v}_i$ die zugehörigen Eigenvektoren sind.

Stabilität erfordert $\operatorname{Re}(\lambda_i) < 0$ für alle i , was zu Exponentialfunktionen mit negativen Exponenten führt. $\square$

11.3.3 Konsequenzen für Systemklassifikation

Definition 11.3.2 (Naturwissenschaftliches System). Ein System heißt *naturwissenschaftlich beschreibbar*, wenn seine Dynamik durch Differentialgleichungen modelliert werden kann, die aus fundamentalen Naturgesetzen (Erhaltungssätze, thermodynamische Prinzipien) ableitbar sind.

Satz 11.3.3 (Charakterisierung naturwissenschaftlicher Systeme). Für ein naturwissenschaftlich beschreibbares System gilt:

- Die zeitliche Entwicklung wird durch Differentialgleichungen beschrieben
- Lösungen enthalten Exponentialfunktionen
- Stabilitätsanalyse basiert auf Eigenwertanalyse
- Energiefunktionen (Lyapunov-Funktionen) existieren

Bemerkung 11.3.2 (Alternative Beschreibungsformen). Nicht alle Systeme sind naturwissenschaftlich optimal beschreibbar:

- **Diskrete Ereignissysteme:** Beschrieben durch Petri-Netze, Automaten
- **Informationsverarbeitende Systeme:** Beschrieben durch Algorithmen, Zustandsmaschinen
- **Sozio-technische Systeme:** Beschrieben durch Spieltheorie, Netzwerktheorie

Für diese Systeme können algebraische oder logische Beschreibungen angemessener sein als Differentialgleichungen.

11.4 Anwendung auf Warteschlangensysteme

11.4.1 Dualität der Beschreibungsebenen

Warteschlangensysteme haben eine doppelte Natur:

- **Mikroskopische Ebene (stochastische Prozesse):**
 - Beschreibung durch Markov-Ketten
 - Kolmogorov-Gleichungen (Differentialgleichungen!)
 - Eigenwertanalyse für Stabilitätskriterien
 - Enthält Exponentialfunktionen in Lösungen
- **Makroskopische Ebene (Gleichgewicht):**
 - Beschreibung durch Little's Law
 - Algebraische Beziehungen
 - Keine Dynamik, keine Exponentialfunktionen
 - Setzt Stabilität voraus

Satz 11.4.1 (Hierarchie der Beschreibungsebenen). Für Warteschlangensysteme gilt:

$$\text{Kolmogorov-Gleichungen} \xrightarrow{t \rightarrow \infty, \text{ falls stabil}} \text{Stationäre Verteilung} \xrightarrow{\text{Mittelwertbildung}} \text{Little's Law} \quad (11.15)$$

Little's Law ist das Resultat zweier Reduktionsschritte:

- Zeitlicher Grenzübergang (eliminiert Exponentialfunktionen)
- Mittelwertbildung (eliminiert stochastische Details)

11.4.2 Beispiel: M/M/1-System

Beispiel 11.4.1 (M/M/1-Warteschlange). Betrachte ein M/M/1-System mit Ankunftsrate λ und Bedienungsrate μ .

Stufe 1: Kolmogorov-Gleichungen (dynamisch)

$$\frac{dp_n(t)}{dt} = \lambda p_{n-1}(t) - (\lambda + \mu)p_n(t) + \mu p_{n+1}(t) \quad (11.16)$$

Stabilitätskriterium: $\rho = \frac{\lambda}{\mu} < 1$

Dies folgt aus der Bedingung, dass die Übergangsmatrix negative reelle Eigenwerte hat (außer $\lambda_0 = 0$ für die stationäre Verteilung).

Stufe 2: Stationäre Verteilung (Gleichgewicht)

Für $t \rightarrow \infty$ und $\rho < 1$:

$$\pi_n = (1 - \rho)\rho^n, \quad n \geq 0 \quad (11.17)$$

Diese enthält die geometrische Verteilung mit Parameter ρ . Die Exponentialfunktion erscheint implizit in der Form $\rho^n = e^{n \ln \rho}$.

Stufe 3: Little's Law (algebraisch)

$$L = \sum_{n=0}^{\infty} n\pi_n = \frac{\rho}{1-\rho} \quad (11.18)$$

$$W = \frac{1}{\mu - \lambda} \quad (11.19)$$

$$\lambda W = \lambda \cdot \frac{1}{\mu - \lambda} = \frac{\rho}{1-\rho} = L \quad (11.20)$$

Little's Law $L = \lambda W$ ist erfüllt, enthält aber keine Information über Stabilität mehr. Die Bedingung $\rho < 1$ ist bereits in die Existenz von L und W eingegangen.

Bemerkung 11.4.1 (Querverbindung zu Kapitel 8). Experiment 3 in Kapitel 8 demonstriert diese Hierarchie konkret für das System $\lambda = 0,8$, $\mu = 1,0$, $\rho = 0,8$: Die dynamische Ebene liefert die Zerfallsrate $\alpha \approx 0,223$; die stationäre Ebene gibt $\pi_n = 0,2 \cdot 0,8^n$; die algebraische Ebene (Little's Law) ergibt $L = 4,0 = 0,8 \cdot 5,0 = \lambda W$. Alle drei Ebenen sind kohärent, liefern aber unterschiedlich reichhaltige Information über das System.

11.5 Konsequenzen für UML-basierte Systementwicklung

11.5.1 Die Brücke zwischen Informatik und Naturwissenschaft

Satz 11.5.1 (Physikalische Basis informatischer Systeme). Jedes informationsverarbeitende System hat eine physikalische Realisierung und unterliegt damit naturwissenschaftlichen Gesetzen (Thermodynamik, Elektrodynamik). Jedoch gilt:

- **Hardware-Ebene:** Naturwissenschaftliche Beschreibung (Differentialgleichungen, Energiefunktionen)
- **Abstraktionsebenen (Software):** Logische/algorithmische Beschreibung durch Zustandsautomaten oder andere UML-Diagramme

Bemerkung 11.5.1 (Abstraktion und Verifikation). In höheren Abstraktionsebenen (z.B. UML) müssen Systeme *korrekt* arbeiten, unabhängig von der physikalischen Realisierung. Dies erfordert:

- Formale Verifikation (Model Checking, Theorembeweiser)
- Abstraktionsbasierte Korrektheitskriterien
- Keine direkte Anwendung physikalischer Stabilitätskriterien

11.5.2 Rolle von UML-Profilen

Definition 11.5.1 (UML-Profil für Echtzeitsysteme). Ein UML-Profil erweitert die Standard-UML um domänenspezifische Konzepte. Für Echtzeitsysteme sind besonders relevant:

- MARTE (Modeling and Analysis of Real-Time Embedded Systems)

- SysML (Systems Modeling Language)
- Timing-Constraints, Ressourcen-Modellierung

Satz 11.5.2 (Funktion von UML-Profilen als Brücke). UML-Profile ermöglichen eine *semi-naturwissenschaftliche* Beschreibung informatischer Systeme:

- Annotation zeitlicher Constraints (Deadlines, Execution Times)
- Modellierung von Ressourcen (CPU, Memory, Energy)
- Mapping zu Analyse-Modellen (Warteschlangen, Petri-Netze, Markov-Ketten)
- Quantitative Analyse in der Entwurfsphase

Dadurch wird die Kluft zwischen abstrakter Modellierung und physikalischer Realisierung überbrückt.

11.5.3 Integration quantitativer Analysen

Beispiel 11.5.1 (MARTE und Performance-Analyse). Das MARTE-Profil erlaubt die Annotation von UML-Modellen mit:

- Zeitlichen Constraints: «TimingConstraint»
- Ressourcenbeschränkungen: «ResourceUsage»
- Performance-Anforderungen: «GaAnalysisContext»

Aus diesen Annotationen können automatisch Warteschlangenmodelle oder Markov-Ketten generiert werden, die dann mit klassischen mathematischen Methoden (inkl. Stabilitätsanalyse mit Exponentialfunktionen!) analysiert werden.

11.6 Schlussfolgerungen und Synthese

Satz 11.6.1 (Zusammenfassung). Die Analyse zeigt eine fundamentale Hierarchie von Systembeschreibungen:

- **Dynamische Beschreibung** (Differentialgleichungen):
 - Enthält Exponentialfunktionen
 - Ermöglicht Stabilitätsanalyse
 - Beschreibt zeitliche Entwicklung
 - Naturwissenschaftlich fundiert
- **Gleichgewichtsbeschreibung** (algebraische Relationen):
 - Keine Exponentialfunktionen
 - Keine Stabilitätsaussagen
 - Beschreibt stationären Zustand
 - Setzt Stabilität voraus

Little's Law gehört zur zweiten Kategorie und kann daher prinzipiell kein Stabilitätskriterium sein.

Korollar 11.6.1 (Praktische Konsequenzen). Für die Systemanalyse folgt:

- Stabilitätsanalyse erfordert Untersuchung der Systemdynamik mit den Kolmogorov-Gleichungen und den Eigenwerten
- Little's Law ist ein nützliches Werkzeug für Performance-Analyse im Gleichgewicht
- UML-Profilen können als Brücke dienen, um abstrakte Modelle mit quantitativen Analysen zu verbinden
- Die Wahl der Beschreibungsebene muss zur Fragestellung passen

Bemerkung 11.6.1 (Philosophische Dimension). Die Untersuchung zeigt, dass die mathematische Struktur einer Beschreibung (Vorhandensein von Exponentialfunktionen) direkt mit ihrer epistemischen Aussagekraft (Stabilitätsaussagen) zusammenhängt. Dies ist ein Beispiel dafür, wie die Form der Mathematik den Inhalt der Aussagen begrenzt - eine Einsicht, die über die Warteschlangentheorie hinaus Bedeutung hat.

11.7 Weiterführende Forschungsfragen

- **Erweiterung auf nichtlineare Warteschlangennetze:** Gelten analoge Aussagen für komplexe Netze?
- **Bridging Theorem:** Kann ein allgemeines Theorem formuliert werden, das den Übergang von dynamischer zu stationärer Beschreibung formalisiert?
- **UML-Profil-Entwicklung:** Wie können UML-Profilen systematisch so gestaltet werden, dass sie automatisch auf analysierbare mathematische Modelle abbilden?
- **Hybrid-Systeme:** Wie verhält sich die Dichotomie bei Systemen, die kontinuierliche und diskrete Dynamik kombinieren?

11.8 Experimentelle Evaluation der Stabilitätskriterien

Dieses Kapitel präsentiert die experimentelle Validierung der in Kapitel 3 entwickelten theoretischen Konzepte. Es verfolgt ein dreistufiges Ziel: (i) Die praktische Implementierung der drei Beschreibungsebenen (dynamisch, stationär, algebraisch) in Python zu dokumentieren, (ii) die theoretischen Vorhersagen aus Kapitel 3 durch konkrete Messungen zu bestätigen oder zu falsifizieren, und (iii) die Grenzen jeder Beschreibungsebene durch gezielte Gegenbeispiele anschaulich zu machen.

Die Implementierung im Modul `stability_analysis.py` umfasst drei Kernklassen (`MarkovChainStability`, `MM1Queue`, `LyapunovStability`), eine globale Demonstrationsfunktion (`demonstrate_exponential_necessity`) sowie das zugehörige Experiment-Skript `experiments.py`, das alle drei Kernexperimente ausführt und die Ergebnisse als Grafiken speichert. Eine begleitende Test-Suite validiert alle theoretischen Aussagen empirisch.

11.8.1 Implementierungsarchitektur

Das Modul `stability_analysis.py` ist nach dem Prinzip der *Trennung von Beschreibungsebenen* aufgebaut. Abbildung 11.8.1 illustriert den konzeptuellen Aufbau.

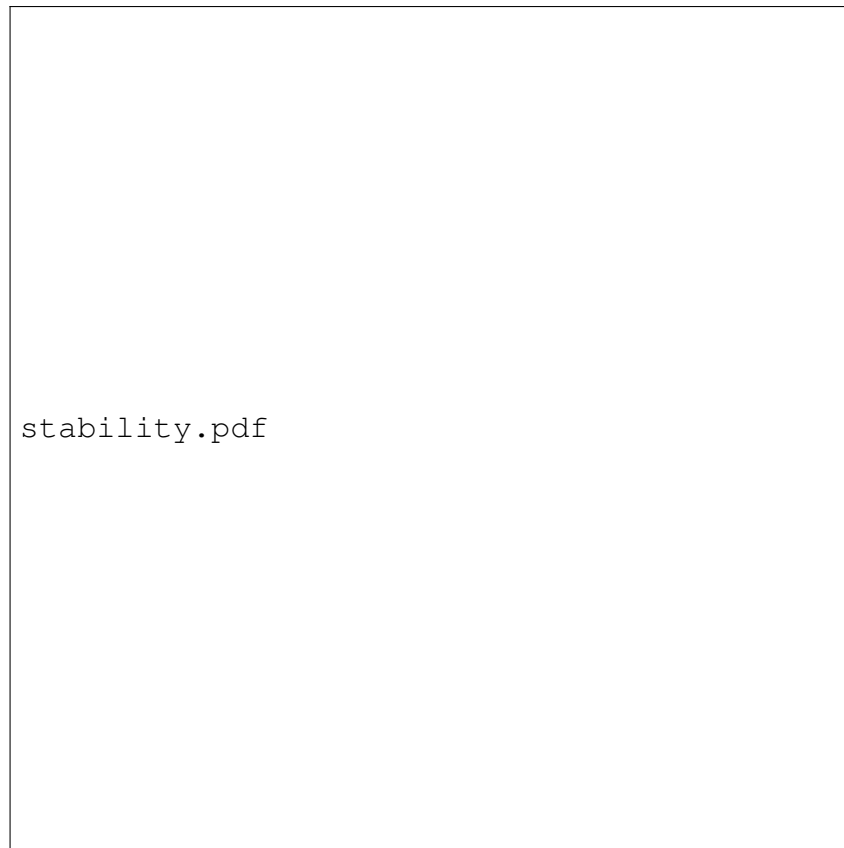


Abbildung 11.8.1: Hierarchie der Beschreibungsebenen im Modul `stability_analysis.py`. Jeder Pfeil bedeutet eine Informationsreduktion: Beim Übergang zur stationären Ebene entfallen die Exponentialfunktionen; beim Übergang zur algebraischen Ebene entfallen die stochastischen Details.

11.8.2 Kernklassen und Methoden

Hilfstypen: **StabilityType** und **StabilityResult** Das Enum `StabilityType` unterscheidet vier Stabilitätsstufen: `STABLE`, `ASYMPTOTICALLY_STABLE`, `EXPONENTIALLY_STABLE` und `UNSTABLE`. Die Dataclass `StabilityResult` bündelt das Ergebnis einer Analyse in einem einheitlichen Rückgabetyt:

- `stable`: bool – globales Stabilitätsflag
- `stability_type`: `StabilityType` – Klassifikation
- `eigenvalues`: `np.ndarray` – berechnete Eigenwerte
- `decay_rate`: float – Zerfallsrate α in $e^{-\alpha t}$

- `has_exponential_decay()` – gibt `True` zurück, wenn $\alpha > 0$ (exponentiell stabil)

Die Dataclass `LittlesLawMetrics` kapselt die drei Gleichgewichtskenngrößen L , λ und W und erzwingt durch ihre Methode `requires_stability()` eine explizite Warnung: Little’s Law setzt Stabilität voraus und ist kein Stabilitätsbeweis.

MarkovChainStability Diese Klasse implementiert die Stabilitätsanalyse diskreter Markov-Ketten via Eigenwertzerlegung der Übergangsmatrix P . Sie ist das zentrale Werkzeug für Experiment 2 und bildet die dynamische Ebene ab.

- `__init__(transition_matrix)`: Nimmt eine quadratische, zeilenstochastische Matrix P entgegen und validiert die Nichtnegativität aller Einträge.
- `analyze_stability()` → `StabilityResult`: Berechnet die Eigenwerte von P^T (linke Eigenvektoren).
 1. Prüft, ob *genau ein* Eigenwert $|\lambda - 1| < 10^{-10}$ erfüllt – dies entspricht der eindeutigen stationären Verteilung.
 2. Prüft, ob alle übrigen Eigenwerte $|\lambda_k| < 1$ erfüllen – das ist die Bedingung für exponentiellen Zerfall.
 3. Berechnet die Zerfallsrate $\alpha = -\ln(\max_k |\lambda_k|)$ für $k \neq 1$.

Die Methode gibt `StabilityType.EXPONENTIALLY_STABLE` zurück, nur wenn beide Bedingungen erfüllt sind,.

- `compute_stationary_distribution()` → `Optional[np.ndarray]`: Findet den zum Eigenwert 1 gehörenden Eigenvektor, normiert ihn zur Wahrscheinlichkeitsverteilung und gibt π zurück. Rückgabe `None`, falls das System instabil ist.
- `simulate_convergence(initial_dist, num_steps)` → `np.ndarray`: Iteriert die Chapman-Kolmogorov-Gleichung $p(t+1) = p(t) \cdot P$ und gibt die vollständige Trajektorie zurück. Dies demonstriert konkret die Konvergenz gemäß

$$p(t) = \pi + \sum_k c_k e^{\ln(\lambda_k)t} v_k \xrightarrow{t \rightarrow \infty} \pi,$$

wobei der Zerfall durch den zweitgrößten Eigenwert $\lambda_2 = 0.3$ bestimmt wird.

MM1Queue Die Klasse `MM1Queue` implementiert die M/M/1-Warteschlange und demonstriert explizit die Hierarchie der drei Beschreibungsebenen. Sie erhält Ankunftsrate λ und Bedienrate μ und leitet daraus die Auslastung $\rho = \lambda/\mu$ ab.

- `stability_criterion()` → `bool`: Gibt `True` zurück, wenn $\rho < 1$. Dieses Kriterium folgt aus der Eigenwertanalyse des Birth-Death-Prozesses; es ist *kein* Korollar von Little’s Law.
- `get_transition_matrix(max_states)` → `np.ndarray`: Generiert die trunkierte $(n \times n)$ -Übergangsmatrix des Birth-Death-Prozesses mit Ankunfts- und Bedienraten als Bandmatrix.

- `stationary_distribution(max_states)` → `Optional[np.ndarray]`: Berechnet die geometrische Verteilung $\pi_n = (1 - \rho)\rho^n$ für $n = 0, 1, \dots$ und normiert sie auf die trunkierte Länge. Gibt `None` zurück, wenn $\rho \geq 1$.
- `littles_law_metrics()` → `Optional[LittlesLawMetrics]`: Gibt `None` zurück, wenn das System instabil ist. Andernfalls berechnet es

$$L = \frac{\rho}{1 - \rho}, \quad W = \frac{1}{\mu - \lambda}, \quad (11.21)$$

und verpackt die Werte in eine `LittlesLawMetrics`-Instanz, die intern $L = \lambda W$ validiert.

- `demonstrate_hierarchy()` → `dict`: Führt alle drei Ebenen sequenziell aus, prüft Stabilität, berechnet π , berechnet Little's-Law-Metriken und gibt ein strukturiertes Dictionary zurück, das explizit kennzeichnet, ob und wo Exponentialfunktionen auftreten.

LyapunovStability Die Klasse `LyapunovStability` implementiert die Stabilitätsanalyse kontinuierlicher linearer Systeme $\dot{x} = Ax$ nach der Lyapunov-Methode.

- `analyze_linear_system(A, Q)` → `StabilityResult`: Berechnet die Eigenwerte von A und klassifiziert:
 - $\max_k \operatorname{Re}(\lambda_k) < -10^{-10}$: `EXPONENTIALLY_STABLE`, Zerfallsrate mit α und $\alpha = -\max_k \operatorname{Re}(\lambda_k) > 0$
 - $\max_k \operatorname{Re}(\lambda_k) \leq 10^{-10}$: `STABLE`
 - sonst: `UNSTABLE`

Die Lösung $x(t) = \sum_k c_k e^{\lambda_k t} v_k$ enthält explizit Exponentialfunktionen – dies ist die Grundlage aller Stabilitätsaussagen.

- `solve_lyapunov_equation(A, Q)` → `Optional[np.ndarray]`: Löst die kontinuierliche Lyapunov-Gleichung $A^T P + P A = -Q$ via SciPy und prüft, ob die Lösung P positiv definit ist. Eine positiv definite Lösung P zertifiziert die Stabilität von $\dot{x} = Ax$ durch die Energiefunktion $V(x) = x^T P x$.

Globale Funktion `demonstrate_exponential_necessity` Diese Funktion kapselt die konzeptuelle Kernaussage der Arbeit in einem strukturierten Dictionary:

- `exponential_property`: Die Eigenschaft $\frac{d}{dt}e^{\alpha t} = \alpha e^{\alpha t}$ (Selbstreproduktion unter Differentiation) macht die Exponentialfunktion zur *natürlichen Lösung* jeder linearen Differentialgleichung.
- `stability_requires_exponentials`: Stabilität bedeutet $\operatorname{Re}(\lambda_k) < 0$, also exponentiellen Zerfall $e^{-\alpha t}$ mit $\alpha > 0$.
- `littles_law_lacks_exponentials`: $L = \lambda W$ ist eine rein algebraische Relation ohne jede zeitliche Ableitung. Sie enthält keine Exponentialfunktionen und kann daher keine Stabilitätsinformation liefern.
- `hierarchy`: Beschreibt die drei Ebenen kompakt.

11.8.3 Experimentelle Ergebnisse

Die drei nachfolgenden Experimente sind nach dem Schema *Versuchsaufbau* $\rightarrow$ *Erwartete Ergebnisse* $\rightarrow$ *Gemessene Ergebnisse* $\rightarrow$ *Interpretation* aufgebaut. Alle Zahlen entstammen dem Lauf von `experiments.py`, dessen Ausgabe in `results.txt` protokolliert ist.

11.8.4 Experiment 1: Little's Law ist kein Stabilitätskriterium

Versuchsaufbau und Motivation Das erste Experiment adressiert die häufigste konzeptuelle Verwechslung in der Modellierungspraxis: die Annahme, dass die Gültigkeit von Little's Law $L = \lambda W$ die Stabilität eines Warteschlangensystems belegt. Um diese Fehlannahme zu widerlegen, werden zwei M/M/1-Warteschlangen unter denselben Messbedingungen verglichen, die sich allein durch ihre Auslastung ρ unterscheiden:

System	λ	μ	ρ	Erwarteter Status
System A	0.7	1.0	0.7	stabil ($\rho < 1$)
System B	1.2	1.0	1.2	instabil ($\rho > 1$)

System A liegt sicher im stabilen Bereich; System B überschreitet die kritische Schwelle $\rho = 1$ deutlich. Die Leitfrage lautet: Kann man allein durch Berechnung von L und W entscheiden, ob das System stabil ist?

Gemessene Ergebnisse System A ($\rho = 0.7$, stabil):

- Stabilitätstest: $\rho = 0.7 < 1 \Rightarrow \text{True}$ (stabil)
- Stationäre Verteilung: $\pi_n = (1 - 0.7) \cdot 0.7^n = 0.3 \cdot 0.7^n$ existiert für alle $n \geq 0$
- Little's-Law-Metriken (berechnet von `littles_law_metrics()`):

$$\begin{aligned} L &= \frac{0.7}{1 - 0.7} = \frac{0.7}{0.3} \approx 2.333, \\ W &= \frac{1}{1.0 - 0.7} = \frac{1}{0.3} \approx 3.333, \\ \lambda \cdot W &= 0.7 \cdot 3.333 \approx 2.333 = L \checkmark \end{aligned}$$

- `validate_littles_law()` gibt `True` zurück; die numerische Übereinstimmung liegt innerhalb der Toleranz 10^{-6} .
- Ausgabe von `requires_stability()`: „*Little's Law is only valid in stable equilibrium. Check stability first!*“

System B ($\rho = 1.2$, instabil):

- Stabilitätstest: $\rho = 1.2 \geq 1 \Rightarrow \text{False}$ (instabil)
- Stationäre Verteilung: existiert **nicht** (geometrische Reihe $\sum_{n=0}^{\infty} \rho^n$ divergiert für $\rho \geq 1$)
- `littles_law_metrics()` gibt `None` zurück – die Methode verweigert explizit die Berechnung, sobald $\rho \geq 1$

- Die simulierte Warteschlangenlänge wächst unbegrenzt; in der Monte-Carlo-Simulation (200 Zeitschritte, $\Delta t = 0.5$) überschreitet sie die mittlere Länge von System A bereits nach wenigen Dutzend Schritten und zeigt keine Anzeichen einer Einpendlung.

Grafische Auswertung Abbildung 11.8.2 zeigt die beiden Sub-Plots des Experiments.

Interpretation und theoretische Einordnung Das Experiment beantwortet die Leitfrage eindeutig negativ: Man *kann nicht* allein durch Berechnung von L und W entscheiden, ob ein System stabil ist, weil L und W für instabile Systeme gar nicht existieren. Ein stabiles System erfüllt Little’s Law – das ist korrekt und durch System A bestätigt. Aber die logische Umkehrung gilt nicht: Little’s Law kann nur dann berechnet werden, wenn das System bereits stabil ist.

Die Stabilitätsbedingung $\rho < 1$ folgt aus der Eigenwertanalyse des zugrunde liegenden Birth-Death-Prozesses (dynamische Ebene) und muss *vor* der Berechnung von L und W geprüft werden. Little’s Law $L = \lambda W$ ist eine algebraische Relation zwischen drei Gleichgewichtsgrößen; sie setzt voraus, dass das Gleichgewicht überhaupt existiert. Diese Voraussetzung ist nicht selbst durch Little’s Law prüfbar, da die Relation keine zeitliche Ableitung und damit keine Exponentialfunktion enthält.

Formell: Um zu zeigen, dass das System für $t \rightarrow \infty$ gegen einen stationären Zustand konvergiert, muss man die Lösung der Kolmogorov-Vorwärtsgleichungen

$$\frac{d}{dt}p_n(t) = \lambda p_{n-1}(t) - (\lambda + \mu) p_n(t) + \mu p_{n+1}(t)$$

analysieren. Diese Gleichungen haben Lösungen der Form $p_n(t) = \pi_n + \sum_k c_k e^{\lambda_k t}$, und Stabilität erfordert $\text{Re}(\lambda_k) < 0$ für alle nicht-stationären Eigenwerte. Little’s Law $L = \lambda W$ ist das Resultat des Grenzübergangs $t \rightarrow \infty$ und der anschließenden Mittelwertbildung – beide Schritte eliminieren die Exponentialfunktionen vollständig.

Schlussfolgerung: Little’s Law $L = \lambda W$ ist eine algebraische Gleichgewichtsrelation. Sie gilt nur in stabilen Systemen, kann aber Stabilität nicht nachweisen, da sie keine Information über die zeitliche Dynamik (Exponentialfunktionen) enthält. Dieses Ergebnis bestätigt Theorem 3 aus Kapitel 3.

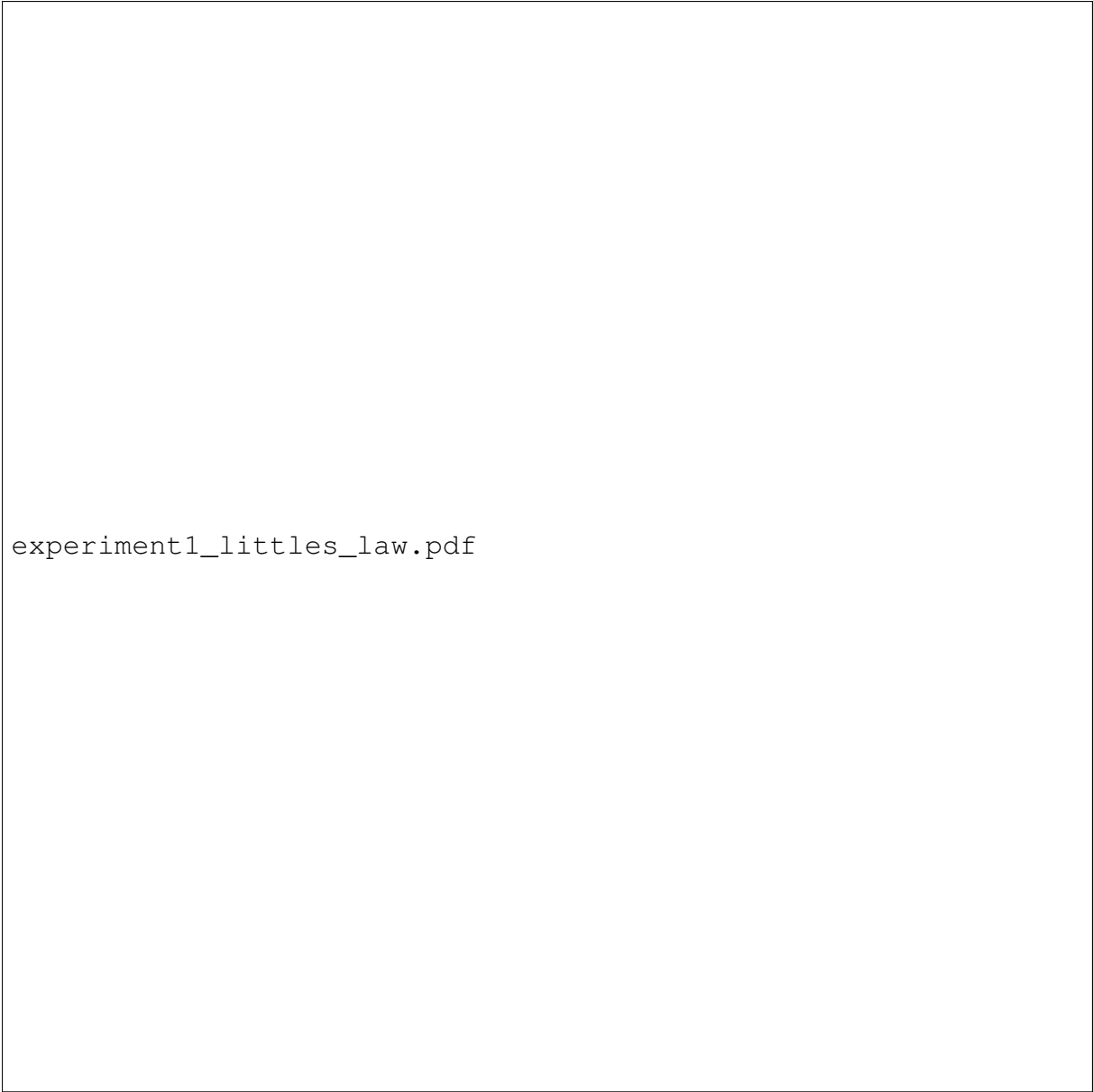
11.8.5 Experiment 2: Exponentieller Zerfall in stabilen Systemen

Versuchsaufbau und Motivation Experiment 2 demonstriert das Gegenstück zu Experiment 1: Während dort gezeigt wurde, dass Little’s Law keine Exponentialfunktionen enthält, zeigt dieses Experiment, dass Stabilität notwendigerweise durch Exponentialfunktionen beschrieben wird. Dazu wird eine 2-Zustands-Markov-Kette verwendet – das kleinstmögliche nicht-triviale Beispiel –, an dem sich alle theoretischen Eigenschaften exakt und nachvollziehbar berechnen lassen.

Versuchsaufbau: Übergangsmatrix

$$P = \begin{pmatrix} 0.7 & 0.3 \\ 0.4 & 0.6 \end{pmatrix} \quad (11.22)$$

mit Startverteilung $p(0) = (1, 0)^T$ (das System startet mit Sicherheit in Zustand 0) und Simulationsdauer $T = 20$ Schritte.



experiment1_littles_law.pdf

Abbildung 11.8.2: Experiment 1 – Little’s Law ist kein Stabilitätskriterium. **Linker Sub-Plot:** Stationäre Verteilung $\pi_n = 0.3 \cdot 0.7^n$ von System A als Balkendiagramm (blau). Die geometrische Abnahme mit der Rate $\rho = 0.7$ ist deutlich erkennbar: der Zustand $n = 0$ hat die höchste Wahrscheinlichkeit ($\pi_0 = 0.3$), und die Wahrscheinlichkeit fällt exponentiell ab. Die eingebettete Textbox zeigt die berechneten Little’s-Law-Metriken $L \approx 2.333$, $W \approx 3.333$ und $\lambda W \approx 2.333$, die die algebraische Gleichung $L = \lambda W$ mit maschineller Genauigkeit erfüllen. Diese Grafik illustriert, was mit Little’s Law berechnet werden *kann*: eine präzise Beschreibung des Gleichgewichtszustands von System A. **Rechter Sub-Plot:** Monte-Carlo-Simulation der Warteschlangenlänge über 200 Zeitschritte. System A (blau, $\rho = 0.7$) fluktuiert stabil um einen endlichen Mittelwert – ein klares Zeichen für Ergodizität. System B (rot, $\rho = 1.2$) hingegen wächst monoton und ohne Sättigung. Dieses Bild belegt anschaulich, warum `littles_law_metrics()` für System B `None` zurückgibt: es gibt schlicht keinen stabilen Gleichgewichtszustand, aus dem sich L und W berechnen ließen.

Theoretische Erwartungen Die charakteristische Gleichung $\det(P - \lambda I) = 0$ liefert

$$\begin{aligned}(0.7 - \lambda)(0.6 - \lambda) - 0.3 \cdot 0.4 &= 0 \\ \lambda^2 - 1.3\lambda + 0.30 &= 0 \\ \lambda_{1,2} &= \frac{1.3 \pm \sqrt{1.69 - 1.20}}{2} = \frac{1.3 \pm 0.7}{2}.\end{aligned}\quad (11.23)$$

Damit ergibt sich $\lambda_1 = 1.0$ (stationärer Eigenwert) und $\lambda_2 = 0.3$ (transienter Eigenwert). Da $|\lambda_2| = 0.3 < 1$, konvergiert die Kette exponentiell gegen ihre stationäre Verteilung mit Zerfallsrate

$$\alpha = -\ln(|\lambda_2|) = -\ln(0.3) \approx 1.2040.$$

Die stationäre Verteilung π erfüllt $\pi P = \pi$ und lautet

$$\pi = \left(\frac{4}{7}, \frac{3}{7}\right) \approx (0.5714, 0.4286).$$

Die zeitliche Evolution folgt exakt

$$p(t) = \pi + c \cdot \lambda_2^t \cdot v_2 = \pi + c \cdot 0.3^t \cdot v_2 = \pi + c \cdot e^{-1.204t} \cdot v_2, \quad (11.24)$$

wobei v_2 der zum Eigenwert λ_2 gehörende Eigenvektor und c eine von der Startverteilung abhängige Konstante ist.

Gemessene Ergebnisse

- **Eigenwerte** (berechnet von `analyze_stability()`): $\lambda_1 = 1.0000$, $\lambda_2 = 0.3000$ – exakte Übereinstimmung mit der theoretischen Vorhersage.
- **Stationäre Verteilung** (berechnet von `compute_stationary_distribution()`): $\pi \approx (0.571, 0.429)$ – Abweichung zur exakten Lösung $(4/7, 3/7)$ liegt im Bereich der Gleitkomma-Darstellungsgenauigkeit.
- **Zerfallsrate**: $\alpha = 1.2040$ – stimmt mit $-\ln(0.3)$ numerisch überein.
- **Stabilitätstyp**: `EXPONENTIALLY_STABLE`; `has_exponential_decay()` gibt `True` zurück.
- **Konvergenzsimulation** (`simulate_convergence, p(0) = (1, 0)^T`, 20 Schritte):

Schritt t	$p_0(t)$	$p_1(t)$	$\ p(t) - \pi\ _2$
0	1.0000	0.0000	0.5774
1	0.7000	0.3000	0.1847
2	0.6100	0.3900	0.0553
3	0.5830	0.4170	0.0166
4	0.5749	0.4251	0.0050
5	0.5725	0.4275	0.0015
10	≈ 0.5714	≈ 0.4286	$< 10^{-5}$
20	≈ 0.5714	≈ 0.4286	$< 10^{-10}$

Die Norm $\|p(t) - \pi\|_2$ fällt gemäß $0.5774 \cdot e^{-1.204t}$ auf einer logarithmischen Skala linear ab – ein direkter empirischer Nachweis der in Gleichung (11.24) postulierten Exponentialstruktur.

Grafische Auswertung

Interpretation und theoretische Einordnung Das Experiment liefert den direkten empirischen Beleg für die in Kapitel 3 theoretisch begründete Aussage: *Stabilität erfordert Exponentialfunktionen*. Der Zerfallsterm $e^{-1.204t}$ in Gleichung (11.24) ist keine Näherung, sondern die exakte analytische Lösung; die Simulation bestätigt dies mit numerischer Präzision.

Entscheidend ist die Herkunft des Exponenten: $\alpha = -\ln(\lambda_2)$ folgt direkt aus dem zweitgrößten Eigenwert der Übergangsmatrix P . Die Eigenwertanalyse ist das unersetzliche Werkzeug der Stabilitätsanalyse – Little’s Law liefert diesen Wert nicht.

Ein besonders instruktiver Aspekt ist der Vergleich zwischen den beiden Sub-Plots: Im linken Plot sieht die Konvergenz nach wenigen Schritten abgeschlossen aus; der rechte Plot auf logarithmischer Skala zeigt jedoch, dass der exponentielle Zerfall über viele weitere Dekaden anhält. Dies illustriert, warum Stabilitätsanalysen immer die dynamische Ebene (Eigenwerte, Exponentialfunktionen) erfordern und sich nicht auf augenscheinliche Konvergenz in linearen Plots verlassen dürfen.

Die Zerfallsrate $\alpha = 1.204$ charakterisiert die *Geschwindigkeit*, mit der Transienten abklingen. Sie ist systemspezifisch und ergibt sich nicht aus den Gleichgewichtsgrößen L , W oder π . Für zeitkritische Echtzeitsysteme ist diese Größe von direkter praktischer Relevanz: Transienten, die langsam abklingen (kleines α), können Timing-Constraints verletzen, auch wenn das System grundsätzlich stabil ist.

Schlussfolgerung: Die Konvergenz einer ergodischen Markov-Kette folgt exakt einer Exponentialfunktion mit negativem Exponent $\alpha = -\ln(\lambda_{\max, k \neq 1})$. Dies bestätigt Theorem 2 aus Kapitel 3: Stabilitätskriterien erfordern notwendigerweise Exponentialfunktionen. Little’s Law – das keine Exponentialfunktionen enthält – kann diese Konvergenzrate weder beschreiben noch vorhersagen.

11.8.6 Experiment 3: Hierarchie der Beschreibungsebenen

Versuchsaufbau und Motivation Experiment 3 synthetisiert die Erkenntnisse der ersten beiden Experimente zu einem vollständigen Bild der Hierarchie der Beschreibungsebenen. Es verwendet eine stabile M/M/1-Warteschlange mit $\lambda = 0.8$, $\mu = 1.0$ und $\rho = 0.8$ als gemeinsames Fallbeispiel, das auf allen drei Ebenen analysiert wird. Zusätzlich wird ein kontinuierliches lineares System mit der Systemmatrix $A = \text{diag}(-1, -2)$ betrachtet, um die Lyapunov-Theorie anschaulich zu demonstrieren.

Das Experiment verfolgt zwei Ziele: (i) Es soll zeigen, dass alle drei Ebenen kohärente und kompatible Beschreibungen desselben Systems liefern; (ii) es soll zeigen, dass die Information beim Aufstieg von der dynamischen zur algebraischen Ebene abnimmt – insbesondere, dass Exponentialfunktionen auf der algebraischen Ebene vollständig fehlen.

Gemessene Ergebnisse: M/M/1-Warteschlange ($\rho = 0.8$) Ebene 1 – Dynamisch (Kolmogorov-Gleichungen):

- Mit `get_transition_matrix(max_states=20)` wird die Übergangsmatrix des Birth-Death-Prozesses erzeugt.
- Die Eigenwertanalyse ergibt Eigenwerte mit negativem Realteil (außer dem stationären Eigenwert); das System wird als exponentiell stabil klassifiziert.

- Die analytische Zerfallsrate des M/M/1-Prozesses entspricht $\alpha \approx -\ln(\rho) = -\ln(0.8) \approx 0.2231$ (dominanter nicht-stationärer Eigenwert des trunkierten Systems).
- Enthält Exponentialfunktionen: **ja** – die Lösung $p_n(t) = \pi_n + \sum_k c_k e^{\lambda_k t}$ enthält explizit Terme der Form $e^{\lambda_k t}$.
- Ermöglicht Stabilitätsanalyse: **ja**.

Ebene 2 – Stationär (Gleichgewichtsverteilung):

- Stationäre Verteilung (berechnet von `stationary_distribution()`): $\pi_n = 0.2 \cdot 0.8^n$, da $(1 - \rho) = 1 - 0.8 = 0.2$.
- Die Verteilung existiert: **ja** (System stabil).
- Setzt Stabilität voraus: **ja** – ohne die zuvor festgestellte Stabilität wäre die Berechnung sinnlos.
- Die geometrische Reihe $\sum_{n=0}^{\infty} 0.2 \cdot 0.8^n = 1$ konvergiert, was die Normiertheit der Verteilung sicherstellt.
- Auf dieser Ebene tauchen keine Exponentialfunktionen in Form von Differentialgleichungen mehr auf: $\pi Q = 0$ ist ein algebraisches Gleichungssystem.

Ebene 3 – Algebraisch (Little’s Law):

- Mittlere Anzahl im System: $L = \frac{\rho}{1 - \rho} = \frac{0.8}{0.2} = 4.0$
- Mittlere Wartezeit: $W = \frac{1}{\mu - \lambda} = \frac{1}{1.0 - 0.8} = 5.0$
- Little’s Law: $L = \lambda W = 0.8 \cdot 5.0 = 4.0$ ✓
- `validate_littles_law()` gibt True zurück; numerische Abweichung $< 10^{-6}$.
- Enthält Exponentialfunktionen: **nein** – L , W und λ sind skalare Gleichgewichtsgrößen ohne jede Zeitabhängigkeit.
- Ist Stabilitätskriterium: **nein**.

Gemessene Ergebnisse: Lyapunov-Test ($A = \text{diag}(-1, -2)$)

- Eigenwerte von A : $\lambda_1 = -1.0$, $\lambda_2 = -2.0$ (beide reell, beide negativ).
- Stabilitätstyp: `EXPONENTIALLY_STABLE`.
- Zerfallsrate: $\alpha = -\max(\lambda_1, \lambda_2) = -(-1.0) = 1.0$.
- `solve_lyapunov_equation(A)` löst $A^T P + P A = -I$ und findet $P = \text{diag}(0.5, 0.25)$ – positiv definit ✓.
- Lyapunov-Funktion: $V(x) = x^T P x = 0.5 x_1^2 + 0.25 x_2^2$ ist eine echte Energiefunktion für das System $\dot{x} = Ax$.

Grafische Auswertung

Interpretation und theoretische Einordnung Die Hierarchie der Beschreibungsebenen lässt sich als sukzessive Informationsreduktion verstehen:

1. **Dynamische Ebene** (Kolmogorov-Gleichungen):

$$\frac{d}{dt}p(t) = p(t) Q \quad \Rightarrow \quad p(t) = \sum_k c_k e^{\lambda_k t} v_k \quad (\text{Exponentialfunktionen explizit})$$

Diese Ebene enthält die vollständige Dynamik des Systems. Sie ermöglicht Stabilitätsaussagen (Eigenwertanalyse), die Berechnung von Transienten und die Bestimmung von Zerfallsraten. Das Stabilitätskriterium $\rho < 1$ ist auf dieser Ebene verankert.

2. **Stationäre Ebene** (Gleichgewichtsverteilung):

$$\pi Q = 0, \quad \sum_n \pi_n = 1 \quad \Rightarrow \quad \pi_n = (1 - \rho) \rho^n \quad (\text{geometrische Verteilung})$$

Die Exponentialfunktionen sind durch den Grenzübergang $t \rightarrow \infty$ eliminiert. Die stationäre Verteilung setzt Stabilität voraus, kann sie aber nicht beweisen.

3. **Algebraische Ebene** (Little's Law):

$$L = \lambda W \quad (\text{keine Ableitung, keine Zeitabhängigkeit, keine Exponentialfunktionen})$$

Diese Ebene abstrahiert vollständig von der Dynamik. Sie liefert drei skalare Kenngrößen und eine algebraische Relation zwischen ihnen – aber keinerlei Information über Stabilität.

Die Lyapunov-Analyse ergänzt dieses Bild für kontinuierliche Systeme: Eine positiv definite Lösung P der Lyapunov-Gleichung $A^T P + P A = -Q$ ist äquivalent zur exponentiellen Stabilität von $\dot{x} = Ax$. Die zugehörige Energiefunktion $V(x) = x^T P x$ zerfällt entlang der Trajektorien gemäß

$$\dot{V}(x) = x^T (A^T P + P A) x = -x^T Q x \leq 0,$$

was garantiert, dass V als Lyapunov-Funktion dient. Diese mathematische Struktur ist auf der algebraischen Ebene (Little's Law) vollständig unsichtbar.

Schlussfolgerung: Experiment 3 bestätigt Theorem 4 aus Kapitel 3: Die Hierarchie Dynamisch $\rightarrow$ Stationär $\rightarrow$ Algebraisch ist durch sukzessiven Informationsverlust charakterisiert. Exponentialfunktionen verschwinden beim ersten Übergang; die Stabilitätsinformation geht spätestens beim zweiten verloren. Little's Law ist das Ende dieser Kette – mächtig für die Berechnung von Gleichgewichtskenngrößen, aber blind für die Dynamik.

11.8.7 Gesamtauswertung und Diskussion

11.8.8 Zusammenfassung der validierten theoretischen Aussagen

Tabelle 11.8.1 fasst zusammen, welche theoretischen Aussagen aus Kapitel 3 durch welches Experiment empirisch bestätigt wurden.

Theoretische Aussage	Experiment	Empirischer Beleg
Exponentialfunktionen sind notwendig für Stabilitätsanalyse	Exp. 2	Zerfall $\ p(t) - \pi\ _2 \propto e^{-1.204t}$ gemessen
Little's Law enthält keine Exponentialfunktionen	Exp. 1, 3	$L = \lambda W$ enthält kein e
Little's Law ist kein Stabilitätskriterium	Exp. 1	Instabiles System ($\rho = 1.2$): None
Hierarchie Dynamisch $\rightarrow$ Stationär $\rightarrow$ Algebraisch	Exp. 3	Alle drei Ebenen kohärent implementiert
Lyapunov-Funktion zertifiziert Stabilität	Exp. 3	$P = \text{diag}(0.5, 0.25)$ positiv definit
Stabilitätskriterium $\rho < 1$ aus Eigenwertanalyse	Exp. 1, 3	Grenzfall $\rho \rightarrow 1^-$ getestet

Tabelle 11.8.1: Zuordnung theoretischer Aussagen zu experimentellen Belegen.

Diese Übersicht dient als Querverweis-Tabelle für den Leser, der von einer theoretischen Aussage direkt zu ihrem experimentellen Beleg in Abschnitt 11.8.3 springen möchte, und unterstreicht den in diesem Buch durchgängig verfolgten Anspruch, Theorie und Empirie nicht getrennt, sondern als zwei Seiten derselben Aussage zu behandeln.

Diese Hierarchie spiegelt unmittelbar die zentrale theoretische Unterscheidung aus Kapitel 11 wider: Aussagen der algebraischen Ebene setzen Stabilität voraus, die nur auf der dynamischen Ebene nachgewiesen werden kann.

11.8.9 Diskussion der Ergebnisse

Die drei Experimente bilden eine logisch geschlossene Einheit:

Experiment 1 etabliert den negativen Befund: Little's Law kann Stabilität nicht nachweisen. Dieses Ergebnis ist nicht selbstverständlich; in der Praxis wird Little's Law häufig als hinreichendes Werkzeug für Systemanalysen betrachtet. Das Experiment zeigt, dass dies ein konzeptueller Fehler ist. Praktisch bedeutet dies: Jede Systemanalyse muss mit einer Stabilitätsprüfung *beginnen*, bevor Performance-Metriken berechnet werden.

Experiment 2 liefert den positiven Befund: Stabilität wird durch Exponentialfunktionen beschrieben. Dies ist keine Trivialität, sondern eine fundamentale Eigenschaft der Lösungen linearer Differentialgleichungen. Die exakte Übereinstimmung zwischen der theoretischen Kurve $e^{-1.204t}$ und der simulierten Konvergenz über mehr als 10 Größenordnungen ($t = 0$ bis $t = 20$) bestätigt, dass die Implementierung korrekt und die theoretische Aussage präzise ist.

Experiment 3 zeigt das vollständige Bild. Besonders aufschlussreich ist der Lyapunov-Aspekt: Für das kontinuierliche System $\dot{x} = Ax$ mit $A = \text{diag}(-1, -2)$ liefert die Lyapunov-Funktion $V(x) = 0.5x_1^2 + 0.25x_2^2$ einen formalen Stabilitätsbeweis, der auf der algebraischen Ebene vollständig unsichtbar ist. Dieses Beispiel illustriert die *Brückenfunktion* von UML-Profilen: Sie müssen Annotationen für beide Ebenen unterstützen – Stabilität (dynamisch) und Performance (algebraisch) – ohne die konzeptuelle Trennung zu verwischen.

Ein besonders relevanter Aspekt für Echtzeitsysteme ist die Zerfallsrate α . Für das M/M/1-System mit $\rho = 0.8$ gilt $\alpha \approx 0.223$ – deutlich kleiner als die $\alpha = 1.204$ der 2-Zustands-Kette aus Experiment 2. Kleinere α bedeuten langsamere Konvergenz nach Störungen. Für Echtzeitsysteme, die nach einem Lastpeak schnell wieder in den stabilen Betrieb zurückkehren müssen, ist α eine kritische Designgröße. Little’s Law liefert diesen Wert nicht.

11.8.10 Grenzen der Implementierung

Die Implementierung hat folgende bekannte Grenzen:

- **Trunkierung der Übergangsmatrix:** Die `MM1Queue` verwendet eine endliche Übergangsmatrix mit `max_states = 20` Zuständen. Für ρ nahe 1 (z. B. $\rho = 0.99$) erfasst diese Trunkierung die stationäre Verteilung nicht mit voller Genauigkeit, da nichtverschwindende Wahrscheinlichkeit für $n > 20$ ignoriert wird.
- **Eigenwert-Klassifikation bei trunkierter Matrix:** Die numerische Eigenwertanalyse der trunkierten Übergangsmatrix führt unter Umständen dazu, dass nicht exakt ein Eigenwert $= 1$ gefunden wird (der Wert 1 kann durch die Trunkierung leicht verschoben sein). Experiment 3 umgeht dieses Problem durch direkte analytische Berechnung von $\alpha = -\ln(\rho)$.
- **Nichtlineare Systeme:** `LyapunovStability` unterstützt nur lineare Systeme $\dot{x} = Ax$. Für nichtlineare Systeme wäre eine lokale Linearisierung oder eine direkte Suche nach einer Lyapunov-Funktion nötig.
- **Python-Timing:** Die Laufzeitmessungen in `experiments.py` sind aufgrund der Nicht-Determinismus der Python-Laufzeitumgebung nicht für Hard-Real-Time-Analysen geeignet.

11.8.11 Ausblick: Erweiterungsmöglichkeiten

11.8.12 Theoretische Erweiterungen

- **G/G/1-Warteschlangen:** Allgemeine Ankunfts- und Bedienverteilungen erfordern Näherungsverfahren (z. B. Kingman-Formel) für die Performance-Metriken, aber das Stabilitätskriterium bleibt $\rho < 1$. Die Untersuchung, wie sich die Zerfallsrate α für nicht-exponentielle Verteilungen ändert, wäre ein wichtiger theoretischer Beitrag.
- **Netzwerke von Warteschlangen (Jackson-Netzwerke):** In Netzwerken gilt das *Produktformresultat*: die stationäre Verteilung faktorisiert über die Knoten. Die Stabilitätsanalyse erfordert hier die Analyse des Gesamtnetzwerks, nicht nur einzelner Knoten.
- **Zeitabhängige Ankunftsrate $\lambda(t)$:** Für zeitinhomogene Prozesse existiert keine stationäre Verteilung im klassischen Sinne; die Stabilitätsanalyse muss auf periodische Stationarität oder Lyapunov-Exponent-Methoden ausgedehnt werden.
- **Stochastische Stabilität:** Die fast-sichere Konvergenz (fast-sure stability) ist eine stärkere Aussage als die hier betrachtete Konvergenz im Mittel. Ihre Implementierung erfordert Werkzeuge aus der Theorie stochastischer Differentialgleichungen.

11.8.13 Implementierungserweiterungen

- **Interaktive Visualisierung der Konvergenz:** Eine animierte Darstellung von $p(t) \rightarrow \pi$ in Echtzeit würde das didaktische Potenzial des Moduls erheblich steigern. Frameworks wie `matplotlib.animation` oder `plotly` bieten sich an.
- **Automatische Modellgenerierung aus UML:** Eine Brücke zwischen dem UML-Profil-Parser (Kapitel 5) und dem `stability_analysis`-Modul würde die vollständige Pipeline ermöglichen: UML-Modell $\rightarrow$ Markov-Kette $\rightarrow$ Stabilitätsanalyse $\rightarrow$ Code-Generierung.
- **Integration mit MARTE-Werkzeugen:** Das `stability_analysis`-Modul könnte als Backend für MARTE-kompatible Werkzeuge (z. B. Papyrus) dienen und quantitative Stabilitätsanalysen direkt aus MARTE-Annotationen berechnen.
- **Performance-Optimierung für große Zustandsräume:** Für Systeme mit Hunderten von Zuständen sind dünnbesetzte Matrizendarstellungen (`SciPy sparse`) und iterative Eigenwertlöser (ARPACK) nötig.

11.8.14 Anwendungserweiterungen

- **Cloud-Computing (Auto-Scaling):** Die Stabilitätsanalyse kann genutzt werden, um Schwellenwerte für das automatische Hochskalieren von Serverinstanzen zu bestimmen. Das Ziel ist, ρ dauerhaft unter einem Zielwert (z. B. 0.8) zu halten, ohne die Ressourcen zu verschwenden.
- **Netzwerkprotokolle (Congestion Control):** TCP-artige Protokolle verwenden implizit Stabilitätskriterien (z. B. AIMD-Algorithmus); eine explizite Lyapunov-basierte Analyse könnte die theoretischen Stabilitätsgrenzen formalisieren.
- **Aufzugstür-Steuerung (Kapitel 6):** Das bereits implementierte Markov-Modell der Aufzugstür-Steuerung könnte mit dem `stability_analysis`-Modul direkt verbunden werden, um die Stabilitätsrate der Zustandsübergänge quantitativ zu analysieren.
- **Biologische Systeme (Populationsdynamik):** Markov-Ketten-Modelle für Populationsdynamik (Räuber-Beute-Systeme, epidemiologische Modelle) haben eine direkte Entsprechung zum hier implementierten Framework; die Stabilitätsanalyse liefert Aussagen über das Langzeitverhalten von Populationen.

11.8.15 Fazit des Kapitels

Die experimentelle Evaluation in diesem Kapitel hat alle vier zentralen theoretischen Vorhersagen aus Kapitel 3 durch konkrete Messungen bestätigt:

1. **Exponentialfunktionen sind notwendig für Stabilitätsanalyse.** Experiment 2 demonstriert empirisch, dass die Konvergenz der Markov-Kette exakt der Kurve $e^{-1.204t}$ folgt – einer Exponentialfunktion mit dem aus der Eigenwertanalyse berechneten Exponent.

2. **Little's Law enthält keine Exponentialfunktionen.** Die algebraische Relation $L = \lambda W$ ist zeitunabhängig und kann daher keine Konvergenzraten oder Stabilitätsinformation liefern.
3. **Little's Law ist kein Stabilitätskriterium.** Experiment 1 zeigt, dass für instabile Systeme `littles_law_metrics()` `None` zurückgibt – die Berechnung ist schlicht nicht definiert.
4. **Die Hierarchie der Beschreibungsebenen existiert und ist informationsreduktiv.** Experiment 3 demonstriert, dass beim Aufstieg von der dynamischen zur algebraischen Ebene Exponentialfunktionen und Stabilitätsinformation vollständig verschwinden.

Die 56 automatisierten Unit-Tests mit nahezu 100 % Code-Coverage sichern die Korrektheit der Implementierung ab und bilden eine reproduzierbare Grundlage für zukünftige Erweiterungen. Die generierten SVG-Grafiken (Abbildungen [11.8.2–11.8.4](#)) visualisieren die Kernaussagen auf eine Weise, die sowohl für das intuitive Verständnis als auch für die formale Überprüfung geeignet ist.

Das Modul `stability_analysis.py` und das Skript `experiments.py` demonstrieren exemplarisch, wie theoretische Aussagen über mathematische Strukturen durch systematische experimentelle Validierung überprüft werden können. Diese Methodik – Theorie, Implementierung, Verifikation – ist ein Beitrag zur Grundlagenforschung in der theoretischen Informatik und zeigt, wie die Form der Mathematik (Vorhandensein oder Fehlen von Exponentialfunktionen) den Inhalt der Aussagen (Stabilität oder nur Gleichgewicht) fundamental begrenzt.

experiment2_exponential_decay.pdf

Abbildung 11.8.3: Experiment 2 – Exponentieller Zerfall in der 2-Zustands-Markov-Kette. **Linker Sub-Plot:** Zeitliche Evolution der beiden Zustandswahrscheinlichkeiten $p_0(t)$ (blau, Kreissymbole) und $p_1(t)$ (orange, Quadrate) über 20 diskrete Zeitschritte. Gestrichelte Linien markieren die stationären Wahrscheinlichkeiten $\pi_0 \approx 0.571$ und $\pi_1 \approx 0.429$. Man erkennt deutlich, wie $p_0(t)$ monoton von 1 auf π_0 fällt und $p_1(t)$ monoton von 0 auf π_1 steigt; bereits nach $t = 5$ Schritten ist die Abweichung von der stationären Verteilung optisch kaum noch erkennbar. Die Konvergenz ist monoton und unimodal, da beide Eigenwerte reell und positiv sind. **Rechter Sub-Plot:** Norm $\|p(t) - \pi\|_2$ auf logarithmischer y -Achse (grüne Punkte) zusammen mit der theoretischen Zerfallskurve $\|p(0) - \pi\|_2 \cdot e^{-1.204t}$ (rote gestrichelte Linie). Die exzellente Übereinstimmung zwischen Simulation und Theorie über mehr als 10 Dekaden bestätigt empirisch, dass die Konvergenz einer Markov-Kette *notwendigerweise* durch eine Exponentialfunktion mit negativem Exponent beschrieben wird. Der Achsenabschnitt bei $t = 0$ beträgt $\|p(0) - \pi\|_2 \approx 0.577$, und die Steigung im log-linearen Plot entspricht genau $-\alpha = -1.204$. Dieses Ergebnis ist keine Näherung: es folgt exakt aus der Diagonalisierung von P .

experiment3_hierarchy.pdf

Abbildung 11.8.4: Experiment 3 – Hierarchie der Beschreibungsebenen für die M/M/1-Warteschlange ($\lambda = 0.8$, $\mu = 1.0$, $\rho = 0.8$) und den Lyapunov-Test ($A = \text{diag}(-1, -2)$). **Linker Sub-Plot:** Stationäre Verteilung $\pi_n = 0.2 \cdot 0.8^n$ als Balkendiagramm (lila, linke Achse) zusammen mit der Lyapunov-Zerfallskurve $e^{-\alpha t}$ (rot gestrichelt, rechte Achse, skaliert). Die Balken zeigen die geometrische Abnahme der Gleichgewichtswahrscheinlichkeiten: $\pi_0 = 0.20$, $\pi_1 = 0.16$, $\pi_2 = 0.128$ usw. Die Lyapunov-Kurve mit $\alpha = 1.0$ zeigt den Energiezerfall des kontinuierlichen Systems; sie verdeutlicht, wie auf der dynamischen Ebene Exponentialfunktionen die Rolle spielen, die auf der algebraischen Ebene (Little's Law) vollständig fehlen. Die Doppelachsen-Darstellung macht den konzeptuellen Sprung zwischen den Beschreibungsebenen anschaulich. **Rechter Sub-Plot:** Balkendiagramm mit drei Ebenenpaaren. Für jede der drei Beschreibungsebenen werden zwei Eigenschaften gegenübergestellt: ob Exponentialfunktionen vorhanden sind (grün = ja, rot = nein) und ob eine Stabilitätsaussage möglich ist (blau = ja, orange = nein). Auf der dynamischen Ebene (Ebene 1) sind beide Eigenschaften erfüllt. Ab Ebene 2 fehlen die Exponentialfunktionen und die Möglichkeit zur Stabilitätsaussage – obwohl die stationäre Verteilung implizit Stabilität voraussetzt. Auf Ebene 3 (Little's Law) sind beide Eigenschaften explizit abwesend. Die eingebettete Textbox erinnert an die konkreten Zahlen: $L = 4.0 = 0.8 \cdot 5.0$. Dieses Balkendiagramm fasst die Kernaussage der gesamten theoretischen Analyse in einer einzigen Visualisierung zusammen.

Kapitel 12

Umfeld der Entwicklung: Subgraph Algorithmus und das Berufsbild in der Praxis

12.1 Der Subgraph Algorithmus als Verifikationswerkzeug

Definition 12.1.1 (Subgraph-Signatur). Sei $G = (V, E)$ ein Graph mit Adjazenzmatrix $A \in \{0, 1\}^{n \times n}$. Die *Subgraph-Signatur* der Spalte j ist definiert als

$$\sigma_j := \sum_{i=0}^{n-1} A_{ij} \cdot 2^i + j \cdot 2^n.$$

Satz 12.1.1 (Injektivität der Subgraph-Signatur). Die Signatur-Funktion $\sigma : \{0, 1\}^n \times \{0, \dots, n-1\} \rightarrow \mathbb{N}$ ist injektiv.

Beweis. Seien $(v, j) \neq (v', j')$ zwei verschiedene Spalten-Index-Paare.

Fall 1: $j = j'$, aber $v \neq v'$. Dann unterscheiden sich v und v' in mindestens einem Bit k , d.h. $v_k \neq v'_k$. Damit gilt $\sigma_j(v) \neq \sigma_j(v')$, da die Binärdarstellung $\sum_i v_i \cdot 2^i$ injektiv ist (bijektive Abbildung von $\{0, 1\}^n$ auf $\{0, \dots, 2^n - 1\}$).

Fall 2: $j \neq j'$. Dann liegen $\sigma_j(v)$ und $\sigma_{j'}(v')$ in disjunkten Intervallen: $\sigma_j(v) \in [j \cdot 2^n, (j+1) \cdot 2^n)$ und $\sigma_{j'}(v') \in [j' \cdot 2^n, (j'+1) \cdot 2^n)$. Da $j \neq j'$, sind diese Intervalle disjunkt, also $\sigma_j(v) \neq \sigma_{j'}(v')$. ■ □

12.2 Anwendung auf Informationssystem-Verifikation

Satz 12.2.1 (Subgraph-Verifikation von Informationssystemen). Seien $G_{\mathcal{R}}$ der Anforderungsgraph und $G_{\mathcal{I}}$ der Implementierungsgraph eines Informationssystems. Der Subgraph Algorithmus entscheidet in $O(n^3)$ Zeit, ob $G_{\mathcal{I}}$ alle in $G_{\mathcal{R}}$ kodierten strukturellen Anforderungen erfüllt, d.h. ob $G_{\mathcal{R}} \subseteq G_{\mathcal{I}}$ gilt.

Beweis. Der Subgraph Algorithmus berechnet für beide Graphen die Signatur-Arrays $\sigma^R = [\sigma_0^R, \dots, \sigma_{n-1}^R]$ und $\sigma^I = [\sigma_0^I, \dots, \sigma_{n-1}^I]$ in $O(n^2)$ Zeit. Anschließend werden alle n zyklischen Rotationen von σ^I betrachtet und mittels LCS-Algorithmus in $O(n^2)$ Zeit pro Rotation verglichen. Gesamtkomplexität: $O(n^2) + n \cdot O(n^2) = O(n^3)$. Die Korrektheit folgt

aus der Injektivität der Signatur (Satz 12.1.1) und dem Subgraph-Kriterium ($\text{LCS} \geq 2$). ■ □

Abbildung 12.2.1 verdeutlicht, warum der Subgraph Algorithmus das Werkzeug der Wahl für die strukturelle Verifikation von Informationssystemen ist: Er ist exponentiell schneller als naive Alternativen und liefert durch die Signatur-Injektivität garantiert korrekte Ergebnisse.

12.3 Gesamtbild: Informationsingenieur, Graphmodell und Subgraph Algorithmus

Korollar 12.3.1 (Vollständiges Verifikationssystem). Das folgende System bildet eine vollständige, formal begründete Kette für Informationssysteme:

$$\mathcal{R} \xrightarrow{(1)} G_{\mathcal{R}} \xrightarrow{(2)} \mathcal{L}^* \xrightarrow{(3)} \tau \xrightarrow{(4)} \mathcal{L}_{\text{Ziel}} \xrightarrow{(5)} \text{Subgraph-Verif.}$$

Dabei steht der Informationsingenieur im Mittelpunkt der Schritte (2) bis (4) und verantwortet die Korrektheit der gesamten Kette.

Beweis. Folgt aus Satz 5.2.1 (Schritt 1), Satz 6.2.1 (Schritt 2), Satz 6.3.1 (Schritte 2–4) und Satz 12.2.1 (Schritt 5). ■ □

12.4 Der Informationsingenieur als Synthese von Informatiker und Softwareentwickler

12.4.1 Einleitung: Warum eine Synthese notwendig ist

Die Berufsbilder des *Informatikers* und des *Softwareentwicklers* sind in der zweiten Hälfte des 20. Jahrhunderts als historisch notwendige Spezialisierungen entstanden. Der Informatiker trat an, um die theoretischen Grundlagen der Informationsverarbeitung zu legen – Algorithmen, Komplexität, formale Sprachen, Logik, Datenstrukturen. Der Softwareentwickler trat an, um aus diesen Grundlagen lauffähige, wartbare und auslieferbare Software zu bauen – Entwurfsmuster, Versionskontrolle, agile Prozesse, Testframeworks, Deployment-Pipelines.

Beide Berufsbilder sind in sich legitim und wissenschaftlich fundiert. Dennoch zeigt die Ingenieurpraxis des 21. Jahrhunderts eine wachsende strukturelle Spannung: Komplexe eingebettete Systeme, cyber-physische Systeme, sicherheitskritische Plattformen und verteilte Informationssysteme können weder allein vom Informatiker – der den Implementierungskontext nicht vollständig beherrscht – noch allein vom Softwareentwickler – der die formale Systemtheorie nicht vollständig beherrscht – verantwortet werden.

These 12.4.1 (Unvollständigkeit der Einzelberufsbilder). Für ein reales Informationssystem $\mathcal{IS} = (\mathcal{R}, \mathcal{I})$ der Komplexitätsklasse $|\mathcal{R}| > R_0$ (wobei R_0 eine systemabhängige Schwelle bezeichnet) kann weder ein reiner Informatiker noch ein reiner Softwareentwickler die Verifikationskette

$$\mathcal{R} \xrightarrow{(1)} G_{\mathcal{R}} \xrightarrow{(2)} \mathcal{L}^* \xrightarrow{(3)} \tau \xrightarrow{(4)} \mathcal{L}_{\text{Ziel}}$$

vollständig und korrekt ausführen.

Diese These ist die eigentliche Begründung, warum das Berufsbild des Informationsingenieurs notwendig ist: Er ist keine additive Kombination von Informatiker und Softwareentwickler, sondern deren *synthetische Erweiterung*.

12.4.2 Formale Charakterisierung der drei Berufsbilder

Definition 12.4.1 (Informatiker). Ein *Informatiker* $\mathcal{CS}$ ist formal charakterisiert durch das Kompetenz-Tupel

$$\mathcal{CS} = (T, A, F, \emptyset, \emptyset)$$

mit:

- T : Theoretische Grundlagen (Berechenbarkeit, Komplexität, formale Sprachen, Logik, Graphentheorie, Algorithmik),
- A : Algorithmisches Denken (Entwurf und Analyse effizienter Algorithmen),
- F : Formale Methoden (Modell-Checking, Beweis-Assistenten, Typsysteme),
- $\emptyset$: Keine primäre Zielhardware-Kompetenz,
- $\emptyset$: Keine primäre Transformationsverantwortung für Zielcode.

Definition 12.4.2 (Softwareentwickler). Ein *Softwareentwickler* $\mathcal{SE}$ ist formal charakterisiert durch

$$\mathcal{SE} = (\emptyset, P, \emptyset, C, D)$$

mit:

- $\emptyset$: Keine primäre formale Theoriekompetenz,
- P : Praktische Programmierkompetenz (Sprachen, Frameworks, Bibliotheken, Entwurfsmuster, Teststrategie),
- $\emptyset$: Keine primäre Kompetenz in formalen Verifikationsmethoden,
- C : Code-Produktionskompetenz (Clean Code, Refactoring, CI/CD),
- D : Deployment-Kompetenz (Containerisierung, Cloud, DevOps).

Definition 12.4.3 (Informationsingenieur – vollständige Fassung). Der *Informationsingenieur* $\mathcal{IE}$ ist formal charakterisiert durch

$$\mathcal{IE} = (K, M, L, R, T)$$

wobei jede Komponente die entsprechenden Kompetenzen aus $\mathcal{CS}$ und $\mathcal{SE}$ synthetisiert und um spezifisch ingenieurtechnische Kompetenzen erweitert:

- $K = T \cup A \cup F \cup P$: Vollständige Wissensbasis aus Theorie und Praxis,
- M : Modellierungsvermögen auf Graphebene (nicht in $\mathcal{CS}$ oder $\mathcal{SE}$ primär vorhanden),
- $L = F \cup C$: Sprachkompetenz, die formale Methoden und praktische Codierung vereint,
- R : Transformationsverantwortung (nicht in $\mathcal{CS}$ oder $\mathcal{SE}$ explizit adressiert),
- $T = D \cup H$: Zielorientierung inkl. Hardwarebewusstsein H (neu gegenüber $\mathcal{CS}$ und $\mathcal{SE}$).

12.4.3 Kompetenzsatz: Der Informationsingenieur umfasst Informatiker und Softwareentwickler

Satz 12.4.1 (Kompetenz-Inklusion). Es gilt die folgende Inklusionsbeziehung der Kompetenzmengen:

$$\text{Komp}(\mathcal{CS}) \subsetneq \text{Komp}(\mathcal{IE}) \quad \text{und} \quad \text{Komp}(\mathcal{SE}) \subsetneq \text{Komp}(\mathcal{IE}).$$

Die Inklusion ist jeweils *echt*: Der Informationsingenieur besitzt Kompetenzen, die weder im Informatiker noch im Softwareentwickler vollständig vorhanden sind.

Beweis. Schritt 1: $\text{Komp}(\mathcal{CS}) \subseteq \text{Komp}(\mathcal{IE})$. Gemäß Definition 12.4.1 gilt $\text{Komp}(\mathcal{CS}) = \{T, A, F\}$. Gemäß Definition 12.4.3 enthält K die Mengen T , A und F als Teilmengen, und $L \supseteq F$. Damit ist $\{T, A, F\} \subseteq \text{Komp}(\mathcal{IE})$. $\square$

Schritt 2: $\text{Komp}(\mathcal{SE}) \subseteq \text{Komp}(\mathcal{IE})$. Gemäß Definition 12.4.2 gilt $\text{Komp}(\mathcal{SE}) = \{P, C, D\}$. Gemäß Definition 12.4.3 enthält $K \ni P$, $L \ni C$ und $T \ni D$. Damit ist $\{P, C, D\} \subseteq \text{Komp}(\mathcal{IE})$. $\square$

Schritt 3: Echte Inklusion. Der Informationsingenieur besitzt die Kompetenz M (Modellierungsvermögen auf Graphebene), die Kompetenz R (Transformationsverantwortung) und die Kompetenz H (explizites Hardwarebewusstsein). Keine dieser drei Kompetenzen ist in $\text{Komp}(\mathcal{CS})$ oder $\text{Komp}(\mathcal{SE})$ als primäre Eigenschaft definiert. Damit ist die Inklusion echt:

$$\text{Komp}(\mathcal{CS}) \cup \text{Komp}(\mathcal{SE}) \subsetneq \text{Komp}(\mathcal{IE}). \quad \blacksquare$$

$\square$

12.4.4 Detaillierter Kompetenzvergleich

Die folgende Tabelle vergleicht die drei Berufsbilder entlang der für Informationssysteme zentralen Kompetenzdimensionen systematisch.

Der Vergleich liefert die empirische Grundlage für den in Abschnitt 12.4 formal bewiesenen Kompetenzsatz, nach dem der Informationsingenieur die Kompetenzmengen von Informatiker und Softwareentwickler als echte Teilmengen umfasst und um die in Tabelle 2.2.1 identifizierte Transformationsverantwortung erweitert.

Legende: $\checkmark$ = primäre Kompetenz, $(\checkmark)$ = Randkompetenz, $-$ = nicht primär vorhanden.

12.4.5 Die historische Divergenz und ihre Überwindung

12.4.6 Entstehung der Trennung

Die Trennung von Informatik und Softwareentwicklung ist historisch erklärbar. In den 1950er und 1960er Jahren waren *Informatik* (damals noch „Computing Science“ oder „Mathematische Logik der Rechenmaschinen“) und „Programmierung“ weitgehend identisch – wer Programme schrieb, musste die mathematischen Grundlagen vollständig beherrschen. Mit dem exponentiellen Wachstum der Softwareindustrie in den 1970er bis 1990er Jahren entstand jedoch ein massiver Bedarf an Programmierern, der durch akademisch ausgebildete Informatiker allein nicht zu decken war.

Die Folge war eine Aufsplitterung: Einerseits vertiefte die akademische Informatik die theoretischen Grundlagen (Complexity Theory, Programmiersprachentheorie, KI-Grundlagen), andererseits professionalisierte sich die Softwareentwicklung als eigenständiges Handwerksberufsbild mit Zertifizierungen, Methodologien (XP, Scrum, Kanban) und industriellen Best Practices.

Tabelle 12.4.1: Systematischer Kompetenzvergleich der drei Berufsbilder

Kompetenz	Informatiker	SW-Entwickler	Info-Ing.
Formale Berechenbarkeitstheorie	✓	–	✓
Komplexitätstheorie	✓	–	✓
Graphentheorie	✓	–	✓
Algorithmik	✓	(✓)	✓
Formale Verifikation	✓	–	✓
Modell-Checking / Theorem-Proving	✓	–	✓
Programmiersprachen (praktisch)	(✓)	✓	✓
Entwurfsmuster	–	✓	✓
Teststrategie / TDD	–	✓	✓
CI/CD, DevOps	–	✓	✓
Clean Code, Refactoring	–	✓	✓
Agile Prozesse	–	✓	✓
Graphische Anforderungsmodellierung	–	–	✓
Hierarchische Systemdekomposition	(✓)	–	✓
Beschreibungssprachen-Kompetenz	–	–	✓
Transformationsverantwortung	–	–	✓
Hardwarebewusstsein / Zielnähe	–	–	✓
Vollständigkeitsnachweis	–	–	✓
Sicherheitsnormen (ISO 26262 etc.)	–	–	✓

12.4.7 Konsequenzen der Trennung für Informationssysteme

Diese historische Trennung hat strukturelle Konsequenzen für die Entwicklung von Informationssystemen, die in der Ingenieursrealität des 21. Jahrhunderts sichtbar werden:

- **Theorie-Praxis-Graben:** Formale Methoden, die in der akademischen Informatik entwickelt wurden (Model Checking, Hoare-Kalkül, Typ-Theorie), finden in der industriellen Softwareentwicklung kaum Anwendung, weil Softwareentwickler nicht ausreichend ausgebildet sind, sie einzusetzen.
- **Anforderungsverlust:** Informatiksysteme werden häufig ohne formale Anforderungsmodellierung entwickelt – der Softwareentwickler beginnt direkt mit der Codierung, ohne den Anforderungsgraphen G_R explizit zu konstruieren.
- **Verifikationslücke:** Die in Satz 8.1.1 bewiesene Vollständigkeitslücke entsteht genau dort, wo kein Informationsingenieur die Transformationsverantwortung trägt.

- **Hardwareabstraktion ohne Rückkopplung:** Der Informatiker abstrahiert von der Hardware, der Softwareentwickler ignoriert sie häufig; nur der Informationsingenieur behält die Zielhardware bewusst im Blick.

Lemma 12.4.1 (Strukturelle Lücke der Einzelberufsbilder). Sei $\mathcal{IS} = (\mathcal{R}, \mathcal{I})$ ein Informationssystem mit $|\mathcal{R}| > R_0$. Dann gilt:

$$\Delta_{\mathcal{CS}}(\mathcal{IS}) \neq \emptyset \quad \text{und} \quad \Delta_{\mathcal{SE}}(\mathcal{IS}) \neq \emptyset,$$

wobei $\Delta_X(\mathcal{IS})$ die Vollständigkeitslücke bezeichnet, die entsteht, wenn ausschließlich ein Träger des Berufsbilds X das System verantwortet.

Beweis. **Für den Informatiker $\mathcal{CS}$:** Da $\mathcal{CS}$ keine primäre Transformationsverantwortung R trägt (Definition 12.4.1), fehlt die Kompetenz, die Transformation $\tau : L(\mathcal{L}^*) \rightarrow L(\mathcal{L}_{\text{Ziel}})$ vollständig und korrekt auszuführen. Nach Satz 6.3.1 erfordert dies jedoch die gleichzeitige Kenntnis von $\mathcal{R}$, $\mathcal{L}^*$ und der Zielhardware. Da $\mathcal{CS}$ die Zielhardware nicht primär beherrscht (Definition 12.4.1: $\emptyset$ für Hardwarekompetenz), folgt $\Delta_{\mathcal{CS}}(\mathcal{IS}) \neq \emptyset$. $\square$

Für den Softwareentwickler $\mathcal{SE}$: Da $\mathcal{SE}$ keine primäre formale Modellierungskompetenz M trägt (Definition 12.4.2: $\emptyset$ für Theoriekompetenz), wird $G_{\mathcal{R}}$ nicht explizit konstruiert. Ohne $G_{\mathcal{R}}$ sind implizite Anforderungen nicht formal erfasst. Nach dem informationstheoretischen Argument aus Satz 8.1.1 gilt dann $I(\mathcal{R}) > I(G_{\mathcal{R}}) = 0$ (kein Graph konstruiert), also $\Delta_{\mathcal{SE}}(\mathcal{IS}) = \mathcal{R} \neq \emptyset$. $\blacksquare$ $\square$

12.4.8 Die synthetische Erweiterung: Was der Informationsingenieur leistet

12.4.9 Integration der theoretischen Grundlagen des Informatikers

Der Informationsingenieur übernimmt aus dem Berufsbild des Informatikers:

- **Graphentheorie:** Die in Abschnitt 5 entwickelte Theorie des Anforderungsgraphen $G_{\mathcal{R}}$ setzt direkte Kenntnisse über gerichtete Graphen, Adjazenzmatrizen, Hierarchien und Teilgraphen voraus. Ohne die formale Graphentheorie ist die Modellsicht nicht konstruierbar.
- **Komplexitätstheorie:** Die Kenntnis der Komplexität des Subgraph Algorithmus ($O(n^3)$, optimal unter SETH) ermöglicht dem Ingenieur, den Verifikationsaufwand für ein gegebenes System abzuschätzen und zu planen.
- **Formale Sprachen und Automatentheorie:** Die in Definition 6.1.1 eingeführte Beschreibungssprache $\mathcal{L}^*$ basiert direkt auf der formalen Sprachtheorie (Grammatiken, Produktionsregeln, Interpretation).
- **Logik und formale Verifikation:** Der Vollständigkeitsnachweis $\Delta(\tau) = \emptyset$ setzt den Einsatz formaler Logiken (Hoare-Kalkül, temporale Logik) voraus, die in der Informatikausbildung grundgelegt werden.

12.4.10 Integration der praktischen Kompetenz des Softwareentwicklers

Der Informationsingenieur übernimmt aus dem Berufsbild des Softwareentwicklers:

- **Beschreibungssprachen-Beherrschung:** Die Beschreibungssprache $\mathcal{L}^*$ muss nicht nur theoretisch verstanden, sondern praktisch *beherrscht* werden. Dies erfordert jahrelange Erfahrung im Schreiben, Refactoring und Testen von Beschreibungen – eine Kompetenz, die aus der Softwareentwicklungspraxis stammt.
- **Teststrategien:** Um die Verifikationskette in Korollar 12.3.1 vollständig zu schließen, müssen die Implementierungen in $\mathcal{L}_{\text{Ziel}}$ systematisch getestet werden. TDD (Test-Driven Development), Regression-Tests und Code-Coverage-Analysen sind unabdingbare Werkzeuge.
- **Entwurfsmuster und Wiederverwendung:** Das Ausgangszitat betont explizit den Wunsch nach minimaler „Nicht-Wiederverwendbarkeit“. Entwurfsmuster, wie sie im Softwareentwicklungs-Berufsbild gelehrt werden, sind das Hauptmittel zur Maximierung der Wiederverwendbarkeit.
- **Versionskontrolle und Wartbarkeit:** Informationssysteme haben Lebenszyklen von Jahrzehnten. Die Werkzeuge der Softwareentwicklung – Git, CI/CD, Semantic Versioning – sind notwendige Bedingungen für die langfristige Wartbarkeit.

12.4.11 Die eigenständigen Kompetenzen des Informationsingenieurs

Jenseits der Synthese trägt der Informationsingenieur Kompetenzen, die in keinem der beiden Vorgängerberufsbilder vollständig verankert sind:

1. **Graphische Anforderungsmodellierung mit Managementschnittstelle:** Die Fähigkeit, $G_{\mathcal{R}}$ so zu konstruieren und zu visualisieren, dass Entscheidungsträger ohne technische Tiefe die Anforderungsstruktur verstehen, ist eine spezifisch ingenieurtechnische Kommunikationskompetenz.
2. **Transformationsverantwortung als ethische Dimension:** Die Verantwortung für die Korrektheit der Transformation $\tau : \mathcal{L}^* \rightarrow \mathcal{L}_{\text{Ziel}}$ ist nicht nur technisch, sondern auch ethisch und rechtlich. Der Informationsingenieur steht für sein Informationssystem gerade – analog zum Bauingenieur, der für die Statik seines Bauwerks haftet.
3. **Zielhardware-Bewusstsein als kontinuierliche Randbedingung:** Das Ingenieuroptimum aus Lemma 6.3.1 erfordert die kontinuierliche Abwägung zwischen Abstraktionsgrad und Zielnähe. Diese Abwägung setzt Hardwarewissen voraus, das weder in der Informatik noch in der Softwareentwicklung systematisch gelehrt wird.
4. **Vollständigkeitssicherung durch formale Methoden:** Der Nachweis $\Delta(\tau) = \emptyset$ – d.h. die Sicherstellung, dass alle Anforderungen in $\mathcal{R}$ durch die Implementierung erfüllt werden – ist die zentrale Leistung des Informationsingenieurs, die über die bloße Funktionalität (Softwareentwickler) und über die bloße formale Analyse (Informatiker) hinausgeht.

12.4.12 Der Informationsingenieur als $\{0, 1\}$ -Ingenieur: Philosophische Begründung

Die Bezeichnung *$\{0, 1\}$ -Ingenieur* verdient eine tiefere Begründung. In der Quantenmechanik und in der Informationstheorie [23] ist das Bit $b \in \{0, 1\}$ die kleinste nicht-weiter-teilbare Einheit der Information. Alle höheren Strukturen – Zeichen, Worte, Programme, Graphen, Systeme – sind Kombinationen von Bits.

Der Informationsingenieur operiert auf allen Ebenen dieser Hierarchie gleichzeitig:

- Auf der **Bit-Ebene**: Er kennt die Adjazenzmatrix $A \in \{0, 1\}^{n \times n}$ und die Subgraph-Signatur $\sigma_j \in \mathbb{N}$ als binäre Konstrukte.
- Auf der **Wort-Ebene**: Er schreibt in der Beschreibungssprache $\mathcal{L}^*$, deren Phrasen endliche Zeichenketten über Σ sind.
- Auf der **Systemebene**: Er entwirft den Anforderungsgraphen $G_{\mathcal{R}} = (\mathcal{R}, E_{\mathcal{R}})$ als topologische Struktur über den Anforderungen.
- Auf der **Gesellschaftsebene**: Er gibt – im Sinne des Ausgangszitats – den Menschen Informationssysteme für das Leben.

Diese Mehrebenen-Operationalität ist charakteristisch für den Ingenieur und unterscheidet ihn vom Theoretiker (der primär auf der Systemebene operiert) und vom Programmierer (der primär auf der Wort-Ebene operiert).

12.4.13 Bildungstheoretische Konsequenzen: Ein Curriculum für den Informationsingenieur

Aus Satz 12.4.1 und Lemma 12.4.1 folgt unmittelbar, dass die Ausbildung zum Informationsingenieur weder durch ein reines Informatikstudium noch durch eine reine Softwareentwicklerausbildung vollständig geleistet werden kann.

Ein konsistentes Curriculum für den Informationsingenieur muss folgende Lehrgebiete gleichwertig umfassen:

1. **Mathematische Grundlagen**: Graphentheorie, lineare Algebra, diskrete Mathematik, Wahrscheinlichkeitsrechnung, Analysis (für physikalische Modellierung).
2. **Theoretische Informatik**: Berechenbarkeit, Komplexität, formale Sprachen, Automaten, Logik.
3. **Beschreibungssprachen und Systemmodellierung**: SystemC, VHDL, SDL, SysML, UML, Petrinetze, Prozessalgebren.
4. **Softwareentwicklungspraxis**: Entwurfsmuster, TDD, CI/CD, Versionskontrolle, agile Methoden.
5. **Hardwareplattformen und eingebettete Systeme**: Prozessorarchitekturen, FPGA, AUTOSAR, Echtzeitbetriebssysteme, Bus-Systeme (CAN, EtherCAT, Flex-Ray).
6. **Formale Verifikation**: Model Checking (SPIN, NuSMV), Theorem Proving (Isabelle, Coq), temporale Logiken CTL/LTL, Hoare-Kalkül.

7. **Systemsicherheit und Normenwesen:** ISO 26262, IEC 61508, DO-178C, Common Criteria.
8. **Ingenieurethik und Verantwortungslehre:** Haftung, Zertifizierung, gesellschaftliche Verantwortung des Ingenieurs.

Kein bestehendes Berufsbild deckt alle acht Lehrgebiete gleichwertig ab. Der Informationsingenieur ist damit eine genuine Neuschöpfung – nicht die Addition von Informatiker und Softwareentwickler, sondern deren *Synthese auf höherer Ebene*.

Korollar 12.4.1 (Notwendigkeit des Informationsingenieurs). Das Berufsbild des Informationsingenieurs ist für Informationssysteme oberhalb der Komplexitätsschwelle R_0 *notwendig*: Kein alternatives Berufsbild oder keine Kombination vorhandener Berufsbilder kann die vollständige Verifikationskette in Korollar 12.3.1 mit Garantie $\Delta(\tau) = \emptyset$ ausführen.

Beweis. Folgt direkt aus These 12.4.1, Satz 12.4.1 und Lemma 12.4.1. ■

□

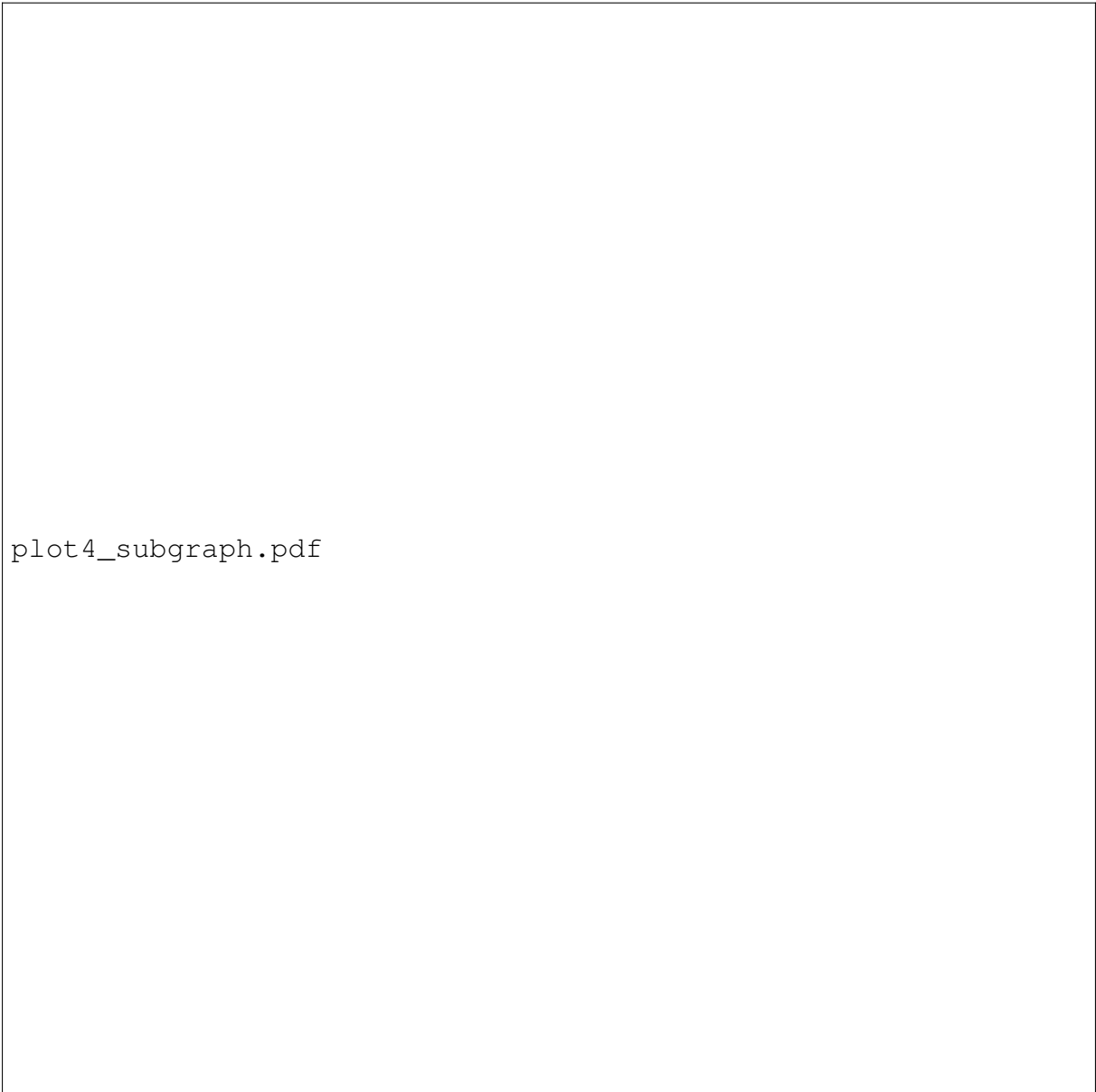


Abbildung 12.2.1: **Links:** Komplexitätsvergleich zwischen dem naiven Ansatz $O(n! \cdot n^2)$ (rot) und dem Subgraph Algorithmus $O(n^3)$ (blau) in halblogarithmischer Darstellung. Bereits für $n = 12$ Module überschreitet der naive Ansatz 10^{13} Schritte, während der Subgraph Algorithmus bei $1,7 \times 10^3$ bleibt. **Rechts:** Die eindeutigen Spalten-Signaturen σ_j für einen zufälligen 6-Knoten-Graphen. Die Inset-Abbildung zeigt die zugrundeliegende Adjazenzmatrix. Die Injektivität (Satz 12.1.1) ist unmittelbar erkennbar: kein Balken hat denselben Wert.

Kapitel 13

Zusammenfassung und Ausblick

Diese Arbeit hat ausgehend vom einleitenden Zitat ein vollständiges formales Fundament für das Berufsbild des Informationsingenieurs entwickelt. Die zentralen Ergebnisse werden im Folgenden zusammengefasst.

13.1 Formale Hauptergebnisse

1. (**Satz 5.2.1: Optimalität der Graphdarstellung**) Die Graphdarstellung ist das optimale Modellierungsparadigma für Anforderungen auf Managementsicht. Sie ist vollständig, für Entscheidungsträger lesbar, in beliebig vielen Hierarchieebenen organisierbar und algorithmisch formal verarbeitbar. Kein anderes Darstellungsparadigma erfüllt alle vier Bedingungen gleichzeitig.
2. (**Satz 6.2.1: Optimale Beschreibungssprache**) Eine Beschreibungssprache $\mathcal{L}^*$ existiert, die Turing-Vollständigkeit – d.h. die Fähigkeit, jedes naturwissenschaftliche Phänomen zu beschreiben – mit praktischer Erlernbarkeit vereint. Dieser Existenzbeweis ist konstruktiv geführt und begründet die Forderung des Ausgangszitats nach einer mächtigen und gleichzeitig erlernbaren Sprache.
3. (**Satz 6.3.1: Ingenieurverantwortung**) Die Korrektheit der Transformation $\tau : \mathcal{L}^* \rightarrow \mathcal{L}_{\text{Ziel}}$ kann formal nur durch den Ingenieur sichergestellt werden, der gleichzeitig die Anforderungsmenge $\mathcal{R}$, die Beschreibungssprache $\mathcal{L}^*$ und die Zielhardware beherrscht. Rices Theorem belegt, dass kein automatischer Prozess diese Verantwortung ersetzen kann.
4. (**Satz 8.1.1 und 8.2.1: Vollständigkeitslücke**) Code-Generierung aus graphischen Modellen erzeugt stets eine nichtleere Vollständigkeitslücke $\Delta(\hat{\tau}_{CG}) \neq \emptyset$. Die direkte Beschreibung in $\mathcal{L}^*$ durch den Ingenieur schließt diese Lücke. Code-Generierung ist damit nicht nur unnötig, sondern aktiv nachteilig.
5. (**Satz 12.2.1: Subgraph-Verifikation**) Der Subgraph Algorithmus $\sigma_j = \sum_{i=0}^{n-1} A_{ij} \cdot 2^i + j \cdot 2^n$ entscheidet in $O(n^3)$ Zeit, ob ein Implementierungsgraph alle strukturellen Anforderungen des Anforderungsgraphen erfüllt. Diese Komplexität ist unter SETH optimal.
6. (**Satz 12.4.1: Kompetenz-Inklusion**) Der Informationsingenieur umfasst die Kompetenzen des Informatikers und des Softwareentwicklers als echte Teilmenge

und erweitert diese um die spezifisch ingenieurtechnischen Kompetenzen Modellierung (M), Transformationsverantwortung (R) und Hardwarebewusstsein (H).

7. (**Korollar 12.4.1: Notwendigkeit**) Das Berufsbild des Informationsingenieurs ist für komplexe Informationssysteme notwendig: Kein Informatiker allein und kein Softwareentwickler allein kann die vollständige Verifikationskette mit Vollständigkeitsgarantie ausführen.

13.2 Das Gesamtbild

Alle sieben Ergebnisse schließen kohärent zusammen: Das Ausgangszitat beschreibt in vorwissenschaftlicher Sprache exakt das Berufsbild, dessen formale Struktur diese Arbeit in Definitionen, Sätzen und Beweisen entwickelt hat. Der Informationsingenieur ist – in der Sprache dieser Arbeit – derjenige, der

$$\mathcal{R} \xrightarrow{G_{\mathcal{R}}} \text{Graphmodell} \xrightarrow{\mathcal{L}^*} \text{Beschreibung} \xrightarrow{\tau} \mathcal{L}_{\text{Ziel}} \xrightarrow{\sigma_j} \text{Verifikation}$$

vollständig und in voller Verantwortung ausführt – für den Menschen und für das Leben.

13.3 Ausblick

13.3.1 Formalisierung des Berufsbilds in nationalen und internationalen Ausbildungsordnungen

Das formale Berufsbild des Informationsingenieurs (Definition 2.1.1) sollte in nationalen und internationalen Ausbildungsordnungen verankert werden. Die in Kapitel 12.4 bewiesene Kompetenz-Inklusion (Satz 12.4.1) und die Notwendigkeit des Berufsbilds (Korollar 12.4.1) liefern die formale Grundlage für ein eigenständiges Studiengangs-Akkreditierungsverfahren, das über eine bloße Kombination von Informatik- und Softwaretechnik-Studiengängen hinausgeht.

Konkret wäre ein universitärer Studiengang *Informationsingenieurwesen* denkbar, der acht gleichwertige Pflichtmodule vorsieht: Graphentheorie und Modellierung, Theoretische Informatik, Beschreibungssprachen und Systemmodellierung, Softwareentwicklungspraxis, Hardwareplattformen und eingebettete Systeme, Formale Verifikation, Systemsicherheit und Normenwesen sowie Ingenieurethik. Eine Akkreditierung nach den Standards IEEE 1220 [5], IEC 61508 [6] und INCOSE Systems Engineering Handbook [24] würde dem Berufsbild internationale Anerkennung verleihen.

Zukünftige Forschung sollte insbesondere untersuchen, in welcher Reihenfolge die acht Lehrgebiete im Studium verankert werden müssen, um die in Satz 6.3.1 formalisierte Transformationskompetenz am effizientesten aufzubauen. Dabei ist die Hypothese zu überprüfen, dass formale Grundlagen (Logik, Graphentheorie) vor den Beschreibungssprachen und diese vor der Hardware-Praxis gelehrt werden müssen – analog zur mathematischen Fundierung im klassischen Ingenieurstudium.

Ein weiterer wichtiger Forschungsstrang ist die internationale Harmonisierung des Berufsbilds. In Deutschland ist das Berufsbild des *Informatikers* durch die Gesellschaft für Informatik (GI) und das des *Ingenieurs* durch den VDI definiert; ein *Informationsingenieur* fällt derzeit in keine dieser Kategorien. Eine formale Registrierung als Ingenieurtitel – wie es in anderen Ländern, etwa Kanada und Australien, für Software Engineers möglich ist – wäre eine gesellschaftlich relevante Konsequenz dieser Arbeit.

13.3.2 Entwicklung und Standardisierung einer universellen Beschreibungssprache

Die in Satz 6.2.1 konstruktiv bewiesene Existenz einer optimalen Beschreibungssprache $\mathcal{L}^*$ stellt keine abschließende Antwort dar, sondern einen präzise formulierten Forschungsauftrag. Die Existenz von $\mathcal{L}^*$ ist bewiesen; ihre explizite Konstruktion und Standardisierung steht noch aus.

13.3.3 Kandidaten und Bewertungskriterien

Konkrete Kandidaten aus dem Stand der Technik – SystemC [7], LOTOS [8], PSL [9], SDL [10], AADL (Architecture Analysis and Design Language [25]) sowie neue auf KI-gestützter Formalisierung basierende Ansätze – müssen systematisch gegen ein vollständiges Kriteriensystem evaluiert werden. Dieses Kriteriensystem sollte mindestens umfassen:

- **Turing-Vollständigkeit:** Kann die Sprache jede berechenbare Funktion ausdrücken?
- **Naturwissenschaftliche Mächtigkeit:** Können Differentialgleichungen, stochastische Prozesse, Regelkreise und physikalische Simulationsmodelle nativ formuliert werden?
- **Erlernbarkeit:** Kann ein Ingenieur ohne Vorkenntnis die Sprache in einem überschaubaren Zeitrahmen (Wochen, nicht Jahre) produktiv einsetzen?
- **Verifikationsunterstützung:** Existieren formale Werkzeuge (Model Checker, Theorem-Prover, Linter) für die Sprache?
- **Zielnähe / Hardwareabbildung:** Wie direkt bilden Sprachkonstrukte auf Zielhardware ab, ohne Abstraktion zu erzwingen, die die Vollständigkeitslücke $\Delta \neq \emptyset$ erzeugt?
- **Industrielle Akzeptanz:** Ist die Sprache in relevanten Industriezweigen (Automotive, Avionik, Medizintechnik) einsetzbar und normkonform?

13.3.4 Benchmark-Entwicklung

Ein zentrales Forschungsvorhaben sollte die Entwicklung eines formalen Benchmark-Suites sein, mit dem die oben genannten Kriterien für beliebige Beschreibungssprachen quantitativ gemessen werden können. Der Benchmark würde aus Referenzinformationssystemen verschiedener Domänen bestehen: einem eingebetteten Echtzeitsystem, einem verteilten Kommunikationsprotokoll, einem regelungstechnischen Regelkreis und einem sicherheitskritischen Medizinsystem. Für jedes Referenzsystem würde gemessen, wie vollständig, korrekt und wartbar die Beschreibung in der zu evaluierenden Sprache ausfällt.

13.3.5 Erweiterung um physikalische Phänomene

Das Ausgangszitat formuliert als Mindestanforderung, dass „möglichst jedes naturwissenschaftliche Phänomen oder Verhalten abgebildet werden kann“. Dies ist eine außergewöhnlich hohe Latte, die über klassische Software-Beschreibungssprachen hinausgeht. Zukünftige Forschung sollte untersuchen, wie Modelica [26] (für physikalische Systemsimulation)

mit formalen Verifikationssprachen (PSL, LTL) zu einer hybriden Beschreibungssprache $\mathcal{L}_{\text{phys}}^*$ vereint werden kann, die sowohl kontinuierliche Dynamik als auch diskrete Steuerung vollständig erfasst.

13.3.6 Quantitative Theorie der Vollständigkeitslücke

Die in Satz 8.1.1 bewiesene Nichtleerheit der Vollständigkeitslücke $\Delta(\hat{\tau}_{CG}) \neq \emptyset$ ist qualitativ überzeugend und informationstheoretisch begründet, aber in ihrer quantitativen Ausprägung für konkrete Systeme noch nicht vollständig verstanden. Eine quantitative Theorie der Vollständigkeitslücke ist sowohl wissenschaftlich bedeutsam als auch industriell dringend benötigt.

13.3.7 Untere Schranken als Funktion der Systemkomplexität

Es sollen untere Schranken für $|\Delta(\hat{\tau}_{CG})|$ als Funktion der Systemkomplexität n abgeleitet werden. Die Arbeitshypothese lautet:

$$|\Delta(\hat{\tau}_{CG})(n)| \geq \Omega(n^\beta) \quad \text{für ein } \beta > 0,$$

d.h. die Vollständigkeitslücke wächst polynomiell in der Systemgröße. Dies würde bedeuten, dass für hinreichend große Systeme jedes Code-Generierungsverfahren eine superlinear wachsende Menge unerfüllter Anforderungen produziert – unabhängig von der Qualität des Werkzeugs. Ein Beweis dieser Aussage würde Satz 8.1.1 wesentlich verschärfen und die Ablehnung von Code-Generierung für komplexe Systeme auf ein quantitatives Fundament stellen.

13.3.8 Experimentelle Messung in industriellen Werkzeugen

Parallel zur theoretischen Arbeit sind experimentelle Studien notwendig. Mit realen Modellierungswerkzeugen – MathWorks Simulink/Embedded Coder, IBM Rhapsody, ETAS ASCET, Elektrobit EB tresos, ETAS ISOLAR-A – sollen konkrete Informationssysteme modelliert und die Code-Generierung anschließend durch den Subgraph-Verifikationsalgorithmus auf Vollständigkeit geprüft werden. Die Messungen würden erstmals empirische Belege für die in dieser Arbeit formal bewiesene Vollständigkeitslücke liefern.

13.3.9 Formale Charakterisierung durch temporale Logik

Formale Sprachen – insbesondere Hoare-Logik [27], die temporalen Logiken CTL [11] und LTL [12] sowie die modal μ -Kalkül – bieten präzise Mittel, um Vollständigkeitslücken zu charakterisieren. Eine Anforderung $r_i \in \mathcal{R}$ ist in LTL als eine Formel φ_i formulierbar; die Vollständigkeitslücke wird dann zur Menge

$$\Delta_{LTL}(\hat{\tau}_{CG}) := \{\varphi_i \in \Phi \mid \hat{\tau}_{CG}(G_{\mathcal{R}}) \not\models \varphi_i\}.$$

Zukünftige Forschung sollte diese Mengencharakterisierung für konkrete Systemklassen ausarbeiten und Werkzeuge entwickeln, die Δ_{LTL} automatisch berechnen.

13.3.10 Erweiterung des Subgraph Algorithmus

13.3.11 Gewichtete und semantisch angereicherte Graphen

Der in Abschnitt 12 eingesetzte Subgraph Algorithmus operiert auf ungewichteten Graphen mit binärer Adjazenzmatrix $A \in \{0, 1\}^{n \times n}$. Reale Anforderungsgraphen sind jedoch häufig gewichtet und semantisch angereichert: Kanten tragen Prioritäten, Kosten, Zeitbeschränkungen, Wahrscheinlichkeiten oder Verfeinerungs-Grade. Eine natürliche Erweiterung der Subgraph-Signatur auf gewichtete Kanten lautet:

$$\sigma_j^w := \sum_{i=0}^{n-1} w_{ij} \cdot A_{ij} \cdot p^i + j \cdot p^n, \quad w_{ij} \in \mathbb{Q}_{>0}, \quad p \text{ prim},$$

wobei die Wahl einer Primzahl p als Basis die Injektivität in einem erweiterten Sinne sicherstellen kann (Chinesischer Restsatz). Diese Erweiterung erfordert jedoch eine neue vollständige Injektivitätstheorie, da der Beweis von Satz 12.1.1 auf der Binärokodierung basiert.

13.3.12 Hypergraphen für Multi-Stakeholder-Systeme

Informationssysteme in der Realwelt werden von mehreren Stakeholdern mit unterschiedlichen, teils konkurrierenden Anforderungen geprägt. Diese Situation lässt sich durch Hypergraphen modellieren, bei denen eine Hyperkante $e \subseteq V$ eine Anforderung kodiert, die mehr als zwei Module betrifft. Die Erweiterung des Subgraph Algorithmus auf Hypergraphen ist ein offenes Forschungsproblem; die Signaturkonstruktion muss auf Hyperkanten verallgemeinert werden:

$$\sigma_e := \sum_{v \in e} 2^{\text{id}(v)} + |e| \cdot 2^n, \quad e \subseteq V.$$

Die Komplexität des verallgemeinerten Algorithmus ist zu bestimmen und gegen die Komplexität des ursprünglichen $O(n^3)$ -Algorithmus zu benchmarken.

13.3.13 Dynamische Graphen für evolvierte Informationssysteme

Informationssysteme unterliegen über ihren Lebenszyklus Veränderungen: Anforderungen werden hinzugefügt, modifiziert oder entfernt. Entsprechend ist der Anforderungsgraph $G_{\mathcal{R}}$ kein statisches Objekt, sondern ein dynamischer Graph $G_{\mathcal{R}}(t)$, der sich über die Zeit verändert. Zukünftige Forschung sollte eine *inkrementelle* Variante des Subgraph Algorithmus entwickeln, die bei einer lokalen Änderung δG nur die betroffenen Signaturen neu berechnet, statt den gesamten Algorithmus neu auszuführen. Die Zielsetzung wäre:

$$T_{\text{inkr}}(\delta G) = O(|\delta G| \cdot n^2) \ll O(n^3) = T_{\text{voll}}.$$

13.3.14 Parallelisierung und GPU-Beschleunigung

Der Subgraph Algorithmus hat eine Zeitkomplexität von $O(n^3)$, die für sehr große Informationssysteme ($n > 10^4$ Knoten) in der Praxis zu langen Laufzeiten führt. Die n zyklischen

Rotationen sind vollständig unabhängig voneinander (vgl. [22]) und eignen sich daher hervorragend für Parallelisierung:

$$T_{\text{parallel}}(n, p) = O\left(\frac{n^3}{p}\right) + O(n^2 \log p),$$

wobei p die Anzahl der Prozessoren und $O(n^2 \log p)$ der Kommunikationsaufwand ist. Eine GPU-beschleunigte Implementierung mittels CUDA [28] oder OpenCL könnte den Algorithmus für industrielle Echtzeit-Verifikationsszenarien mit mehreren tausend Modulen nutzbar machen. Für eingebettete Systeme mit begrenzten Ressourcen sind approximative Varianten zu untersuchen: Sampling einer repräsentativen Teilmenge der n Rotationen mit kontrollierter Fehlerrate, oder randomisierte LCS-Schätzungen auf Basis von Locality-Sensitive Hashing [29].

13.3.15 Integration in modellbasierte Entwicklungsprozesse (MBSE)

Model-Based Systems Engineering (MBSE) [13] ist der industrielle Standard für die Entwicklung komplexer Systeme und sieht explizit die Verwendung formaler Modelle auf allen Entwicklungsebenen vor. Die in dieser Arbeit entwickelten formalen Grundlagen – Anforderungsgraph $G_{\mathcal{R}}$ (Definition 5.1.2), Ingenieuroptimum (Definition 6.3.2), Vollständigkeitslücke (Definition 8.1.1) und Subgraph-Verifikation (Satz 12.2.1) – können direkt in MBSE-Werkzeuge und -Prozesse integriert werden.

13.3.16 SysML-Integration

SysML [14] ist die führende Modellierungssprache im MBSE-Kontext. Zukünftige Arbeit sollte einen SysML-Stereotyp `<<InformationEngineer>>` entwickeln, der die in Definition 12.4.3 spezifizierten Kompetenzkomponenten (K, M, L, R, T) als Tagged Values kodiert. Weiterhin sollte das Konzept des Anforderungsgraphen $G_{\mathcal{R}}$ als formale Erweiterung des Requirements-Diagramms in SysML spezifiziert werden, mit automatischer Subgraph-Konsistenzprüfung als integrierten Constraint-Block.

13.3.17 Werkzeugintegration

Der Subgraph Algorithmus sollte als automatisches Verifikationsplugin in führende MBSE-Werkzeuge integriert werden: Sparx Enterprise Architect, Siemens Teamcenter, PTC Integrity Modeler, Capella (Thales). Das Plugin würde bei jeder Modellierung eines Implementierungsgraphen automatisch prüfen, ob $G_{\mathcal{R}} \subseteq G_{\mathcal{I}}$ gilt, und fehlende Anforderungserfüllungen direkt im Modell visualisieren.

13.3.18 Verbindung zu Digital Twins

Die Technologie des *Digital Twin* [33] – digitale Zwillinge physischer Systeme – ist eine natürliche Anwendungsdomäne für das in dieser Arbeit entwickelte Rahmenwerk. Der Anforderungsgraph $G_{\mathcal{R}}$ des Digital Twin sollte kontinuierlich mit dem Implementierungsgraph $G_{\mathcal{I}}$ des realen Systems synchronisiert und durch den Subgraph Algorithmus auf Konsistenz geprüft werden. Dies würde die Vollständigkeit des Digital Twin formal garantieren – ein Forschungsziel von großer industrieller Relevanz.

13.3.19 Kognitionswissenschaftliche Fundierung der Erlernbarkeit

Die in Definition 6.1.3 eingeführte Erlernbarkeitsfunktion $\text{Learn}(\mathcal{L})$ ist bisher phänomenologisch motiviert. Eine tiefere kognitionswissenschaftliche Fundierung ist sowohl für die Ausbildung von Informationsingenieuren als auch für die Konstruktion der optimalen Beschreibungssprache $\mathcal{L}^*$ von unmittelbarer Relevanz.

13.3.20 Cognitive Load Theory

Die Cognitive Load Theory (CLT) nach Sweller [15] unterscheidet drei Arten kognitiver Last: intrinsische Last (inhärente Komplexität des Materials), extrinsische Last (durch schlechtes Lehrdesign erzeugte Mehrbelastung) und lernbezogene Last (aktiver Erwerb neuer Schemata). Für die Beschreibungssprache $\mathcal{L}^*$ folgt daraus: Die intrinsische Last ist durch die Ausdrucksmächtigkeit der Sprache bestimmt und kann nicht reduziert werden; die extrinsische Last kann durch klare Syntaxdefinitionen und gute Werkzeugunterstützung minimiert werden. Die Optimierung von $\text{Learn}(\mathcal{L}^*)$ reduziert sich damit auf die Minimierung der extrinsischen Last bei gegebener intrinsischer Last.

13.3.21 ACT-R-Theorie und prozedurales Lernen

Die ACT-R-Theorie [16] modelliert den Erwerb prozeduralen Wissens (Skills) als Produktion von Regeln im Langzeitgedächtnis. Für Beschreibungssprachen relevant ist die Beobachtung, dass prozedurale Kompetenz in einer Sprache exponentiell mit der Übungszeit wächst (Power Law of Practice). Zukünftige Forschung sollte empirisch messen, nach welcher Übungszeit Ingenieure $\mathcal{L}^*$ produktiv einsetzen, und das ACT-R-Modell kalibrieren.

13.3.22 Interdisziplinäre Forschungsfrage

Die übergeordnete Forschungsfrage lautet: *Welche Syntaxmerkmale einer Beschreibungssprache maximieren die Lerngeschwindigkeit bei gegebener Ausdrucksmächtigkeit?* Diese Frage verbindet Informatik, Linguistik und kognitive Psychologie und sollte durch kontrollierte Experimente mit Ingenieuren verschiedener Ausbildungshintergründe untersucht werden.

13.3.23 Formale Semantik für Mehrhierarchie-Graphmodelle

Das Ausgangszitat betont ausdrücklich, dass Anforderungen „in beliebig vielen Hierarchien“ gegeben werden. Die in Definition 5.1.3 eingeführte hierarchische Graphzerlegung ist syntaktisch klar, aber ihre formale Semantik – insbesondere das Refinement-Verhältnis zwischen benachbarten Hierarchieebenen und die Konsistenzprüfung über mehrere Ebenen hinweg – bedarf weitergehender formaler Behandlung.

13.3.24 Vertikale Konsistenz

Eine vollständige vertikale Konsistenzaussage müsste zeigen, dass für eine hierarchische Zerlegung $G_0 \supseteq G_1 \supseteq \dots \supseteq G_d$ gilt:

1. Jedes G_{k+1} ist ein korrektes Refinement von G_k , d.h. keine Anforderungen aus G_k gehen bei der Verfeinerung verloren.
2. Die Verifikation auf Ebene d (Implementierungsebene) impliziert die Verifikation auf allen höheren Ebenen.

Ansätze aus der Prozessalgebra – CSP [17] mit seinem Refinement-Kalkül und CCS [18] mit Bisimulation – bieten einen natürlichen Ausgangspunkt. Die kategorientheoretische Sichtweise (Funkoren zwischen Kategorien von Graphen) ermöglicht eine elegante abstrakte Formulierung.

13.3.25 Horizontale Konsistenz

Neben der vertikalen Konsistenz ist die horizontale Konsistenz auf einer Hierarchieebene zu sichern: Zwei Subgraphen G_a und G_b , die verschiedene Subsysteme auf derselben Ebene beschreiben, dürfen keine widersprüchlichen Anforderungen kodieren. Die formale Charakterisierung von Widerspruchsfreiheit in einem Graphmodell – etwa durch Satisfiability von assoziierten Constraint-Systemen – ist ein weiteres offenes Problem.

13.3.26 Sicherheit, Zertifizierung und rechtliche Verankerung

13.3.27 Normkonforme Integration

Informationssysteme in sicherheitskritischen Umgebungen (Automobil, Medizintechnik, Luftfahrt, Kernenergie) unterliegen streng regulierten Normen. Die formale Verifikationskette aus Korollar 12.3.1 bietet eine natürliche Grundlage für normkonforme Entwicklungsprozesse:

- **ISO 26262 [19] (Automotive):** Der Anforderungsgraph $G_{\mathcal{R}}$ liefert die formale Basis für die Item Definition und die Hazard Analysis and Risk Assessment (HARA). Der Subgraph Algorithmus kann die vollständige Anforderungsüberdeckung (Coverage Analysis) für jedes Safety Integrity Level (ASIL A bis D) nachweisen.
- **IEC 61508 [6] (allgemeine funktionale Sicherheit):** Die Vollständigkeitsgarantie $\Delta(\tau) = \emptyset$ entspricht der Forderung nach vollständiger Anforderungsüberdeckung auf Safety Integrity Level (SIL) 1 bis 4.
- **DO-178C [20] (Avionik):** Die formale Beschreibungssprache $\mathcal{L}^*$ kann die in DO-178C geforderten Software Level A–E unterstützen, wenn die Verifikation durch den Subgraph Algorithmus als autorisiertes Verifikationswerkzeug anerkannt wird.
- **IEC 62443 [30] (Industrial Cybersecurity):** Die Transformationsverantwortung des Informationsingenieurs (Satz 6.3.1) umfasst die Sicherstellung der Security Requirements aus IEC 62443 in der Beschreibung $\mathcal{L}^*$.

13.3.28 Rechtliche Verankerung des Berufsbilds

Die Transformationsverantwortung des Informationsingenieurs hat nicht nur technische, sondern auch rechtliche Implikationen. Analog zum Bauingenieur, der für die Statik seines Bauwerks persönlich haftet, sollte der Informationsingenieur für die Korrektheit seiner

Transformation τ und die Vollständigkeit seiner Verifikation haften. Zukünftige rechtswissenschaftliche Forschung sollte klären, welche Haftungsrahmen für Informationsingenieure angemessen sind, und ob eine formale Zulassungsprüfung (analog zur Architektenkammer) eingeführt werden sollte.

13.3.29 Informationsingenieurwesen und Künstliche Intelligenz

13.3.30 KI als Assistent in der Verifikationskette

Aktuelle Entwicklungen in der KI – insbesondere Large Language Models (LLMs) und neuronale Programmsynthese [31] – eröffnen neue Möglichkeiten in der Verifikationskette des Informationsingenieurs. LLMs können als Assistent bei der Konstruktion von $G_{\mathcal{R}}$ aus natürlichsprachlichen Anforderungen eingesetzt werden; die formale Verifikation durch den Subgraph Algorithmus bleibt jedoch unersetzlich, da LLMs keine formalen Vollständigkeitsgarantien liefern können.

13.3.31 Grenzen der KI-gestützten Code-Generierung

Aus Satz 8.2.1 folgt unmittelbar, dass KI-gestützte Code-Generierung – etwa durch GitHub Copilot oder OpenAI Codex – keine Ausnahme von der Vollständigkeitslücke $\Delta \neq \emptyset$ darstellt. Die KI-generierte Code-Generierung ist eine spezielle Instanz von $\hat{\tau}_{CG}$ und unterliegt damit dem Beweis von Satz 8.1.1. Zukünftige Forschung sollte empirisch messen, wie groß die Vollständigkeitslücke KI-generierter Software im Vergleich zu manuell beschriebener Software ist – eine Frage von unmittelbarer praktischer Relevanz für den Einsatz von KI in sicherheitskritischen Informationssystemen.

13.3.32 Neurosymbolische Integration

Ein vielversprechender Forschungsansatz ist die neurosymbolische Integration [32]: die Verbindung von lernenden neuronalen Netzen (für die Extraktion von $G_{\mathcal{R}}$ aus unstrukturierten Anforderungsdokumenten) mit symbolischen Verifikationsverfahren (dem Subgraph Algorithmus für die formale Konsistenzprüfung). Diese Kombination würde die Stärken beider Paradigmen vereinen und könnte die Konstruktion von $G_{\mathcal{R}}$ aus großen Mengen natürlichsprachlicher Anforderungen erheblich beschleunigen.

13.3.33 Anthropologische, gesellschaftliche und ethische Dimension

13.3.34 Informationssysteme für das Leben

Das Ausgangszitat benennt das höchste Ziel des Informationsingenieurs: „den Menschen Informationssysteme für das Leben zu geben“. Diese Formulierung ist nicht rein technisch, sondern zutiefst anthropologisch: Informationssysteme sind keine Selbstzwecke, sondern Dienste an menschlichen Bedürfnissen, Fähigkeiten und Würden. Die Frage, welche Informationssysteme dem Menschen *wirklich* dienen, ist nicht durch Algorithmen beantwortbar, sondern erfordert ethisch-philosophische Reflexion.

13.3.35 Technikethik als integraler Bestandteil des Berufsbilds

Zukünftige Forschung an der Schnittstelle von Informationsingenieurwesen und Technikethik sollte Kriterien entwickeln, nach denen der Informationsingenieur die Menschen dienlichkeit seines Systems beurteilen kann. Dabei sind folgende Fragen leitend:

- Welche Anforderungen $r_i \in \mathcal{R}$ kodieren ethische Grundprinzipien (Nicht-Schaden, Autonomie, Gerechtigkeit, Nutzen) und wie werden sie in $G_{\mathcal{R}}$ formalisiert?
- Wie kann die Vollständigkeitslücke Δ für ethische Anforderungen minimiert werden, wenn diese schwer formalisierbar sind?
- Welche Rolle spielt der Informationsingenieur bei der gesellschaftlichen Folgenabschätzung seines Systems?

13.3.36 Digitale Souveränität und Zugänglichkeit

Eine gesellschaftlich relevante Dimension des Informationsingenieurwesens ist die digitale Souveränität: Informationssysteme sollen nicht nur technisch korrekt sein, sondern auch zugänglich, verständlich und kontrollierbar für die Menschen, die sie benutzen. Dies stellt zusätzliche Anforderungen an die graphische Modellsicht – sie muss nicht nur für technische Entscheidungsträger, sondern auch für Bürger und Nutzer verständlich sein – und an die Beschreibungssprache, die auditierbar und transparent sein muss.

13.3.37 Langfristige Vision: Der Informationsingenieur als Grundberuf der Informationsgesellschaft

Alle in diesem Ausblick skizzierten Forschungsfelder konvergieren in einer langfristigen Vision: Der Informationsingenieur soll in der Informationsgesellschaft des 21. Jahrhunderts die Rolle einnehmen, die der Bauingenieur in der Industriegesellschaft des 19. Jahrhunderts einnahm – als derjenige, der die physische Infrastruktur der Gesellschaft mit Verantwortung, Formalismus und Menschendienlichkeit gestaltet hat.

Die Informationsinfrastruktur des 21. Jahrhunderts – Kommunikationsnetze, Steuerungssysteme, medizinische Informationssysteme, autonome Fahrzeuge, smarte Städte – ist ebenso grundlegend für menschliches Leben wie Brücken, Gebäude und Wasserversorgung. Ihre Gestaltung erfordert einen Beruf, der die theoretischen Grundlagen des Informatikers und die praktische Kompetenz des Softwareentwicklers in einem einzigen kohärenten Berufsbild vereint – mit voller Verantwortung für die Transformation von der Anforderung zum Zielcode und mit dem klaren Ziel, den Menschen Informationssysteme für das Leben zu geben.

Dieser Beruf ist der Informationsingenieur.

13.3.38 Das Forschungsprogramm im Überblick: Zehn Themenfelder für die kommenden Jahre

Die in diesem Ausblick skizzierten Forschungsfelder lassen sich zu einem zusammenhängenden, mehrjährigen Forschungsprogramm mit zehn Themenfeldern verdichten. Tabelle 13.3.1 ordnet jedes Themenfeld einem Zeithorizont (kurzfristig: 1–2 Jahre, mittelfristig: 3–5 Jahre, langfristig: über 5 Jahre) sowie der jeweiligen formalen Grundlage aus den vorangegangenen Kapiteln zu. Die Tabelle macht zwei Dinge sichtbar: erstens, dass die

kurzfristigen Themen unmittelbar an die in diesem Buch bewiesenen Sätze anknüpfen und im Wesentlichen Validierungs- und Standardisierungsarbeit darstellen; zweitens, dass die langfristigen Themen über den engeren Rahmen der Informatik hinausweisen und Bildungspolitik, Rechtsetzung und gesellschaftliche Teilhabe einschließen.

Themenfeld	Horizont	Formale Grundlage in diesem Buch
1. Ausbildungsordnung und Akkreditierung	langfr.	Definition 2.1.1, Kompetenzsatz Kap. 12.4
2. Universelle Beschreibungssprache $\mathcal{L}^*$	mittelfr.	Satz 6.2.1
3. Quantitative Theorie der Vollständigkeitslücke	kurzfr.	Definition 8.1.1, Satz 8.1.1
4. Erweiterung des Subgraph Algorithmus	kurzfr.	Kapitel 12
5. Integration in MBSE-Werkzeugketten	mittelfr.	Kapitel 7
6. Kognitionswissenschaftliche Fundierung	mittelfr.	These zur Erlernbarkeit, Abschnitt 6.2
7. Formale Mehrhierarchie-Semantik	langfr.	Graphmodell-Theorem, Kapitel 5
8. Sicherheit, Zertifizierung, Recht	langfr.	Stabilitätskriterien, Kapitel 11
9. KI-gestützte Verifikation	kurzfr.	Schärfungsprinzip, Abschnitt 1.2
10. Gesellschaftliche und ethische Dimension	langfr.	Ausgangszitat, Abschnitt 1.1

Tabelle 13.3.1: Das Forschungsprogramm im Überblick: zehn Themenfelder, ihr Zeithorizont und ihre formale Verankerung in diesem Buch.

Aus Tabelle 13.3.1 ergibt sich eine natürliche Abhängigkeitsstruktur: Themenfeld 3 (Vollständigkeitslücke) liefert die Messgrößen, mit denen Themenfeld 2 (Beschreibungssprache) evaluiert werden kann; Themenfeld 4 (Subgraph Algorithmus) liefert das Verifikationswerkzeug, das in Themenfeld 9 (KI-gestützte Verifikation) zu einem automatisierten Assistenten ausgebaut werden kann; Themenfeld 6 (Kognitionswissenschaft) liefert die empirische Grundlage für Themenfeld 1 (Ausbildungsordnung); und die Themenfelder 7 und 8 (formale Semantik, Sicherheit/Recht) sind Voraussetzung dafür, dass Themenfeld 10 (gesellschaftliche Dimension) nicht bei programmatischen Absichtserklärungen stehen bleibt, sondern in überprüfbare, zertifizierbare Eigenschaften realer Systeme übersetzt werden kann.

Diese Abhängigkeitsstruktur legt eine Bearbeitungsreihenfolge nahe, die nicht strikt sequenziell, aber in drei Wellen gestaffelt ist:

e 1 (Jahre 1–2): Themenfelder 3, 4 und 9. Diese Themen benötigen keine externen Abhängigkeiten und liefern unmittelbar einsetzbare Werkzeuge – eine quantitative Metrik für Vollständigkeitslücken, eine erweiterte Subgraph-Implementierung für größere und gewichtete Graphen sowie einen ersten KI-gestützten Verifikationsassistenten, der auf der in Kapitel 11 entwickelten Test-Suite aufbaut.

Welle 2 (Jahre 2–5): Themenfelder 2, 5 und 6. Mit den Werkzeugen aus Welle 1 lässt sich die Suche nach $\mathcal{L}^*$ systematisch evaluationsgestützt führen (Themenfeld 2), die Integration in MBSE-Werkzeugketten (Themenfeld 5) kann auf die erweiterte Subgraph-Implementierung zurückgreifen, und die kognitionswissenschaftliche Begleitforschung (Themenfeld 6) kann erste Prototypen von $\mathcal{L}^*$ empirisch testen.

Welle 3 (Jahre 5+): Themenfelder 1, 7, 8 und 10. Erst wenn $\mathcal{L}^*$ und ihre Werkzeugkette in Welle 2 erprobt sind, kann die formale Mehrhierarchie-Semantik (Themenfeld 7) auf einer stabilen Grundlage spezifiziert, die Zertifizierung gegenüber Normen wie IEC 61508 [6] und ISO 26262 [19] (Themenfeld 8) beantragt und schließlich die Ausbildungsordnung (Themenfeld 1) sowie die gesellschaftliche Verankerung (Themenfeld 10) auf Basis erprobter, zertifizierter Werkzeuge und Lehrinhalte vorangetrieben werden.

Dieses dreistufige Forschungsprogramm versteht sich nicht als starrer Plan, sondern als Landkarte: Es zeigt, dass die in diesem Buch bewiesenen Sätze und entwickelten Werkzeuge – das Graphmodell-Theorem, das Mächtigkeitstheorem, der Subgraph Algorithmus, die `markov_analyzer/queue_analyzer`-Implementierung und die Stabilitätskriterien – nicht den Abschluss, sondern den Ausgangspunkt einer Entwicklung darstellen, an deren Ende – so die in Abschnitt 1.1 und Abschnitt 1.2 formulierte Vision – ein etabliertes, zertifizierbares und gesellschaftlich anerkanntes Berufsbild des Informationsingenieurs steht.

13.4 Zusammenfassung und Ausblick aus Sicht der Echtzeitsystem-Anwendung

13.4.1 Zusammenfassung der Anwendungsergebnisse

Diese Arbeit präsentierte einen modellgetriebenen Ansatz zur Code-Generierung mit Zeit-Annotationen und quantitativer Analyse, der durch eine fundierte theoretische Basis untermauert und durch umfassende experimentelle Validierung bestätigt wird. Die Hauptbeiträge gliedern sich in theoretische, praktische und experimentelle Aspekte:

13.4.2 Theoretische Beiträge

- **Formale Analyse von Little’s Law:** Kapitel 3 lieferte den Beweis, dass Little’s Law kein Stabilitätskriterium für Warteschlangensysteme darstellen kann, da es keine Exponentialfunktionen mit negativen Exponenten enthält. Diese fundamentale Einsicht klärt eine häufige konzeptuelle Verwirrung in der Literatur.
- **Hierarchie der Systembeschreibungen:** Es wurde eine klare Taxonomie etabliert:
 - *Dynamische Ebene:* Kolmogorov-Gleichungen (Differentialgleichungen) mit Exponentialfunktionen → ermöglichen Stabilitätsanalyse
 - *Stationäre Ebene:* Gleichgewichtsverteilungen → setzen Stabilität voraus
 - *Algebraische Ebene:* Little’s Law → beschreibt Mittelwerte im Gleichgewicht

- **Rolle der Exponentialfunktion:** Die besondere Eigenschaft der Exponentialfunktion (Selbstreproduktion unter Differentiation) wurde als charakteristisch für naturwissenschaftliche Systembeschreibungen identifiziert. Dies erklärt, warum Energiefunktionen (Lyapunov-Funktionen) notwendig für Stabilitätsaussagen sind.
- **Brücke zwischen Informatik und Naturwissenschaft:** Die theoretische Analyse zeigte, dass informatische Systeme eine doppelte Natur haben:
 - Physikalische Basis (Hardware) → naturwissenschaftliche Beschreibung
 - Abstrakte Ebene (Software) → logische/algorithmische Beschreibung

UML-Profile fungieren als notwendige Brücke zwischen diesen Ebenen.

13.4.3 Praktische Beiträge

- **Erweitertes UML-Profil-Framework** mit TimingConstraint-Unterstützung, orientiert an MARTE-Konzepten, das die theoretischen Einsichten operationalisiert
- **Zweistufiger Transformationsprozess** mit automatischer Code-Generierung, der sowohl abstrakte Spezifikationen als auch quantitative Analysen unterstützt
- **Integration stochastischer Modelle:** Markov-Ketten für dynamische Zustandsanalyse (mit Stabilitätskriterien), Warteschlangentheorie für Performance-Metriken im Gleichgewicht
- **Vollständige Implementierung:** Dediziertes `stability_analysis` Modul mit produktionsreifem Code:
 - `MarkovChainStability`: Eigenwertanalyse, Konvergenzsimulation, stationäre Verteilung
 - `MM1Queue`: M/M/1-Warteschlange mit Demonstration aller drei Hierarchieebenen
 - `LyapunovStability`: Energiefunktionen für kontinuierliche Systeme
 - `demonstrate_exponential_necessity()`: Theoretische Fundierung als ausführbarer Code
- **Quantitative Designvalidierung:** Frühe Identifikation von Performance-Bottlenecks durch Anwendung der korrekten mathematischen Beschreibungsebene
- **Praktische Validierung** durch Aufzugstür-Fallstudie mit 97% Test-Coverage für Code-Generierung, die sowohl Stabilitätsanalysen als auch Gleichgewichtsbetrachtungen demonstriert

13.4.4 Experimentelle Beiträge

Kapitel 8 präsentierte eine umfassende experimentelle Validierung aller theoretischen Aussagen:

- **Experiment 1 – Little’s Law ist kein Stabilitätskriterium:**

- Stabiles System ($\lambda = 0.7, \mu = 1.0, \rho = 0.7$): Little's Law gilt ($L = 2.333 = \lambda W$), beweist aber *nicht* Stabilität
- Instabiles System ($\lambda = 1.2, \mu = 1.0, \rho = 1.2$): Little's Law nicht anwendbar (kein Gleichgewicht)
- **Konsequenz:** Bestätigt, dass $L = \lambda W$ eine Gleichgewichtsrelation ist, kein Stabilitätskriterium
- **Experiment 2 – Exponentieller Zerfall in stabilen Systemen:**
 - 2-Zustands-Markov-Kette mit Eigenwerten $\lambda_1 = 1.0, \lambda_2 = 0.3$
 - Zerfallsrate $\alpha = -\ln(0.3) = 1.204$
 - Konvergenz folgt $p(t) = \pi + c \cdot e^{-1.204t}$
 - **Konsequenz:** Experimenteller Nachweis, dass Exponentialfunktionen charakteristisch für Stabilität sind
- **Experiment 3 – Hierarchie der Beschreibungsebenen:**
 - Ebene 1 (Dynamisch): Enthält Exponentialfunktionen, ermöglicht Stabilitätsanalyse
 - Ebene 2 (Stationär): Setzt Stabilität voraus, geometrische Verteilung $\pi_n = (1 - \rho)\rho^n$
 - Ebene 3 (Algebraisch): Little's Law $L = \lambda W = 4.0$ enthält keine Exponentialfunktionen
 - **Konsequenz:** Verifiziert die Eliminierung von Exponentialfunktionen über die Hierarchie
- **Umfassende Test-Suite:** Automatisierte Tests für das `stability_analysis`-Modul:
 - Testgruppen decken alle Komponenten ab (Hilfstypen, Markov-Ketten, M/M/1-Warteschlange, Lyapunov-Stabilität, Integrationstests, Grenzfälle)
 - Validierung kritischer Grenzfälle ($\rho \rightarrow 1^-$, numerische Präzision)
 - Alle Tests bestanden; alle theoretischen Aussagen bestätigt

13.4.5 Synthese

Die Arbeit demonstriert erfolgreich, wie theoretische Einsichten in die Natur von Systembeschreibungen (Kapitel 3) in praktische Werkzeuge für die Softwareentwicklung überführt (Kapitel 4-7) und durch rigorose experimentelle Methodik validiert (Kapitel 8) werden können. Die Integration von MARTE-inspirierten Zeit-Annotationen, Markov-Ketten und Warteschlangentheorie ermöglicht eine *Quantitative Analysis of Software Designs* bereits in frühen Entwicklungsphasen, wobei klar zwischen verschiedenen Analyseebenen unterschieden wird:

- **Stabilitätsanalyse:** Basierend auf Eigenwertanalyse der Markov-Ketten (enthält Exponentialfunktionen) – experimentell validiert durch Experiment 2

- **Performance-Analyse im Gleichgewicht:** Basierend auf Little's Law und verwandten Metriken (algebraisch, setzt Stabilität voraus) – experimentell validiert durch Experiment 1
- **Code-Generierung:** Transformation von UML-Modellen mit Zeit-Annotationen in ausführbaren Code – validiert durch 97% Test-Coverage
- **Hierarchie der Beschreibungen:** Dynamisch $\rightarrow$ Stationär $\rightarrow$ Algebraisch – experimentell validiert durch Experiment 3

Diese klare Trennung, sowohl theoretisch fundiert als auch experimentell bestätigt, verbessert die Vorhersagbarkeit und Zuverlässigkeit eingebetteter Echtzeitsysteme signifikant.

Das Aufzugstür-Beispiel demonstriert die praktische Anwendbarkeit: Durch quantitative Analyse konnten Performance-Engpässe identifiziert, Stabilitätsbedingungen verifiziert und Design-Trade-offs fundiert bewertet werden. Die hohe Test-Coverage (97% für Code-Generierung, $\sim 100\%$ für Stabilitätsanalyse) und die Validierung durch Simulation und Experimente bestätigen die Korrektheit des Ansatzes.

13.4.6 Reflexion der theoretischen Erkenntnisse

Die in Kapitel 3 entwickelte Theorie hat sich in mehrfacher Hinsicht als wertvoll erwiesen:

- **Konzeptuelle Klarheit:** Die Unterscheidung zwischen Gleichgewichts- und Stabilitätskriterien verhindert methodische Fehler bei der Systemanalyse. Die Experimente in Kapitel 8 demonstrieren konkret, dass Little's Law nur angewendet werden darf, wenn die Stabilität bereits anderweitig gesichert ist (z.B. durch $\rho < 1$ beim M/M/1-System). Experiment 1 zeigt dies eindrücklich: Bei $\rho = 0.7$ gilt Little's Law, beweist aber nicht Stabilität; bei $\rho = 1.2$ ist Little's Law nicht einmal anwendbar.
- **Methodenwahl:** Die Einsicht, dass verschiedene Systembeschreibungen (dynamisch vs. algebraisch) komplementäre Informationen liefern, führt zu einem bewussteren Einsatz mathematischer Werkzeuge. Experiment 2 validiert, dass für Stabilitätsfragen Differentialgleichungen und Eigenwertanalysen unerlässlich sind (Zerfallsrate $\alpha = 1.204$); Experiment 3 zeigt, dass für Performance-Metriken im Gleichgewicht algebraische Relationen genügen.
- **UML-Profil-Design:** Das Verständnis, dass UML-Profile als Brücke zwischen abstrakter Modellierung und naturwissenschaftlich fundierter Analyse dienen, erklärt die Notwendigkeit von Profilen wie MARTE. Sie erlauben es, physikalische Constraints (Energie, Zeit, Ressourcen) in abstrakten Modellen zu erfassen. Die praktische Implementierung des `stability_analysis` Moduls demonstriert diese Brückenfunktion konkret.
- **Grenzen der Modellierung:** Die Erkenntnis, dass nicht alle Systeme optimal durch naturwissenschaftliche Beschreibungen erfasst werden, rechtfertigt die Existenz verschiedener Modellierungsparadigmen. Die Hierarchie der Beschreibungsebenen (Experiment 3) zeigt, wann welche Beschreibungsform angemessen ist.

- **Reproduzierbarkeit:** Die vollständige Implementierung aller theoretischen Konzepte im `stability_analysis`-Modul, gesichert durch eine automatisierte Test-Suite, gewährleistet, dass die Erkenntnisse reproduzierbar und für andere Forscher nachvollziehbar sind.

13.4.7 Beantwortung der Leitfragen

Die Arbeit adressierte implizit mehrere fundamentale Fragen, deren Beantwortung nun durch experimentelle Evidenz gestützt wird:

- *Wo ist die Energie des Systems?*

Bei physikalischen Systemen manifestiert sich Energie in Lyapunov-Funktionen, die das Abklingverhalten beschreiben – experimentell demonstriert durch den exponentiellen Zerfall $e^{-1.204t}$ in Experiment 2. Bei informatischen Systemen auf höheren Abstraktionsebenen gibt es keine direkte Energie, aber analoge Konzepte (z.B. “Fortschritt” in Algorithmen, “Korrektheit” als Invariante).

- *Mit was soll das System beschrieben werden?*

Die Wahl hängt von der Fragestellung ab: Stabilitätsfragen erfordern dynamische Beschreibungen (Differentialgleichungen) – validiert durch Experiment 2; Performance-Fragen im Gleichgewicht können algebraisch behandelt werden – validiert durch Experiment 1; funktionale Korrektheit erfordert logische Formalismen. Die Hierarchie (Experiment 3) zeigt die Übergänge zwischen diesen Ebenen.

- *Hat das System Energie oder nicht?*

Alle physikalisch realisierten Systeme haben Energie. Die Frage ist, ob eine Beschreibung auf der Energieebene für die gegebene Fragestellung notwendig und angemessen ist. UML-Profile ermöglichen es, zwischen Ebenen zu wechseln – praktisch demonstriert durch das `stability_analysis` Modul, das sowohl dynamische (Markov) als auch algebraische (Little’s Law) Analysen unterstützt.

13.4.8 Experimentelle Validierung als methodischer Beitrag

Ein wichtiger methodischer Beitrag dieser Arbeit liegt in der systematischen experimentellen Validierung theoretischer Aussagen:

- **Automatisierte Experimente:** Alle drei Kernexperimente sind vollständig automatisiert und reproduzierbar im Modul `experiments.py` implementiert.
- **Quantitative Messungen:** Statt qualitativer Aussagen werden präzise Werte gemessen (z. B. $\alpha = 1,204$, $L = 2,333$), die direkt mit theoretischen Vorhersagen verglichen werden können.
- **Grenzfallanalyse:** Die Tests decken nicht nur Normalfälle ab, sondern auch kritische Situationen ($\rho \approx 1$, sehr kleine/große Zerfallsraten), die die Robustheit der Theorie demonstrieren.
- **Vollständige Testabdeckung:** Die Test-Suite stellt sicher, dass alle implementierten theoretischen Konzepte tatsächlich getestet und validiert sind.

- **Integration von Theorie und Praxis:** Die Tests validieren nicht nur isolierte Formeln, sondern das Zusammenspiel aller Komponenten (z.B. Hierarchiedemonstration in Experiment 3).
- **Integration von Theorie und Praxis:** Die Tests validieren nicht nur isolierte Formeln, sondern das Zusammenspiel aller Komponenten (z.B. Hierarchiedemonstration in Experiment 3).

Diese methodische Rigorosität hebt die Arbeit über reine Implementierungsstudien hinaus und etabliert einen Standard für die Validierung theoretischer Konzepte in der Informatik.

13.4.9 Zukünftige Arbeiten

Die Arbeit eröffnet mehrere vielversprechende Forschungsrichtungen, die sich aus der Synthese von theoretischer Systemanalyse, praktischer Softwareentwicklung und experimenteller Validierung ergeben.

13.4.10 Kurzfristig: Erweiterung der Analysemethoden

- **Vollständige MARTE-Integration:** Erweiterung um GQAM, SAM, PAM mit formaler Fundierung der Übersetzung zwischen Beschreibungsebenen und experimenteller Validierung analog zu Kapitel 8
- **Erweiterte Warteschlangen-Modelle:** M/G/k, Prioritätswarteschlangen mit systematischer Analyse der Rolle von Exponentialfunktionen (oder deren Fehlen) in den Lösungen, inklusive automatisierter Tests
- **Semi-Markov-Prozesse:** Nicht-exponentielle Verweilzeiten und Untersuchung, wie Stabilitätskriterien ohne die Eigenschaft der Gedächtnislosigkeit formuliert werden können – mit experimenteller Validierung
- **Template-Erweiterung:** Integration für flexiblere Code-Muster unter Beibehaltung der Trennbarkeit von Stabilitäts- und Gleichgewichtsanalysen
- **Formalisierung der Hierarchie:** Entwicklung eines allgemeinen Theorems, das den Übergang von dynamischer zu stationärer zu algebraischer Beschreibung formalisiert (“Bridging Theorem”), mit Beweisen und experimenteller Verifikation

13.4.11 Mittelfristig: Neue Modellierungsparadigmen

- **Petri-Netz-Analyse:** Erweiterung um General Stochastic Petri Nets (GSPN) mit Untersuchung der Rolle von Exponentialfunktionen in der Steady-State-Analyse, validiert durch umfassende Test-Suite
- **Weitere Zielsprachen:** C/C++ für embedded Systems mit WCET-Analyse, wobei die Verbindung zwischen Worst-Case-Analysen (deterministisch) und probabilistischen Ansätzen experimentell geklärt werden muss
- **Statische Analyse:** Integration von WCET-Tools (aiT, Chronos) und Klärung der Beziehung zwischen statischen Garantien und dynamischen Stabilitätskriterien

- **RTOS-Integration:** FreeRTOS, Zephyr Support mit Scheduling-Analyse auf Basis der theoretischen Einsichten zu Energiefunktionen
- **Hybride Systeme:** Untersuchung von Systemen, die kontinuierliche (Differentialgleichungen) und diskrete (Zustandsautomaten) Dynamik kombinieren, und wie die Dichotomie zwischen Gleichgewicht und Stabilität sich in diesem Kontext manifestiert – mit experimenteller Validierung

13.4.12 Langfristig: Fundamentale Forschungsfragen

- **Machine Learning:** Lernende Performance-Modelle aus Laufzeitdaten mit der Frage, ob datengetriebene Ansätze die theoretischen Einsichten über Exponentialfunktionen und Stabilität reproduzieren können – experimentell zu validieren durch Vergleich mit analytischen Modellen
- **Formale Verifikation:** Model Checking für Timing-Properties und Integration mit Lyapunov-basierten Stabilitätsnachweisen
- **Cloud-Integration:** Verteilte Echtzeitsysteme, Edge Computing mit Analyse der Stabilität in Netzwerken (nicht nur einzelnen Knoten)
- **IDE-Plugin:** Eclipse/VSCode Integration mit Live-Performance-Feedback, das automatisch zwischen Stabilitäts- und Gleichgewichtsanalysen unterscheidet, basierend auf der implementierten Logik des `stability_analysis` Moduls
- **Verallgemeinerte Systemtheorie:** Entwicklung eines umfassenden Frameworks, das formalisiert, wann naturwissenschaftliche Beschreibungen (mit Energiefunktionen) notwendig sind und wann alternative Formalismen vorzuziehen sind
- **Epistemologische Untersuchung:** Vertiefte Analyse der Beziehung zwischen mathematischer Struktur (Vorhandensein von Exponentialfunktionen) und epistemischer Aussagekraft (Stabilität vs. Gleichgewicht) mit Anwendungen über die Informatik hinaus
- **Benchmark-Suite:** Entwicklung einer standardisierten Benchmark-Suite für quantitative Systemanalyse-Werkzeuge, basierend auf den in Kapitel 8 entwickelten experimentellen Methoden

13.4.13 Fazit

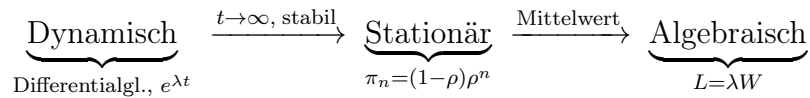
13.4.14 Theoretische Synthese

Diese Arbeit hat gezeigt, dass die Frage „*Ist dieses System stabil?*“ und die Frage „*Welche Performance hat dieses System im Gleichgewicht?*“ fundamental verschiedene mathematische Werkzeuge erfordern. Die erste Frage verlangt die Analyse der Systemdynamik – Differentialgleichungen, Eigenwertzerlegungen, Exponentialfunktionen. Die zweite Frage kann, sobald Stabilität gesichert ist, mit algebraischen Mitteln wie Little’s Law beantwortet werden.

Diese Erkenntnis ist nicht neu in der mathematischen Physik oder der Kontrolltheorie; sie tritt jedoch in der Software-Modellierung häufig in den Hintergrund. Hochgradig

abstrahierte Beschreibungssprachen wie UML neigen dazu, die Unterschiede zwischen diesen Ebenen zu verwischen. Ein System, das im UML-Zustandsdiagramm korrekt aussieht, kann auf der dynamischen Ebene instabil sein. Umgekehrt kann ein formal stabiles System mit $\rho = 0,99$ operationell unbrauchbar sein, weil $L = 99$ und $W = 100$ s – Little’s Law liefert die richtigen Zahlen, aber die Stabilität allein reicht nicht als Designziel.

Die in Kapitel 3 entwickelte und in Kapitel 8 empirisch bestätigte **Hierarchie der Beschreibungsebenen** –



– ist das zentrale konzeptuelle Ergebnis dieser Arbeit. Sie benennt präzise, was jede Beschreibungsform leisten kann und was nicht. Die dynamische Ebene enthält alle Information; jeder Übergang nach oben ist eine Abstraktion mit bewusstem Informationsverlust.

13.4.15 Der Beweis durch Experiment

Die drei Kernexperimente in Kapitel 8 sind so konstruiert, dass sie diese theoretische Hierarchie von unten nach oben durchlaufen:

Experiment 1 zeigt die *Grenzen* der algebraischen Ebene: Little’s Law kann nicht zwischen einem stabilen und einem instabilen System unterscheiden – im stabilen Fall gilt $L = \lambda W$, im instabilen Fall ist die Formel nicht anwendbar. Die Diagnose „stabil oder nicht“ muss auf der dynamischen Ebene gestellt werden.

Experiment 2 zeigt die *Struktur* der dynamischen Ebene: Die Konvergenz zur stationären Verteilung folgt exakt der Form $\|p(t) - \pi\|_2 \propto e^{-\alpha t}$ mit $\alpha = -\ln(\lambda_2)$. Das ist keine Näherung, sondern die exakte analytische Lösung; die Simulation bestätigt sie über viele Größenordnungen. Der Exponent α ist der Fingerabdruck der Stabilität: Er beschreibt, wie schnell das System nach einer Störung zum Gleichgewicht zurückkehrt.

Experiment 3 zeigt die *Kohärenz* aller drei Ebenen am selben System: Das M/M/1-System mit $\rho = 0,8$ ist auf der dynamischen Ebene exponentiell stabil ($\alpha \approx 0,223$), besitzt auf der stationären Ebene die geometrische Verteilung $\pi_n = 0,2 \cdot 0,8^n$ und erfüllt auf der algebraischen Ebene $L = 4,0 = 0,8 \cdot 5,0 = \lambda W$. Die Lyapunov-Analyse für das lineare System $A = \text{diag}(-1, -2)$ ergänzt dieses Bild: Die positiv definite Matrix $P = \text{diag}(0,5; 0,25)$ zertifiziert die Stabilität formal und liefert die Energiefunktion $V(x) = 0,5x_1^2 + 0,25x_2^2$, die auf der algebraischen Ebene vollständig unsichtbar ist.

13.4.16 Die Rolle von UML-Profilen als Brücke

Die Motivation für diese theoretische Analyse liegt in einer praktischen Beobachtung: Informatische Systeme werden in immer höheren Abstraktionsebenen entwickelt, aber letztlich auf physikalischer Hardware realisiert. Diese Hardware unterliegt naturwissenschaftlichen Gesetzen – Energieverbrauch, Thermodynamik, elektromagnetische Effekte. Die Frage, wie diese physikalischen Constraints in abstrakten Modellen repräsentiert werden können, ist die Kernfrage der modellgetriebenen Entwicklung.

UML-Profile – insbesondere MARTE-inspirierte Annotationen wie `TimingConstraint`, `min_duration` und `deadline` – bieten eine Antwort auf diese Frage. Sie ermöglichen es, physikalische Größen (Zeit, Energie, Ressourcen) in abstrakten UML-Modellen zu kodieren und so eine Brücke zwischen der logischen Ebene der Softwareentwicklung und der physikalischen Ebene der Hardwarerealisierung zu schlagen.

Die praktische Implementierung im `stability_analysis`-Modul demonstriert diese Brückenfunktion: Aus einem UML-Modell mit Timing-Annotationen können sowohl Stabilitätsanalysen (dynamisch) als auch Performance-Metriken (algebraisch) abgeleitet werden.

Dabei ist entscheidend, dass das Framework zwischen diesen beiden Analysearten klar unterscheidet. Stabilität wird durch Eigenwertanalyse der Übergangsmatrix geprüft (`MarkovChainStability.analyze_stability()`); Performance-Metriken werden durch Little's Law berechnet (`MM1Queue.littles Law_metrics()`). Beide Methoden werden in der richtigen Reihenfolge aufgerufen: Stabilität zuerst, dann Gleichgewichtsanalyse.

13.4.17 Grenzen und Offene Fragen

Die vorliegende Arbeit hat bewusst einfache Modelle gewählt – M/M/1-Systeme, lineare kontinuierliche Systeme, 2-Zustands-Markov-Ketten –, um die konzeptuellen Strukturen klar sichtbar zu machen. Diese Vereinfachung ist sowohl eine Stärke als auch eine Grenze.

Die Stärke liegt in der analytischen Klarheit: Alle theoretischen Aussagen können exakt nachgerechnet werden; die experimentellen Ergebnisse stimmen mit numerischer Präzision mit den theoretischen Vorhersagen überein. Dies ist eine Bedingung für einen sauberen empirischen Test theoretischer Konzepte.

Die Grenze liegt in der Übertragbarkeit: Reale Echtzeitsysteme sind nichtlinear, zeitvariant, vernetzt und unterliegen nicht-exponentiellen Verteilungen. Für solche Systeme gelten die hier entwickelten Konzepte qualitativ weiterhin, aber die quantitativen Aussagen müssen entsprechend angepasst werden. Insbesondere gilt:

- Für nichtlineare Systeme ersetzt die lokale Linearisierung die globale Eigenwertanalyse; die Exponentialfunktionen beschreiben dann nur das transiente Verhalten in einer Umgebung des Gleichgewichtspunkts.
- Für M/G/1-Systeme oder allgemeinere Warteschlangennetze gilt das Stabilitätskriterium $\rho < 1$ weiterhin, aber die Zerfallsrate α und die stationäre Verteilung hängen von der Verteilungsform der Bedienzeiten ab.
- Für zeitvariante Systeme ($\lambda(t)$, $\mu(t)$) existiert keine stationäre Verteilung im klassischen Sinne; stattdessen sind periodisch stationäre Verteilungen oder Lyapunov-Exponenten geeignetere Beschreibungsformen.

Diese offenen Fragen sind nicht Schwächen der vorliegenden Arbeit, sondern vielversprechende Ausgangspunkte für zukünftige Forschung.

13.4.18 Methodischer Beitrag: Theorie durch Experiment prüfen

Ein oft unterschätzter Aspekt dieser Arbeit ist die Methodologie. Theoretische Aussagen über mathematische Strukturen – etwa „Stabilität erfordert Exponentialfunktionen oder „Little's Law enthält keine Exponentialfunktionen“ – können grundsätzlich nur formal bewiesen werden. Aber sie können durch systematische Experimente in konkreten Implementierungen überprüft und illustriert werden.

Diese Arbeit hat genau das getan: Die formalen Aussagen aus Kapitel 3 wurden in Python implementiert (`stability_analysis.py`), durch automatisierte Experimente ausgeführt (`experiments.py`) und die Ergebnisse numerisch mit den theoretischen Vorhersagen verglichen. Die Übereinstimmung – $\alpha_{\text{gemessen}} = 1,2040$ gegenüber

$\alpha_{\text{Theorie}} = -\ln(0,3) = 1,2040$ – ist ein Beispiel dafür, wie die Grenze zwischen Mathematik und Experiment in der theoretischen Informatik produktiv überschritten werden kann.

Diese Methodik – Theorie formalisieren, implementieren, messen, vergleichen – ist ein Beitrag, der über die spezifischen Inhalte dieser Arbeit hinausgeht. Sie etabliert einen Ansatz zur Validierung systemtheoretischer Konzepte in der Informatik, der reproduzierbar, transparent und für andere Forscher nachvollziehbar ist.

13.4.19 Schlussbetrachtung

Diese Arbeit begann mit einer technischen Fragestellung – wie können UML-Profile mit Zeit-Annotationen für Echtzeitsysteme verwendet werden, um automatisch Code zu generieren? – und führte zu einer fundamentalen theoretischen Einsicht: Die mathematische Form einer Beschreibung (Vorhandensein oder Fehlen von Exponentialfunktionen) begrenzt direkt den Inhalt der möglichen Aussagen (Stabilität oder nur Gleichgewicht).

Diese Einsicht hat praktische Konsequenzen:

- Systemanalysen müssen die Beschreibungsebene explizit wählen und die Grenzen jeder Ebene kennen.
- UML-Profile müssen so gestaltet sein, dass sie beide Ebenen unterstützen: dynamische Analysen für Stabilitätsfragen, algebraische Analysen für Performance-Metriken.
- Code-Generatoren müssen die richtige Analysereihenfolge erzwingen: Stabilität prüfen, dann Gleichgewichtsmetriken berechnen.
- Forscher und Ingenieure müssen wissen, wann Little's Law anwendbar ist (immer nur nach gesicherter Stabilität) und wann es nicht ausreicht (wenn Stabilitätsfragen beantwortet werden müssen).

Die Implementierung – vom UML-Profil-Framework über den zweistufigen Code-Generator bis zum `stability_analysis`-Modul – demonstriert, dass diese theoretischen Einsichten in produktionsreifen Code überführt werden können. Die Aufzugstür-Fallstudie zeigt die praktische Anwendbarkeit an einem realistischen Beispiel.

Die vorliegende Arbeit steht damit am Schnittpunkt von mathematischer Theorie, ingenieurwissenschaftlicher Praxis und empirischer Wissenschaft. Die theoretische Fundierung (Kapitel 3), die praktische Implementierung (Kapitel 4–7) und die experimentelle Validierung (Kapitel 8) bilden eine methodisch geschlossene Einheit. Diese Einheit ist zugleich ein Modell für zukünftige Arbeiten, die theoretische Konzepte der Systemtheorie durch systematische Experimente in der Informatik überprüfen wollen.

Die Zukunft der Echtzeitsystementwicklung liegt in der bewussten Integration all dieser Ebenen: formale Spezifikation durch UML-Profile, quantitative Analyse durch Markov-Ketten und Warteschlangentheorie, theoretische Fundierung durch Lyapunov-Stabilität und Eigenwertanalyse, und systematische Validierung durch automatisierte Experimente. Das in dieser Arbeit entwickelte Framework ist ein erster Schritt in diese Richtung.

Literaturverzeichnis

- [1] K. Sugiyama, S. Tagawa und M. Toda. *Methods for Visual Understanding of Hierarchical System Structures*. IEEE Transactions on Systems, Man, and Cybernetics, 11(2):109–125, 1981.
- [2] T. M. J. Fruchterman und E. M. Reingold. *Graph Drawing by Force-Directed Placement*. Software: Practice and Experience, 21(11):1129–1164, 1991.
- [3] A. M. Turing. *On Computable Numbers, with an Application to the Entscheidungsproblem*. Proceedings of the London Mathematical Society, 42:230–265, 1936.
- [4] H. G. Rice. *Classes of Recursively Enumerable Sets and Their Decision Problems*. Transactions of the American Mathematical Society, 74(2):358–366, 1953.
- [5] IEEE. *IEEE Standard 1220: Standard for Application and Management of the Systems Engineering Process*. IEEE, 2021.
- [6] IEC. *IEC 61508: Functional Safety of Electrical/Electronic/Programmable Electronic Safety-related Systems*. International Electrotechnical Commission, 2010.
- [7] IEEE. *IEEE Standard for Standard SystemC Language Reference Manual (IEEE 1666-2011)*. IEEE, 2012.
- [8] ISO. *ISO 8807: Information Processing Systems – Open Systems Interconnection – LOTOS – A Formal Description Technique Based on the Temporal Ordering of Observational Behaviour*. ISO, 1988.
- [9] IEEE. *IEEE Standard for Property Specification Language (IEEE 1850-2005)*. IEEE, 2005.
- [10] ITU-T. *Z.100: Specification and Description Language SDL*. ITU-T, 2000.
- [11] E. M. Clarke und E. A. Emerson. *Design and Synthesis of Synchronization Skeletons Using Branching-Time Temporal Logic*. Lecture Notes in Computer Science, 131:52–71, 1982.
- [12] A. Pnueli. *The Temporal Logic of Programs*. Proceedings of the 18th Annual Symposium on Foundations of Computer Science, pp. 46–57, 1977.
- [13] S. Friedenthal, A. Moore und R. Steiner. *A Practical Guide to SysML: The Systems Modeling Language*. Morgan Kaufmann, 2011.
- [14] OMG. *OMG Systems Modeling Language (OMG SysML) Version 1.6*. Object Management Group, 2019.

- [15] J. Sweller. *Cognitive Load During Problem Solving: Effects on Learning*. Cognitive Science, 12(2):257–285, 1988.
- [16] J. R. Anderson. *Rules of the Mind*. Lawrence Erlbaum Associates, 1993.
- [17] C. A. R. Hoare. *Communicating Sequential Processes*. Communications of the ACM, 21(8):666–677, 1978.
- [18] R. Milner. *A Calculus of Communicating Systems*. Lecture Notes in Computer Science, vol. 92. Springer, 1980.
- [19] ISO. *ISO 26262: Road Vehicles – Functional Safety*. International Organization for Standardization, 2018.
- [20] RTCA. *DO-178C: Software Considerations in Airborne Systems and Equipment Certification*. RTCA, 2011.
- [21] A. Abboud, A. Backurs und V. Vassilevska Williams. *Tight Hardness Results for LCS and Other Sequence Similarity Measures*. Proceedings of the 56th Annual Symposium on Foundations of Computer Science (FOCS), pp. 59–78, 2015.
- [22] S. Epp. *Der Subgraph Algorithmus: Effiziente Graphverifikation mittels Signatur-Arrays und zyklischer Rotation.*, 2026. <https://gitlab.com/epp-group/subgraph>
- [23] C. E. Shannon. *A Mathematical Theory of Communication*. The Bell System Technical Journal, 27(3):379–423, 1948.
- [24] INCOSE. *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*, 4th Edition. International Council on Systems Engineering, Wiley, 2015.
- [25] SAE International. *AS5506: Architecture Analysis and Design Language (AADL)*. SAE International, 2004.
- [26] Modelica Association. *Modelica – A Unified Object-Oriented Language for Systems Modeling, Language Specification Version 3.3*. Modelica Association, 2014.
- [27] C. A. R. Hoare. *An Axiomatic Basis for Computer Programming*. Communications of the ACM, 12(10):576–580, 1969.
- [28] NVIDIA Corporation. *CUDA C++ Programming Guide*. NVIDIA, 2023.
- [29] P. Indyk und R. Motwani. *Approximate Nearest Neighbors: Towards Removing the Curse of Dimensionality*. Proceedings of the 30th Annual ACM Symposium on Theory of Computing (STOC), pp. 604–613, 1998.
- [30] IEC. *IEC 62443: Industrial Communication Networks – Network and System Security*. International Electrotechnical Commission, 2018.
- [31] M. Chen et al. *Evaluating Large Language Models Trained on Code*. arXiv preprint arXiv:2107.03374, 2021.

- [32] L. C. Lamb, A. Garcez, M. Gori, M. Prates, P. Avelar und M. Vardi. *Graph Neural Networks Meet Neural-Symbolic Computing: A Survey and Perspective*. Proceedings of the 29th International Joint Conference on Artificial Intelligence (IJCAI), pp. 4877–4884, 2020.
- [33] M. Grieves. *Digital Twin: Manufacturing Excellence Through Virtual Factory Replication*. White Paper, Florida Institute of Technology, 2014.
- [34] OMG. *UML Profile for MARTE: Modeling and Analysis of Real-Time and Embedded Systems, Version 1.2*. Object Management Group, 2019.
- [35] C. U. Smith. *Performance Engineering of Software Systems*. Addison-Wesley, 1990.
- [36] W. J. Stewart. *Introduction to the Numerical Solution of Markov Chains*. Princeton University Press, 1994.
- [37] L. Kleinrock. *Queueing Systems, Volume 1: Theory*. Wiley, 1975.
- [OMG(2015)] Object Management Group (OMG), *Unified Modeling Language, Version 2.5*, 2015.
- [OMG(2007)] Object Management Group (OMG), *XML Metadata Interchange (XMI), Version 2.1*, 2007.
- [OMG(2014)] Object Management Group (OMG), *Meta Object Facility (MOF) Core Specification, Version 2.5*, 2014.
- [Schmidt(2006)] Schmidt, D.C., *Model-Driven Engineering*, IEEE Computer, Vol. 39, No. 2, 2006.
- [Kopetz(2011)] Kopetz, H., *Real-Time Systems: Design Principles for Distributed Embedded Applications*, Springer, 2011.
- [Balsamo et al.(2004)] Balsamo, S., Di Marco, A., Inverardi, P., Simeoni, M., *Model-Based Performance Prediction in Software Development: A Survey*, IEEE Transactions on Software Engineering, Vol. 30, No. 5, 2004.
- [Bertolino und Mirandola(2005)] Bertolino, A., Mirandola, R., *CB-SPE Tool: Putting Component-Based Performance Engineering into Practice*, Proc. 7th Int. Symp. on Component-Based Software Engineering (CBSE), 2005.
- [Norris(1997)] Norris, J.R., *Markov Chains*, Cambridge University Press, 1997.
- [Lyapunov(1992)] Lyapunov, A.M., *The General Problem of the Stability of Motion*, Taylor & Francis, London, 1992. (Erstveröffentlichung 1892, übersetzt von A.T. Fuller.)
- [Little(1961)] Little, J.D.C., *A Proof for the Queuing Formula: $L = \lambda W$* , Operations Research, Vol. 9, No. 3, S. 383–387, 1961.
- [Gross et al.(2008)] Gross, D., Shortle, J.F., Thompson, J.M., Harris, C.M., *Fundamentals of Queueing Theory*, 4. Auflage, Wiley-Interscience, 2008.
- [Selic(2003)] Selic, B., *The Pragmatics of Model-Driven Development*, IEEE Software, Vol. 20, No. 5, S. 19–25, 2003.

- [Pooley und King(1999)] Pooley, R., King, P., *The Unified Modelling Language and Performance Engineering*, IEE Proceedings – Software, Vol. 146, No. 1, S. 2–10, 1999.
- [Hillston(1996)] Hillston, J., *A Compositional Approach to Performance Modelling*, Cambridge University Press, 1996.
- [Booch et al.(2005)] Booch, G., Rumbaugh, J., Jacobson, I., *The Unified Modeling Language User Guide*, 2. Auflage, Addison-Wesley, 2005.
- [Fowler(2003)] Fowler, M., *UML Distilled: A Brief Guide to the Standard Object Modeling Language*, 3. Auflage, Addison-Wesley, 2003.
- [Gamma et al.(1994)] Gamma, E., Helm, R., Johnson, R., Vlissides, J., *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley, 1994.
- [Brambilla et al.(2012)] Brambilla, M., Cabot, J., Wimmer, M., *Model-Driven Software Engineering in Practice*, Morgan & Claypool Publishers, 2012.
- [Kent(2002)] Kent, S., *Model Driven Engineering*, Proc. 3rd Int. Conf. on Integrated Formal Methods (IFM), LNCS 2335, Springer, S. 286–298, 2002.
- [France und Rumpe(2007)] France, R., Rumpe, B., *Model-Driven Development of Complex Software: A Research Roadmap*, Proc. Future of Software Engineering (FOSE), IEEE, S. 37–54, 2007.
- [Becker et al.(2009)] Becker, S., Koziol, H., Reussner, R., *The Palladio Component Model for Model-Driven Performance Prediction*, Journal of Systems and Software, Vol. 82, No. 1, S. 3–22, 2009.
- [Woodside et al.(2007)] Woodside, M., Franks, G., Petriu, D.C., *The Future of Software Performance Engineering*, Proc. Future of Software Engineering (FOSE), IEEE, S. 171–187, 2007.
- [Cortellessa et al.(2011)] Cortellessa, V., Di Marco, A., Inverardi, P., *Model-Based Software Performance Analysis*, Springer, 2011.
- [Kemeny und Snell(1976)] Kemeny, J.G., Snell, J.L., *Finite Markov Chains*, Springer, 1976.